📖

Elasticsearch란? / Elasticsearch 주요 활용 사례

✅ Elasticsearch란?

Elasticsearch의 의미를 인터넷에 검색해보면 아래와 같이 나온다.

> Elasticsearch는 오픈 소스 분산, RESTful 검색 및 분석 엔진, 확장 가능한 데이터 저장소 및 벡터 데이터베이스이다.
> - Elasticsearch 공식 홈페이지 -

너무 어렵게 적혀져있다. First Word 법칙에 따라 쉽게 바꿔서 이해해보자.

> Elasticsearch는 **검색, 데이터 분석에 최적화된 데이터베이스**이다.

이렇게 기억하고 있어도 충분하다.

✅ Elasticsearch의 주요 활용 사례

최근 현업에서 Elasticsearch는 크게 2가지 용도로 사용된다.

1. **데이터 수집 및 분석**

   Elasticsearch는 대규모 데이터(ex. 로그 등)를 수집 및 분석하는 데 최적화되어 있다. 주로 Elasticsearch(데이터 저장), Logstash(데이터 수집 및 가공), Kibana(데이터 시각화)를 같이 활용해 데이터를 수집 및 분석한다.

2. **검색 최적화**

   Elasticsearch는 데이터가 많더라도 뛰어난 검색 속도를 가지고 있고, 오타나 동의어를 고려해서 유연하게 검색할 수 있는 기능을 가지고 있다. 쿠팡이나 배달의민족의 검색 기능도 전부 Elasticsearch를 활용해 구현되어있다.

   (네이버나 구글은 자체적인 검색 엔진을 구축해서 사용한다.)

Elasticsearch를 처음 배우는 입장에서 2가지 활용 사례를 한 번에 익히려고 하면 방대한 학습량에 압도당할 것이다. 따라서 이 2가지 활용 사례 중에서 **'검색 최적화'** 사례에 초점을 맞춰서 배워볼 것이다.

이렇게 1가지 사례에 초점을 맞춰서 학습하다보면, 자연스럽게 Elasticsearch의 기본 기능을 빠르게 익힐 수 있게 된다. 그런 뒤에 Elasticsarch의 디테일한 기능이나 작동 원리에 대해서 조금씩 학습하면서 살을 붙여나가면 된다.

Elasticsearch 설치하기

✅ Elasticsearch 설치하기

OS 환경과 상관없이 간편하고 일관되게 설치하기 위해 Docker를 활용하자.

** 아래 과정은 Docker가 설치되어 있다는 전제 하에서 진행한다.

1. **compose.yml 파일 작성**

   compose.yml

```
services:
  elastic:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.4 # Elastic 8.17.4 버전의 이미지
    ports:
    - 9200:9200 # 9200번 포트에서 Elasticsearch 실행
    environment:
    # 아래 설정은 개발/테스트 환경에서 간단하게 테스트하기 위한 옵션 (운영 환경에서는 설정하면 안 됨)
    - discovery.type=single-node # 단일 노드 (지금은 알 필요 없음)
    - xpack.security.enabled=false # 보안 설정
    - xpack.security.http.ssl.enabled=false # 보안 설정
```

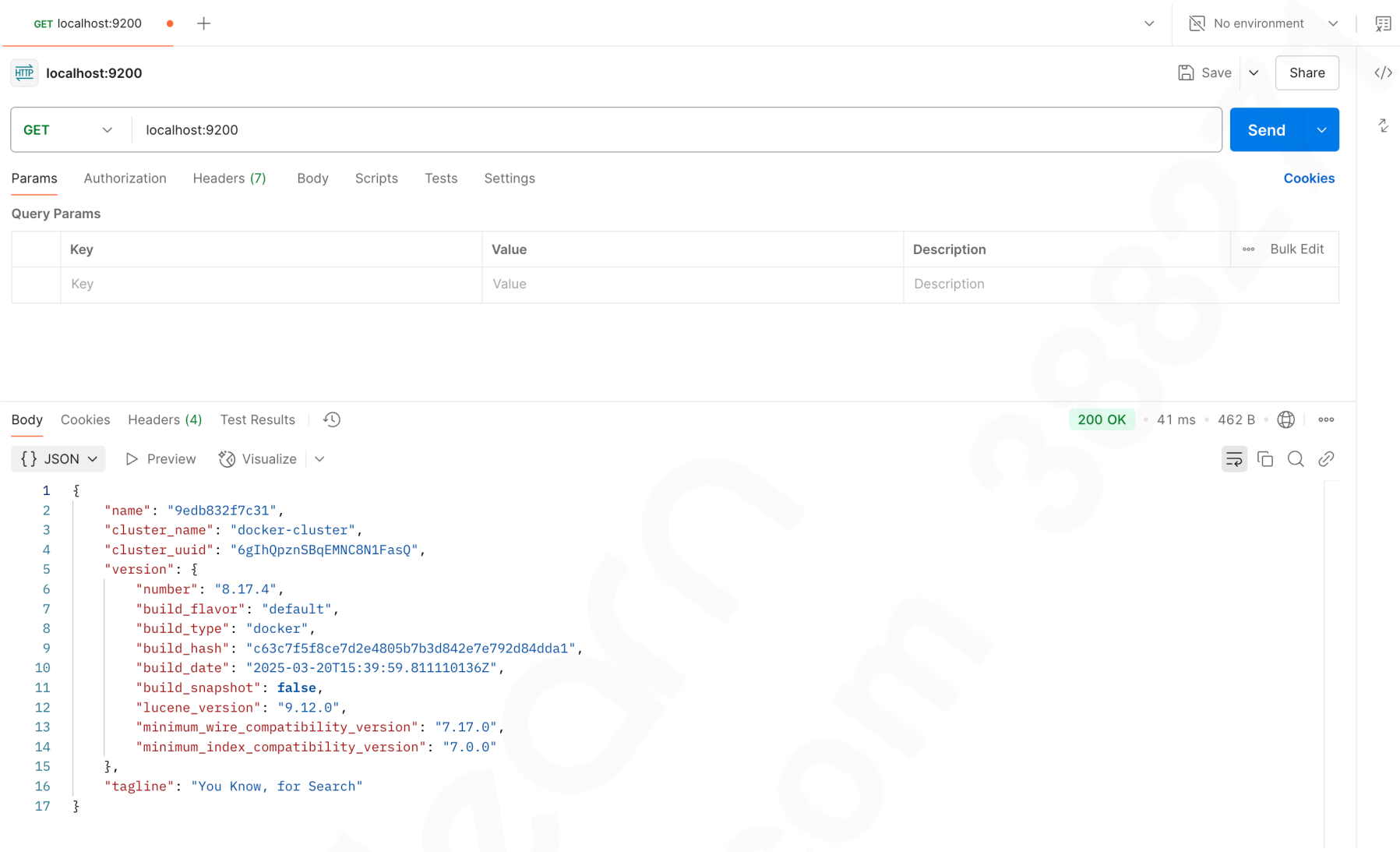
2. **compose 실행**

```
# compose.yml 파일이 존재하는 경로에서 아래 명령어를 실행시켜야 함
$ docker compose up
```

3. **잘 실행됐는 지 확인하기**

   Elasticsearch는 9200번 포트에서 실행되므로 Postman에서 localhost:9200으로 요청을 보내보자. 아래와 같이 응답을 받았다면 정상적으로 잘 실행이 된 것이다.

khb3025@gmail.com 3882171



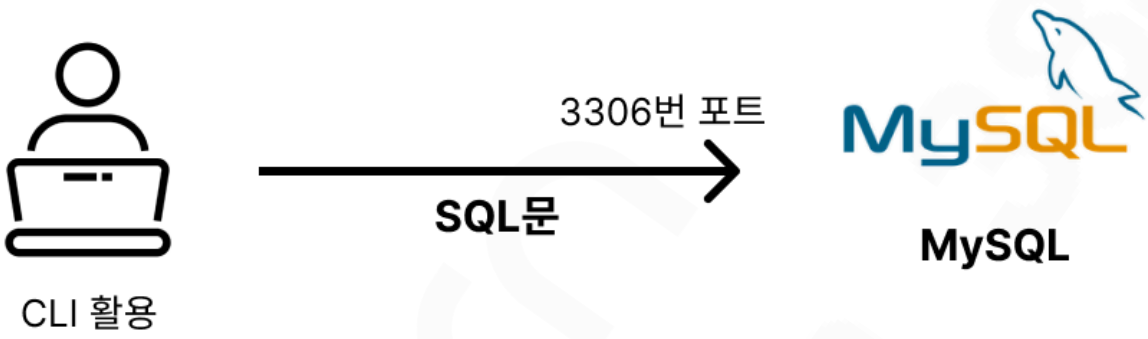
참고) 위 응답값에는 Elasitcsearch의 기본적인 정보가 포함되어 있다. 각 값이 어떤 의미를 가지는 지는 아직까지 알 필요 없다. 우선 Elasticsearch가 잘 작동하는 지 확인하는 용도로 쓰는 API라고만 정리해두자.

`GET /` : Elasticsearch 작동 여부 확인

khb302500@gmail.com

Elasticsearch 작동 방식 / GUI 툴

✅ Elasticsearch 작동 방식



MySQL과 소통하려면 SQL문이라는 방식으로 통신해야 한다. 이와 비슷하게 **Elasticsearch와 소통하려면 REST API라는 방식으로 통신**해야 한다. 통신 방식은 제작사에서 편리하다고 생각하는 방식을 자기네들이 판단해서 정한다.

[예시]

1. **데이터 삽입**

   MySQL

   ```
   INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');
   ```

   Elasticsearch

   ```
   curl -X POST "localhost:9200/users/_doc" -H 'Content-Type: application/json' -d'
   {
     "name": "Alice",
     "email": "alice@example.com"
   }'
   ```

2. **데이터 조회**

   MySQL

   ```
   SELECT * FROM users;
   ```

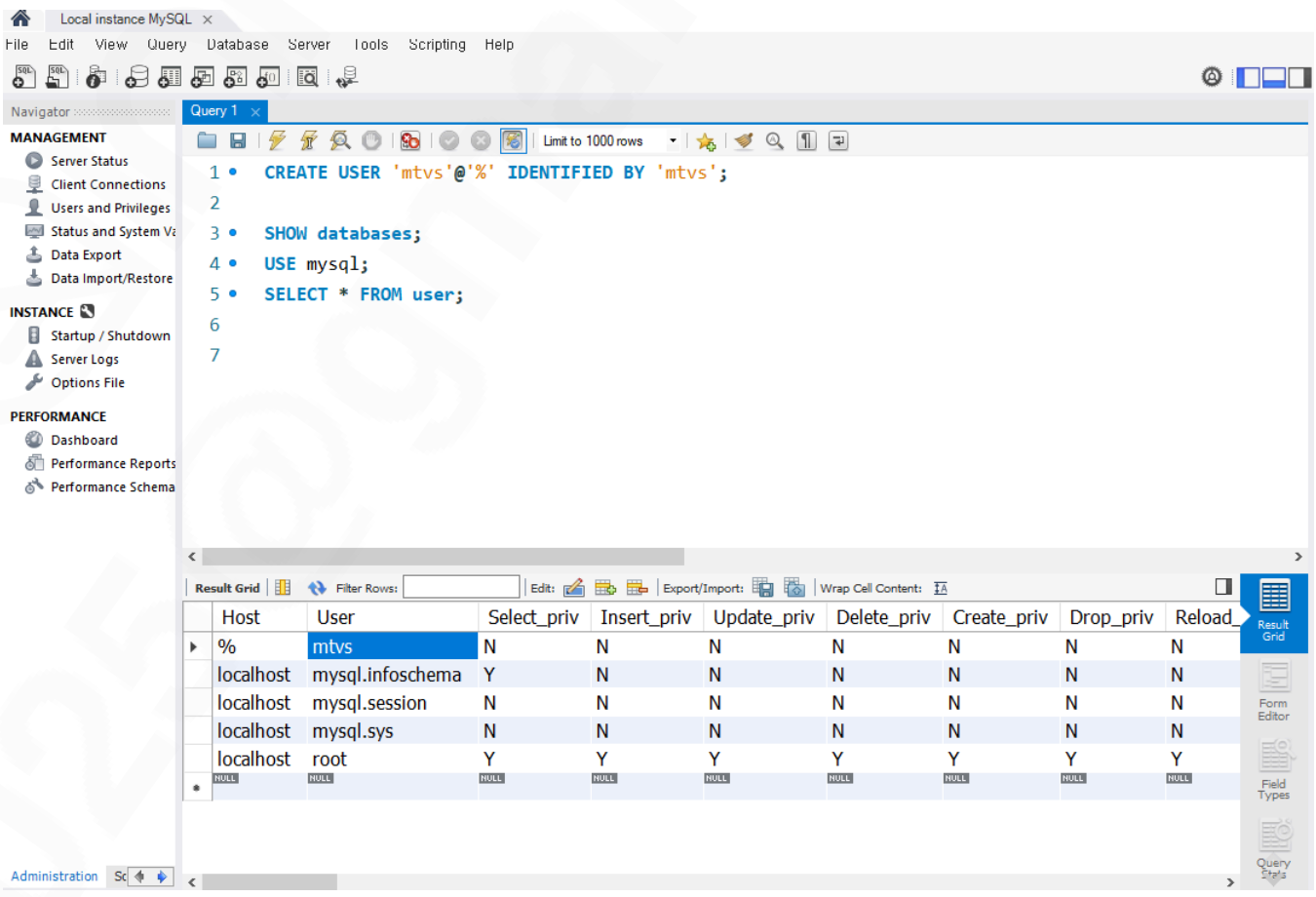
   Elasticsearch

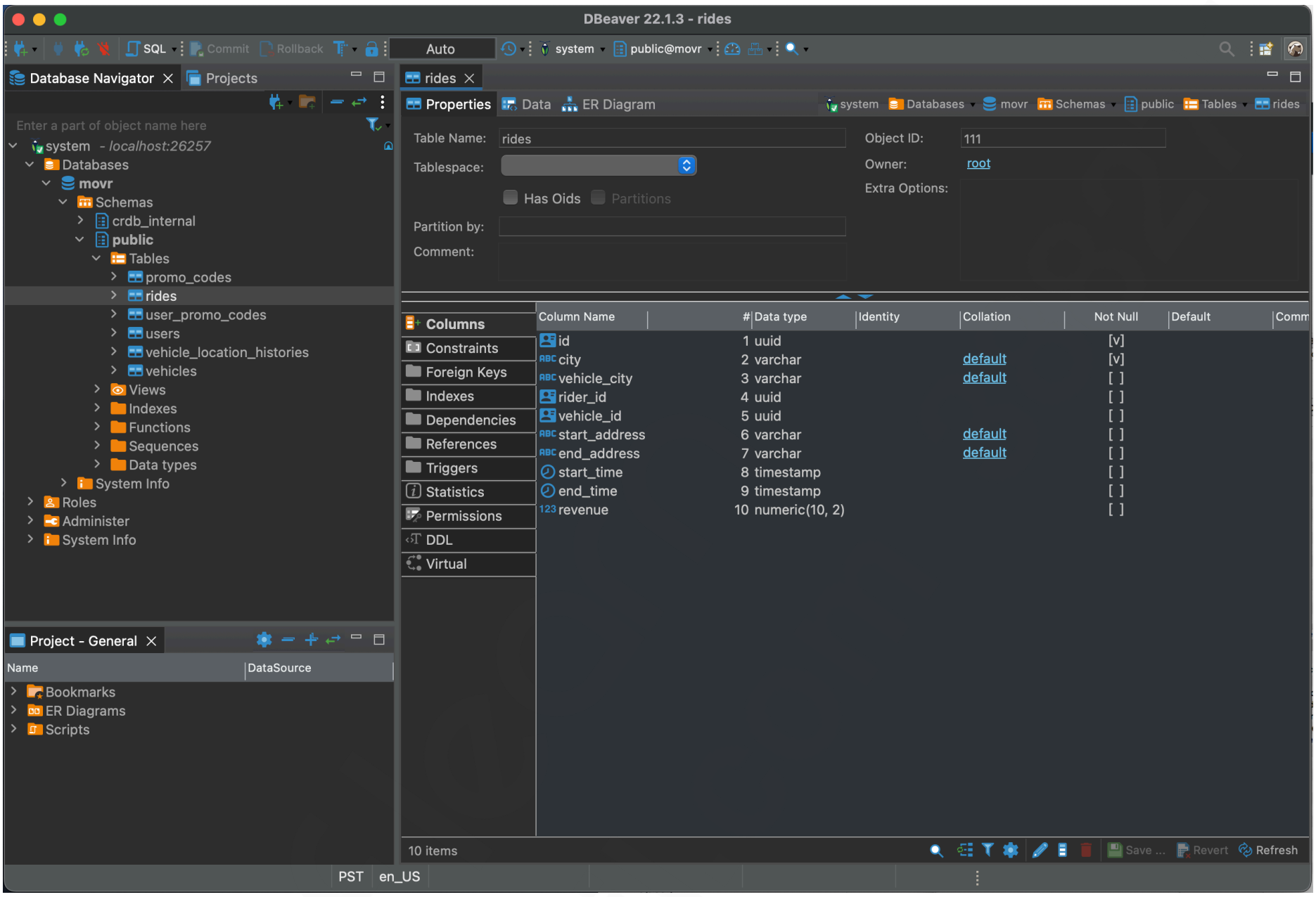
```
curl -X GET "localhost:9200/users/_search" -H 'Content-Type: application/json' -d'
{
  "query": {
    "match_all": {}
  }
}'
```

✅ Elasticsearch GUI 툴

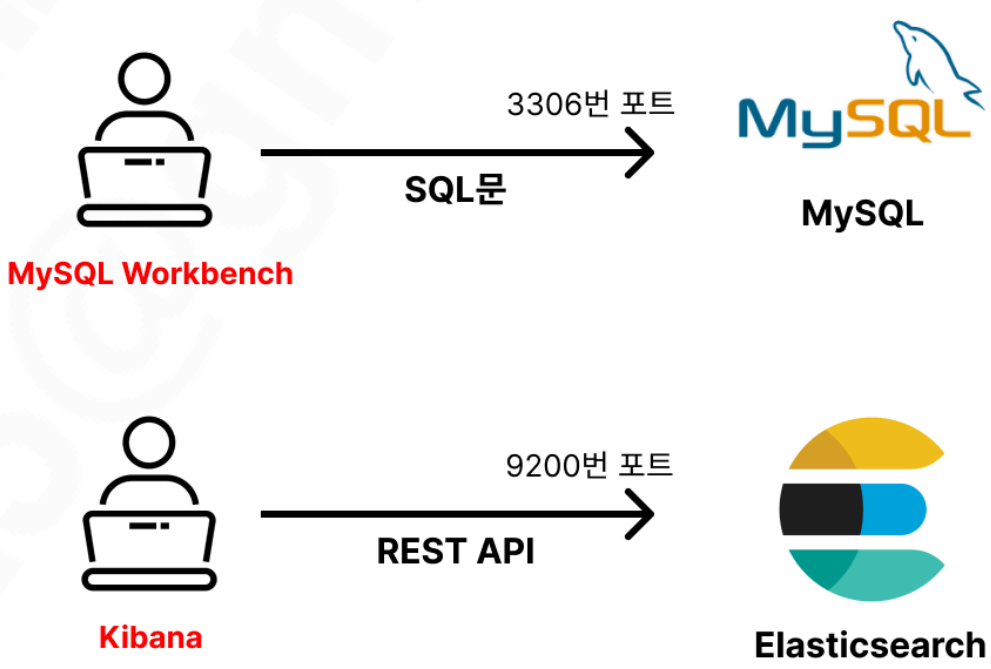
```
mysql> use db;
Database changed
mysql> SELECT t.* FROM pelanggan AS t;
+------+------------------+------------+
| kode | nama             | kode_pusat |
+------+------------------+------------+
|    1 | Pelanggan 1      |       NULL |
|    2 | Pelanggan 2      |       NULL |
|    3 | Anak Pelanggan 1 |          1 |
|    4 | Anak Pelanggan 1 |          1 |
|    5 | Anak Pelanggan 2 |          2 |
+------+------------------+------------+
5 rows in set (0.00 sec)

mysql>
```

MySQL을 사용할 때 위와 같이 매번 CLI로 SQL문을 입력하기가 불편하다. 그리고 조회된 데이터를 알아보는 것도 쉽지 않다. 그래서 MySQL Workbench, Dbeaver, Datagrip과 같은 GUI 툴을 많이 활용한다.





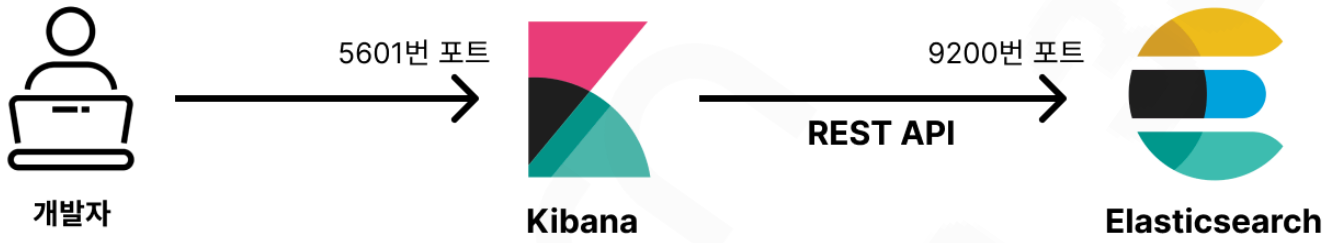
Elasticsearch도 매번 cURL이나 Postman으로 요청을 보내기가 불편한 편이라서, 조금 더 편리하게 조작할 수 있는 GUI 툴이 존재한다. 대표적인 GUI 툴이 **Kibana**이다.



다음 강의에서는 Kibana를 활용해 Elasticsearch를 조작해보자.

GUI 툴로 Elasticsearch 훨씬 쉽게 조작하기 (feat. Kibana)

✅ Kibana, Elasticsearch 아키텍처



✅ Kibana, Elasticsearch 설치 및 실행

1. **compose.yml 파일 작성**

   compose.yml

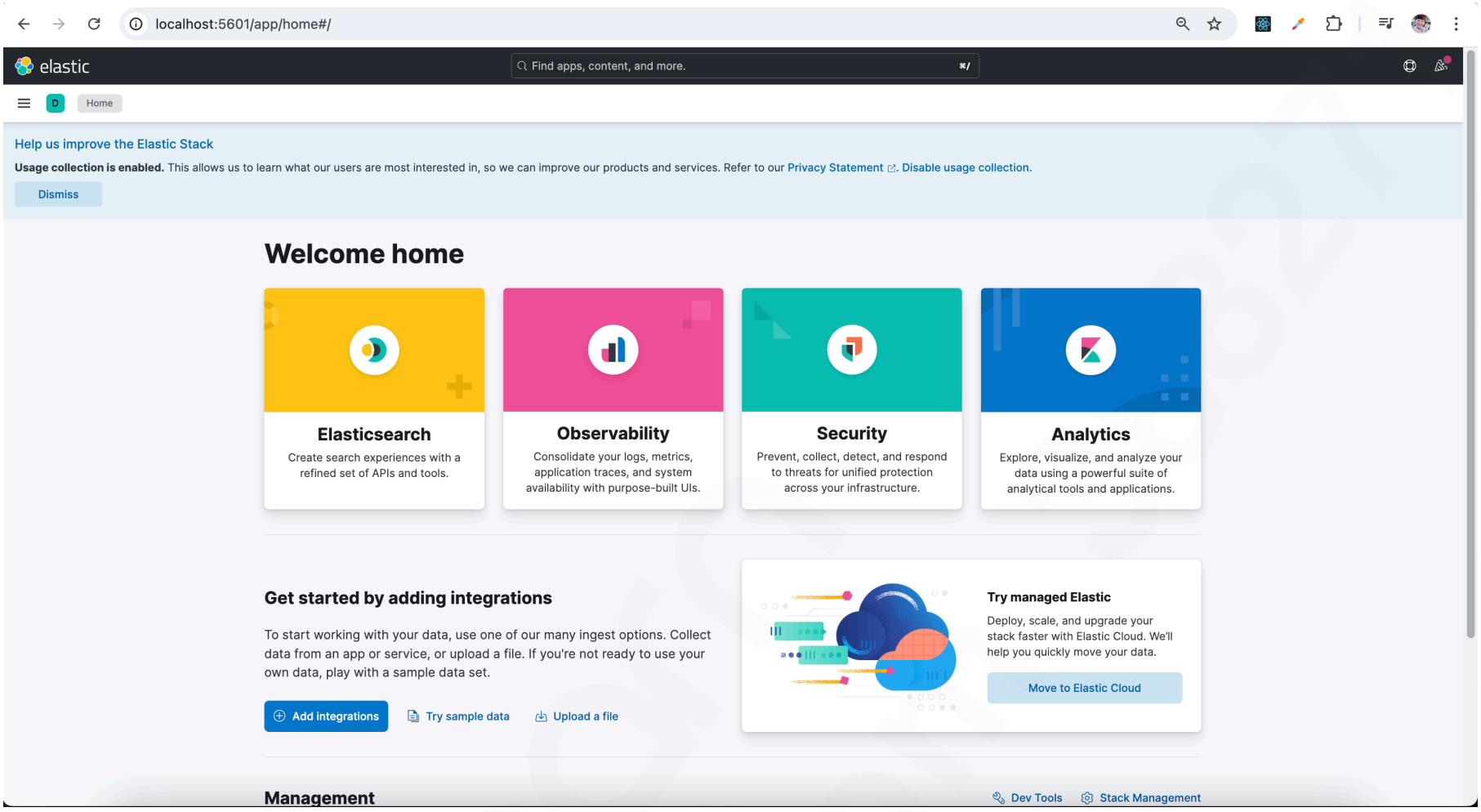
```
services:
  elastic:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.4 # 8.17.4 버전
    ports:
      - 9200:9200 # 9200번 포트에서 Elasticsearch 실행
    environment:
      # 아래 설정은 개발/테스트 환경에서 간단하게 테스트하기 위한 옵션 (운영 환경에서는 설정하면 안 됨)
      - discovery.type=single-node # 단일 노드 (지금은 알 필요 없음)
      - xpack.security.enabled=false # 보안 설정
      - xpack.security.http.ssl.enabled=false # 보안 설정
  kibana:
    image: docker.elastic.co/kibana/kibana:8.17.4 # 8.17.4 버전
    ports:
      - 5601:5601 # 5601번 포트에서 kibana 실행
    environment:
      - ELASTICSEARCH_HOSTS=http://elastic:9200 # kibana가 통신할 Elasticsearch 주소 알려주기
```

2. **compose 실행**

```
# compose.yml 파일이 존재하는 경로에서 아래 명령어를 실행시켜야 함
$ docker compose up -d # -d : 백그라운드에서 실행
```

3. **잘 실행됐는 지 확인하기**

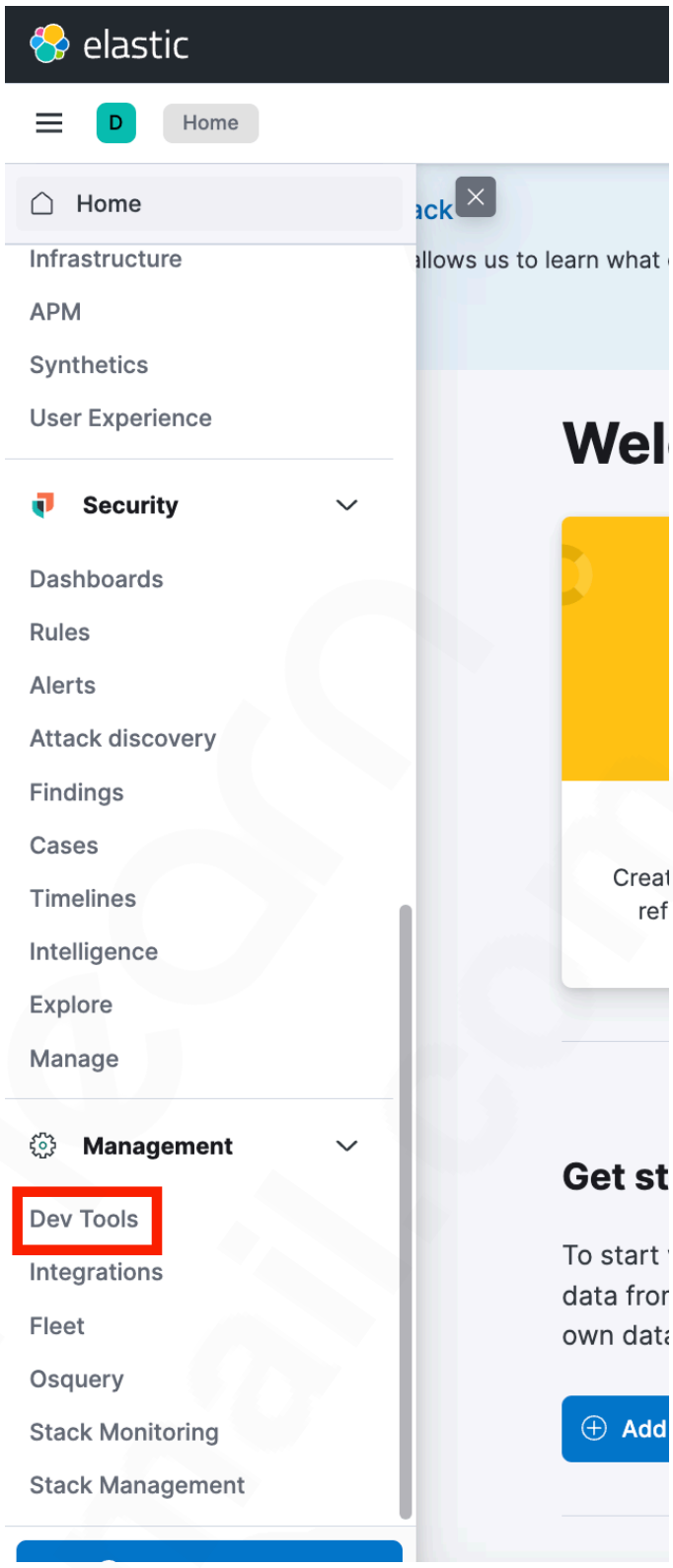
   Kibana는 5601번 포트에서 실행되므로 브라우저에서 http://localhost:5601로 접속해보자. 아래와 같은 화면이 뜬다면 잘 실행된 것이다.



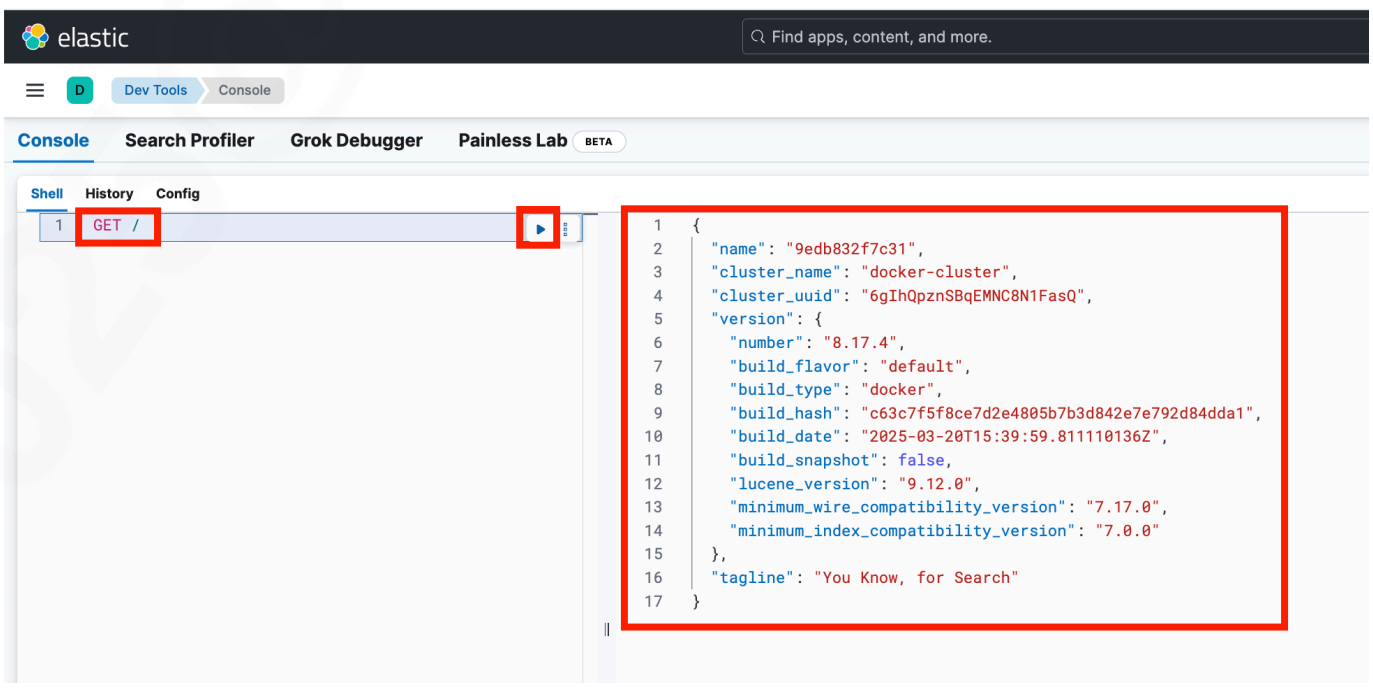
✅Kibana로 Elasticsearch 조작하기

1. Kibana의 `Dev Tools` 메뉴로 들어가기

khb302@gmail.com



2. **Kibana로 Elasticsearch에 요청 보내보기**



- 이전에 Elasitcsearch가 잘 작동하는 지 확인하기 위해 Postman을 활용해 http://localhost:9200으로 요청을 보냈다. Kibana에서는 http://localhost:9200 부분은 쓰지 않고 훨씬 간단하게 입력해서 요청을 보낼 수 있다.
- 요청을 보내면 오른쪽 화면에서 Elasitcsearch가 응답한 결과값을 보여준다.

👨🏻‍🏫 Kibana를 활용해 Elasticsearch를 조작하는 방법에 대해 알아봤다. 그럼 다음 강의에서는 Elasticsearch의 기본 개념에 대해 알아보자.

kkb3025@gmail.com

📖

Elasticsearch의 기본 용어 정리 (인덱스, 도큐먼트, 매핑, 필드)

✅ Elasticsearch의 기본 용어 정리

👨🏻‍🏫 MySQL과 Elasticsearch를 비교하면서 공부하면 Elasticsearch를 훨씬 빠르게 배울 수 있다.

MySQL은 데이터베이스이다.

데이터를 **저장/조회/수정/삭제**할 수 있다.

데이터를 저장하기 위해 **가장 먼저 테이블(table)을 만든다.**

테이블을 만들 때 **어떤 유형의 데이터를 넣을 지 스키마(schema)를 정의**한다.

테이블의 컬럼(column)에 맞게 데이터를 저장한다.

Elasitcsearch도 데이터베이스다.

Elasticsearch에 데이터를 **저장/조회/수정/삭제**할 수 있다.

데이터를 저장하기 위해 **가장 먼저 인덱스(index)를 만든다.**

인덱스를 만들 때 **어떤 유형의 데이터를 넣을 지 매핑(mapping)을 정의**한다.

테이블의 필드(field)에 맞게 데이터를 저장한다.

MySQL	Elasticsearch
테이블(table)	인덱스(index)
컬럼(column)	필드(field)
레코드(record), 로우(row)	도큐먼트(document)
스키마(schema)	매핑(mapping)

참고) ‘MySQL에서의 인덱스(index)’와 ‘Elasticsearch의 인덱스(index)’는 완전 별개의 개념이니 구분해서 이해하도록 하자.

✅ Elasticsearch의 검색 기능을 빨리 배워서 써먹고 싶어요!

Elasticsearch는 검색에 최적화된 기능을 많이 가지고 있다.

- 동의어를 고려한 검색
- 오타를 고려한 검색
- 검색어 자동 완성 기능
- 검색어와 가장 관련성이 높은 순서대로 데이터를 조회하는 기능

이런 기능을 어떻게 구현하는 지가 제일 궁금하겠지만 이 기능들은 나중에 알아보자.

우선 Elasitcsearch의 가장 기본적인 기능인 CRUD(삽입/조회/수정/삭제) 기능을 먼저 알아보자.

그리고 Elasticsearch의 가장 기본적인 CRUD 기능을 활용해 Spring Boot 프로젝트를 만들어볼 것이다.

“최소한의 개념만 배운 뒤에 실전에서 연습해봐야 한다. 이렇게 배우면 정말 빨리 배운다.”

> 헬스를 처음 배울 때부터 50kg, 100kg와 같이 무거운 무게를 들면 다친다. 작은 무게부터 드는 걸 연습하고 차근차근 무게를 늘려나가야 한다.

Elasticsearch를 활용해 CRUD 기능을 만들기 위해, 가장 먼저 **인덱스(index) 만드는 방법**과 **매핑(mapping) 정의 방법, 도큐먼트(document) 삽입 방법**을 알아보자.

@kfc3825.com
kfc3825

📖

인덱스 생성하기 / 매핑 정의하기 / 도큐먼트 삽입하기

✅ 인덱스(index) 생성, 조회 및 삭제

> **Elasticsearch에서 인덱스(index)를 생성**하는 건 **MySQL에서 테이블(table)을 생성**하는 것과 같다.

인덱스 생성 및 조회

```
# 인덱스 생성
# PUT /{인덱스명}
PUT /users

# 인덱스 생성됐는 지 확인하기
# GET /{인덱스명}
GET /users

# 없는 인덱스를 조회했을 때 응답값 확인하기
GET /abc
```

참고) 위 명령어를 입력했을 때 나오는 응답값의 모든 의미를 처음부터 다 알 필요없다. 그러니 부담갖지 말고 핵심 기능에 초점을 맞춰 학습하자. (파레토의 법칙)

인덱스 삭제

```
# 인덱스 생성
PUT /boards

# 인덱스 생성됐는 지 확인하기
GET /boards

# 인덱스 삭제
# DELETE /{인덱스명}
DELETE /boards

# 삭제됐는 지 확인하기
GET /boards
```

✅ 매핑(mapping) 정의

> **Elasticsearch에서 매핑(mapping)을 정의**하는 건 **MySQL에서 테이블의 스키마(schema)를 정의하는 것**과 같다.

```
# 매핑 정의하기
# PUT /{인덱스명}/_mappings
PUT /users/_mappings
{
  "properties": {
    "name": { "type": "keyword" },
    "age": { "type": "integer" },
```

```
    "is_active": { "type": "boolean" }
  }
}


# 정의한 매핑 확인하기
GET /users
```

참고) 매핑(mapping)의 **타입**이나 **특징**에 대해서는 나중에 다시 한 번 자세히 다룰 예정이다.

✅ 도큐먼트(document) 삽입 및 조회

> **Elasticsearch에서 도큐먼트(document)을 삽입**하는 건 **MySQL에서 테이블에 레코드(record)를 삽입하는 것**과 같다.

```
# 도큐먼트 삽입
# POST /{인덱스명}/_doc
POST /users/_doc
{
  "name": "Alice",
  "age": 28,
  "is_active": true
}

POST /users/_doc
{
  "name": "Bob",
  "age": 30,
  "is_active": false
}

# 모든 도큐먼트 조회
# GET /{인덱스명}/_search
GET /users/_search
```

도큐먼트 조회 시 응답값

```
{
  "took": 1,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 2,
      "relation": "eq"
    },
    "max_score": 1,
    "hits": [
```

```
{
  # _index : 인덱스명
  "_index": "users",

  # 데이터를 저장하면서 자동으로 생성된 '랜덤 고유 ID'
  "_id": "0imoJ5YBVwYHNUofQSQc",

  "_score": 1,

  # _source : 저장한 데이터가 들어있는 필드
  "_source": {
    "name": "Alice",
    "age": 28,
    "is_active": true
  }
},
{
  "_index": "users",
  "_id": "0ymoJ5YBVwYHNUofRSRX",
  "_score": 1,
  "_source": {
    "name": "Bob",
    "age": 30,
    "is_active": false
  }
 }
 ]
 }
}
```

참고) 지금으로써는 주석으로 표시한 필드의 의미만 해석할 수 있으면 된다. (파레토의 법칙)

이번 강의에서는 인덱스를 생성하고 도큐먼트를 넣어봄으로써 데이터를 넣는 전체적인 흐름을 맛만 봤다. 다음 강의에서 도큐먼트를 조작하는 방법(CRUD)에 대해 조금 더 자세히 알아보자.

kb302500@gmail.com

📖

도큐먼트 저장, 조회, 수정, 삭제하기

✅ 도큐먼트 저장

1. **id를 자동으로 생성해서 저장**

```
# 도큐먼트 저장 (id 자동 생성)
# POST /{인덱스명}/_doc
POST /users/_doc
{
  "name": "Jaeseong",
  "age": 20,
  "is_active": true
}

# 잘 저장됐는 지 확인하기
GET /users/_search
```

2. **id를 직접 지정해서 저장**

```
# 도큐먼트 저장 (id 직접 지정)
# POST /{인덱스명}/_create/{id}
POST /users/_create/1
{
  "name": "jscode",
  "age": 30,
  "is_active": true
}

# 잘 저장됐는 지 확인하기
GET /users/_search

# 이미 존재하는 id로 데이터 저장하면 어떻게 되는 지 확인하기
POST /users/_create/1
{
  "name": "jscode",
  "age": 30,
  "is_active": true
}
```

3. **id를 직접 지정해서 저장 (이미 id가 존재한다면 데이터 덮어씌움)** = UPSERT

```
# 도큐먼트 저장 및 업데이트
# PUT /{인덱스명}/_doc/{id}
PUT /users/_doc/2
{
  "name": "jason",
  "age": 30,
  "is_active": true
}
```

```
# 잘 저장됐는 지 확인하기
GET /users/_search

# 이미 존재하는 id로 데이터 저장하면 어떻게 되는 지 확인하기
POST /users/_doc/2
{
  "name": "jason2",
  "age": 30,
  "is_active": true
}

# 어떻게 됐는 지 확인하기
GET /users/_search
```

**** HTTP 메서드가 POST가 아니라 PUT임에 주의하자**

✅ 도큐먼트 조회

1. **(특정 인덱스의) 모든 도큐먼트 조회**

```
# GET /{인덱스명}/_search
GET /users/_search
```

2. **id로 특정 도큐먼트 조회**

```
# 특정 도큐먼트 조회
# GET /{index}/_doc/{id}
GET /users/_doc/1
GET /users/_doc/2
```

이 외에도 다양한 조건을 활용해 데이터를 조회하는 방법이 있다. 하지만 이 부분은 나중에 자세히 다룰 예정이다. 우선 기본적인 기능을 빠르게 배워 실전에서 빨리 써먹어보자. 그래야 전체 흐름을 빨리 파악할 수 있다.

✅ 도큐먼트 수정

1. **도큐먼트를 통째로 덮어씌우기**

```
# 기존 도큐먼트 확인
GET /users/_doc/1

# 특정 도큐먼트 수정
# 이 API는 데이터 저장 시에도 사용하는 API이다. (바로 위에서 언급했었음)
# PUT /{인덱스명}/_doc/{id}
PUT /users/_doc/1
{
  "name": "new"
}
```

```
# 수정됐는 지 확인하기
GET /users/_doc/1
```

2. **일부 필드만 수정**

```
# 기존 도큐먼트 확인
GET /users/_doc/2

# 특정 도큐먼트 수정
# POST /{인덱스명}/_update/{id}
POST /users/_update/2
{
  "doc": {
     "age": 10,
     "is_active": false
  }
}

# 수정됐는 지 확인하기
GET /users/_doc/2
```

✅ 도큐먼트 삭제하기

```
# 기존 도큐먼트 확인
GET /users/_doc/2

# id로 도큐먼트 삭제하기
# DELETE /{인덱스명}/_doc/{id}
DELETE /users/_doc/2

# 삭제됐는 지 확인하기
GET /users/_doc/2
```

Elasticsearch는 데이터베이스의 일종이라고 설명했었다. 데이터베이스의 핵심 기능인 CRUD(저장, 조회, 수정, 삭제)를 알아봤다. 이 기능만 가지고 실제 프로젝트에서 써먹어보자.

🎯 [실습] Spring Boot에 Elasitcsearch 연결하기 (feat. Spring Data Elasticsearch)

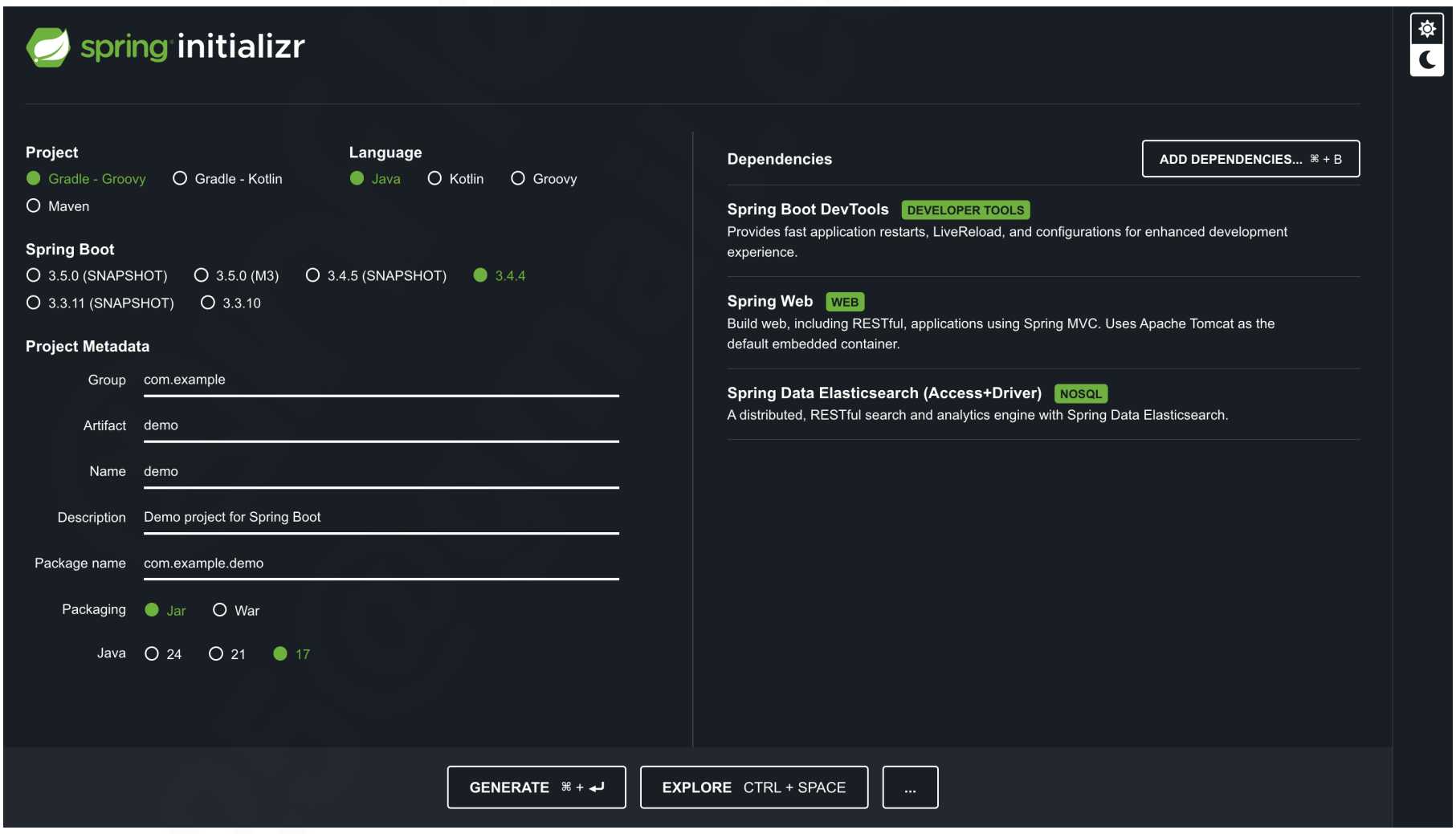
✅ Spring Boot에 Elasitcsearch 연결하기

> 동일한 환경에서 실습을 진행하기 위해 아래 버전을 사용할 것을 권장한다.
> - Spring Boot는 3.x.x 버전
> - JDK 17

1. **Spring Boot 프로젝트 셋팅**

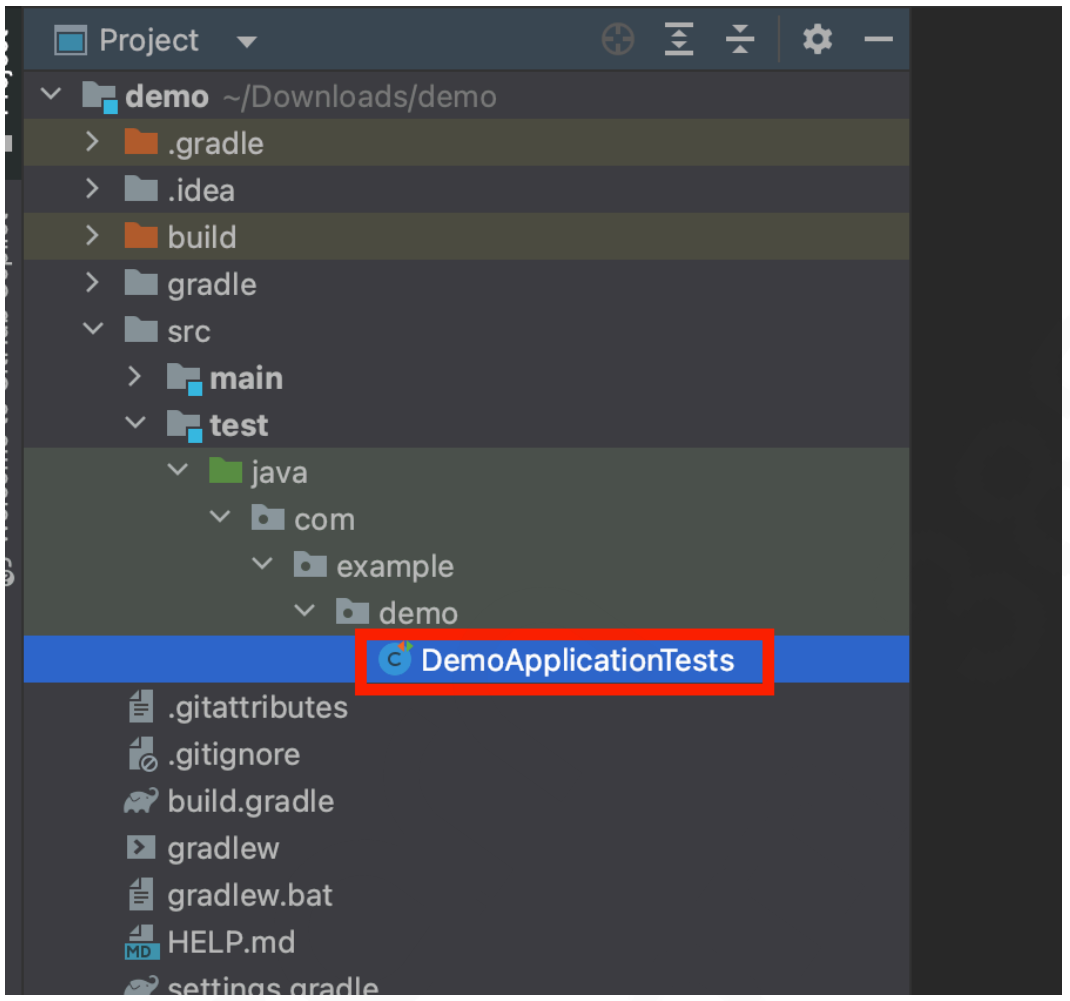
start.spring.io

https://start.spring.io/



- Java 17 버전을 선택하자. 아래 과정을 Java 17 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Spring Web`, `Spring Data Elasticsearch (Access+Driver)`를 선택해라.
  - `Spring Data Elasticsearch`는 `Spring Data JPA`와 비슷한 원리로 작동하는 Elasticsearch 라이브러리이다.

2. **불필요한 테스트 코드 삭제**



3. **application.yml에 Elasticsearch 연결을 위한 정보 작성하기**

   > 이 프로젝트에서는 `application.properties`를 지우고 `application.yml`을 생성했다.

   application.yml

```
spring:
  elasticsearch:
    uris: http://localhost:9200 # 연결할 Elasticsearch 주소

logging:
  level:
    org.elasticsearch.client: TRACE # Elasticsearch의 쿼리 로그를 출력
```

4. **Document 생성하기**

   UserDocument

```
@Document(indexName = "users") // users 인덱스의 Document임을 명시
public class UserDocument {
  @Id
  private String id; // Elasticsearch에서는 Document ID를 문자열(String)으로 다룸

  @Field(type = FieldType.Keyword) // 매핑 정의 : keyword 타입
  private String name;

  @Field(type = FieldType.Long) // 매핑 정의 : long 타입
  private Long age;

  @Field(type = FieldType.Boolean) // 매핑 정의 : boolean 타입
  private Boolean isActive;

  public UserDocument(String id, String name, Long age, Boolean isActive) {
    this.id = id;
    this.name = name;
```

```
  this.age = age;
  this.isActive = isActive;
}

public String getId() {
  return id;
}

public String getName() {
  return name;
}

public Long getAge() {
  return age;
}

public Boolean getIsActive() {
  return isActive;
  }

  public void setId(String id) {
    this.id = id;
  }

  public void setName(String name) {
    this.name = name;
  }

  public void setAge(Long age) {
    this.age = age;
  }

  public void setIsActive(Boolean isActive) {
    this.isActive = isActive;
  }
}
```

- **참고)** 응답값의 형태를 위해 `getActive()`라고 만들지 말고 `getIsActive()`라고 메서드명을 지어주자. 그리고 일관성을 위해 `setActive()`라고 만들지 말고 `setIsActive()`라고 메서드명을 지어주자.

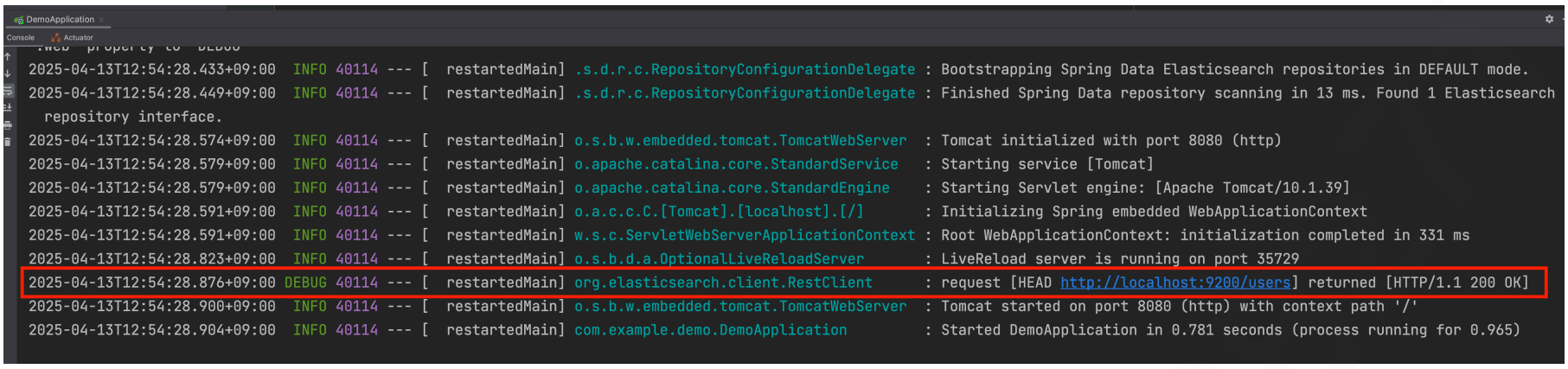
5. **Repository 생성하기**

   UserDocumentRepository

   ```
   public interface UserDocumentRepository extends ElasticsearchRepository<UserDocument, String> {
   }
   ```

   참고) 최소 1개 이상의 ElasticsearchRepository가 존재해야, Spring Boot를 실행시킬 때 Elasticsearch와 연결을 시도한다.

6. **Spring Boot 프로젝트 실행시키기**



Spring Boot 프로젝트를 실행시켰을 때 에러가 뜨지 않고 위 로그가 찍혀있다면 정상적으로 Elasticsearch와 연결이 된 것이다.

✅ Spring Boot와 Elasticsearch가 연결이 안 됐을 때 발생하는 에러

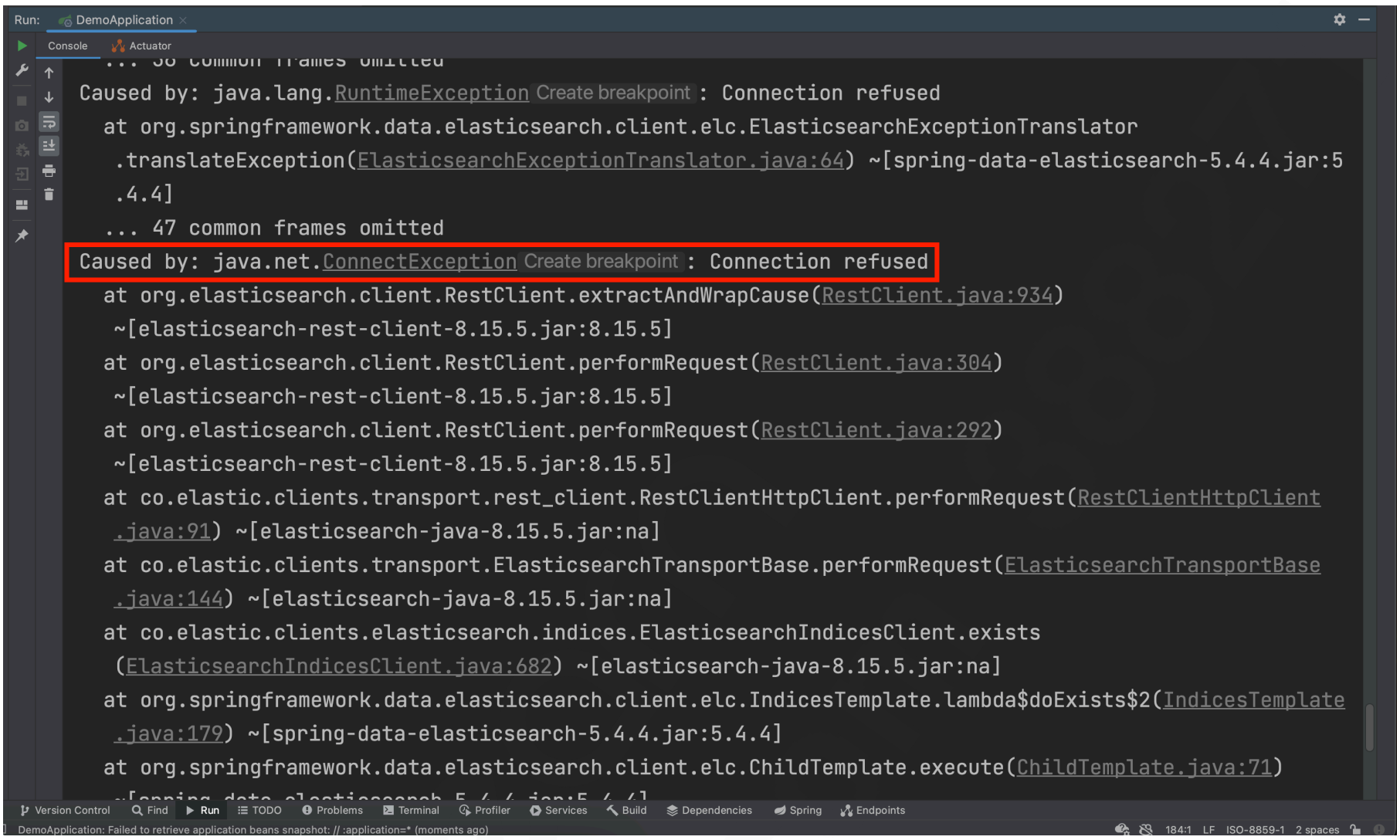
에러를 일부러 발생시켜서 어떤 식으로 에러 메시지가 뜨는 지 테스트해보자. 이런 방식으로 에러 형태를 미리 봐두면 나중에 트러블 슈팅할 때 도움이 많이 된다.

1. **application.yml 수정하기**

```
spring:
  elasticsearch:
      # Elasticsearch가 실행이 안 되고 있다는 걸 가정 or 주소를 잘못 입력했을 때를 가정
    uris: http://localhost:9999

logging:
  level:
    org.elasticsearch.client: TRACE
```

2. **Spring Boot 프로젝트 실행시켜보기**

khb3025@gmail.com



Elasticsearch와 연결이 안 될 경우 `Connection refused` 라는 에러 메시지가 출력된다.

3. **다시 원래대로 application.yml 수정하기**

```
spring:
  elasticsearch:
    uris: http://localhost:9200

logging:
  level:
    org.elasticsearch.client: TRACE
```

✅ users 인덱스를 지우고 실행시켜보자.

이렇게도 작동시켜보고 저렇게도 작동시켜보면서, Spring Data Elasticsearch가 어떤 식으로 작동하는 지 파악하는 습관을 가지자.

1. **Kibana로 users 인덱스 삭제하기**

```
DELETE /users
```

2. **Spring Boot 프로젝트 실행시켜보기**

```
Run:  DemoApplication
Console  Actuator
2025-04-13T13:49:34.664+09:00  INFO 43252 --- [  restartedMain] o.apache.catalina.core.StandardService   :
 Starting service [Tomcat]
2025-04-13T13:49:34.664+09:00  INFO 43252 --- [  restartedMain] o.apache.catalina.core.StandardEngine    :
 Starting Servlet engine: [Apache Tomcat/10.1.39]
2025-04-13T13:49:34.676+09:00  INFO 43252 --- [  restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]       :
 Initializing Spring embedded WebApplicationContext
2025-04-13T13:49:34.676+09:00  INFO 43252 --- [  restartedMain] w.s.c.ServletWebServerApplicationContext :
 Root WebApplicationContext: initialization completed in 314 ms
2025-04-13T13:49:34.907+09:00  INFO 43252 --- [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       :
 LiveReload server is running on port 35729
2025-04-13T13:49:34.953+09:00 DEBUG 43252 --- [  restartedMain] org.elasticsearch.client.RestClient      :
 request [HEAD http://localhost:9200/users] returned [HTTP/1.1 404 Not Found]
2025-04-13T13:49:35.119+09:00 DEBUG 43252 --- [  restartedMain] org.elasticsearch.client.RestClient      :
 request [PUT http://localhost:9200/users] returned [HTTP/1.1 200 OK]
2025-04-13T13:49:35.155+09:00  INFO 43252 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  :
 Tomcat started on port 8080 (http) with context path '/'
2025-04-13T13:49:35.159+09:00  INFO 43252 --- [  restartedMain] com.example.demo.DemoApplication         :
 Started DemoApplication in 0.931 seconds (process running for 1.119)
Version Control  Find  Run  TODO  Problems  Terminal  Profiler  Services  Build  Dependencies  Spring  Endpoints
```

로그를 보면 users 인덱스를 조회해보고 인덱스가 없다고 판단해서, users 인덱스를 생성하는 것 같은 로그가 찍혀있다. 정말 users 인덱스를 조회해보자.

3. **Kibana에서 users 인덱스 조회해보기**

```
GET /users
```

Shell History Config

```
GET /users
```

```
{
  "users": {
    "aliases": {},
    "mappings": {
      "properties": {
        "_class": {
          "type": "keyword",
          "index": false,
          "doc_values": false
        },
        "age": {
          "type": "long"
        },
        "isActive": {
          "type": "boolean"
        },
        "name": {
          "type": "keyword"
        }
      }
    },
    "settings": {
      "index": {
        "routing": {
          "allocation": {
            "include": {
              "_tier_preference": "data_content"
            }
          }
        },
        "refresh_interval": "1s",
        "number_of_shards": "1",
        "provided_name": "users",
        "creation_date": "1744519775043",
        "number_of_replicas": "1",
        "uuid": "FnbUnfg0QHWepqTBZEy5zQ",
        "version": {
          "created": "8521000"
        }
      }
    }
  }
}
```

응답값

```
{
  "users": {
    "aliases": {},
    "mappings": {
      "properties": {
        "_class": {
          "type": "keyword",
          "index": false,
          "doc_values": false
        },
        # UserDocument에서 정의한 Field에 맞게 매핑(mapping)이 정의되어있다.
        "age": {
          "type": "long"
        },
        "isActive": {
          "type": "boolean"
        },
        "name": {
          "type": "keyword"
        }
      }
    },
    "settings": {
      "index": {
        "routing": {
          "allocation": {
            "include": {
              "_tier_preference": "data_content"
            }
          }
        },
        "refresh_interval": "1s",
        "number_of_shards": "1",
        "provided_name": "users",
        "creation_date": "1744519775043",
        "number_of_replicas": "1",
        "uuid": "FnbUnfg0QHWepqTBZEy5zQ",
        "version": {
          "created": "8521000"
        }
      }
    }
  }
}
```


Spring Boot에서 정의한 `UserDocument`에 맞게 인덱스가 생성됐다는 걸 알 수 있다.

정리

Spring Data Elasticsearch는 Spring Boot 내부에서 정의해둔 Document의 인덱스가 존재하는 지 확인한다.

i. 만약 존재하지 않는다면 Document에서 정의해둔 설정에 맞게 인덱스를 추가한다.

ii. 만약 이미 존재한다면 아무런 일도 일어나지 않는다.

👨🏻‍🏫 지금까지 Spring Boot 프로젝트에 Elasticsearch를 연동시켜보고, Spring Data Elasticsearch가 어떤 식으로 작동하는 지도 간단히 알아봤다. 다음 강의에 이어서 Elasticsearch를 활용한 CRUD API를 구축해보자.

@kfc38821@gmail.com
khb302

🎯 [실습] Elasticsearch를 활용해 CRUD API 만들기

> 심플하게 만들기 위해 Service Layer 없이 Controller만 생성해서 CRUD API를 만들 것이다.

✅ 사용자 생성 API 만들기

1. **Controller 틀 만들기**

```
@RestController
@RequestMapping("users")
public class UserController {
  private UserDocumentRepository userDocumentRepository;

  public UserController(UserDocumentRepository userDocumentRepository) {
    this.userDocumentRepository = userDocumentRepository;
  }

  @PostMapping()
  public UserDocument createUser() {
  }
}
```

2. **Request DTO 만들기**

   UserCreateRequestDto

```
public class UserCreateRequestDto {
  private String id;
  private String name;
  private Long age;
  private Boolean isActive;

  public String getId() {
    return id;
  }

  public String getName() {
    return name;
  }

  public Long getAge() {
    return age;
  }

  public Boolean getIsActive() {
    return isActive;
  }
}
```

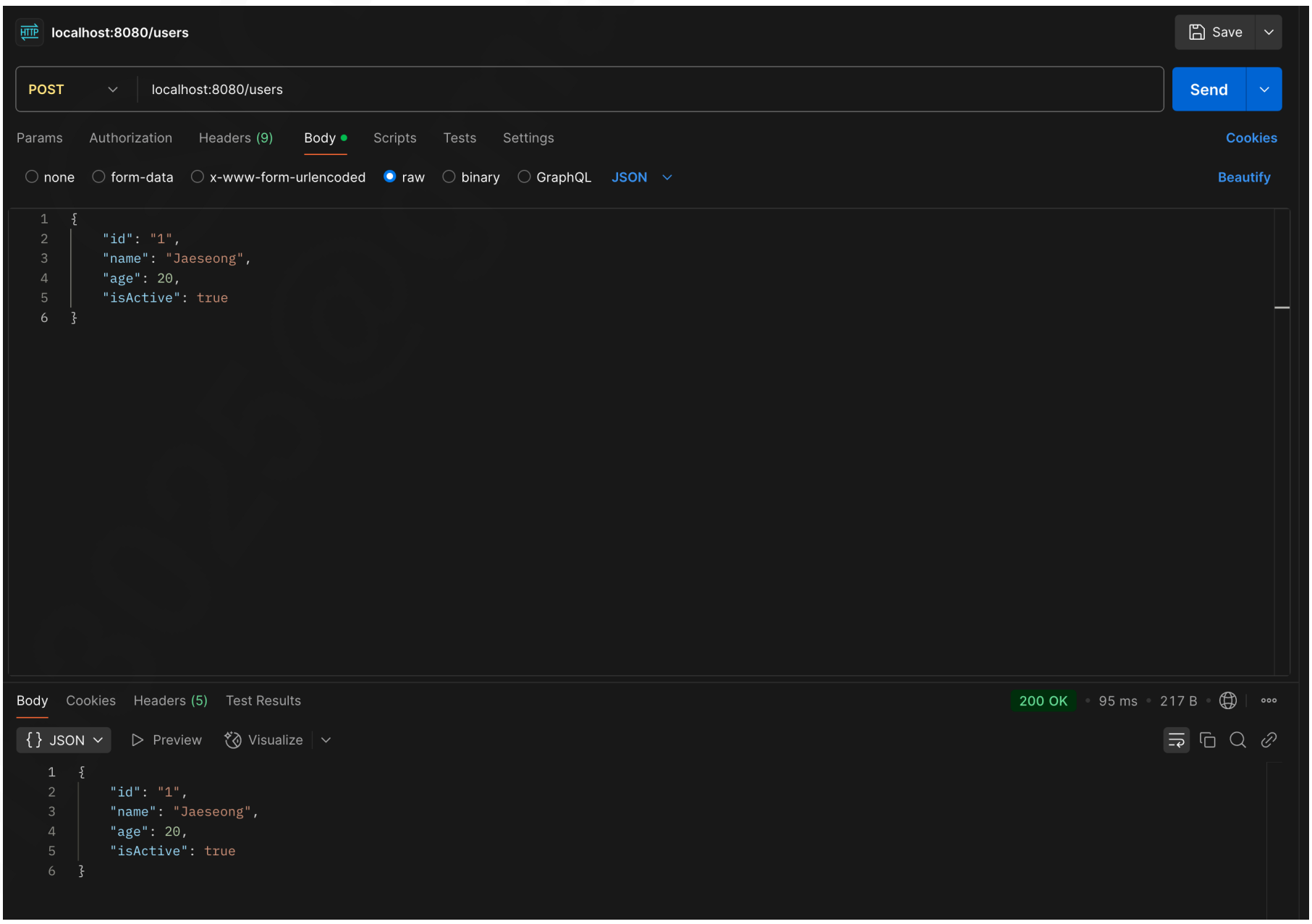
- **참고)** 일관성을 위해 메서드명을 `getIsActive()`라고 지어주자.

3. **Controller 완성하기**

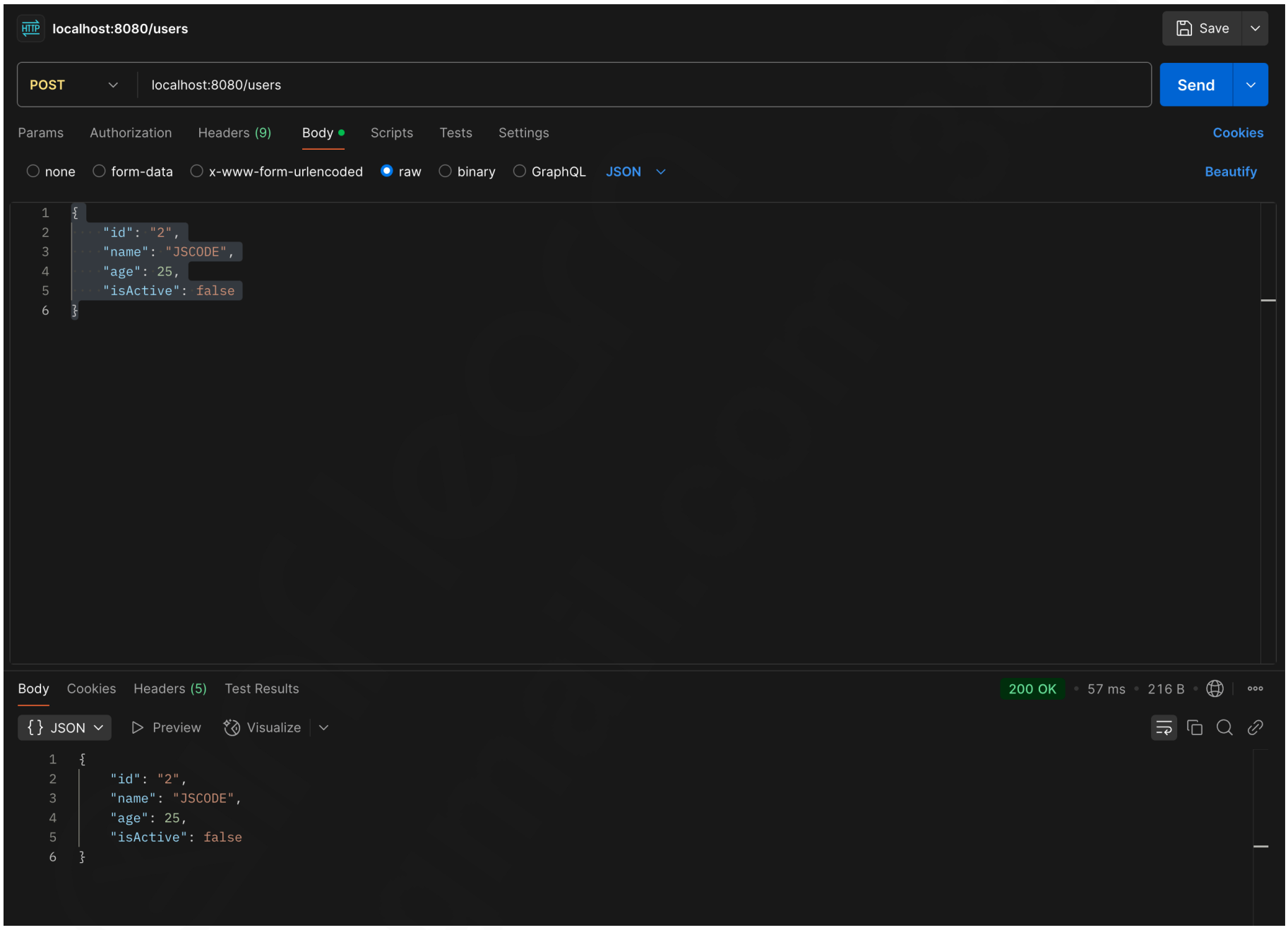
```
@RestController
@RequestMapping("users")
public class UserController {
  private UserDocumentRepository userDocumentRepository;

  public UserController(UserDocumentRepository userDocumentRepository) {
    this.userDocumentRepository = userDocumentRepository;
  }

  @PostMapping()
  public UserDocument createUser(@RequestBody UserCreateRequestDto requestDto) {
    UserDocument user = new UserDocument(requestDto.getId(), requestDto.getName(), requestDto.getAge(), reques
    return userDocumentRepository.save(user);
  }
}
```

4. **Postman으로 테스트해보기**

```
{
    "id": "1",
    "name": "Jaeseong",
    "age": 20,
    "isActive": true
}
```



```
{
   "id": "2",
   "name": "JSCODE",
   "age": 25,
   "isActive": false
}
```

✅ 사용자 조회 API 만들기

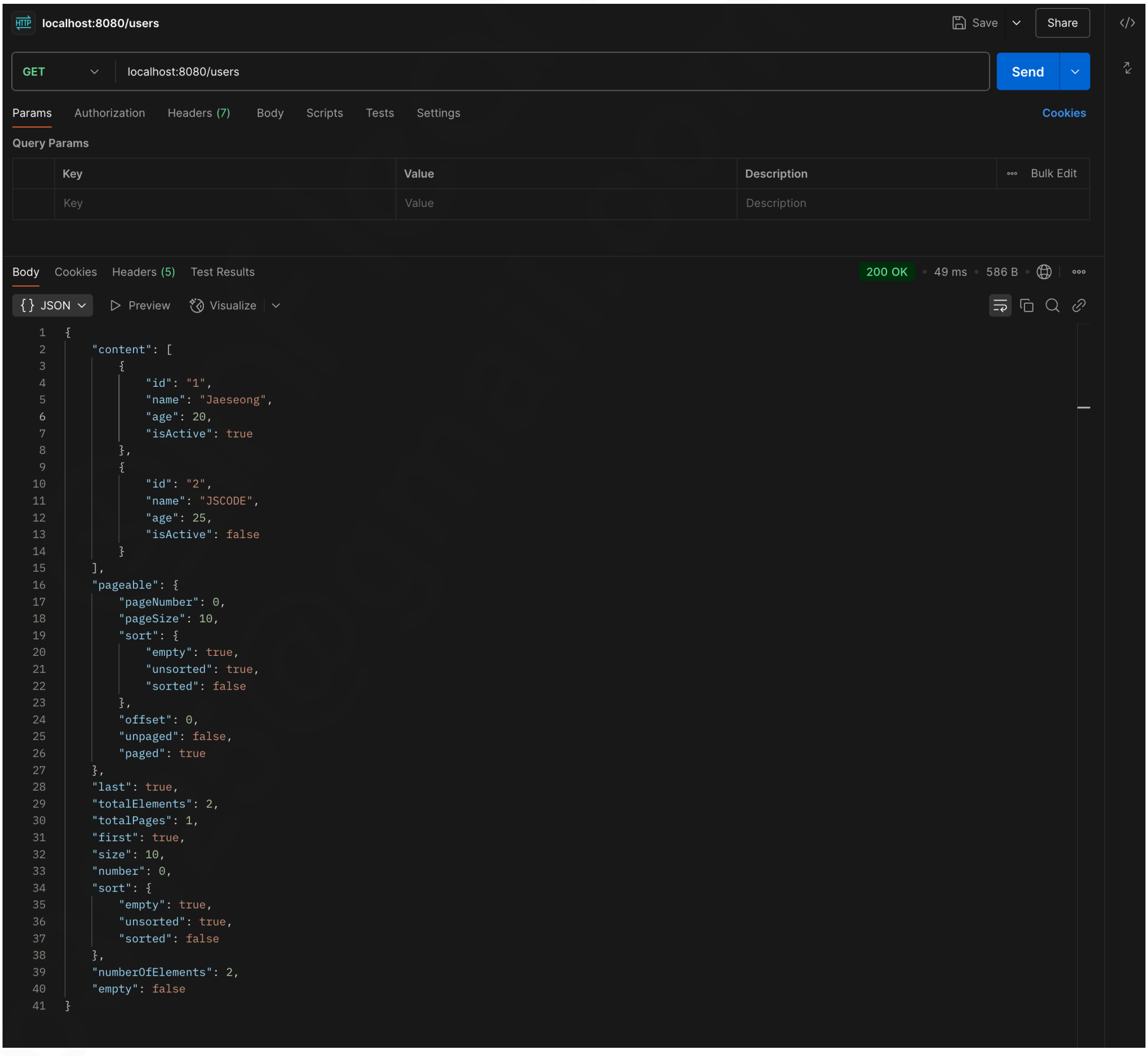
1. **Controller 만들기**

```
@RestController
@RequestMapping("users")
public class UserController {
 private UserDocumentRepository userDocumentRepository;

 public UserController(UserDocumentRepository userDocumentRepository) {
  this.userDocumentRepository = userDocumentRepository;
 }

 @PostMapping()
 public UserDocument createUser(@RequestBody UserCreateRequestDto requestDto) {
  UserDocument user = new UserDocument(requestDto.getId(), requestDto.getName(), requestDto.getAge(), reques
  return userDocumentRepository.save(user);
 }
```

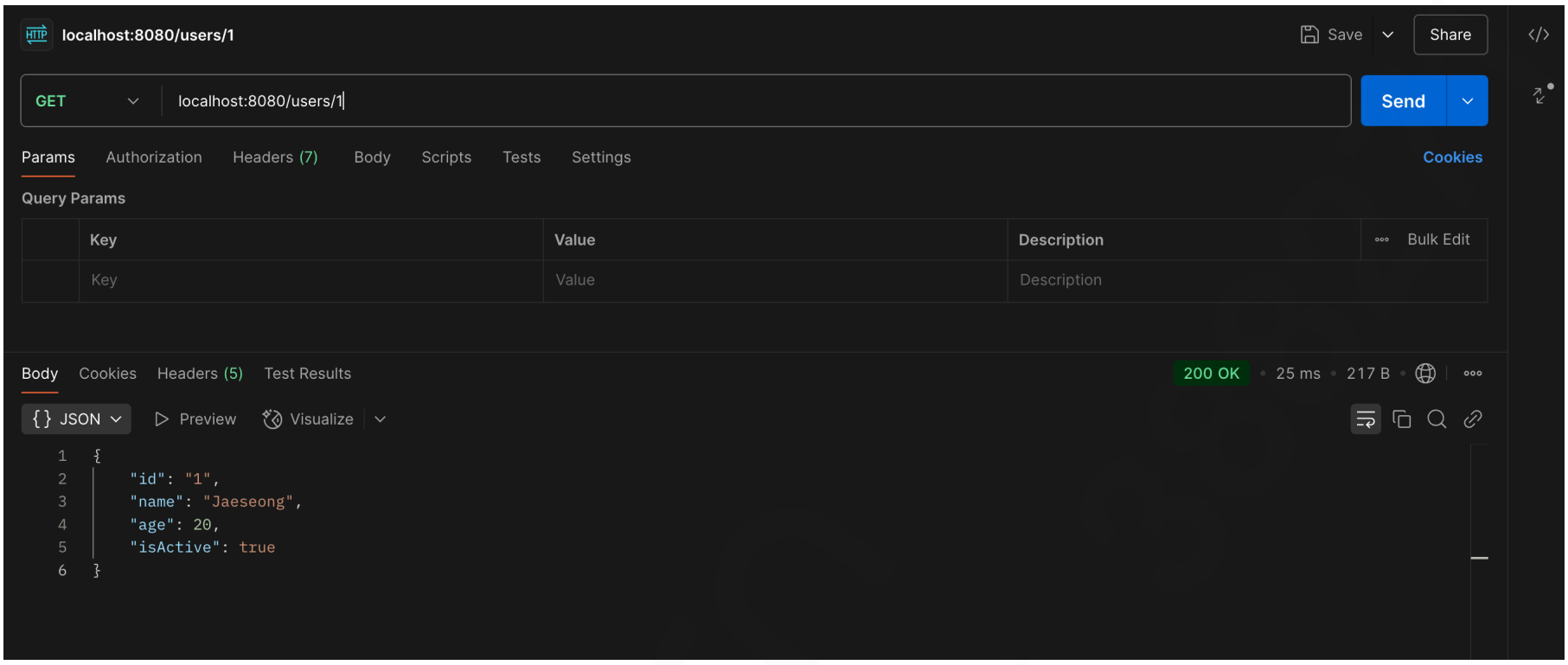
```
  @GetMapping()
  public Page<UserDocument> findUsers() {
    return userDocumentRepository.findAll(PageRequest.of(0, 10));
  }

  @GetMapping("/{id}")
  public UserDocument findUserById(@PathVariable String id) {
    return userDocumentRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("존재하지 않는 사용자입니다."));
  }
}
```

2. **Postman으로 테스트해보기**

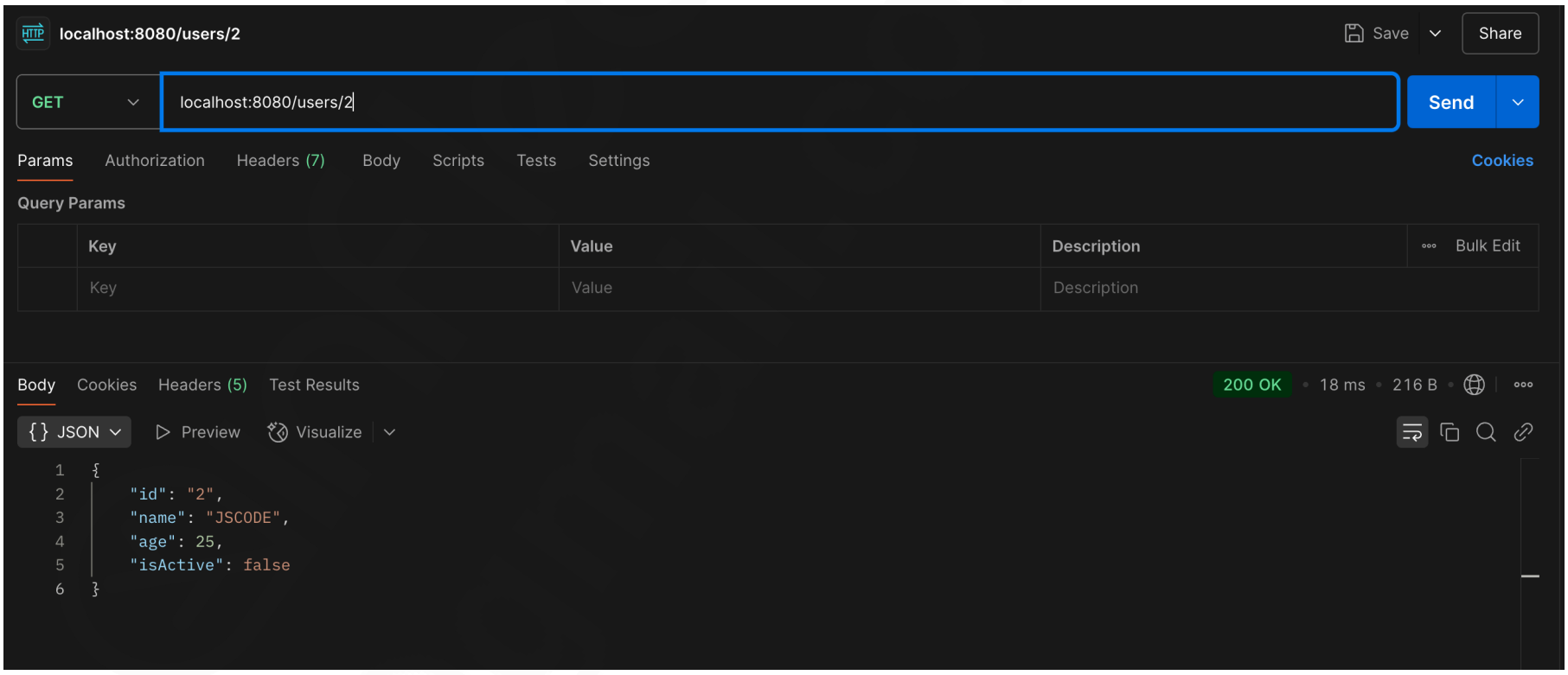
GET localhost:8080/users



GET localhost:8080/users/1



GET localhost:8080/users/2



✅ 사용자 정보 수정 API 만들기

1. **Request DTO 만들기**

   UserUpdateRequestDto

```
public class UserUpdateRequestDto {
  private String name;
  private Long age;
  private Boolean isActive;

  public String getName() {
    return name;
  }

  public Long getAge() {
    return age;
  }
```

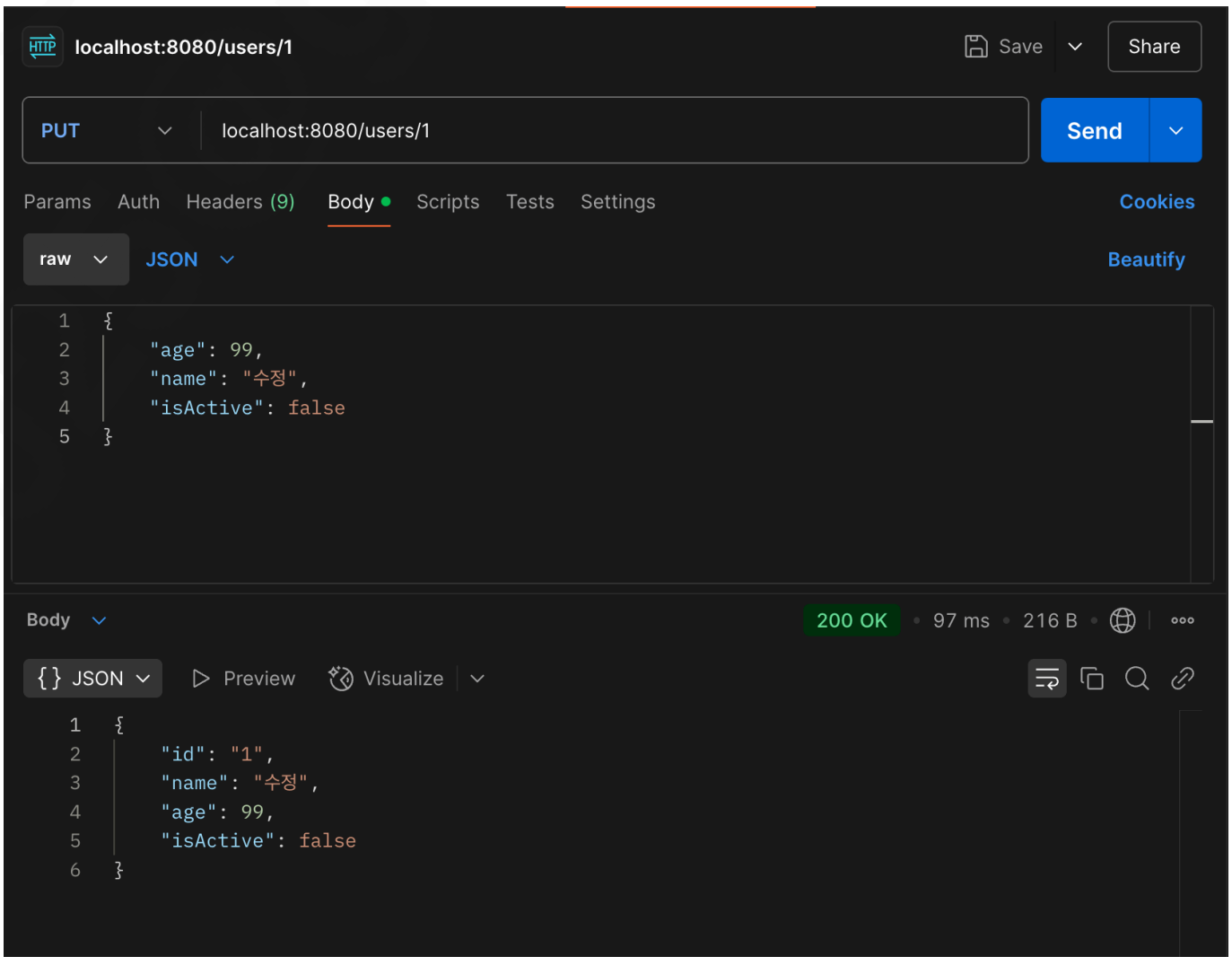
```
  public Boolean getIsActive() {
    return isActive;
  }
}
```

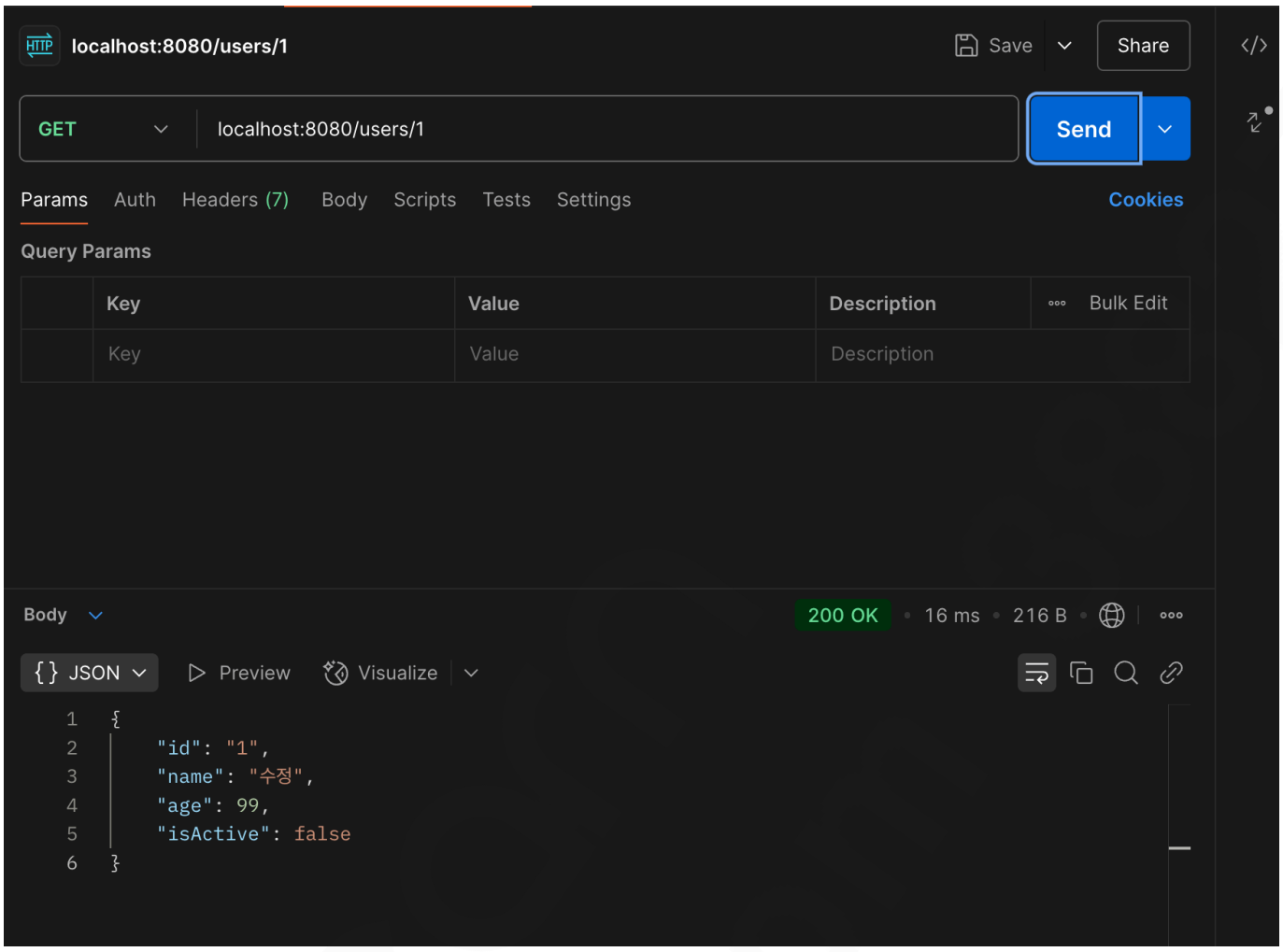
- **참고)** 일관성을 위해 메서드명을 `getIsActive()`라고 지어주자.

3. **Controller 완성하기**

```
@RestController
@RequestMapping("users")
public class UserController {
  private UserDocumentRepository userDocumentRepository;

  public UserController(UserDocumentRepository userDocumentRepository) {
    this.userDocumentRepository = userDocumentRepository;
  }

  ...

  @PutMapping("/{id}")
  public UserDocument updateUser(@PathVariable String id, @RequestBody UserUpdateRequestDto requestDto) {
    UserDocument existingUser = userDocumentRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("존재하지 않는 사용자입니다."));
    existingUser.setAge(requestDto.getAge());
    existingUser.setName(requestDto.getName());
    existingUser.setIsActive(requestDto.getIsActive());
    return userDocumentRepository.save(existingUser);
  }
}
```

3. **Postman으로 테스트해보기**



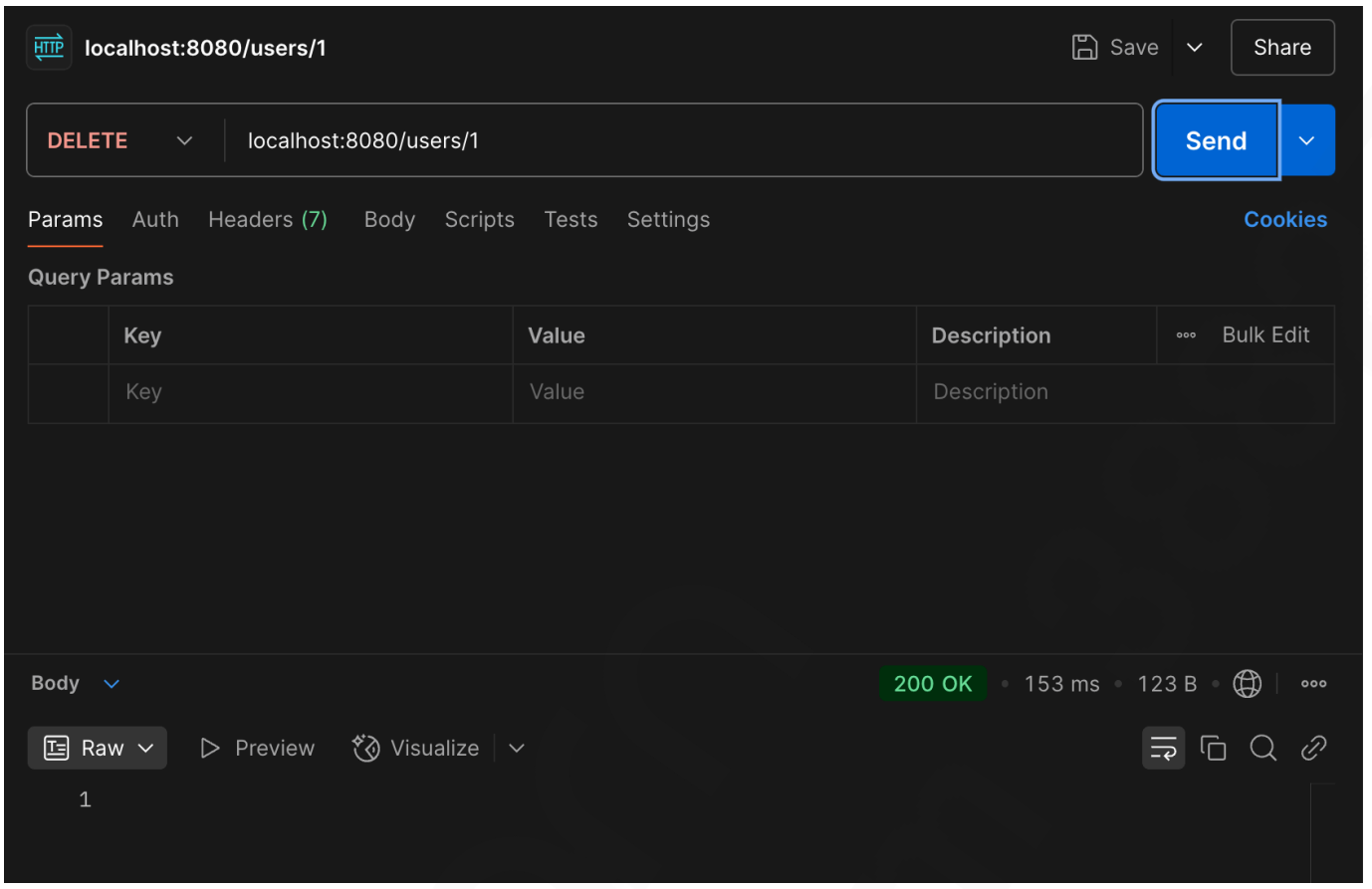


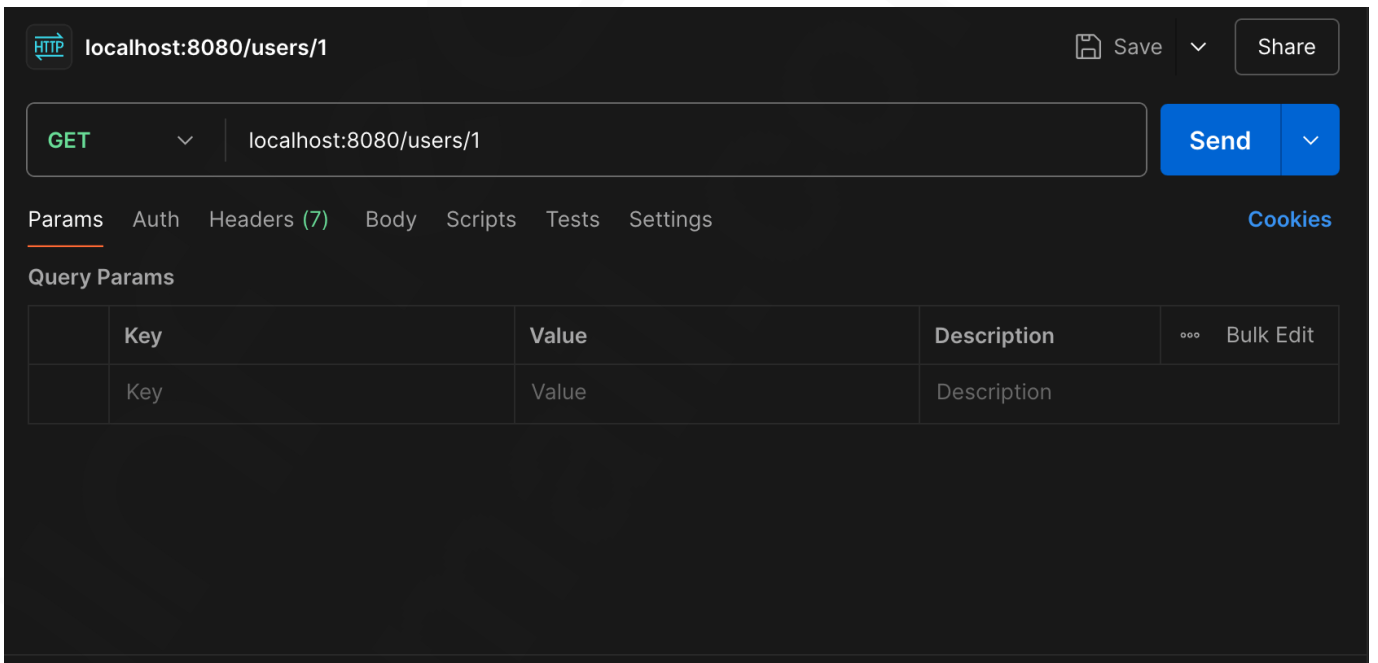
✅ 사용자 삭제 API 만들기

1. **Controller 완성하기**

```
@RestController
@RequestMapping("users")
public class UserController {
  private UserDocumentRepository userDocumentRepository;

  public UserController(UserDocumentRepository userDocumentRepository) {
    this.userDocumentRepository = userDocumentRepository;
  }

  ...

  @DeleteMapping("/{id}")
  public void deleteUser(@PathVariable String id) {
    UserDocument user = userDocumentRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("존재하지 않는 사용자입니다."));

    userDocumentRepository.delete(user);
  }
}
```

2. **Postman으로 테스트해보기**





지금까지 Spring Boot에 Elasticsearch를 연동시켜 간단한 CRUD API를 만들어봤다. 그런데 실제 현업에서는 Elasticsearch만 단독으로 쓰지 않고, 주로 MySQL과 같은 RDB와 Elasticsearch를 같이 사용하는 편이다. 하지만 이번 실습에서 Elasticsearch만 활용해 API를 만든 이유는, Spring Boot에서 Elasticsearch를 활용하는 걸 빠르게 경험하기 위해서였다.

그럼 다음 강의에서는 Elasticsearch의 핵심 기능인 검색 기능에 대해 본격적으로 알아보자.

(나중에 실제 현업 프로젝트와 유사하게 RDB와 Elasticsearch를 같이 활용해서 API를 만들어 볼 예정이니 조급해하지 말고 기초부터 하나씩 차근차근 배워나가자.)

📖

단어의 순서가 바뀌어도 검색이 가능하다 ?!

✅ 단어의 순서가 바뀌어도 검색이 가능하다 ?!

MySQL의 `products` 테이블의 `name` 컬럼에 `맥북 에어 13 M4`라고 저장해놓고, 특정 키워드로 데이터를 검색하려면 아래와 같이 SQL문을 작성한다.

```
SELECT * FROM products WHERE name = "%맥북 에어 13%";
```

만약 MySQL에서 `맥북 13 에어`라고 검색하면 데이터가 검색되지 않는다.

```
SELECT * FROM products WHERE name = "%맥북 13 에어%";
```

하지만 실제로 우리가 쿠팡에서 물건을 검색할 때 단어의 순서를 정확하게 기억하고 검색하진 않는다.

Apple 2025 맥북 에어 13 M4, 스카이 블루, M4 10코어, 16GB, 256GB, 한글

1,589,990원 로켓와우

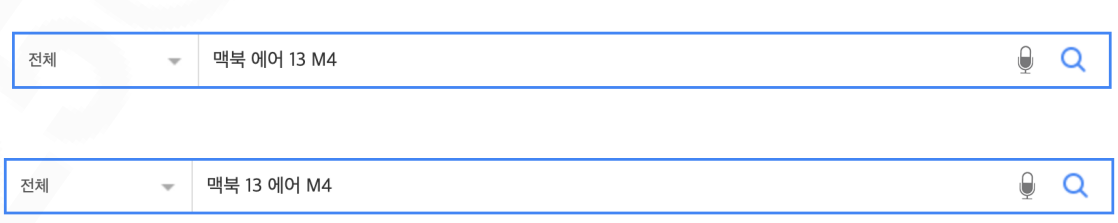
내일(금) 새벽 도착 보장

무료배송 · 무료반품

★★★★★ (55)

최대 1% 카드 즉시할인

최대 10,000원 적립



위 제품을 쿠팡에 검색해서 구매하고 싶다고 할 때, 우린 쿠팡에 `맥북 에어 13 M4`라고 검색할 수도 있지만, `맥북 13 에어 M4`라고 검색할 수도 있다. 이와 같이 쿠팡에서는 제품 이름의 단어 순서를 지키지 않고 검색해도 같은 결과를 얻을 수 있다. 위와 같은 검색 기능을 구현하려면 Elasticsearch를 활용하면 된다.

👨‍🏫 다음 강의에서 Elasticsearch를 활용하면 단어의 순서가 바뀌더라도 검색이 가능한 지 실제로 테스트해보고 눈으로 확인해보자.

🎯 [실습] Elasitcsearch의 검색 기능 테스트해보기

✅ Elasitcsearch의 검색 기능 테스트해보기

1. **인덱스 생성하기**

```
PUT /products
```

참고) 이렇게 배운 기능을 계속해서 써먹어봐야 Elasticsearch에 금방 익숙해질 수 있다.

2. **매핑 정의하기**

```
PUT /products/_mappings
{
  "properties": {
    "name": {
      "type": "text"
    }
  }
}
```

참고) **매핑(mapping)**의 **타입**이나 **특징**에 대해서는 나중에 다시 한 번 자세히 다룰 예정이다.

3. **도큐먼트 삽입**

```
# 도큐먼트 삽입
# POST /{인덱스명}/_doc
POST /products/_doc
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}
```

4. **도큐먼트가 잘 저장됐는지 확인하기**

```
# 모든 도큐먼트 조회
GET /products/_search
```

5. **도큐먼트 검색해보기**

```
# 특정 조건을 만족하는 도큐먼트 조회
# (name 필드에 '맥북 에어 13 M4'가 포함된 도큐먼트 조회)
GET /products/_search
{
  "query": {
    "match": {
      "name": "맥북 에어 13 M4"
    }
  }
}
```

참고) 특정 조건을 만족하는 도큐먼트를 조회할 때 사용하는 **match 쿼리**는 정말 많이 사용되니 까먹지 않게 따로 정리해두자.

응답값

```
{
  "took": 4,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "max_score": 1.1507283,

    # 조건을 만족하는 도큐먼트
    "hits": [
     {
       # _index : 인덱스명
       "_index": "products",

       # 데이터를 저장하면서 자동으로 생성된 '랜덤 고유 ID'
       "_id": "hU2MZ5YB5XfvBpblxof6",

       "_score": 1.1507283,

       # _source : 저장한 데이터가 들어있는 필드
       "_source": {
         "name": "Apple 2025 맥북 에어 13 M4 10코어"
       }
     }
    ]
  }
}
```

참고) 지금으로써는 주석으로 표시한 필드의 의미만 해석할 수 있으면 된다. (파레토의 법칙)

6. **단어의 순서를 바꿔서 검색해보기**

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "맥북 13 에어 M4"
    }
  }
}
```

응답값

```
{
  "took": 3,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "max_score": 1.1507283,
    "hits": [
      {
        "_index": "products",
        "_id": "hU2MZ5YB5XfvBpblxof6",
        "_score": 1.1507283,
        "_source": {
          "name": "Apple 2025 맥북 에어 13 M4 10코어"
        }
      }
    ]
  }
}
```


분명 단어의 순서를 바꿔서 검색했는데도 Elasticsearch가 찰떡같이 도큐먼트를 조회했다.

다음 강의에서는 어떻게 Elasticsearch가 단어의 순서를 바꿨는데도 도튜먼트를 조회해올 수 있는 지에 대한 원리를 알아보자.

kh3025@naver.com

📖

역인덱스(Inverted Index)란?

✅ 역인덱스(Inverted Index)란?

역인덱스(Inverted Index)는 **필드 값을 단어마다 쪼개서 찾기 쉽게 정리해놓은 목록**이다. 글로 이해하려면 어렵다. 예시를 바로 살펴보자.

```
POST /products/_create/1
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}

POST /products/_create/2
{
  "name": "Apple 2024 에어팟 4세대"
}

POST /products/_create/3
{
  "name": "Apple 2024 아이패드 mini A17 Pro"
}
```

위와 같은 3개의 도큐먼트를 Elasticsearch에 저장한다고 가정하자. 그러면 Elasticsearch는 내부적으로 아래와 같이 데이터를 저장해둔다.

1. **단어 단위로 자르기**

   `"name": "Apple 2025 맥북 에어 13 M4 10코어"`

   → [Apple, 2025, 맥북, 에어, 13, M4, 10코어]

   `"name": "Apple 2024 에어팟 4세대"`

   → [Apple, 2024, 에어팟, 4세대]

   `"name": "Apple 2024 아이패드 mini A17 Pro"`

   → [Apple, 2024, 아이패드, mini, A17, Pro]

2. **역인덱스로 저장**

   `products`**의** `name` **필드에 대한 역인덱스**

토큰(token)	도큐먼트 id
Apple	[1, 2, 3]
2025	[1]
맥북	[1]
에어	[1]
13	[1]
M4	[1]
10코어	[1]
2024	[2, 3]
에어팟	[2]
4세대	[2]
아이패드	[3]

토큰(token)	도큐먼트 id
mini	[3]
A17	[3]
Pro	[3]

- 필드값에서 추출돼 역인덱스에 저장된 단어를 보고 **토큰(token)**이라고 부른다.
- 생성된 역인덱스를 직접 눈으로 확인할 수는 없다. 시스템 내부적으로 생성될 뿐이다.

3. **검색을 할 경우 역인덱스를 활용**

`Apple 2024 아이패드`라고 검색하면 역인덱스를 활용해 일치하는 단어가 많은 도큐먼트를 우선적으로 조회한다.

토큰(token)	도큐먼트 id
Apple	**[1, 2, 3]**
2025	[1]
맥북	[1]
에어	[1]
13	[1]
M4	[1]
10코어	[1]
2024	**[2, 3]**
에어팟	[2]
4세대	[2]
아이패드	**[3]**
mini	[3]
A17	[3]
Pro	[3]

- id가 1인 도큐먼트 : 단어 1개 일치
- id가 2인 도큐먼트 : 단어 2개 일치
- id가 3인 도큐먼트 : 단어 3개 일치

이와 같은 Elasticsearch의 자체적인 로직으로 **score(점수)**를 매겨 score가 높은 순(`id=3` → `id=2` → `id=1`)으로 도큐먼트를 조회한다.
일반적으로 검색어와 관련성이 높으면 높을수록 **score(점수)**가 높게 측정된다.

▼ **참고)** 대략적인 score 계산 로직

1. **Term Frequency (TF)**

> 문서 내에서 검색어가 얼마나 자주 등장하냐
>
> → 많이 등장할수록 점수↑

2. **Inverse Document Frequency (IDF)**

> 검색어가 전체 문서 중 얼마나 희귀하냐
>
> → 희귀할수록 점수↑ (흔한 단어일수록 점수↓)

3. **Field Length Normalization**

> 문서(필드)가 짧을수록 점수↑
>
> → 검색어가 짧은 문서에서 등장하면 더 관련성 높다고 판단

> Elasticsearch가 역인덱스(Inverted Index)의 기능을 가지고 있기 때문에, 단어의 순서랑 상관없이 도큐먼트를 조회해올 수 있는 것이다.
>
> **참고)** 이런 작동 구조는 데이터 타입의 `text` 타입에 한해서만 적용된다.

다음 강의에서는 방금 설명한 역인덱스를 활용해 데이터를 직접 조회해보자.

@kfc3802@gmail.com
kfc3802

🎯

[실습] 역인덱스(Inverted Index)를 활용한 데이터 조회

✅ 역인덱스(Inverted Index)를 활용한 데이터 조회

1. **기존 products 인덱스 삭제**

```
# 기존 products 인덱스 삭제
DELETE /products
```

2. **인덱스 생성**

[기존에 배웠던 방법]

```
# 인덱스 생성
PUT /products

# 매핑 정의
PUT /products/_mappings
{
  "properties": {
    "name": {
      "type": "text"
    }
  }
}
```

[조금 더 편리한 방법]

```
# 인덱스 생성 + 매핑 정의 (자주 쓰이니 정리해두자!)
PUT /products
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      }
    }
  }
}
```

3. **인덱스 잘 생성됐는 지 확인**

```
# 인덱스 잘 생성됐는 지 확인
GET /products
```

4. **도큐먼트 삽입**

```
# 데이터 넣기
POST /products/_create/1
```

```
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}

POST /products/_create/2
{
  "name": "Apple 2024 에어팟 4세대"
}

POST /products/_create/3
{
  "name": "Apple 2024 아이패드 mini A17 Pro"
}
```

5. **도큐먼트 조회**

```
# 'Apple 2024 아이패드'로 검색
GET /products/_search
{
  "query": {
    "match": {
      "name": "Apple 2024 아이패드"
    }
  }
}
```

응답값

```
{
  "took": 7,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 3,
      "relation": "eq"
    },
    "max_score": 1.5471337,
    "hits": [
      {
        "_index": "products",
        "_id": "3",
        "_score": 1.5471337,
        "_source": {
          "name": "Apple 2024 아이패드 mini A17 Pro"
        }
      },
      {
        "_index": "products",
        "_id": "2",
        "_score": 0.6860854,
```

```
      "_source": {
        "name": "Apple 2024 에어팟 4세대"
      }
    },
    {
      "_index": "products",
      "_id": "1",
      "_score": 0.12180668,
      "_source": {
        "name": "Apple 2025 맥북 에어 13 M4 10코어"
      }
    }
  ]
 }
}
```


score가 높은 순으로 도큐먼트가 조회됐다. score가 대략적으로 어떻게 계산됐는 지 이전 강의의 내용을 다시 떠올려보자.

[복습]

`Apple 2024 아이패드` 라고 검색하면 역인덱스를 활용해 일치하는 단어가 많은 도큐먼트를 우선적으로 조회한다.

토큰(token)	도큐먼트 id
Apple	**[1, 2, 3]**
2025	[1]
맥북	[1]
에어	[1]
13	[1]
M4	[1]
10코어	[1]
2024	**[2, 3]**
에어팟	[2]
4세대	[2]
아이패드	**[3]**
mini	[3]
A17	[3]
Pro	[3]

- id가 1인 도큐먼트 : 단어 1개 일치
- id가 2인 도큐먼트 : 단어 2개 일치
- id가 3인 도큐먼트 : 단어 3개 일치

이와 같은 Elasticsearch의 자체적인 로직으로 **score(점수)**를 매겨 score가 높은 순(`id=3` → `id=2` → `id=1`)으로 도큐먼트를 조회한다.

▼ **참고)** 대략적인 score 계산 로직

1. **Term Frequency (TF)**

   > 문서 내에서 검색어가 얼마나 자주 등장하냐
   >
   > → 많이 등장할수록 점수↑

2. **Inverse Document Frequency (IDF)**

> 검색어가 전체 문서 중 얼마나 희귀하냐
>
> → 희귀할수록 점수↑ (흔한 단어일수록 점수↓)

3. **Field Length Normalization**

> 문서(필드)가 짧을수록 점수↑
>
> → 검색어가 짧은 문서에서 등장하면 더 관련성 높다고 판단

다음 강의에서는 역인덱스를 생성하는 과정을 조금 더 깊이 파헤쳐보자.

@kfc3882[illegible]
@gmail.com
khb3025[illegible]

애널라이저(Analyzer)란?

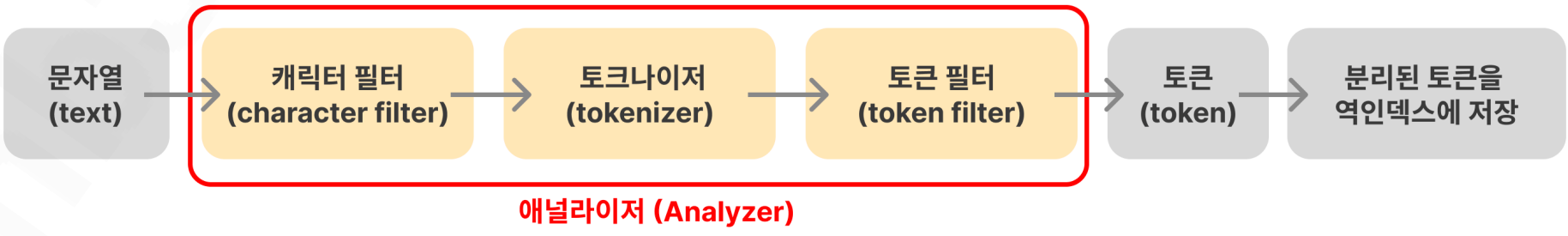
✅ 애널라이저(Analyzer)란?

이전 강의에서 `products`의 `name` 필드값을 토큰(token)으로 분리해 역인덱스(Inverted Index)로 저장하는 과정을 살펴봤다. 이 과정에서 **문자열(text)을 토큰으로 변환시켜주는 장치**를 보고 **애널라이저(Analyzer)**라고 부른다.

```
POST /products/_create/1
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}

POST /products/_create/2
{
  "name": "Apple 2024 에어팟 4세대"
}

POST /products/_create/3
{
  "name": "Apple 2024 아이패드 mini A17 Pro"
}
```

토큰(token)	도큐먼트 id
Apple	[1, 2, 3]
2025	[1]
맥북	[1]
에어	[1]
13	[1]
M4	[1]
10코어	[1]
2024	[2, 3]
에어팟	[2]
4세대	[2]
아이패드	[3]
mini	[3]
A17	[3]
Pro	[3]

그림으로 나타내면 다음과 같다.



이전에 간단하게 배울 때는 문자열을 단어 단위로 잘라서 토큰으로 만든다고만 배웠다. 사실은 문자열을 간단하게 단어 단위로만 자르는 게 아니라 **애널라이저(Anylyzer)**가 여러 가지의 작업을 거쳐서 토큰을 만들어낸다.

애널라이저(Analyzer)는 내부적으로 **캐릭터 필터(character filter)**, **토크나이저(tokenizer)**, **토큰 필터(token filter)**라는 걸 활용해 문자열을 토큰으로 변환시킨다. 이 3가지의 개념에 대해 조금 더 자세히 알아보자.

✅ 캐릭터 필터(character filter)

캐릭터 필터(character filter)는 **문자열을 토큰으로 자르기 전에 문자열을 다듬는 역할**을 한다.

다양한 종류의 필터가 존재하며, 여러 개의 필터를 적용시킬 수 있다.

[예시]

- **html_strip 필터 적용 (HTML 태그를 제거)**

  `<h1>아이폰 15 사용 후기</h1>` → `아이폰 15 사용 후기`

- (mapping 필터, pattern replace 필터)

✅ 토크나이저(tokenizer)

토크나이저(tokenizer)는 **문자열을 토큰으로 자르는 역할**을 한다.

[예시]

- **standard 토크나이저 (공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 자름)**

  `The Brown-Foxes jumped over the roof.`

  → [`The` , `Brown` , `Foxes` , `jumped` , `over` , `the` , `roof`]

- (classic 토크나이저, keyword 토크나이저, pattern 토크나이저 등등)

✅ 토큰 필터(token filter)

토큰 필터(token filter)는 **잘린 토큰을 최종적으로 다듬는 역할**을 한다.

다양한 종류의 필터가 존재하며, 여러 개의 필터를 적용시킬 수 있다.

[예시]

1. **lowercase 필터 적용 (소문자로 변환)**

   [`The` , `Brown` , `Foxes` , `jumped` , `over` , `the` , `roof`]

   → [`the` , `brown` , `foxes` , `jumped` , `over` , `the` , `roof`]

2. **stop 필터 적용 (`a` , `the` , `is` 와 같은 특별한 의미를 가지지 않는 단어 제거)**

   [`the` , `brown` , `foxes` , `jumped` , `over` , `the` , `roof`]

   → [`brown` , `foxes` , `jumped` , `roof`]

3. **stemmer 필터 적용 (단어의 원래 형태로 변환)**

   [`brown` , `foxes` , `jumped` , `roof`]

   → [`brown` , `fox` , `jump` , `roof`]

- (asciifolding 필터, synonym 필터, edge_ngram 필터, kstem 필터 등등)

그럼 다음 강의에서는 Elasticsearch에 기본값으로 설정되어 있는 애널라이저의 character filter, tokenizer, token filter가 어떻게 설정되어 있는 지 알아보자.

@naver.com
khb3025

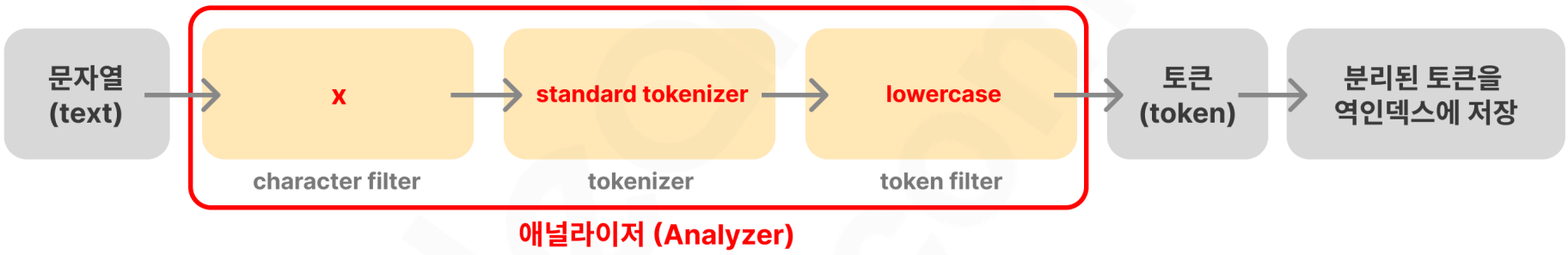
📖

Elasticsearch에 기본값으로 설정되어 있는 애널라이저(Analyzer)

✅ 기본값으로 설정되어 있는 애널라이저(Analyzer)

Elasticsearch에서 제공하는 애널라이저로는 `standard`, `simple`, `whitespace` 등 다양한 종류가 있다. 그 중 기본값으로 설정되어 있는 애널라이저는 **standard analyzer**이다.

이전 강의에서 애널라이저는 3가지(character filter, tokenizer, token filter)로 이루어져있다고 설명했다. **standard analyzer**는 charcater filter는 설정되어 있지 않고, tokenizer는 `standard`로 설정되어 있고, token filter는 `lowercase`인 애널라이저를 뜻한다.



- `tokenizer(standard)` : 공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름
- `token filter(lowercase)` : 소문자로 변환

✅ standard analyzer 표현 방식

Elasitcsearch에서 standard analyzer를 표현하는 방식은 크게 2가지가 있다. 1번의 방식으로 표현하든 2번의 방식으로 표현하든 똑같이 작동한다. 다음 실습에서 쓰일 문법이니 봐두자.

```
// 방법 1
"analyzer": "standard"

// 방법 2
"char_filter": [],
"tokenizer": "standard",
"filter": ["lowercase"]
```

✅ 이전 실습 다시 되짚어보기

이전 실습에서는 아래와 같이 토큰이 나뉜다고 설명했다. 얼추 비슷하긴 하지만 틀린 부분이 존재한다.

```
POST /products/_create/1
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}

POST /products/_create/2
{
  "name": "Apple 2024 에어팟 4세대"
```

```
}

POST /products/_create/3
{
  "name": "Apple 2024 아이패드 mini A17 Pro"
}
```

토큰(token)	도큐먼트 id
Apple	[1, 2, 3]
2025	[1]
맥북	[1]
에어	[1]
13	[1]
M4	[1]
10코어	[1]
2024	[2, 3]
에어팟	[2]
4세대	[2]
아이패드	[3]
mini	[3]
A17	[3]
Pro	[3]

[제대로 된 토큰 형태]

기본값으로 설정되어 있는 standard analyzer는 `lowercase`라는 character filter가 있기 때문에 소문자로 변환해서 토큰을 저장한다. 따라서 아래와 같이 역인덱스가 생성됐다고 설명하는 게 정확한 설명이다.

토큰(token)	도큐먼트 id
apple	[1, 2, 3]
2025	[1]
맥북	[1]
에어	[1]
13	[1]
m4	[1]
10코어	[1]
2024	[2, 3]
에어팟	[2]
4세대	[2]
아이패드	[3]
mini	[3]
a17	[3]
pro	[3]

애널라이저(Analyzer)가 문자열을 어떻게 토큰으로 나누는 지 이론적으로 알아봤다. 개발 공부를 할 때는 배운대로 작동하는 지 눈으로 확인하는 과정이 굉장히 중요하다. 그리고 제대로 작동하는 지 확인하는 방법을 알아야만 디버깅을 훨씬 쉽게 할 수 있다.

따라서 다음 강의에서는 애널라이저(Analyer)가 문자열을 어떤 형태의 토큰으로 잘랐는지 확인하는 방법을 알아보자.

@naver.com
kb302

🎯 [실습] 애널라이저(Analyzer)가 토큰을 어떻게 나누는 지 확인하는 방법

✅ standard analyzer가 토큰을 어떻게 나누는 지 확인하는 방법

문법 (Analyze API)

```
// 방법 1
GET /_analyze
{
  "text": "_____",
  "analyzer": "standard"
}

// 방법 2 (standard analyer의 구성을 직접 명시)
GET /_analyze
{
  "text": "_____",
  "char_filter": [],
  "tokenizer": "standard",
  "filter": ["lowercase"]
}
```

실제 적용

```
// 방법 1
GET /_analyze
{
  "text": "Apple 2025 맥북 에어 13 M4 10코어",
  "analyzer": "standard"
}

// 방법 2
GET /_analyze
{
  "text": "Apple 2025 맥북 에어 13 M4 10코어",
  "char_filter": [],
  "tokenizer": "standard",
  "filter": ["lowercase"]
}
```

위 2가지 방식은 완전히 똑같이 작동한다. 그런데 앞으로 실습에서 Analyze의 각 요소에 따라 어떻게 작동하는 지 하나씩 뜯어보기 위해 2번째 방법을 활용해 가지고 놀아볼 것이다.

응답값 분석

```
{
  "tokens": [
    {
      "token": "apple",
```

```
    "start_offset": 0,
    "end_offset": 5,
    "type": "<ALPHANUM>",
    "position": 0
  },
  {
    "token": "2025",
    "start_offset": 6,
    "end_offset": 10,
    "type": "<NUM>",
    "position": 1
  },
  {
    "token": "맥북",
    "start_offset": 11,
    "end_offset": 13,
    "type": "<HANGUL>",
    "position": 2
  },
  {
    "token": "에어",
    "start_offset": 14,
    "end_offset": 16,
    "type": "<HANGUL>",
    "position": 3
  },
  {
    "token": "13",
    "start_offset": 17,
    "end_offset": 19,
    "type": "<NUM>",
    "position": 4
  },
  {
    "token": "m4",
    "start_offset": 20,
    "end_offset": 22,
    "type": "<ALPHANUM>",
    "position": 5
  },
  {
    "token": "10코어",
    "start_offset": 23,
    "end_offset": 27,
    "type": "<ALPHANUM>",
    "position": 6
  }
 ]
}
```


토큰으로 분리한 결과를 보면 `standard tokenizer`(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름)와 `lowercase token filter`(소문자로 변환)가 적용된 채로 토큰이 생성된 것을 확인할 수 있다.

다음 강의에서는 Analyze의 구성 요소를 이것저것 써보면서 어떤 식으로 토큰을 분리할 수 있는 지 알아보자.

🎯 [실습] 대소문자 구분없이 검색하는 방법 (lowercase)

✅ 대소문자 구분없이 검색하는 방법

1. **Custom Analyzer를 활용해 인덱스 생성하기**

   아무 설정 없이 인덱스를 생성하면 standard analyzer가 설정된다. 이번에는 기본으로 설정되는 standard analyzer가 아닌, Analyzer의 구성 요소를 직접 설정하는 Custom Analyzer를 활용해서 인덱스를 생성해볼 것이다.

```
// 기존 인덱스 삭제
DELETE /products

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /products
{
  "settings": {
    "analysis": {
      "analyzer": {
        "products_name_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": []
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "name": {
          "type": "text",
          "analyzer": "products_name_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /products
```

   `products` 인덱스에서 `name` 필드에 `products_name_analyzer`라는 Custom Analyzer를 적용시켰다. 이번 Analyzer에는 `standard tokenizer`(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름)만 설정했고, `lowercase token filter`(소문자로 변환)은 설정하지 않았다.

2. **데이터 삽입하기**

```
POST /products/_create/1
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}
```

3. **검색해보기**

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "apple"
    }
  }
}
```

위 쿼리로 검색해보면 아무 검색 결과가 안 뜨는 걸 확인할 수 있다.

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "Apple"
    }
  }
}
```

`apple`이 아닌 `Apple`로 바꿔서 다시 검색해보면 아까 저장한 도큐먼트가 조회되는 걸 확인할 수 있다. 왜 그런 지 Analyze API를 활용해서 디버깅해보자.

4. **Analyze API 사용하기**

```
// 특정 인덱스의 필드에 적용된 Analyzer를 활용해 분석
GET /products/_analyze
{
  "field": "name"
  "text": "Apple 2025 맥북 에어 13 M4 10코어"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "Apple",
      "start_offset": 0,
      "end_offset": 5,
      "type": "<ALPHANUM>",
      "position": 0
    },
      ...
  ]
}
```

응답값을 보면 토큰으로 `apple`이라고 저장되어 있지 않고 `Apple`이라고 저장되어 있다. 이 때문에 `apple`이라고 검색했을 때 검색이 안 된 것이다.

5. **Custom Analyzer를 활용해 인덱스 다시 생성하기**

이번에는 대소문자를 구분하지 않고 검색할 수 있게 만들기 위해 `lowercase token filter`를 추가해서 인덱스를 생성해보자.

```
// 기존 인덱스 삭제
DELETE /products

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /products
{
  "settings": {
    "analysis": {
      "analyzer": {
        "products_name_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "name": {
          "type": "text",
          "analyzer": "products_name_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /products
```

6. **데이터 삽입하기**

```
POST /products/_create/1
{
  "name": "Apple 2025 맥북 에어 13 M4 10코어"
}
```

6. **검색해보기**

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "apple"
    }
  }
}

GET /products/_search
{
  "query": {
    "match": {
      "name": "Apple"
    }
```

```
  }
}
```

둘 다 잘 검색이 되는 걸 확인할 수 있다. 왜 검색이 잘 됐는 지 디버깅하기 위해 Analyze API를 사용해보자.

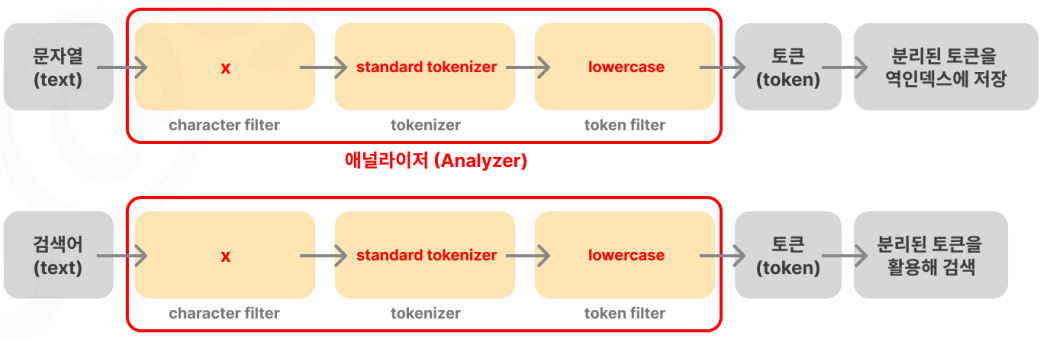
8. **Analyze API 사용하기**

```
// 특정 인덱스의 필드에 적용된 Analyzer를 활용해 분석
GET /products/_analyze
{
  "field": "name"
  "text": "Apple 2025 맥북 에어 13 M4 10코어"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "apple",
      "start_offset": 0,
      "end_offset": 5,
      "type": "<ALPHANUM>",
      "position": 0
    },
        ...
  ]
}
```

응답값을 보면 토큰으로 `apple`이라고 저장되어 있다. 이 때문에 `apple`이라고 검색했을 때 조회가 잘 된 것이다. 하지만 `Apple`이라고 검색했는데도 도큐먼트가 조회된 이유가 뭘까?



도큐먼트를 생성할 때 Analyzer가 문자열을 토큰으로 분리해 역인덱스를 생성한다. 그런데 검색을 할 때도 Analyzer가 검색어로 입력한 문자열을 토큰으로 분리해 검색한다.

이 때문에 `Apple`이라고 검색어를 입력하더라도 `lowercase token filter`에 의해 `apple`로 바뀐 채로 검색을 하게 된다. 그래서 `Apple`이라고 검색했는데도 불구하고 도큐먼트가 조회된 것이다.

✅ 정리

결론적으로 Analyzer에서 `lowercase token filter`를 활용함으로써 대소문자에 상관없이 데이터를 검색할 수 있게 된 것이다.

그럼 다음 강의에서는 Analyzer의 또 다른 기능에 대해 배워보도록 하자.

@gmail.com
khb302

🎯 [실습] 검색할 때 필요없는 HTML 태그 제거하기 (html_strip)

✅ 검색 시 필요없는 HTML 태그 제거하기

> 대부분의 게시글 서비스는 굵게, 기울임, 링크 등을 포함해서 작성할 수 있게 되어 있다. 그러려면 HTML 태그를 포함해서 그대로 DB에 저장해야 하는 경우가 많다. 이렇게 HTML 태그가 포함된 형태의 글은 어떻게 검색을 해야할 지 알아보도록 하자.

1. **Custom Analyzer를 활용해 인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /boards
```

이번 Analyzer에는 `standard tokenizer`(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름)와 `lowercase token filter`(소문자로 변환)만 설정했다.

2. **데이터 삽입하기**

```
// id를 자동 생성해서 도큐먼트 저장하기
POST /boards/_doc
{
```

```
  "content": "<h1>Running cats, jumping quickly — over the lazy dogs!</h1>"
}
```

3. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "running"
    }
  }
}
```

위 쿼리로 검색해보면 정상적으로 잘 조회가 된다.

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "h1"
    }
  }
}
```

실제 게시글의 내용과는 상관없는 `h1` 이라는 키워드에도 게시글이 조회됐다. 즉, HTML 태그에 대해서도 검색이 되다보니 검색 품질이 떨어지게 된다. 또 다른 단점을 알아보기 위해 Analyze API를 사용해서 분석해보자.

4. **Analyze API 사용하기**

```
GET /boards/_analyze
{
  "field": "content",
  "text": "<h1>Running cats, jumping quickly — over the lazy dogs!</h1>"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "h1",
      "start_offset": 1,
      "end_offset": 3,
      "type": "<ALPHANUM>",
      "position": 0
    },
       ...
  ]
}
```

토큰으로 저장될 필요 없는 HTML 태그까지도 역인덱스에 저장하다보니 디스크 공간을 낭비하게 된다. 그리고 역인덱스에 불필요한 토큰까지 저장되므로 검색 속도도 떨어지게 된다. 이러한 이유 때문에 토큰으로 저장할 때 HTML 태그는 제거한 뒤에 저장을 한다.

5. **Custom Analyzer를 활용해 인덱스 다시 생성하기**

   이번에는 HTML 태그를 제거한 뒤에 토큰을 만들어 저장하기 위해 `html_strip character filter`를 추가해서 인덱스를 생성해보자.

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": ["html_strip"],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /boards
```

6. **데이터 삽입하기**

```
POST /boards/_doc
{
  "content": "<h1>Running cats, jumping quickly — over the lazy dogs!</h1>"
}
```

6. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "running"
    }
  }
}

GET /boards/_search
{
```

```
  "query": {
    "match": {
      "content": "h1"
    }
  }
}
```

`running`이라는 키워드로는 검색이 잘 되지만, `h1`이라는 키워드로는 검색이 잘 안되는 걸 확인할 수 있다. 검색 결과가 이렇게 나온 이유를 디버깅하기 위해 Analyze API를 사용해보자.

8. **Analyze API 사용하기**

```
GET /boards/_analyze
{
  "field": "content",
  "text": "<h1>Running cats, jumping quickly — over the lazy dogs!</h1>"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "running",
      "start_offset": 4,
      "end_offset": 11,
      "type": "<ALPHANUM>",
      "position": 0
    },
       ...
  ]
}
```

응답값을 보면 `h1` 이 토큰으로 저장되어 있지 않다. 이 때문에 `h1`이라고 검색하더라도 조회가 안 된 것이다.

✅ 정리

HTML 태그가 포함된 데이터를 검색에 사용할 때는 character filter로 `html_strip` 적용을 고려해보자.

👨🏻‍🏫 다음 강의에서는 Analyzer의 또 다른 기능에 대해 배워보도록 하자.

🎯 [실습] 검색할 때 필요없는 불용어(a, an, the, or, but) 제거하기 (stop)

✅ 검색할 때 필요없는 불용어(a, an, the, or, but 등) 제거하기 (stop)

> 영어로 작성된 게시글을 보면 검색어로 잘 사용하지 않는 a, an, the, or과 같은 불용어(= 의미없는 단어)가 많이 포함되어 있다. 역인덱스의 효율적인 저장과 활용을 위해 불용어를 제거하고 관리하는 방법을 알아보자.

1. **Custom Analyzer를 활용해 인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase", "stop"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /boards
```

이번 Analyzer에는 `standard tokenizer`(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름)와 `lowercase token filter`(소문자로 변환)에다가 `stop token filter`(불용어를 제거)를 추가했다.

2. **데이터 삽입하기**

```
POST /boards/_doc
{
  "content": "The cat and the dog are friends."
}
```

3. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "the"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "and"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "are"
    }
  }
}
```

위 쿼리로 검색해보면 아무 데이터도 조회되지 않는다. 왜 그런지 Analyze API를 사용해서 분석해보자.

4. **Analyze API 사용하기**

```
GET /boards/_analyze
{
  "field": "content",
  "text": "The cat and the dog are friends."
}
```

응답값

```
{
  "tokens": [
    {
      "token": "cat",
      "start_offset": 4,
      "end_offset": 7,
      "type": "<ALPHANUM>",
      "position": 1
    },
    {
      "token": "dog",
      "start_offset": 16,
      "end_offset": 19,
```

```
      "type": "<ALPHANUM>",
      "position": 4
    },
    {
      "token": "friends",
      "start_offset": 24,
      "end_offset": 31,
      "type": "<ALPHANUM>",
      "position": 6
    }
  ]
}
```


토큰으로 저장될 필요 없는 불용어(`the`, `and`, `are`)를 제거한 뒤에 역인덱스에 저장했다. 그래서 `the`, `and`, `are`로 검색했을 때 아무 데이터도 조회되지 않은 것이다.

✅정리

- 불용어(a, the, are, is 등)를 활용해 검색할 일이 없는 데이터라면, 역인덱스의 효율성을 위해 token filter로 `stop`을 활용하도록 하자.
- 만약 노래 제목(ex. 비틀즈의 Let It Be)과 같이 불용어를 포함해서 검색하는 게 중요할 때는 token filter로 `stop`을 적용시키지 않아야 한다.

다음 강의에서는 Analyzer의 또 다른 기능에 대해 배워보도록 하자.

kb302500@gmail.com

🎯 [실습] 단어의 형태(-ed, -ing, -s 등)에 상관없이 검색하는 방법 (stemmer)

✅ 단어의 형태(-ed, -ing, -s, -er 등)에 상관없이 검색하는 방법

> 영어 단어를 보면 play, playing, played, player 등 다양한 형태로 사용된다. 검색을 할 때 단어의 형태에 상관없이 검색이 가능하다면 훨씬 편하게 원하는 데이터를 조회할 수 있을 것이다. 지금부터 단어의 형태에 상관없이 검색하는 방법에 대해 알아보자.

1. **Custom Analyzer를 활용해 인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
  }
}

// 잘 생성됐는 지 확인
GET /boards
```

이번 Analyzer에는 `standard tokenizer`(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 문자열을 자름)와 `lowercase token filter`(소문자로 변환)만 설정했다.

2. **데이터 삽입하기**

```
POST /boards/_doc
{
```

```
  "content": "Running cats, jumping!"
}
```

3. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "run"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "cat"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "jumped"
    }
  }
}
```

위 쿼리로 검색해보면 데이터가 조회되지 않는다. 왜 데이터가 조회되지 않는 지 Analyze API로 분석해보자.

4. **Analyze API 사용하기**

```
GET /boards/_analyze
{
  "field": "content"
  "text": "Running cats, jumping!"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "running",
      "start_offset": 0,
      "end_offset": 7,
      "type": "<ALPHANUM>",
      "position": 0
    },
    {
```

```
      "token": "cats",
      "start_offset": 8,
      "end_offset": 12,
      "type": "<ALPHANUM>",
      "position": 1
    },
    {
      "token": "jumping",
      "start_offset": 14,
      "end_offset": 21,
      "type": "<ALPHANUM>",
      "position": 2
    }
  ]
}
```

분리된 토큰을 살펴보면 검색할 때 영어 단어의 형태까지 일치시켜서 정확하게 입력해야만 검색이 된다는 걸 알 수 있다. 하지만 실제 검색을 하다보면 정확한 단어 형태가 떠오르지 않아서 비슷한 단어를 입력해서 검색을 하는 경우가 대부분이다. 이 불편함을 해결해보자.

5. **Custom Analyzer를 활용해 인덱스 다시 생성하기**

   이번에는 token filter에 `stemmer`을 추가해서 인덱스를 생성해보자. `stemmer`은 영단어를 기본형으로 변환한 후에 토큰으로 저장한다.

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase", "stemmer"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /boards
```

6. **데이터 삽입하기**

```
GET /boards/_doc
{
  "content": "Running cats, jumping!"
}
```

6. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "run"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "cat"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "jumped"
    }
  }
}
```

token filter로 `stemmer`를 추가한 이후로 변형된 단어로도 검색이 잘 된다. 왜 그런지 Analyze API로 확인해보자.

8. **Analyze API 사용하기**

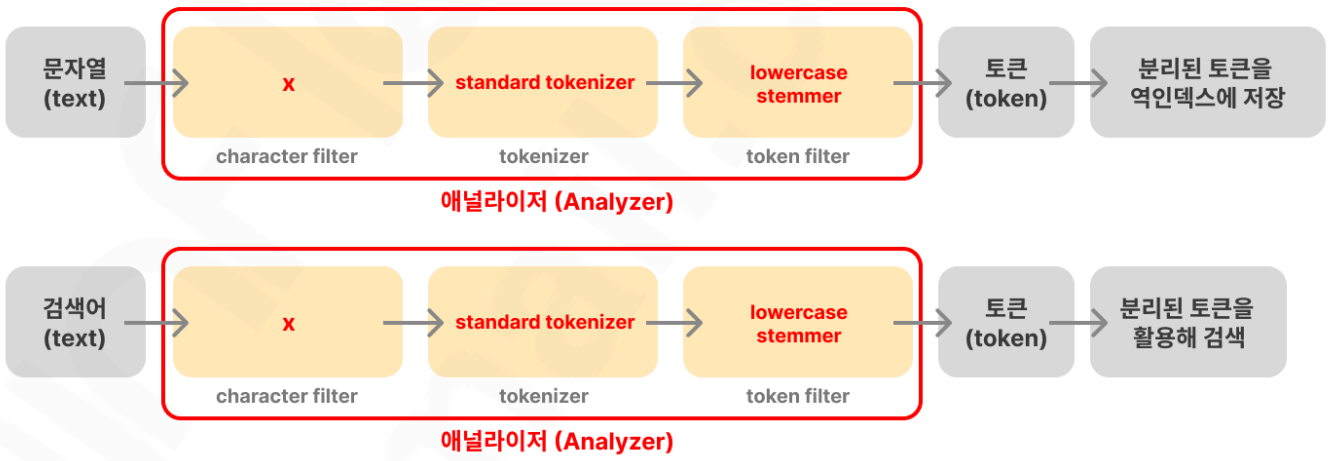
```
GET /boards/_analyze
{
   "field": "content",
   "text": "Running cats, jumping!"
}
```

응답값

```
{
  "tokens": [
    {
      "token": "run",
      "start_offset": 0,
      "end_offset": 7,
      "type": "<ALPHANUM>",
```

```
      "position": 0
    },
    {
      "token": "cat",
      "start_offset": 8,
      "end_offset": 12,
      "type": "<ALPHANUM>",
      "position": 1
    },
    {
      "token": "jump",
      "start_offset": 14,
      "end_offset": 21,
      "type": "<ALPHANUM>",
      "position": 2
    }
  ]
}
```


응답값을 보면 영단어의 기본 형태로 저장되어 있는 걸 확인할 수 있다. 여기서 `jumped` 라고 검색했는데도 데이터가 잘 조회된 이유를 다시 한 번 짚어보자.



도큐먼트를 생성할 때 Analyzer가 문자열을 토큰으로 변환해 역인덱스를 생성한다. 그런데 검색을 할 때도 Analyzer가 검색어로 입력한 문자열을 토큰으로 변환해 검색한다.

이 때문에 `jumped` 이라고 검색어를 입력하더라도 `stemmer` 에 의해 `jump` 로 바뀐 채로 검색을 하게 된다. 그래서 `jumped` 라고 검색했는데도 불구하고 도큐먼트가 조회된 것이다.

✅ 정리

play, playing, played, player와 같이 영단어의 형태에 상관없이 검색을 할 수 있게 만들고 싶다면 token filter로 `stemmer` 적용을 고려해보자.

👨🏻‍🏫 다음 강의에서는 Analyzer의 또 다른 기능에 대해 배워보도록 하자.

🎯 [실습] 동의어로 검색하는 방법 (synonym)

✅ 동의어로 검색하는 방법 (synonym)

> `Samaung Notebook`이라는 상품이 있다고 가정하자. 사람들은 검색할 때 `notebook`이라고 정확하게 검색할 수도 있지만, `노트북`, `랩탑`, `휴대용 컴퓨터`, `laptop`과 같이 동의어로 검색할 수도 있다. 그리고 `samsung`이라고 검색할 수도 있지만 `삼성`이라고 검색할 수도 있다. 이렇게 동의어로 검색하더라도 `Samsung notebook`이 나오게 하려면 어떻게 셋팅해야 하는 지 알아보자.

1. **Custom Analyzer를 활용해 인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /products

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /products
{
  "settings": {
    "analysis": {
      "filter": {
        "products_synonym_filter": {
          "type": "synonym",
          "synonyms": [
            "notebook, 노트북, 랩탑, 휴대용 컴퓨터, laptop",
            "samsung, 삼성"
          ]
        }
      },
      "analyzer": {
        "products_name_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": [
            "lowercase",
            "products_synonym_filter"
          ]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "name": {
          "type": "text",
          "analyzer": "products_name_analyzer"
        }
      }
    }
}
```

```
// 잘 생성됐는 지 확인
GET /products
```

2. **데이터 삽입하기**

```
POST /products/_doc
{
  "name": "Samsung Notebook"
}
```

3. **검색해보기**

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "노트북"
    }
  }
}

GET /products/_search
{
  "query": {
    "match": {
      "name": "랩탑"
    }
  }
}

GET /products/_search
{
  "query": {
    "match": {
      "name": "휴대용 컴퓨터"
    }
  }
}

GET /products/_search
{
  "query": {
    "match": {
      "name": "laptop"
    }
  }
}

GET /products/_search
{
  "query": {
    "match": {
      "name": "삼성"
    }
```

```
  }
}
```

`synonym` 설정 덕분에 위 쿼리로 검색해보면 데이터가 잘 조회된다. 왜 잘 조회가 된 건지 Analyze API를 활용해 디버깅해보자.

4. **Analyze API 사용하기**

```
GET /products/_analyze
{
   "field": "name",
   "text": "Samsung Notebook"
}
```

응답값

```
{
"tokens": [
 {
  "token": "samsung",
  "start_offset": 0,
  "end_offset": 7,
  "type": "<ALPHANUM>",
  "position": 0
 },
 {
  "token": "삼성",
  "start_offset": 0,
  "end_offset": 7,
  "type": "SYNONYM",
  "position": 0
 },
 {
  "token": "notebook",
  "start_offset": 8,
  "end_offset": 16,
  "type": "<ALPHANUM>",
  "position": 1
 },
 {
  "token": "노트북",
  "start_offset": 8,
  "end_offset": 16,
  "type": "SYNONYM",
  "position": 1
 },
 {
  "token": "랩탑",
  "start_offset": 8,
  "end_offset": 16,
  "type": "SYNONYM",
  "position": 1
 },
 {
  "token": "휴대용",
  "start_offset": 8,
  "end_offset": 16,
```

```
      "type": "SYNONYM",
      "position": 1
    },
    {
      "token": "laptop",
      "start_offset": 8,
      "end_offset": 16,
      "type": "SYNONYM",
      "position": 1
    },
    {
      "token": "컴퓨터",
      "start_offset": 8,
      "end_offset": 16,
      "type": "SYNONYM",
      "position": 2
    }
  ]
}
```


분리된 토큰을 살펴보면 동의어까지 포함해서 토큰이 등록된 걸 확인할 수 있다. 이 때문에 동의어로 검색을 하더라도 원하는 데이터가 잘 조회된다.

✅ 정리

동의어로 검색할 수 있게 만들고 싶다면 token filter로 `synonym`을 활용하면 된다.

지금까지 Elasticsearch에서 기본적으로 제공하면서도 많이 사용하는 Analyzer 기능에 대해 알아봤다. 다음 강의에서는 한글에서 자주 사용하는 Analyzer 기능들을 위주로 알아보자.

kb302500@gmail.com

📖

한글(korean)이 제대로 검색되지 않는 현상

✅ 한글(korean)이 제대로 검색되지 않는 현상

> 한글로 이루어진 데이터가 검색이 잘 되는 지 확인해보자.

1. **인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "standard",
          "filter": ["lowercase", "stop", "stemmer"]
        }
      }
    }
  },
  "mappings": {
      "properties": {
        "content": {
          "type": "text",
          "analyzer": "boards_content_analyzer"
        }
      }
    }
}

// 잘 생성됐는 지 확인
GET /boards
```

2. **데이터 삽입하기**

```
POST /boards/_doc
{
  "content": "백화점에서 쇼핑을 하다가 친구를 만났다."
}
```

3. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
```

```
    "content": "백화점"
   }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "쇼핑"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "친구"
    }
  }
}
```

위 쿼리로 검색해보면 아무 데이터도 조회되지 않는다. 왜 그런지 Analyze API를 사용해서 분석해보자.

4. **Analyze API 활용해 디버깅하기**

   영어는 **띄어쓰기로 단어가 명확하게 구분**되다보니 **standard tokenizer**(공백 또는 `,`, `.`, `!`, `?`와 같은 문장 부호를 기준으로 자름)로도 잘 나눌 수 있다.

```
GET /boards/_analyze
{
  "field": "content",
  "text": "I like bananas"
}
```

응답값

```
{
"tokens": [
 {
  "token": "i",
  "start_offset": 0,
  "end_offset": 1,
  "type": "<ALPHANUM>",
  "position": 0
 },
 {
  "token": "like",
  "start_offset": 2,
  "end_offset": 6,
  "type": "<ALPHANUM>",
  "position": 1
 },
 {
  "token": "banana",
```

```
    "start_offset": 7,
    "end_offset": 14,
    "type": "<ALPHANUM>",
    "position": 2
   }
  ]
}
```

하지만 한글은 좀 다르다. **조사(-는, -를), 어미(-다, -해요)를 붙여서 쓰는 말**이 많다. 그리고 **띄어쓰기가 비교적 자유롭다.** 그러다보니 **standard tokenizer**로 단어를 나눠보면 제대로 잘 나누지 못하는 문제가 생긴다.

```
POST /boards/_analyze
{
  "field": "content",
  "text": "백화점에서 쇼핑을 하다가 친구를 만났다."
}
```

응답값

```
{
  "tokens": [
    {
      "token": "백화점에서",
      "start_offset": 0,
      "end_offset": 5,
      "type": "<HANGUL>",
      "position": 0
    },
    {
      "token": "쇼핑을",
      "start_offset": 6,
      "end_offset": 9,
      "type": "<HANGUL>",
      "position": 1
    },
    {
      "token": "하다가",
      "start_offset": 10,
      "end_offset": 13,
      "type": "<HANGUL>",
      "position": 2
    },
    {
      "token": "친구를",
      "start_offset": 14,
      "end_offset": 17,
      "type": "<HANGUL>",
      "position": 3
    },
    {
      "token": "만났다",
      "start_offset": 18,
      "end_offset": 21,
      "type": "<HANGUL>",
```

```
    "position": 4
   }
  ]
}
```

위와 같이 토큰이 나뉘기 때문에 `백화점`, `쇼핑`, `친구`로 검색하더라도 검색이 안 되는 문제가 발생한 것이다. 이 문제를 해결하려면 한글에 맞는 전용 Analyzer인 **Nori(노리) Analyzer**를 써야 한다.

다음 강의에서는 Nori Analyzer를 적용시키는 방법에 대해 알아보자.

@kfc3882.com
khb3025

🎯 [실습] Nori Analyzer를 활용해 한글(korean)이 제대로 검색되게 만들기

✅ Nori Analyzer를 활용해 한글(korean)이 제대로 검색되게 만들기

1. **기존 컨테이너 종료하기**

   Nori Analyzer를 사용하려면 플러그인을 설치해야 한다. 플러그인을 설치한 채로 Docker 컨테이너를 다시 띄우기 위해 기존에 실행하던 컨테이너를 종료한다.

```
# 실행되고 있는 컨테이너 확인
$ docker ps

# compose.yml 파일이 있는 경로에서 아래 명령어 입력
$ docker compose down

# 잘 종료됐나 확인
$ docker ps
```

2. **파일 작성하기**

   Dockerfile

```
FROM docker.elastic.co/elasticsearch/elasticsearch:8.17.4

# Nori Analyzer 플러그인 설치
RUN bin/elasticsearch-plugin install analysis-nori
```

   compose.yml

```
services:
  elastic:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.4 # 8.17.4 버전
    build:
      context: .
      dockerfile: Dockerfile
  ports:
    - 9200:9200 # 9200번 포트에서 Elasticsearch 실행
  environment:
    # 아래 설정은 개발/테스트 환경에서 간단하게 테스트하기 위한 옵션 (운영 환경에서는 설정하면 안 됨)
    - discovery.type=single-node # 단일 노드 (지금은 알 필요 없음)
    - xpack.security.enabled=false # 보안 설정
    - xpack.security.http.ssl.enabled=false # 보안 설정
kibana:
  image: docker.elastic.co/kibana/kibana:8.17.4 # 8.17.4 버전
  ports:
    - 5601:5601 # 5601번 포트에서 kibana 실행
  environment:
    - ELASTICSEARCH_HOSTS=http://elastic:9200 # kibana에게 통신할 Elasticsearch 주소 알려주기
```

3. **컨테이너 띄우기**

```
$ docker compose up -d

# 잘 실행됐나 확인
$ docker ps
```

4. **Analyze API 활용해 디버깅해보기**

```
// 방법 1
GET /_analyze
{
  "text": "백화점에서 쇼핑을 하다가 친구를 만났다.",
  "analyzer": "nori"
}

// 방법 2 (nori analyzer의 구성을 직접 명시)
GET /_analyze
{
  "text": "백화점에서 쇼핑을 하다가 친구를 만났다.",
  "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
}
```

- `nori_part_of_speech` : 의미 없는 조사(`을`, `의` 등), 접속사 등을 제거
- `nori_readingform` : 한자를 한글로 바꿔서 토큰으로 저장

응답값

```
{
  "tokens": [
    {
      "token": "백화",
      "start_offset": 0,
      "end_offset": 2,
      "type": "word",
      "position": 0
    },
    {
      "token": "점",
      "start_offset": 2,
      "end_offset": 3,
      "type": "word",
      "position": 1
    },
    {
      "token": "쇼핑",
      "start_offset": 6,
      "end_offset": 8,
      "type": "word",
      "position": 3
    },
    {
      "token": "하",
```

```
    "start_offset": 10,
    "end_offset": 11,
    "type": "word",
    "position": 5
  },
  {
    "token": "친구",
    "start_offset": 14,
    "end_offset": 16,
    "type": "word",
    "position": 7
  },
  {
    "token": "만나",
    "start_offset": 18,
    "end_offset": 20,
    "type": "word",
    "position": 9
  }
 ]
}
```

응답값을 확인해보니 이전에 standard analyze를 썼을 때보다 훨씬 한글의 의미 단위로 토큰이 잘 나뉘어졌다. 그럼 검색을 했을 때도 잘 나오는 지 확인해보자.

5. **인덱스 생성하기**

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성 + 매핑 정의 + Custom Analyzer 적용
PUT /boards
{
  "settings": {
    "analysis": {
      "analyzer": {
        "boards_content_analyzer": {
          "char_filter": [],
          "tokenizer": "nori_tokenizer",
          "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "content": {
        "type": "text",
        "analyzer": "boards_content_analyzer"
      }
    }
  }
}

// 잘 생성됐는 지 확인
GET /boards
```

6. **데이터 삽입하기**

```
POST /boards/_doc
{
  "content": "백화점에서 쇼핑을 하다가 친구를 만났다."
}
```

7. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "content": "백화점"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "쇼핑"
    }
  }
}

GET /boards/_search
{
  "query": {
    "match": {
      "content": "친구"
    }
  }
}
```

위 3가지 전부 다 데이터가 잘 조회되는 걸 확인할 수 있다.

Nori analyzer를 활용하면 한글로 구성된 문장도 토큰으로 잘 분해해서 역인덱스에 저장하는 걸 직접 눈으로 확인했다. 그럼 다음 강의에서는 한글과 영어가 섞인 글은 어떻게 처리해야 하는 지 알아보자.

[실습] 한글(korean)과 영어(english)가 섞인 글을 검색 가능하게 만들기

✅ 한글(korean)과 영어(english)가 섞인 글을 검색 가능하게 만들기

> "오늘 영어 책에서 'It depends on the results.'이라는 문구를 봤다."
>
> 우리가 글을 쓰다보면 한글과 영어를 같이 쓰는 경우가 많다. 그럼 이런 글은 어떻게 검색할 수 있을 지 알아보자.

1. **Analyze API 활용해 한글과 영어 섞인 문장을 어떻게 토큰으로 분해하는 지 디버깅해보기**

```
// Nori analyzer의 구성을 직접 명시
GET /_analyze
{
  "text": "오늘 영어 책에서 'It depends on the results.'이라는 문구를 봤다.",
  "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
}
```

응답값

```
{
  "tokens": [
    {
      "token": "영어",
      "start_offset": 3,
      "end_offset": 5,
      "type": "word",
      "position": 1
    },
    {
      "token": "책",
      "start_offset": 6,
      "end_offset": 7,
      "type": "word",
      "position": 2
    },
    {
      "token": "it",
      "start_offset": 11,
      "end_offset": 13,
      "type": "word",
      "position": 4
    },
    {
      "token": "depends",
      "start_offset": 14,
      "end_offset": 21,
```

```
    "type": "word",
    "position": 5
  },
  {
    "token": "on",
    "start_offset": 22,
    "end_offset": 24,
    "type": "word",
    "position": 6
  },
  {
    "token": "the",
    "start_offset": 25,
    "end_offset": 28,
    "type": "word",
    "position": 7
  },
  {
    "token": "results",
    "start_offset": 29,
    "end_offset": 36,
    "type": "word",
    "position": 8
  },
  {
    "token": "이",
    "start_offset": 38,
    "end_offset": 39,
    "type": "word",
    "position": 9
  },
  {
    "token": "문구",
    "start_offset": 42,
    "end_offset": 44,
    "type": "word",
    "position": 11
  },
  {
    "token": "보",
    "start_offset": 46,
    "end_offset": 47,
    "type": "word",
    "position": 13
  }
 ]
}
```


- Nori analyzer만 쓰더라도 한글과 영어 전부다 의미 단위로 잘 나누는 걸 확인할 수 있다.
- lowercase가 적용되어 있어서 영어 토큰을 소문자로 변환시켜 토큰으로 저장했다는 것도 알 수 있다.

> 아쉬운 점은 영어의 불용어(`it`, `on`, `the`)가 포함되어 있다는 점과 영단어의 기본 형태를 안 쓰고 있다는 점(`depends`, `results`)이다. 이 문제를 해결하기 위해 이전에 배웠던 token filter를 활용해보자.

2. **Analyzer에 token filter 추가하기**

```
// Nori analyzer의 구성을 직접 명시
GET /_analyze
{
  "text": "오늘 영어 책에서 'It depends on the results.'이라는 문구를 봤다.",
  "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": ["nori_part_of_speech", "nori_readingform", "lowercase", "stop", "stemmer"]
}
```

응답값


```
{
  "tokens": [
    {
      "token": "영어",
      "start_offset": 3,
      "end_offset": 5,
      "type": "word",
      "position": 1
    },
    {
      "token": "책",
      "start_offset": 6,
      "end_offset": 7,
      "type": "word",
      "position": 2
    },
    {
      "token": "depend",
      "start_offset": 14,
      "end_offset": 21,
      "type": "word",
      "position": 5
    },
    {
      "token": "result",
      "start_offset": 29,
      "end_offset": 36,
      "type": "word",
      "position": 8
    },
    {
      "token": "이",
      "start_offset": 38,
      "end_offset": 39,
      "type": "word",
      "position": 9
    },
    {
      "token": "문구",
      "start_offset": 42,
      "end_offset": 44,
      "type": "word",
      "position": 11
```

```
    },
    {
      "token": "보",
      "start_offset": 46,
      "end_offset": 47,
      "type": "word",
      "position": 13
    }
  ]
}
```

`stop`, `stemmer`를 쓰니 불용어가 제거되고 영단어가 기본 형태로 바뀌었다.

✅정리

한글(korean)과 영어(english)가 섞여있는 글이라면 Nori Analyzer를 활용하면 된다. 거기서 필드 값의 특징에 따라 character filter나 token filter를 추가해서 사용하면 된다.

kb302500@gmail.com

📖

매핑(mapping)이란? / 데이터 타입(data type)

✅ 매핑(mapping)이란?

매핑(mapping)이란 **도큐먼트(document)의 각 필드가 어떤 데이터 타입을 가지고 있는 지 정의하는 설정**을 의미한다.

MySQL에서 테이블을 만들 때 어떤 유형의 데이터를 넣을 지 스키마(schema)를 정의하는 것처럼, Elasticsearch에서 인덱스를 만들 때 어떤 유형의 데이터를 넣을 지 **매핑(mapping)**을 정의할 수 있다.

✅ 데이터 타입(data type)

Elasticsearch에서 데이터 타입의 종류는 아주 많다. 그 중 우리는 자주 사용하는 데이터 타입에 대해서만 알아볼 것이다.

[숫자]

- 10억 이하의 정수만 저장하면 되는 경우 : `integer`
- 10억이 넘어가는 정수를 저장해야 될 수도 있는 경우 : `long`
- 실수(소숫점을 가진 숫자 포함)를 저장해야 될 수도 있는 경우 : `double`

[문자]

- 문자열을 토큰으로 쪼개서 저장하고 싶은 경우 : `text`
  - **유연한 검색이 필요할 때 (완전히 일치하지 않아도 비슷한 데이터를 조회해오는 방식)**
- 문자열을 토큰으로 쪼개지 않고, 문자열 그대로 저장하고 싶은 경우 : `keyword`
  - **정확한 검색이 필요할 때 (정확하게 일치할 때만 데이터를 조회해오는 방식)**
  - ex) 휴대폰 번호(`010-1234-5678`), 이메일(`jscode@gmail.com`), 주문번호(`156293920`)
  - **주의)** 계산에서 쓰는 값이 아니라면 숫자가 아닌 문자로 저장해야 한다. 예를 들어, 휴대폰 번호나 주민등록번호에 숫자를 더하거나 빼지 않는다. 즉, 휴대폰 번호나 주민등록번호는 숫자가 아닌 문자로 저장해야 한다.

[기타]

- 날짜 데이터를 저장해야 하는 경우 : `date`
- true, false를 저장해야 하는 경우 : `boolean`

👨🏽‍🏫 다음 강의에서는 매핑(mapping)이 가지는 특징에 대해 알아보자.

📖

매핑(mapping)의 특이한 특징 (null 허용, Array 허용)

✅ null 허용 (기본적으로 필수로 값을 넣을 필요가 없음)

Elasticsearch는 매핑을 정의하더라도 해당 필드가 반드시 존재하거나 null이면 안 된다는 제약을 두지 않는다. 다음과 같은 매핑(mapping)을 설정했다고 치자.

```
"mappings": {
  "properties": {
    "title": { "type": "text" }
    "content": { "type": "text" }
  }
}
```

아래와 같이 데이터를 넣어도 들어간다.

```
{
  "title": null,
  "content": "산다는 것 그것은 치열한 전투이다."
}
```

```
{
  "content": "산다는 것 그것은 치열한 전투이다."
}
```

✅ 배열(array) 허용

Elasticsearch는 별도의 설정 없이 배열(array) 형태의 데이터를 삽입할 수 있다. 그리고 배열(array)에 특화된 매핑 타입(mapping type)이 따로 없다. 다음과 같이 매핑(mapping)을 설정했다고 치자.

```
"mappings": {
  "properties": {
    "hashtags": { "type": "text" }
  }
}
```

아래와 같이 데이터를 넣어도 들어간다.

```
{
  "hashtags": "여행"
}
```

```
{
  "hashtags": ["여행", "요리"]
}
```

`여행`이라는 키워드로 검색하면 위 2가지 데이터 전부 다 조회된다.

다음 강의에서는 데이터 타입(data type)을 고려해서 매핑을 정의하고 인덱스를 생성해보자.

@kfkc3882@gmail.com
kb302500

🎯 [실습] 저장할 데이터에 맞게 매핑 정의하기

✅ 저장할 데이터에 맞게 매핑 정의하기

아래 데이터를 Elasticsearch에 저장해보자.

```
{
  "review_id": 923874239847,
  "user_id": 1024,
  "user_name": "김영희",
  "categories": ["가전", "청소기"],
  "rating": 4.5,
  "is_verified_purchase": true,
  "review_date": "2024-09-21T14:32:00Z",
  "review_text": "배송이 빨랐고 제품도 만족스러워요."
}
```

1. **매핑 정의 + 인덱스 생성**

```
PUT /product_reviews
{
  "mappings": {
    "properties": {
      "review_id": {
        "type": "long" // 현업에서 확장성을 고려해 integer가 아닌 long으로 셋팅하는 편
      },
      "user_id": {
        "type": "long" // 현업에서 확장성을 고려해 integer가 아닌 long으로 셋팅하는 편
      },
      "user_name": {
        "type": "text" // 일부분으로도 검색할 수 있게 설정 (만약 부분 검색이 필요 없다면 keyword로 설정)
      },
      "categories": {
        "type": "keyword" // 정확한 비교로만 사용하기 때문에 keyword로 설정
      },
      "rating": {
        "type": "double" // 소수점을 가진 숫자이므로 double로 설정
      },
      "is_verified_purchase": {
        "type": "boolean"
      },
      "review_date": {
        "type": "date" // 날짜 데이터
      },
      "review_text": {
        "type": "text" // 유연하게 검색할 수 있도록 text 설정
      }
    }
  }
}
```

2. **인덱스 잘 생성됐는 지 확인**

```
GET /product_reviews
```

2. **데이터 삽입**

```
POST /product_reviews/_doc
{
  "review_id": 923874239847,
  "user_id": 1024,
  "user_name": "김영희",
  "categories": ["가전", "청소기"],
  "rating": 4.5,
  "is_verified_purchase": true,
  "review_date": "2024-09-21T14:32:00Z",
  "review_text": "배송이 빨랐고 제품도 만족스러워요."
}
```

3. **데이터가 잘 삽입됐는 지 확인하기**

```
GET /product_reviews/_search
```

저장해야 하는 데이터의 형태에 따라 매핑(mapping)을 정의해봤다. 다음 강의에서는 자주 사용하는 검색 기능에 대해 알아보도록 하자.

khb3025@gmail.com

📖

검색 키워드가 포함된 데이터를 조회하고 싶을 때 (match)

✅ 검색 키워드가 포함된 데이터를 조회하고 싶을 때 (match)

- match 쿼리는 검색 키워드가 포함된 모든 도큐먼트를 조회한다.
- 이전 강의에서 역인덱스(Inverted Index)와 애널라이저(Analyzer)를 설명하면서 작동 원리까지 설명했었다.
- match 쿼리는 text 타입의 필드에서만 사용하는 쿼리이다.
  (keyword를 포함한 long, integer와 같은 타입의 필드에서는 사용하지 않는다.)

> match 쿼리를 직접 사용해보자.

1. **인덱스 생성 및 매핑 정의하기**

```
DELETE /boards

PUT /boards
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text"
      }
    }
  }
}
```

2. **데이터 삽입하기**

```
POST /boards/_doc
{
  "title": "편의점 과자 내돈내산 후기"
}
```

3. **도큐먼트 검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "title": "편의점 후기"
    }
  }
}
```

text 타입의 필드는 match 쿼리를 활용해 검색을 자주 하는 편이다. 하지만 text 타입이 아닌 필드에서는 match 쿼리를 활용하면 안 된다. 그럼 다른 text 이외의 타입에서는 어떤 쿼리를 사용해 데이터를 조회할 수 있는 지 다음 강의에서 알아보자.

@kfc3802@gmail.com
kfc3802@gmail.com

📖

특정 값과 정확하게 일치하는 데이터를 조회하고 싶을 때 (term, terms)

✅ 특정 값과 정확하게 일치하는 데이터를 조회하고 싶을 때 (term)

- term 쿼리는 특정 값과 정확히 일치하는 모든 도큐먼트를 조회한다.
- term 쿼리는 text를 제외한 모든 타입에서 사용한다.

  (text 타입에서도 사용은 가능하나 의도대로 작동하지 않을 수 있어서 사용하지 않는 걸 권장한다.)

> term 쿼리를 직접 사용해보자.

1. **인덱스 생성 및 매핑 정의하기**

```
DELETE /boards

PUT /boards
{
  "mappings": {
    "properties": {
      "board_id": {
        "type": "long"
      },
      "category": {
        "type": "keyword"
      }
    }
  }
}
```

2. **데이터 삽입하기**

```
POST /boards/_doc
{
  "board_id": 1,
  "category": "자유 게시판"
}

POST /boards/_doc
{
  "board_id": 2,
  "category": "익명 게시판"
}

POST /boards/_doc
{
  "board_id": 3,
```

```
  "category": "광고 게시판"
}
```

3. **도큐먼트 검색해보기**

   데이터가 가진 값과 정확히 일치하게 검색하지 않으면 도큐먼트가 조회되지 않는다.

```
GET /boards/_search
{
  "query": {
    "term": {
      "category": "자유"
    }
  }
}

GET /boards/_search
{
  "query": {
    "term": {
      "category": "자유게시판"
    }
  }
}

GET /boards/_search
{
  "query": {
    "term": {
      "board_id": 13
    }
  }
}
```

정확한 값으로 검색하면 도큐먼트가 잘 조회된다.

```
GET /boards/_search
{
  "query": {
    "term": {
      "category": "자유 게시판"
    }
  }
}

GET /boards/_search
{
  "query": {
    "term": {
      "board_id": 1
    }
  }
}
```

SQL문으로 표현하자면 아래와 같이 표현할 수 있다.

→ `SELECT * FROM boards WHERE category = "자유 게시판"`

→ `SELECT * FROM boards WHERE board_id = 152`

✅ 여러 개의 값 중 하나라도 일치하는 도큐먼트 조회 (terms)

- terms 쿼리는 여러 개의 값 중 하나라도 일치하는 모든 도큐먼트를 조회한다.
- SQL문의 `IN`과 비슷한 역할을 한다.

> terms 쿼리를 직접 사용해보자.

`SELECT * FROM boards WHERE category IN ("자유 게시판", "익명 게시판")`처럼 `category`가 `자유 게시판` 또는 `익명 게시판`인 모든 데이터를 조회하고 싶다고 가정하자.

```
GET /boards/_search
{
  "query": {
    "terms": {
      "category": ["자유 게시판", "익명 게시판"]
    }
  }
}
```

✅ 여러 개의 조건을 동시에 만족하는 데이터를 조회하고 싶을 때

`SELECT * FROM boards WHERE category = “자유 게시판” AND board_id = 1`과 같이 2가지 이상의 조건을 활용해 검색하고 싶을 수도 있다. 아래와 같이 작성하면 되지 않을까라고 생각할 수 있다.

```
GET /boards/_search
{
  "query": {
    "term": {
      "category": "자유 게시판",
      "board_id": 1
    }
  }
}
```

하지만 위와 같이 쿼리를 실행시켜보면 아래와 같이 에러가 발생한다.

```
{
  "error": {
    "root_cause": [
      {
        "type": "parsing_exception",
        "reason": "[term] query doesn't support multiple fields, found [category] and [board_id]",
        "line": 1,
        "col": 82
      }
    ],
    "type": "parsing_exception",
    "reason": "[term] query doesn't support multiple fields, found [category] and [board_id]",
    "line": 1,
    "col": 82
  },
  "status": 400
}
```


그럼 2가지 이상의 조건을 활용해서 데이터를 검색하고 싶을 때는 어떻게 쿼리를 작성해야 할까? 다음 강의에서 알아보자.

kkb3025@gmail.com

📖

2가지 이상의 조건을 만족시키는 데이터를 조회하고 싶을 때 (bool : filter, must)

✅ 2가지 이상의 조건을 만족시키는 데이터를 조회하고 싶을 때 (bool : filter, must)

이전 강의에서 아래와 같이 쿼리를 했을 때 에러가 발생하는 걸 확인했다.

```
GET /boards/_search
{
  "query": {
    "term": {
      "category": "자유 게시판",
      "board_id": 1
    }
  }
}
```


```
{
  "error": {
    "root_cause": [
      {
        "type": "parsing_exception",
        "reason": "[term] query doesn't support multiple fields, found [category] and [board_id]",
        "line": 1,
        "col": 82
      }
    ],
    "type": "parsing_exception",
    "reason": "[term] query doesn't support multiple fields, found [category] and [board_id]",
    "line": 1,
    "col": 82
  },
  "status": 400
}
```


1가지 조건이 아닌 2가지 이상의 조건을 만족시키는 쿼리를 작성하려면 **bool 쿼리**를 활용해야 한다. 즉, **bool 쿼리**는 여러 쿼리를 조합하기 위해서 사용하는 개념이다. **bool 쿼리**에는 크게 4가지 기능이 있다.

- must : SQL문에서의 `AND` 역할을 함
- filter : SQL문에서의 `AND` 역할을 함
- must_not : SQL문에서의 `NOT` 역할을 함
- should : 조건을 만족하면 좋고, 아니면 말고.

이 중에서 2가지 조건을 전부 만족시키는 데이터를 조회하기 위해서 `must`와 `filter`에 대해 알아보도록 하자.

예시를 바로 살펴보자.

```
GET /boards/_search
{
  "query": {
    "bool": {
      "filter": [
        {
            "term": {
              "category": "자유 게시판"
```

```
          }
         },
         {
          "term": {
          "board_id": 1
         }
        }
       ]
    }
  }
}

GET /boards/_search
{
  "query": {
    "bool": {
      "must": [
       {
          "term": {
           "category": "자유 게시판"
          }
         },
         {
          "term": {
          "board_id": 1
         }
        }
       ]
    }
  }
}
```

위 두 쿼리를 실행시켜보면 정상적으로 원하는 데이터를 잘 불러오는 것처럼 보이지만, 2번째 쿼리는 `must`의 용도에 맞지 않게 잘못 사용한 쿼리이다. 그럼 `filter`와 `must`의 차이에 대해 알아보고 각 기능은 언제 사용해야 하는 지까지 알아보자.

✅ `filter` vs `must`

`filter`와 `must`의 가장 큰 차이는 `filter`는 score(점수)에 영향을 주지 않고, `must`는 score(점수)에 영향을 준다는 점이다. 여기서 얘기하는 **score(점수)**는 검색을 할 때 관련도가 얼마나 높은 지를 나타내는 수치이다.

(참고 : 📖 역인덱스(Inverted Index)란? , 🎯 [실습] 역인덱스(Inverted Index)를 활용한 데이터 조회)

score(점수)가 활용되는 대표적인 쿼리가 `match`이다. score(점수)를 활용해 관련도가 높은 도큐먼트를 우선적으로 조회해온다. 즉, 정확하게 일치하지 않아도 도큐먼트를 조회해온다. 반면, `term` 쿼리와 같이 조건을 정확하게 일치시키지 않으면 도큐먼트를 아예 조회해오지 않는다. 이 때문에 score(점수)를 사용하지 않는다.

`filter`와 `must`의 사용 용도는 score(점수)에 영향을 주는 쿼리인지 아닌지로 구분해서 사용해야 한다.

- `filter` : score(점수)와 상관없는 쿼리인 경우 → ex) `term`
- `must` : score(점수)와 상관있는 쿼리인 경우 → ex) `match`

따라서 아래 예시에서의 `term` 쿼리는 score(점수)와 상관없는 쿼리이므로 `must`가 아닌 `filter` 내부에서 사용하는 것이 올바른 사용 방식이다.

```
// 올바른 사용 방식
GET /boards/_search
```

```
{
  "query": {
    "bool": {
      "filter": [
        {
            "term": {
              "category": "자유 게시판"
            }
          },
          {
            "term": {
            "board_id": 1
          }
        }
      ]
    }
  }
}

// 잘못된 방식
GET /boards/_search
{
  "query": {
    "bool": {
      "must": [
        {
            "term": {
              "category": "자유 게시판"
            }
          },
          {
            "term": {
            "board_id": 1
          }
        }
      ]
    }
  }
}
```

✅ `filter`와 `must`를 쉽게 구분하기

점수(score)를 활용하는 쿼리는 `text` 타입을 기반으로 검색을 유연하게 해주는 쿼리(`match`, `match_phrase`, `multi_match` 등) 밖에 없다. 즉, 정확하게 일치하지 않더라도 관련성이 있는 데이터까지 조회하는 쿼리에 대해서만 `must`를 사용하면 된다. 그 외에는 전부 `filter`를 쓰면 된다.

머릿속에 아래와 같이 연결을 시켜도 이해하기가 쉽다.

- **유연한 검색 필요**

  → `text` 타입, `match` 쿼리

  → `bool`의 `must`

- **정확한 검색 필요 (정확한 값 비교)**

  → `text` 이외의 타입, `term` 쿼리

  → `bool`의 `filter`

다음 강의에서 조금 더 복잡한 쿼리에서 `filter`와 `must`를 구분해서 사용하는 연습을 해보자.

@kfc 388271
@gmail.com
khb3025

🎯 [예제] filter와 must 구분해서 사용하기

✅ filter와 must 구분해서 사용하기

[사전 셋팅]

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성
PUT /boards
{
  "mappings": {
    "properties": {
      "board_id": {
        "type": "long"
      },
      "title": {
        "type": "text", // 유연한 검색이 필요하므로 text 타입으로 선언
        "analyzer": "nori" // 한글 데이터를 토큰으로 정확하게 나누기 위해
      },
      "category": {
        "type": "keyword" // 유연한 검색이 필요없으므로 keyword 타입으로 선언
      },
      "is_notice": { // (공지글 여부)
        "type": "boolean"
      },
      "created_at": {
        "type": "date"
      }
    }
  }
}

// 데이터 삽입
POST /boards/_doc
{
  "board_id": 1,
  "title": "엘라스틱서치는 정말 강력한 검색엔진이에요",
  "category": "자유 게시판",
  "is_notice": false,
  "created_at": "2025-05-01T12:00:00"
}
```

[검색 쿼리 사용해보기]

`자유 게시판`의 게시글 중에서 `검색엔진`과 관련된 글을 찾고 싶다. 그런데 `공지글`이 아닌 게시글 중에서만 검색하고 싶다고 가정하자. 그럼 어떻게 검색 쿼리를 작성하면 될까?

```
GET /boards/_search
{
  "query": {
```

```
    "bool": {
      "must": [
        { "match": { "title": "검색엔진" } } // 유연한 검색이 필요 (must)
      ],
      "filter": [
        { "term": { "category": "자유 게시판" } }, // 정확한 검색이 필요 (filter)
        { "term": { "is_notice": false } } // 정확한 검색이 필요 (filter)
      ]
    }
  }
}
```

위와 같이 쿼리를 작성해야 용도에 맞게 잘 작성한 것이다. 실제 쿼리를 실행시켜봐도 원하는 값이 출력된다.

지금까지 bool 쿼리 중 `filter`와 `must`에 대해 알아봤다. 다음 강의에서는 bool 쿼리의 `must_not`에 대해 알아보자.

kb302500@gmail.com

📖

특정 조건을 만족하지 않는 데이터를 조회하고 싶을 때 (bool : must_not)

✅ 특정 조건을 만족하지 않는 데이터를 조회하고 싶을 때 (bool : must_not)

[사전 셋팅]

```
// 기존 인덱스 삭제
DELETE /boards

// 인덱스 생성
PUT /boards
{
  "mappings": {
    "properties": {
      "board_id": {
        "type": "long"
      },
      "title": {
        "type": "text", // 유연한 검색이 필요하므로 text 타입으로 선언
        "analyzer": "nori" // 한글 데이터를 토큰으로 정확하게 나누기 위해
      },
      "category": {
        "type": "keyword" // 유연한 검색이 필요없으므로 keyword 타입으로 선언
      },
      "is_notice": { // (공지글 여부)
        "type": "boolean"
      },
      "created_at": {
        "type": "date"
      }
    }
  }
}

// 데이터 삽입
POST /boards/_doc
{
  "board_id": 1,
  "title": "엘라스틱서치는 정말 강력한 검색엔진이에요",
  "category": "자유 게시판",
  "is_notice": false,
  "created_at": "2025-05-01T12:00:00"
}

POST /boards/_doc
{
  "board_id": 2,
  "title": "이벤트 참여 방법 안내드립니다",
  "category": "광고 게시판",
  "is_notice": false,
  "created_at": "2025-05-02T10:30:00"
}
```

```
POST /boards/_doc
{
  "board_id": 3,
  "title": "익명으로 질문하고 답변 받을 수 있어요",
  "category": "익명 게시판",
  "is_notice": true,
  "created_at": "2025-05-03T08:20:00"
}
```

[검색 쿼리 사용해보기]

`광고 게시판`의 글이 아니면서, `공지 글`이 아니면서, `검색엔진`의 키워드와 관련된 게시글을 조회하고 싶다고 가정하자.

```
// 방법 1
GET /boards/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "검색엔진" } } // 유연한 검색이 필요 (must)
      ],
      "filter": [
        { "term": { "is_notice": false } } // 정확한 검색이 필요 (filter)
      ],
      "must_not": [
        { "term": { "category": "광고 게시판" } }
      ]
    }
  }
}

// 방법 2
GET /boards/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "검색엔진" } } // 유연한 검색이 필요 (must)
      ],
      "must_not": [
        { "term": { "category": "광고 게시판" } },
        { "term": { "is_notice": true } }
      ]
    }
  }
}
```

쿼리를 실행시켜보면 원하는 값이 출력된다.

지금까지 bool 쿼리 중 `filter`, `must`, `must_not`에 대해 알아봤다. 다음 강의에서는 마지막으로 bool 쿼리의 `should`에 대해 알아보자.

📖

숫자/날짜의 값에 대해 범위 조건으로 데이터를 조회하고 싶을 때 (range)

✅ 숫자/날짜의 값에 대해 범위 조건으로 데이터를 조회하고 싶을 때 (range)

> bool의 should의 실습에서 range 쿼리를 사용하기 때문에 range 쿼리를 먼저 알아보자.

예시를 바로 보는 게 이해가 빠르기 때문에 예시를 바로 살펴보자.

1. **인덱스 생성**

```
DELETE /users

PUT /users
{
  "mappings": {
    "properties": {
      "name": {
        "type": "keyword"
      },
      "age": {
        "type": "integer"
      },
      "created_at": {
        "type": "date"
      }
    }
  }
}
```

2. **데이터 생성**

```
POST /users/_doc
{
  "name": "kim_jisoo",
  "age": 28,
  "created_at": "2024-09-01"
}

POST /users/_doc
{
  "name": "lee_joon",
  "age": 35,
  "created_at": "2024-12-15"
}

POST /users/_doc
{
  "name": "park_saejin",
  "age": 32,
```

```
  "created_at": "2025-03-25"
}
```

3. **검색하기**

   나이가 30살 이상이면서 회원가입 날짜가 2025년 1월 1일 이후인 사용자를 조회해보자.

```
GET /users/_search
{
  "query": {
    "bool": {
      "filter": [
        {
          "range": {
            "age": {
              "gte": 30
            }
          }
        },
        {
          "range": {
            "created_at": {
              "gte": "2025-01-01"
            }
          }
        }
      ]
    }
  }
}
```

이렇게 숫자 또는 날짜 데이터에 대해 `range` 쿼리를 활용해 범위로 검색할 수 있다.

참고) `range` 쿼리에서 사용하는 연산자

연산자	의미	예시 조건	의미
`gte`	이상 (≥)	`"gte": 30`	30 **이상**인 값 (30, 31, 32...) 포함
`lte`	이하 (≤)	`"lte": 100`	100 **이하**인 값 (100, 99, 98...) 포함
`gt`	초과 (>)	`"gt": 18`	18 **초과**인 값 (19, 20...) **18은 제외**
`lt`	미만 (<)	`"lt": 10`	10 **미만**인 값 (9, 8...) **10은 제외**

다음 강의에서 bool 쿼리의 `should`에 대해 알아보자.

📖

특정 조건을 만족하는 데이터 위주로 상위 노출 시키고 싶을 때 (bool : should)

✅ bool 쿼리의 should는 언제 어떻게 쓰일까?

bool 쿼리에서 `must`와 `filter`는 반드시 조건을 만족하는 데이터만 조회되지만, `should`는 조건을 만족하지 않는 데이터도 조회되기도 한다. 다만, `should`의 조건을 충족시키는 데이터일 경우 score(점수)에 가산점을 부여해 상위 노출될 가능성이 높아진다. 그래서 `must`, `filter`는 **'무조건'**의 느낌이지만 `should`는 **'있으면 좋고, 아니면 말고'**의 느낌이다.

> 그럼 `should`는 언제 쓸까?

실제 우리가 사용하고 있는 많은 서비스에서 `should` 기능이 많이 활용되고 있다. 여러 예시를 살펴보자.

[사례 1]

고객이 쇼핑몰에서 "노트북"을 검색했을 때, 우리는 다음 조건을 만족하는 상품을 **상위에 노출**시키고 싶은 경우

- 최근 출시된 제품
- 평점이 높은 제품
- 특정 브랜드(예: Apple)

하지만, 이 조건을 **필수**로 강제하진 않고 만족하면 가산점을 부여하기

[사례 2]

사용자가 “백엔드 개발자”를 검색했을 때, 우리는 다음 조건을 만족하는 공고를 더 상위에 노출시키고 싶은 경우

- 원격 근무 가능
- 연봉 6,000만 원 이상
- 유명 기업(예: 네이버, 카카오, 당근)

이 조건은 필수는 아니고, 만족하면 더 매력적인 공고이므로 `should`로 가산점만 줍니다.

[사례 3]

사용자가 네이버에서 "챗GPT"로 블로그를 검색했을 때, 우리는 다음 조건을 만족하는 글을 더 상위에 노출시키고 싶은 경우

- 제목에 "챗GPT"가 포함된 글
- 작성일이 최근일수록
- 좋아요 수가 많은 글

이 조건들을 `should`로 설정해 **더 좋은 글**을 먼저 보여주는 전략이다.

👨🏻‍🏫 그럼 다음 강의에서 이 내용을 실습하면서 직접 눈으로 확인해보자.

🎯 [실습] 검색 결과 중 평점이 높고 좋아요 수가 많은 글을 상위에 노출시키고 싶은 경우

✅ 검색 결과 중 평점이 높고 좋아요 수가 많은 글을 상위에 노출시키고 싶은 경우

사용자가 특정 키워드(예: "무선 이어폰")로 검색했을 때, 키워드와 관련된 데이터를 조회해오게 만들자. 이 때, 평점이 높거나 좋아요 수가 많은 상품이 상단에 노출되게 만들어보자.

1. **인덱스 생성**

```
DELETE /products

PUT /products
{
  "mappings": {
    "properties": {
      "name":   {
        "type": "text",
        "analyzer": "nori"
      },
      "rating": { "type": "double" },
      "likes":  { "type": "integer" }
    }
  }
}
```

2. **데이터 삽입**

```
// 키워드("무선 이어폰")와 관련성은 적지만, 좋아요 수가 높고, 평점이 좋은 경우
POST /products/_doc
{
  "name": "무선 충전기 C타입",
  "rating": 4.9,
  "likes": 300
}

// 키워드("무선 이어폰")와 관련성은 높지만, 좋아요 수가 낮고, 평점도 낮은 경우
POST /products/_doc
{
  "name": "소니 무선 이어폰 WF",
  "rating": 3.8,
  "likes": 15
}


// 키워드("무선 이어폰")와 관련성도 높고, 좋아요 수도 높고, 평점도 높은 경우
POST /products/_doc
{
  "name": "갤럭시 버즈2 무선 이어폰",
  "rating": 4.8,
  "likes": 310
```

```
}

// 키워드("무선 이어폰")와 관련성이 아예 없는데, 좋아요 수는 높고, 평점도 높은 경우
POST /products/_doc
{
  "name": "삼성 노트북 13인치",
  "rating": 5.0,
  "likes": 1000
}
```

3. **should 조건 없이 검색해보기**

```
GET /products/_search
{
   "query": {
      "match": {
         "name": "무선 이어폰"
      }
   }
}
```

응답값

```
{
  "took": 2,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 3,
      "relation": "eq"
    },
"max_score": 0.6409958,
"hits": [
  {
    "_index": "products",
    "_id": "z0-AIZYBWhYNXJPWkb6p",
    "_score": 0.6409958,
    "_source": {
      "name": "소니 무선 이어폰 WF",
      "rating": 3.8,
      "likes": 15
    }
  },
  {
    "_index": "products",
    "_id": "0E-AIZYBWhYNXJPWlr56",
    "_score": 0.5863999,
    "_source": {
      "name": "갤럭시 버즈2 무선 이어폰",
```

```
        "rating": 4.8,
        "likes": 310
      }
    },
    {
      "_index": "products",
      "_id": "zk-AlZYBWhYNXJPWjb70",
      "_score": 0.12974027,
      "_source": {
        "name": "무선 충전기 C타입",
        "rating": 4.9,
        "likes": 300
      }
    }
  ]
 }
}
```

- 쿼리의 조건에 따라 `"무선 이어폰"` 키워드와 아예 관련이 없는 데이터는 조회되지 않았다.
- `should`를 활용하지 않고 조회를 하니 저품질(평점이 낮거나 좋아요 수가 적은 상품)의 상품이 상위에 노출됐다. 이런 경우 사용자 경험에 좋지 않을 수 있다. `should`를 활용해 저품질 상품이 상위에 노출되지 않도록 만들어보자.

4. **should 활용해서 검색해보기**

```
GET /products/_search
{
 "query": {
  "bool": {
   "must": [
    {
     "match": {
      "name": "무선 이어폰"
     }
    }
   ],
   "should": [
    {
     "range": {
      "rating": {
       "gte": 4.5 // 4.5 이상의 평점의 상품일 경우 score에 가산점 부여
      }
     }
    },
    {
     "range": {
      "likes": {
       "gte": 100 // 좋아요 수가 100개 이상인 상품일 경우 score에 가산점 부여
      }
     }
    }
   ]
  }
 }
}
```

응답값


```
{
  "took": 6,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 3,
      "relation": "eq"
    },
    "max_score": 2.5864,
    "hits": [
      {
        "_index": "products",
        "_id": "0E-AlZYBWhYNXJPWlr56",
        "_score": 2.5864,
        "_source": {
          "name": "갤럭시 버즈2 무선 이어폰",
          "rating": 4.8,
          "likes": 310
        }
      },
      {
        "_index": "products",
        "_id": "zk-AlZYBWhYNXJPWjb70",
        "_score": 2.1297402,
        "_source": {
          "name": "무선 충전기 C타입",
          "rating": 4.9,
          "likes": 300
        }
      },
      {
        "_index": "products",
        "_id": "z0-AlZYBWhYNXJPWkb6p",
        "_score": 0.6409958,
        "_source": {
          "name": "소니 무선 이어폰 WF",
          "rating": 3.8,
          "likes": 15
        }
      }
    ]
  }
}
```


- 1번째 데이터 : 검색 키워드와 관련성이 높고, 평점이 좋고, 좋아요 수가 높은 상품이 상위에 노출됐다.
- 2번째 데이터 : 검색 키워드와 관련성이 조금 낮더라도, 평점이 좋고, 좋아요 수가 높아서 검색 키워드와 관련성이 높은 3번째 상품보다 상단에 노출됐다.
- 3번째 데이터 : 검색 키워드와 관련성은 높았으나, 평점이 낮고 좋아요 수가 낮아서 하단에 배치됐다.

이와 같이 `score`에 가산점을 주는 bool 쿼리의 `should`를 활용해 원하는 조건을 갖추고 있는 데이터를 상단에 노출되게 만들 수 있다.

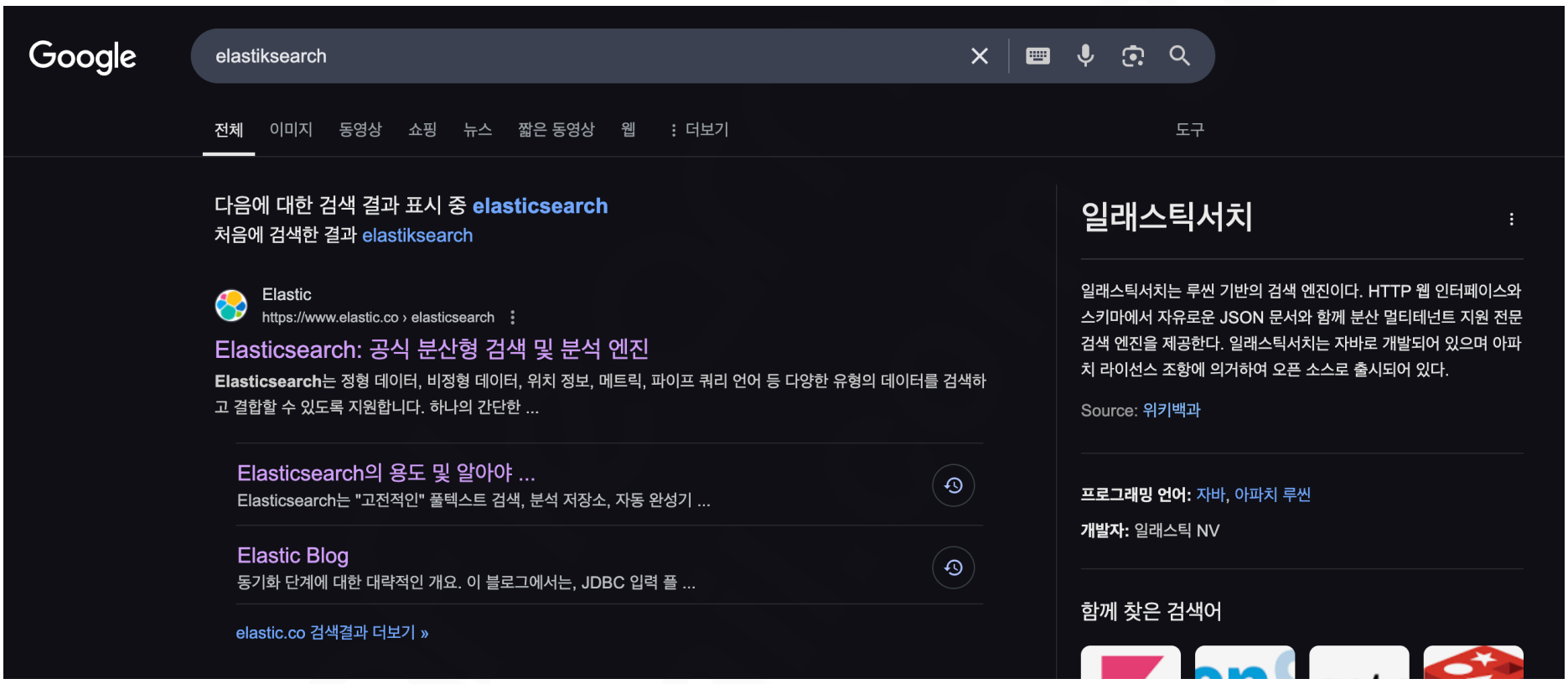
지금까지 bool 쿼리의 4가지 기능인 `filter`, `must`, `must_not`, `should`에 대해 알아봤다. 다음 강의에서는 다른 검색 기능을 알아보자.

@kfkc3882@gmail.com
kfb30250

오타가 있더라도 유사한 단어를 포함한 데이터를 조회하고 싶을 때 (fuzziness)

✅ 오타가 있더라도 유사한 단어를 포함한 데이터를 조회하고 싶을 때 (fuziness)

구글에서 검색을 해보면 일부 오타를 내더라도 원하는 데이터가 잘 검색이 된다.



우리도 오타가 일부 있더라도 원하는 데이터가 잘 검색되도록 만들어보자.

✅ 실습

1. **인덱스 생성**

```
DELETE /boards

PUT /boards
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text"
      }
    }
  }
}
```

2. **데이터 삽입**

```
POST /boards/_doc
{ "title": "elasticsearch 사용법" }
```

3. **검색해보기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "title": "elasticsearch"
    }
  }
}
```

위와 같이 오타없이 키워드로 검색하면 데이터가 잘 조회된다.

```
GET /boards/_search
{
  "query": {
    "match": {
      "title": "elastiksearch"
    }
  }
}
```

하지만 위와 같이 오타가 포함된 키워드로 검색하면 아무 데이터도 조회되지 않는다.

4. **오타가 어느 정도 포함되어 있어도 검색되게 만들기**

```
GET /boards/_search
{
  "query": {
    "match": {
      "title": {
        "query": "elastiksearch",
        "fuzziness": "AUTO"
      }
    }
  }
}
```

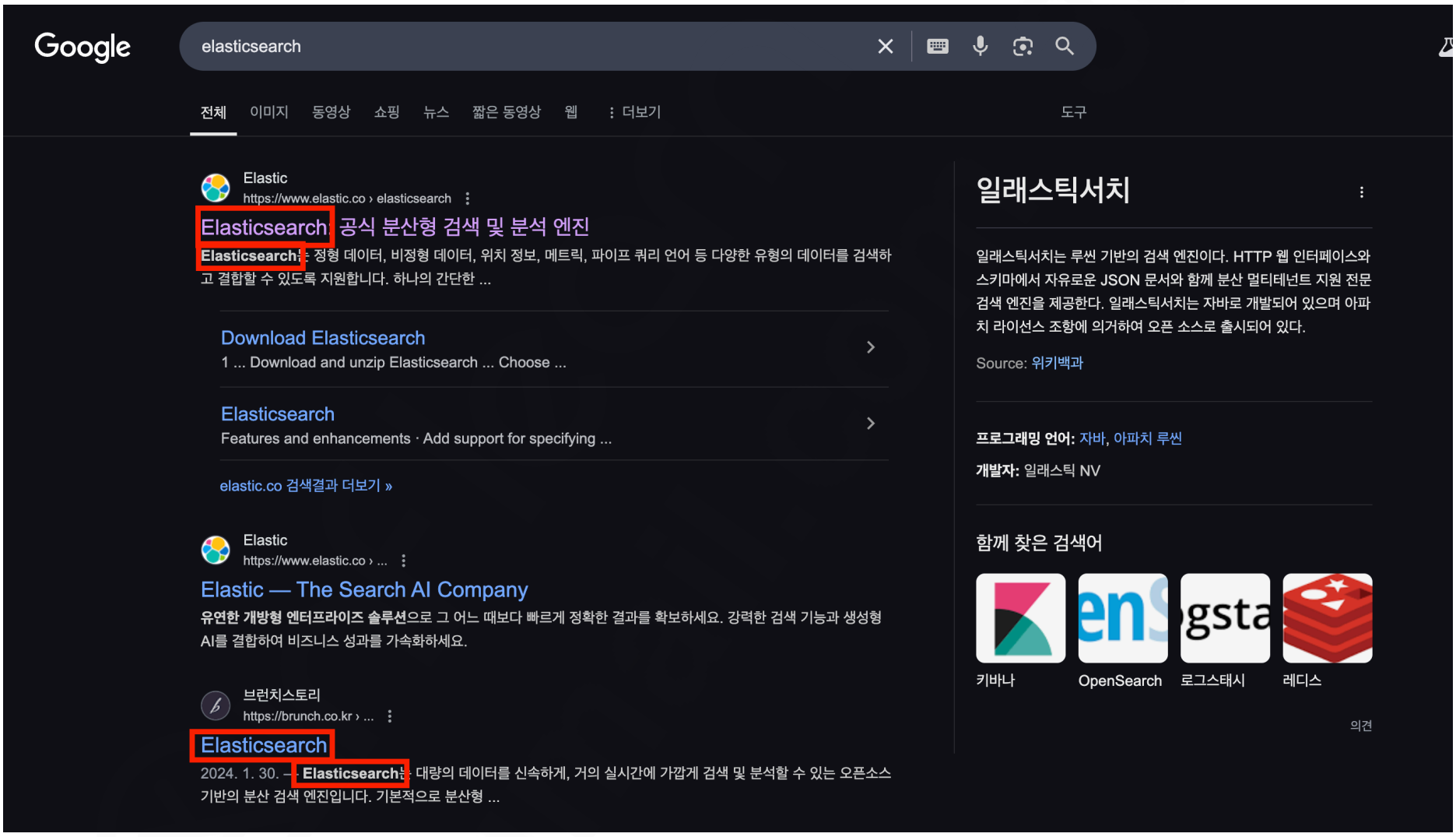
- `fuzziness : AUTO` : 단어 길이에 따라 오타 허용 개수를 자동으로 설정

오타를 어느 정도 허용해서 검색하고 싶을 때 `fuzziness`를 활용하면 된다는 걸 배웠다. 다음 강의에서는 다른 검색 기능을 배워보자.

여러 필드(ex. 제목, 내용)에서 검색 키워드가 포함된 데이터를 조회하고 싶을 때 (multi_match)

✅ 여러 필드에서 검색 키워드가 포함된 데이터를 조회하고 싶을 때 (multi_match)

구글에서 특정 키워드로 검색해보면 사이트의 `제목(title)` 뿐만 아니라 `내용(content)`을 포함해서 검색한다.



이와 같이 여러 필드에서 검색 키워드(`엘라스틱서치 적용 후기`)가 포함된 데이터를 조회하고 싶을 때 어떻게 쿼리를 작성하면 되는 지 살펴보자.

✅ 실습

1. **인덱스 생성**

```
DELETE /boards

PUT /boards
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "nori"
      },
      "content": {
        "type": "text",
        "analyzer": "nori"
      }
    }
```

```
  }
}
```

2. **데이터 삽입**

```
// title, content 둘 다에 키워드 포함
POST /boards/_doc
{
  "title": "엘라스틱서치 적용 후기",
  "content": "회사 프로젝트에 엘라스틱서치를 적용한 후기를 공유합니다."
}

// title에만 키워드 포함
POST /boards/_doc
{
  "title": "엘라스틱서치를 사용해보니",
  "content": "검색 엔진 도입 후 성능이 향상되었습니다."
}

// content에만 키워드 포함
POST /boards/_doc
{
  "title": "검색엔진 도입 사례",
  "content": "이번 프로젝트에 엘라스틱서치를 적용한 후 많은 개선 효과가 있었습니다."
}

// title, content 둘 다 포함 안 됨
POST /boards/_doc
{
  "title": "레디스 캐시 사용기",
  "content": "서비스 속도 개선을 위해 캐시 시스템을 사용했습니다."
}
```

3. **검색하기**

```
GET /boards/_search
{
  "query": {
    "multi_match": {
      "query": "엘라스틱서치 적용 후기",
      "fields": ["title", "content"]
    }
  }
}
```

응답값

```
{
  "took": 33,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
```

```
},
"hits": {
  "total": {
    "value": 3,
    "relation": "eq"
  },
  "max_score": 3.79424,
  "hits": [
    // title, content 둘 다에 키워드 포함
    {
      "_index": "boards",
      "_id": "2k-SmJYBWhYNXJPWbr6u",
      "_score": 3.79424,
      "_source": {
        "title": "엘라스틱서치 적용 후기",
        "content": "회사 프로젝트에 엘라스틱서치를 적용한 후기를 공유합니다."
      }
    },

// content에만 키워드 포함
{
  "_index": "boards",
  "_id": "3E-SmJYBWhYNXJPWd740",
  "_score": 1.7749425,
  "_source": {
    "title": "검색엔진 도입 사례",
    "content": "이번 프로젝트에 엘라스틱서치를 적용한 후 많은 개선 효과가 있었습니다."
  }
},

// title에만 키워드 포함
{
  "_index": "boards",
  "_id": "20-SmJYBWhYNXJPWc75z",
  "_score": 1.3862942,
  "_source": {
    "title": "엘라스틱서치를 사용해보니",
    "content": "검색 엔진 도입 후 성능이 향상되었습니다."
  }
}
    ]
  }
}
```

- `title` 또는 `content` 필드에 검색 키워드가 포함된 데이터를 조회했다.
  - `title`이랑 `content` 둘 다에 키워드가 포함되지 않은 데이터는 제외됐다.
- 검색 키워드와 데이터의 관련성을 score로 점수를 매겨 조회했다.
  - 검색 키워드가 문서에서 자주 등장할 수록 점수가 높게 책정된다.
  - 전체 문서 중 검색어가 희귀한 검색어가 일치할 수록 점수가 높게 책정된다.
  - 필드 값의 길이가 작은데도 불구하고 키워드가 등장했다면 점수가 높게 책정된다.

4. 가중치 활용해서 검색해보기

`내용(content)`에만 키워드가 포함된 글보다 `제목(title)`에만 키워드가 포함된 글을 더 상위노출 시키고 싶을 수 있다. 즉, `제목(title)`에 검색 키워드가 등장한다면 더 관련성 높은 데이터라고 판단하고 싶을 수 있다. 그럴 때 가중치를 활용한다.

```
GET /boards/_search
{
  "query": {
    "multi_match": {
      "query": "엘라스틱서치 적용 후기",
      "fields": ["title^2", "content"] // title에 2배 더 높은 score를 부여
    }
  }
}
```

응답값

```
{
  "took": 10,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 3,
      "relation": "eq"
    },
    "max_score": 7.58848,
    "hits": [
      // title, content 둘 다에 키워드 포함
      {
        "_index": "boards",
        "_id": "2k-SmJYBWhYNXJPWbr6u",
        "_score": 7.58848,
        "_source": {
          "title": "엘라스틱서치 적용 후기",
          "content": "회사 프로젝트에 엘라스틱서치를 적용한 후기를 공유합니다."
        }
      },

      // title에만 키워드 포함
      {
        "_index": "boards",
        "_id": "20-SmJYBWhYNXJPWc75z",
        "_score": 2.7725885,
        "_source": {
          "title": "엘라스틱서치를 사용해보니",
          "content": "검색 엔진 도입 후 성능이 향상되었습니다."
        }
      },

      // content에만 키워드 포함
      {
        "_index": "boards",
        "_id": "3E-SmJYBWhYNXJPWd740",
        "_score": 1.7749425,
        "_source": {
```

```
        "title": "검색엔진 도입 사례",
        "content": "이번 프로젝트에 엘라스틱서치를 적용한 후 많은 개선 효과가 있었습니다."
      }
    }
   ]
  }
}
```

의도한 순서대로 게시글이 잘 조회됐다.

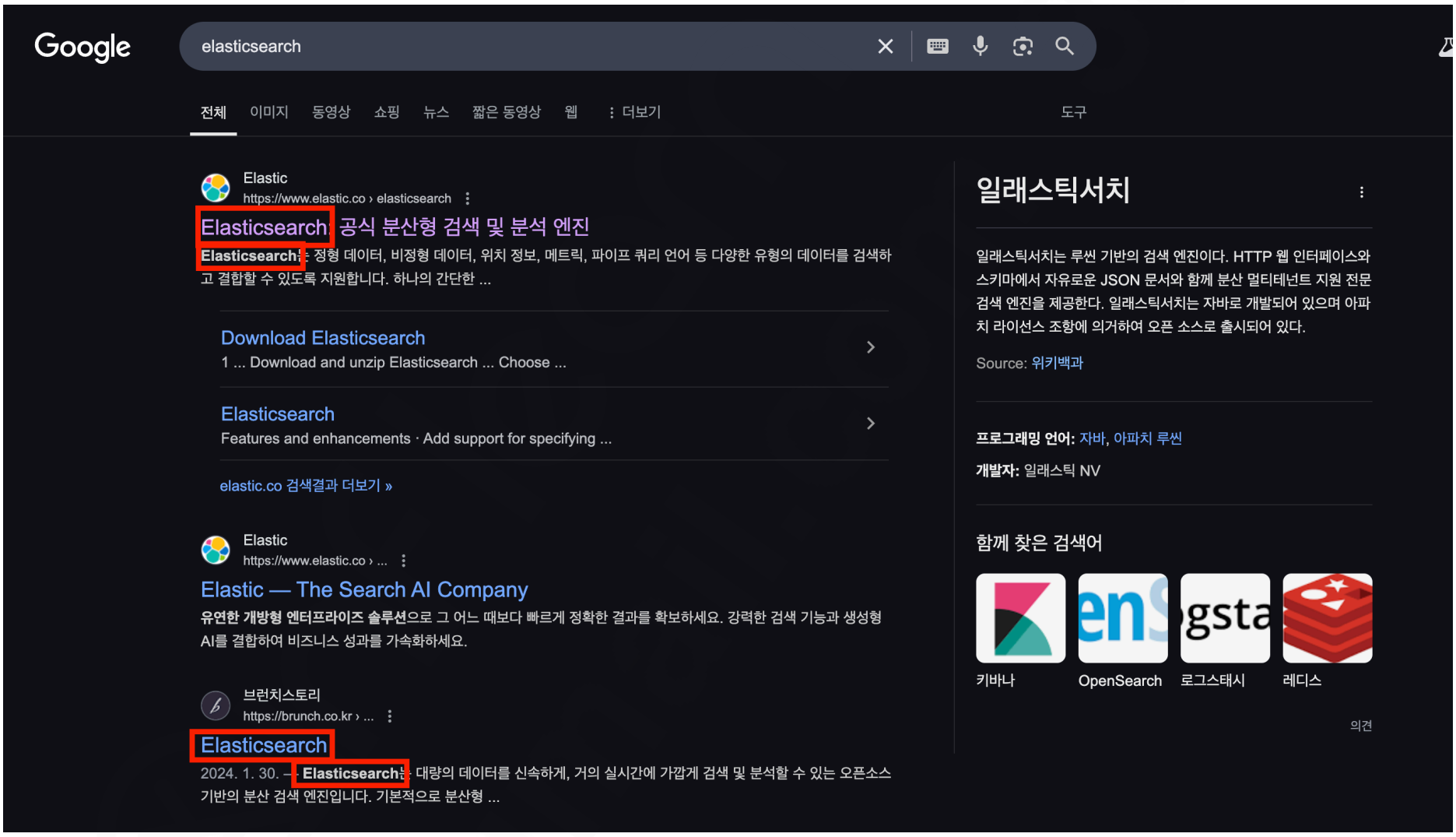
여러 필드에서 검색 키워드가 포함된 데이터를 조회할 때는 `multi_match` 키워드를 사용하면 된다. 다음 강의에서는 다른 검색 기능을 배워보자.

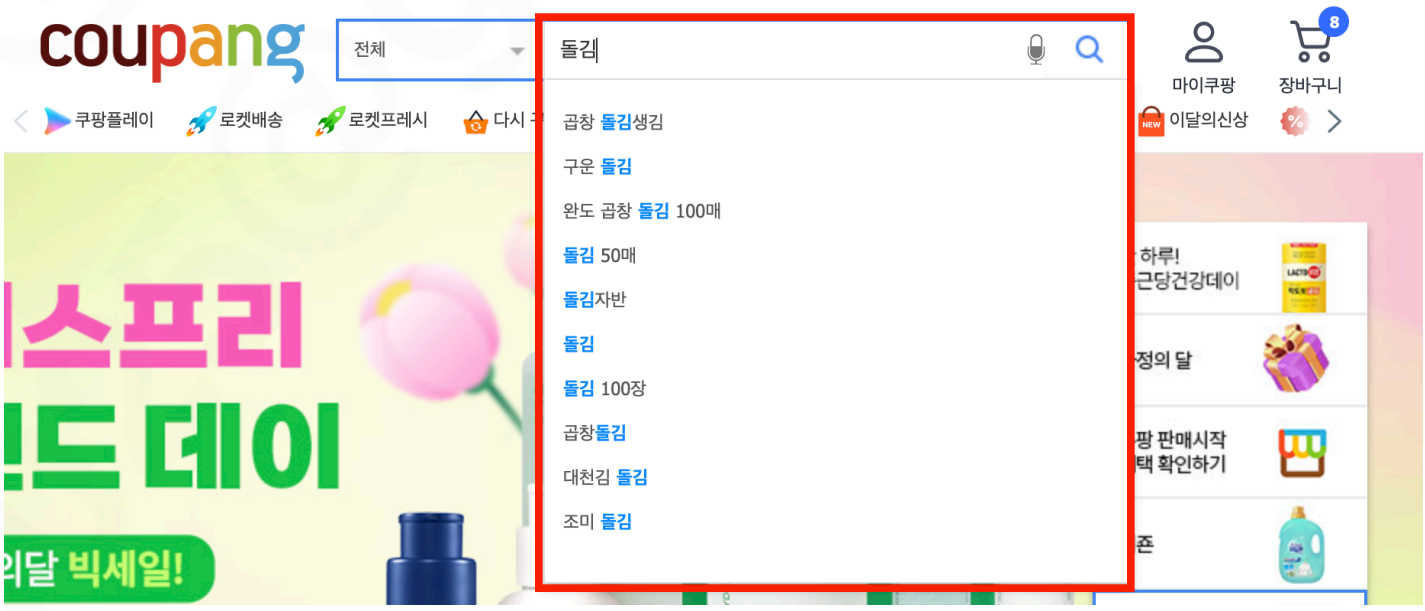
@kfc1@gmail.com
kfc302502

검색한 키워드를 하이라이팅 처리하고 싶을 때 (highlight)

✅ 검색한 키워드를 하이라이팅 처리하고 싶을 때 (highlight)

구글이나 쿠팡에서 검색을 해보면 검색한 키워드가 하이라이팅 처리가 되있는 걸 확인할 수 있다. 이 기능을 구현해보자.





✅ 실습

> 이전 강의에 셋팅해놓은 환경을 그대로 사용하자.

검색하기

```
GET /boards/_search
{
  "query": {
    "multi_match": {
      "query": "엘라스틱서치 적용 후기",
      "fields": ["title", "content"]
    }
  },
  "highlight": {
    "fields": {
      "title": {
        "pre_tags": ["<mark>"],
        "post_tags": ["</mark>"]
      },
      "content": {
        "pre_tags": ["<b>"],
        "post_tags": ["</b>"]
      }
    }
  }
}
```

응답값

```
{
  "took": 19,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 3,
      "relation": "eq"
    },
    "max_score": 3.79424,
    "hits": [
      {
        "_index": "boards",
        "_id": "2k-SmJYBWhYNXJPWbr6u",
        "_score": 3.79424,
        "_source": {
          "title": "엘라스틱서치 적용 후기",
          "content": "회사 프로젝트에 엘라스틱서치를 적용한 후기를 공유합니다."
        },
        "highlight": {
          "title": [
            "<mark>엘라스틱</mark><mark>서치</mark> <mark>적용</mark> <mark>후기</mark>"
          ],
          "content": [
            "회사 프로젝트에 <b>엘라스틱</b><b>서치</b>를 <b>적용</b>한 <b>후기</b>를 공유합니다."
```

```
      ]
     }
    },
    {
     "_index": "boards",
     "_id": "3E-SmJYBWhYNXJPWd740",
     "_score": 1.7749425,
     "_source": {
      "title": "검색엔진 도입 사례",
      "content": "이번 프로젝트에 엘라스틱서치를 적용한 후 많은 개선 효과가 있었습니다."
     },
     "highlight": {
      "content": [
       "이번 프로젝트에 <b>엘라스틱</b><b>서치</b>를 <b>적용</b>한 후 많은 개선 효과가 있었습니다."
      ]
     }
    },
    {
     "_index": "boards",
     "_id": "20-SmJYBWhYNXJPWc75z",
     "_score": 1.3862942,
     "_source": {
      "title": "엘라스틱서치를 사용해보니",
      "content": "검색 엔진 도입 후 성능이 향상되었습니다."
     },
     "highlight": {
      "title": [
       "<mark>엘라스틱</mark><mark>서치</mark>를 사용해보니"
      ]
     }
    }
   ]
  }
}
```


검색 결과에 HTML 태그로 감싼 뒤 결과값을 반환했다. 이 결과값을 활용해서 웹 페이지에서 하이라이팅 처리를 하면 된다.

📖

페이지네이션 (Pagination), 정렬(Sorting)

✅ 페이지네이션 (Pagination)

서버의 과부하를 방지하기 위해 데이터 조회 시 필수적으로 적용해야 하는 기능이 **페이지네이션(Pagination)**이다. Elasticsearch에서 페이지네이션(Pagination)을 활용해 데이터 조회하는 방법을 알아보자.

1. **인덱스 생성**

```
DELETE /boards

PUT /boards
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "nori"
      },
      "likes": { "type": "long" }
    }
  }
}
```

2. **데이터 생성**

```
POST /boards/_doc
{
  "title": "1번 글",
  "likes": 12
}

POST /boards/_doc
{
  "title": "2번 글",
  "likes": 35
}

POST /boards/_doc
{
  "title": "3번 글",
  "likes": 1
}

POST /boards/_doc
{
  "title": "4번 글",
  "likes": 11
}

POST /boards/_doc
{
```

```
  "title": "5번 글",
  "likes": 72
}

POST /boards/_doc
{
  "title": "6번 글",
  "likes": 10
}

POST /boards/_doc
{
  "title": "7번 글",
  "likes": 5
}
```

3. **페이지네이션 적용시켜서 데이터 불러오기**

```
// 1 페이지
GET /boards/_search
{
  "query": {
    "match": {
      "title": "글"
    }
  },
  "size": 3,
  "from": 0
}

// 2 페이지
GET /boards/_search
{
  "query": {
    "match": {
      "title": "글"
    }
  },
  "size": 3,
  "from": 3
}

// 3 페이지
GET /boards/_search
{
  "query": {
    "match": {
      "title": "글"
    }
  },
  "size": 3,
  "from": 6
}
```

- `size` : 한 페이지에 불러올 데이터 개수 (SQL문의 `LIMIT`과 동일)
- `from` : 몇 번째 데이터부터 불러올 지 (SQL문의 `OFFSET`과 동일, `0`부터 시작)

- `from` = (페이지 수 - 1) x `size`

✅ 정렬 (Sorting)

> 좋아요 수(likes)를 내림차순으로 정렬해보자.

```
GET /boards/_search
{
  "query": {
    "match": {
      "title": "글"
    }
  },
  "sort": [
    {
      "likes": {
        "order": "desc"
      }
    }
  ]
}
```

khb302571@gmail.com

📖

하나의 필드에 text와 keyword 타입을 동시에 사용하고 싶을 때 (Multi Field)

✅ 하나의 필드에 text와 keyword 타입을 동시에 사용하고 싶을 때 (Multi Field)

이전 강의(📖 매핑(mapping)이란? / 데이터 타입(data type))에서 데이터 타입을 설명할 때 `text`는 유연한 검색이 필요할 때 사용한다고 얘기했다. 반면에 `keyword`는 정확한 검색이 필요할 때 사용한다고 했다.

그런데 **유연한 검색**과 **정확한 검색** 둘 다 하고 싶은 경우가 있을 수 있다. 예를 들어, `상품 카테고리`와 `이름` 필드에 저장된 필드 값에 대해 키워드로 검색할 때는 유연하게 검색되게 하고 싶고, 정확한 비교를 통해 특정 `카테고리`를 기준으로 필터링을 하고 싶을 수 있다. 즉, `text` 타입과 `keyword` 타입을 동시에 사용하고 싶은 경우이다.

이 문제를 어떻게 해결할 수 있는 지 바로 알아보자.

✅ 문제 상황

먼저 위에서 설명한 문제 상황부터 경험해보자.

1. **인덱스 생성**

```
DELETE /products

PUT /products
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text",
        "analyzer": "nori"
      },
      "category": {
        "type": "text",
        "analyzer": "nori"
      }
    }
  }
}
```

2. **데이터 삽입**

```
POST /products/_doc
{
  "name": "삼성 세탁기",
  "category": "특수 가전제품"
}
```

3. **검색하기**

[`name` , `category` 을 활용해 유연하게 검색하고 싶을 때]

```
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "가전",
      "fields": ["name", "category"]
    }
  }
}
```

[`특수 가전제품` 카테고리의 상품만 검색하고 싶을 때]

```
GET /products/_search
{
  "query": {
    "term": {
      "category": "특수 가전제품"
    }
  }
}
```

⇒ 제대로 작동하지 않는다! 이걸 해결하기 위해서는 인덱스의 `category` 의 데이터 타입을 설정할 때 `text` 타입과 `keyword` 타입을 둘 다 선언해야 한다. 그 방법을 알아보자.

✅ 해결 방법

하나의 필드에 여러 개의 타입을 선언할 수 있는 방법을 활용할 것이다. 이 방법을 보고 Elasticsearch에서는 **multi-fields**라고 얘기한다.

1. **인덱스 생성, 데이터 삽입**

```
DELETE /products

PUT /products
{
"mappings": {
  "properties": {
    "name": {
      "type": "text",
      "analyzer": "nori"
    },
    "category": {
      "type": "text",
      "analyzer": "nori",
      "fields": {
        "raw": { // 서브 필드명 (다른 이름으로 설정해도 됨)
          "type": "keyword"
        }
      }
    }
  }
```

```
  }
 }
```

위와 같이 설정하면 `category` 필드에 하나의 데이터를 삽입하더라도 `text`, `keyword` 타입의 형태로 나눠서 보관하게 된다.

2. **데이터 삽입**

```
POST /products/_doc
{
  "name": "삼성 세탁기",
  "category": "특수 가전제품"
}
```

`text` 타입은 토큰으로 분리해서 저장하지만, `keyword` 타입은 값 자체를 통째로 저장한다. 그래서 `category`에 `특수 가전제품`이라는 값을 삽입했을 때 아래와 같이 저장된다.

category (text 타입)	category.raw (keyword 타입)
특수	특수 가전제품
가전	
제품	

정말 위와 같이 저장되는 지 확인해보자.

3. **Anaylze API로 디버깅해보기**

```
POST /products/_analyze
{
  "field": "category",
  "text": "특수 가전제품"
}

POST /products/_analyze
{
  "field": "category.raw",
  "text": "특수 가전제품"
}
```

위에서 예상한대로 토큰이 분리되는 걸 확인할 수 있다. 그럼 이제 실제 검색을 해보자.

4. **검색해보기**

[`name`, `category`을 활용해 유연하게 검색하고 싶을 때]

```
POST /products/_search
{
  "query": {
    "multi_match": {
      "query": "가전",
      "fields": ["name", "category"]
    }
  }
}
```

[`특수 가전제품` 카테고리의 상품만 검색하고 싶을 때]

```
POST /products/_search
{
  "query": {
    "term": {
      "category.raw": "특수 가전제품"
    }
  }
}
```

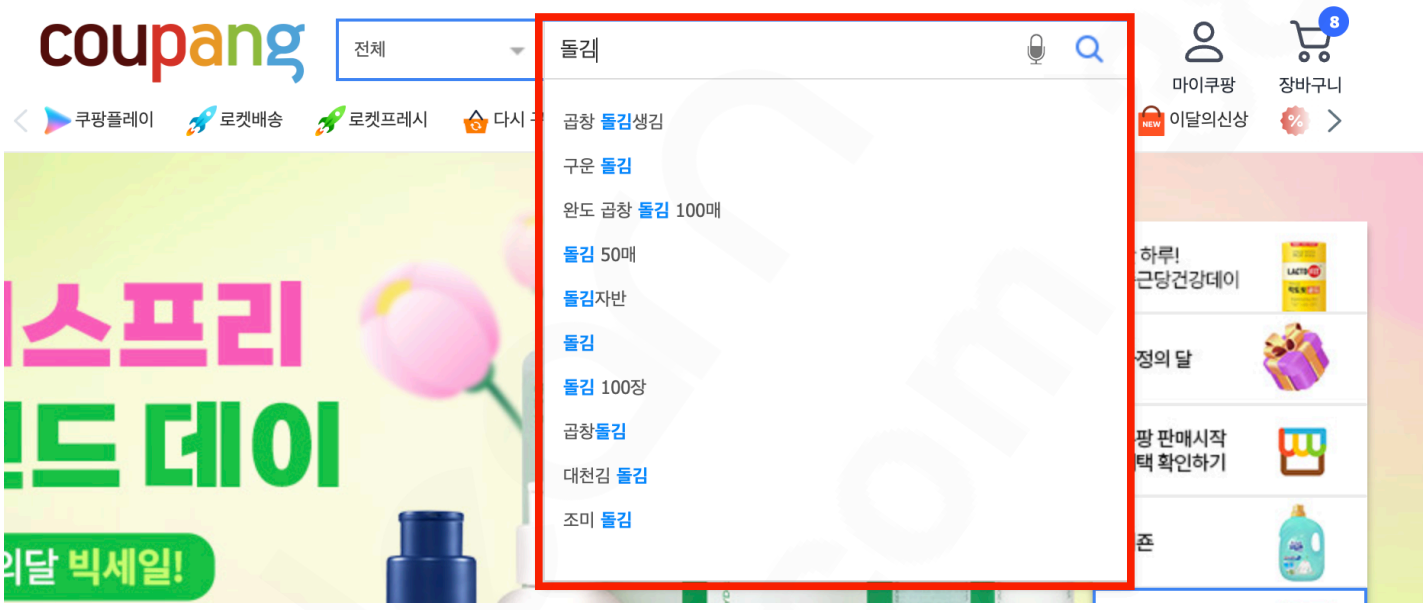
정상적으로 잘 응답하는 걸 볼 수 있다.

@naver.com
khb3025

📖

검색 키워드를 일부 입력했을 때 검색어를 추천해주는 기능 (자동 완성 기능)

✅ 검색 키워드를 일부 입력했을 때 검색어를 추천해주는 기능 (자동 완성 기능)



쿠팡 서비스에 들어가서 검색 창에 글자를 입력하면 글자에 해당하는 검색어를 추천해준다. 자동 완성 기능을 구현하는 방법에는 정말 다양한 방법이 존재한다. 그 중에서 가성비가 좋은 방법인 `search_as_you_type`이라는 걸 활용해서 구현하는 방법을 배워보자.

✅ `search_as_you_type` 이란?

- Elasticsearch에서 **자동 완성 기능을 쉽게 구현할 수 있게 설계된 데이터 타입**이다.
- `text` 타입처럼 애널라이저(Analyzer)를 거쳐 토큰으로 분리된다.
- 이 타입을 활용해 필드를 만들면 내부적으로 `_2gram`, `_3gram`이라는 멀티 필드(Multi Field)도 같이 만든다.

✅ `_2gram`, `_3gram` 은 어떻게 구성되어 있을까?

`search_as_you_type`의 `_2gram`은 두 단어씩 묶어서 토큰을 만든다. `_3gram`은 세 단어씩 묶어서 토큰을 만든다. 무슨 말인지 Analyze API를 활용해 직접 눈으로 확인해보자.

1. **인덱스 생성**

```
DELETE /products

PUT /products
{
  "mappings": {
    "properties": {
      "name": {
        "type": "search_as_you_type",
        "analyzer": "nori"
      }
    }
```

```
  }
}
```

2. **Analyze API로 디버깅하기**

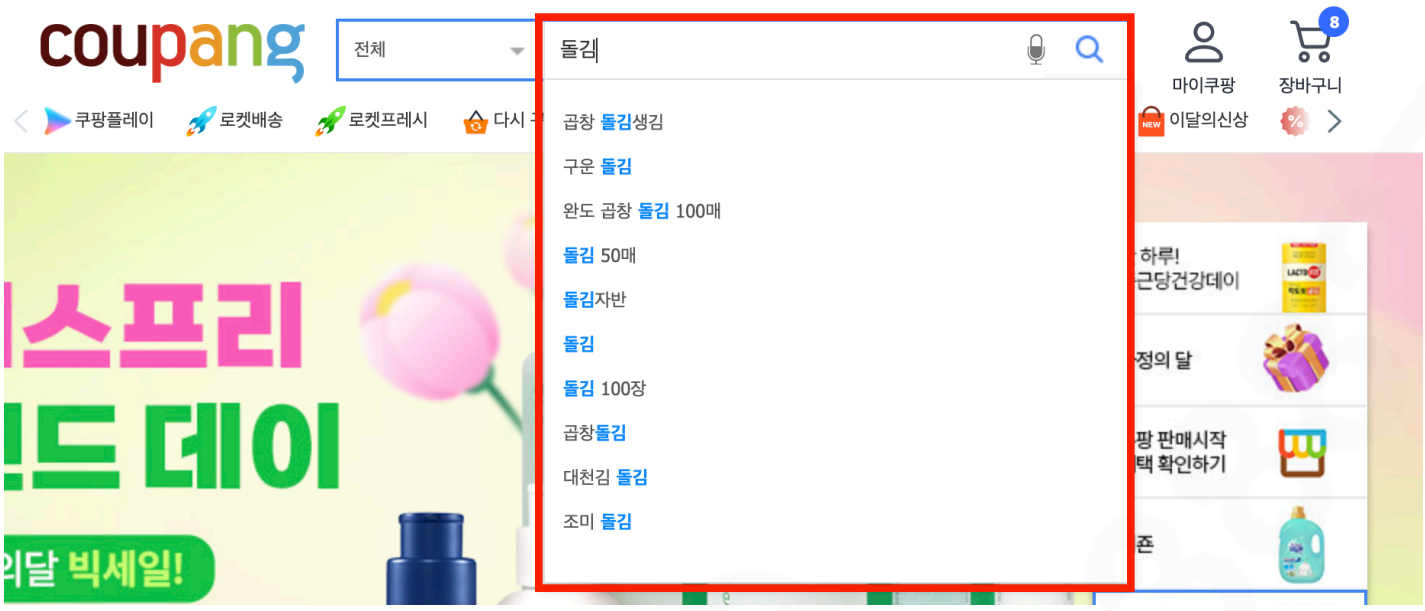
   영어 문장

```
GET /products/_analyze
{
  "field": "name",
  "text": "You have the big banana"
}

GET /products/_analyze
{
  "field": "name._2gram",
  "text": "you have the big banana"
 }

 GET /products/_analyze
 {
   "field": "name._3gram",
   "text": "you have the big banana"
 }
```

   한글 문장

```
GET /products/_analyze
{
  "field": "name",
  "text": "프리미엄 감귤 선물세트"
}

GET /products/_analyze
{
  "field": "name._2gram",
  "text": "프리미엄 감귤 선물세트"
}

GET /products/_analyze
{
  "field": "name._3gram",
  "text": "프리미엄 감귤 선물세트"
}
```

✅ 검색해보기



1. **인덱스 생성**

```
DELETE /products

PUT /products
{
  "mappings": {
    "properties": {
      "name": {
        "type": "search_as_you_type",
        "analyzer": "nori"
      }
    }
  }
}
```

2. **데이터 삽입**

```
POST /products/_doc
{
  "name": "곱창 돌김생김"
}

POST /products/_doc
{
  "name": "구운 돌김"
}

POST /products/_doc
{
  "name": "완도 곱창 돌김 100매"
}

POST /products/_doc
{
  "name": "삼성 노트북"
}

POST /products/_doc
{
```

```
  "name": "Nike 신발"
}
```

3. **데이터 검색하기**

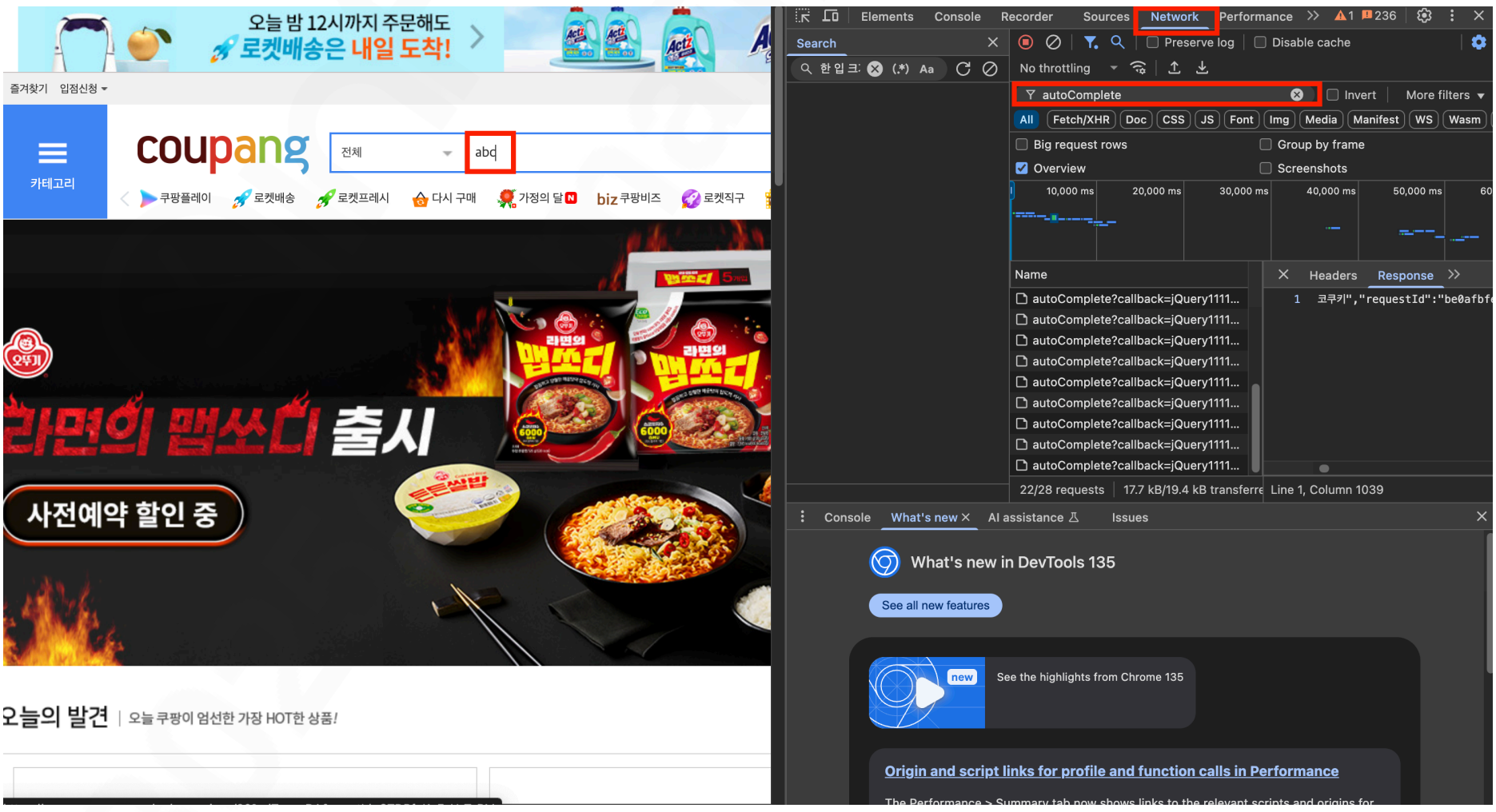
```
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "돌김",
      "type": "bool_prefix",
      "fields": [
        "name",
        "name._2gram",
        "name._3gram"
      ]
    }
  }
}
```

- `multi_match` : 여러 필드에 걸쳐서 데이터 검색
- `bool_prefix` : 앞쪽 단어는 match, 마지막 단어는 prefix match
  - `you have th`라고 검색하면 앞쪽 단어인 `you`와 `have`는 역인덱스에 저장된 토큰과 일치하는 데이터를 찾고, 마지막 단어인 `th`는 역인덱스에 저장된 토큰 중에 `th`로 시작하는 데이터를 찾는다.
- `name._2gram`, `name._3gram` 필드에서도 검색을 하는 이유는 연속적으로 단어가 일치할수록 score를 더 높게 측정해 상위 노출되도록 만들기 위함이다.

```
// 1글자로 검색해도 검색 잘 됨
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "돌",
      "type": "bool_prefix",
      "fields": [
        "name",
        "name._2gram",
        "name._3gram"
      ]
    }
  }
}

// 순서 바뀌어도 검색 잘 됨
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "돌김 구운",
      "type": "bool_prefix",
      "fields": [
        "name",
        "name._2gram",
        "name._3gram"
```

```
    ]
   }
  }
}

// 순서까지 일치하면 score가 더 높게 측정됨
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "구운 돌김",
      "type": "bool_prefix",
      "fields": [
        "name",
        "name._2gram",
        "name._3gram"
      ]
    }
  }
}
```

✅ 자동 완성 기능 구현 방식



글씨를 타이핑할 때마다 API 서버로 요청을 보내 자동 완성에 보여줄 데이터를 반환한다.

📖 프로젝트 설명 / 요구 사항 (상품 검색 기능, 자동 완성 기능)

👨🏻‍🏫 지금까지 배운 Elasticsearch의 기능을 활용해 쿠팡의 상품 검색 기능을 구현해보자.

✅ 프로젝트 설명

https://github.com/JSCODE-EDU/elasticsearch-coupang-project

Spring boot(gradle), Spring Data JPA, MySQL을 활용해 아래 API를 가진 서버를 운영하고 있었다고 가정하자.

- 상품 전체 조회 API
- 상품 등록 API
- 상품 삭제 API

그런데 기획자가 `검색 기능`과 `검색어 자동 완성 기능`을 구현해달라고 요청한 것이다. 기존 프로젝트에서 `검색 기능`과 `검색어 자동 완성 기능`을 구현해야 하는데 MySQL로는 한계가 있어 Elasticsearch를 활용하기로 결정한 상황이라고 하자.

> 기획자의 구체적인 요구 사항은 다음과 같다.

✅ 요구 사항

1. **검색 기능**
   - ☐ 검색을 했을 때 `상품명`, `상품 설명`, `카테고리명`을 기반으로 관련된 상품을 검색해야 한다.
     - ☐ **상품명 : 상품 설명 : 카테고리명**의 score 가중치를 **3 : 1 : 2**로 설정하기
   - ☐ 대소문자 구분없이 검색할 수 있어야 한다.
   - ☐ 단어의 순서에 상관없이 검색할 수 있어야 한다.
     - ex) `삼성 노트북`이라는 상품을 `노트북 삼성`이라고 검색해도 조회가 되야 한다.
   - ☐ `상품 설명`에 HTML 태그가 섞여서 저장되어 있는데, 검색할 때는 HTML 태그로 검색되지 않아야 한다.
   - ☐ 한글, 영어 둘 다 검색이 잘 돼야 한다.
   - ☐ `상품명`에 대해서는 아래 값을 동의어로 인식할 수 있어야 한다.

     ```
     samsung, 삼성
     apple, 애플
     노트북, 랩탑, 컴퓨터, computer, laptop, notebook
     전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone
     아이폰, iphone
     맥북, 맥, macbook, mac
     ```

   - ☐ 카테고리, 가격으로 필터링을 할 수 있어야 한다.

- 특정 카테고리 내에서만 검색할 수 있어야 한다.
- 특정 가격 범위 내에서만 검색할 수 있어야 한다.

☐ 되도록이면 검색 결과 중 평점이 4.0이 넘는 상품을 상위에 노출시킬 수 있도록 한다.

☐ 검색어에 어느 정도 오타가 있더라도 유사한 상품을 조회할 수 있어야 한다.

- ex) `samsung notebook`이라는 상품을 `simsung notebook`이라고 검색해도 조회가 되야 한다.

☐ `상품명`에서 일치하는 검색 키워드는 하이라이팅 처리(`<b></b>`)를 해야 한다.

☐ 상품이 5개씩 조회되도록 페이지네이션 해야 한다.

2. **자동 완성 기능**

☐ `상품명`으로만 자동 완성이 되도록 해야 한다.

☐ 자동 완성 결과가 5개만 조회되도록 설정해야 한다.

@kh3025.com

📖

기존 프로젝트 구조 살펴보기

✅ 기존 프로젝트 구조 살펴보기

> https://github.com/JSCODE-EDU/elasticsearch-coupang-project

1. **Product 엔티티**
   - `id`
   - `name` : 상품명
   - `description` : 상품 설명 (HTML로 상품 설명을 작성)
   - `price` : 가격
   - `rating` : 평점
   - `category` : 카테고리명

2. **API 구성**
   - 상품 전체 조회 API
   - 상품 등록 API
   - 상품 삭제 API

3. **아키텍처**

4. **특징**
   - Controller - Service - Repository 구성 활용
   - DTO 활용

5. **의존성**
   - Spring Boot 3.4.5
   - JDK 17
   - gradle 기반
   - Spring Data JPA, MySQL

다음 강의에서는 기존 프로젝트 구조에서 `검색 기능`과 `자동 완성 기능`을 도입하기 위해 아키텍처를 어떻게 구성해야 하는 지 고민해보자.

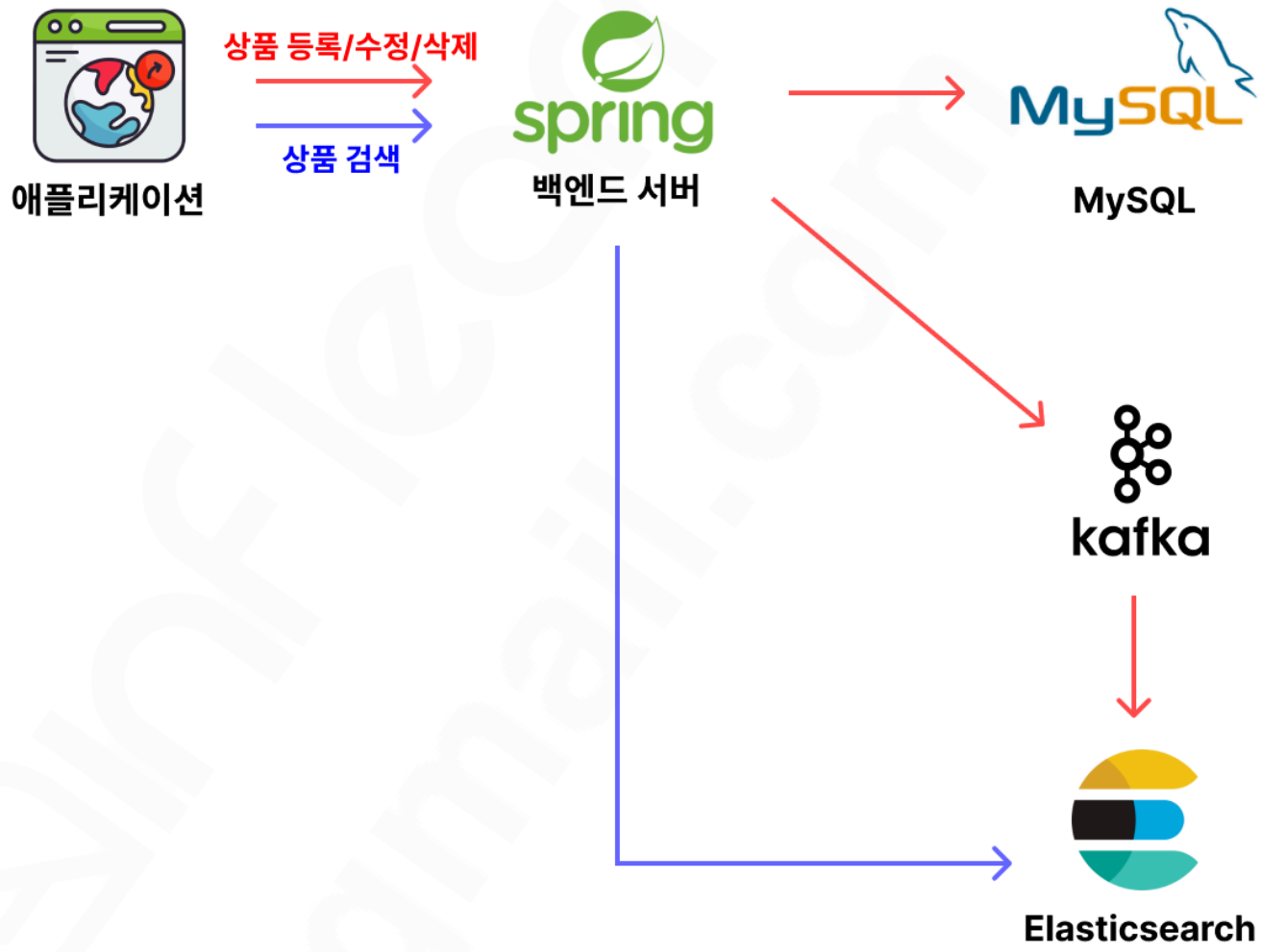
@gmail.com
khb302

인프라 아키텍처 설계하기

✅ 인프라 아키텍처 설계하기

MySQL과 Elasticsearch의 장단점은 명확하다. MySQL은 트랜잭션을 기반으로 데이터의 정합성을 보존해주는 게 큰 장점이다. 반면에 Elasticsearch는 빠른 검색에 최적화 되어 있다. 현업에서는 이 2가지의 장점을 둘 다 활용하기 위해서 MySQL과 Elasticsearch를 둘 다 활용하는 경우가 많다.

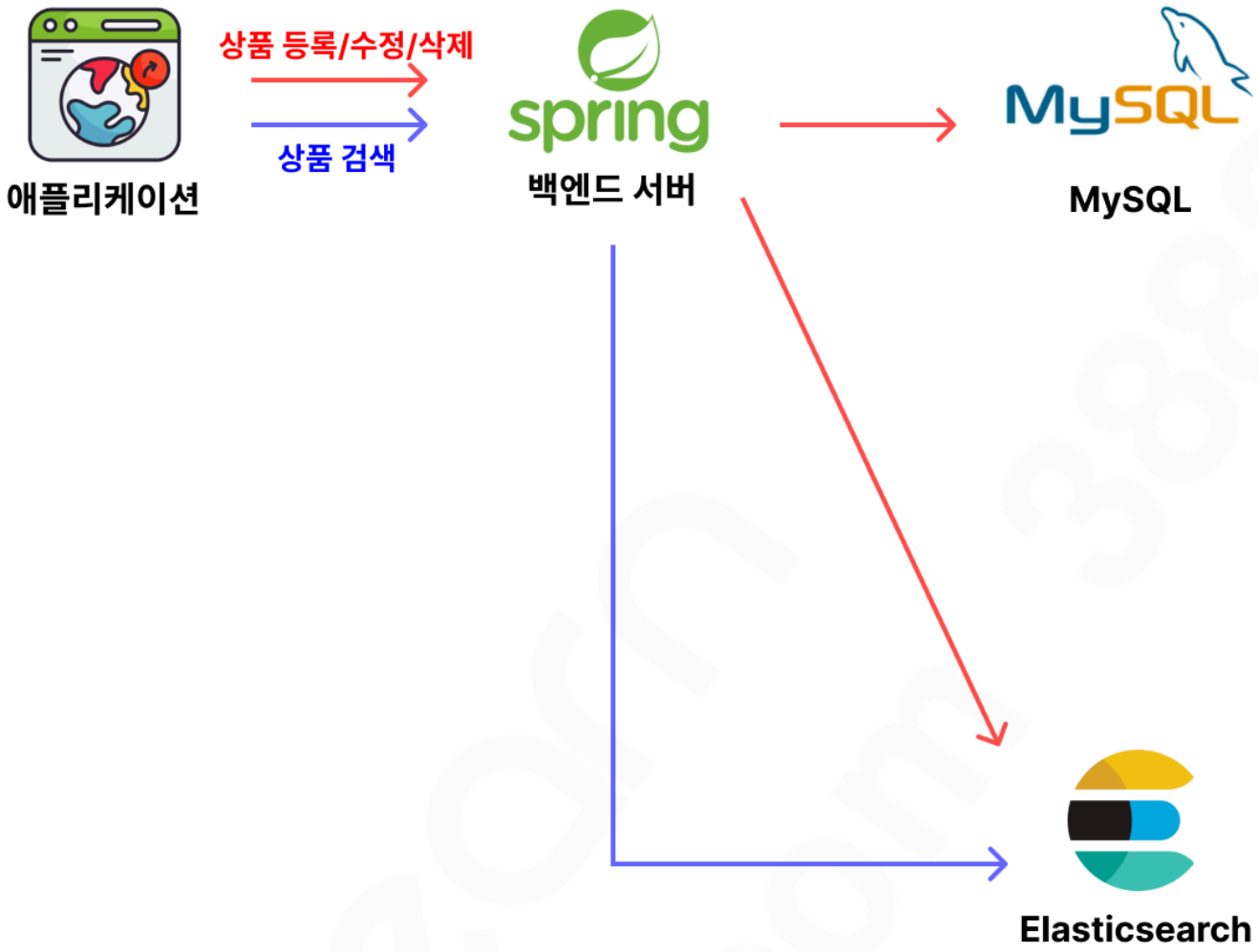
이 때, 문제되는 점이 데이터를 삽입할 때 MySQL과 Elasticsearch에 둘 다 삽입해주어야 한다는 점이다. 즉, **데이터 동기화 문제**를 해결해야 한다. 검색해보면 카프카(kafka)와 같은 메시지 큐를 활용해 처리하는 경우가 대부분이다.



하지만 소규모 서비스에서는 카프카(kafka)를 도입하고 관리하는 것 자체가 부담인 경우가 많다. 이런 경우에는 애플리케이션 레벨에서 아래와 같이 직접 동기화를 하기도 한다.

```
public class ProductService {
  ...

  @Transactional
  public Product createProduct(Product product) {
    // MySQL에 데이터 저장
    productRepository.save(product);

    // Elasticsaerch에 데이터 저장
    productSearchRepository.save(product);
  }
}
```

아키텍처 그림으로 표현하자면 다음과 같다.



이번 프로젝트에서는 카프카(kafka)를 활용하지 않고 구성할 것이기 때문에 이 아키텍처에 맞게 프로젝트를 진행해보자.

❗ 인프라 구성은 정답이라는 게 없다. 장단점이랑 기회 비용(trade-off)만 있을 뿐이다. 만약 '`Elasticsearch는 무조건 카프카를 써서 데이터를 동기화 해야 돼`'라는 생각을 가지고 있었다면, 카프카를 쓰지 않았을 때 정말 서비스를 운영하지 못할 정도의 치명적인 문제점이 있는 지 다시 한 번 찾아보고 고민해보자.

khb3025@gmail.com

🎯 [실습] 인덱스 생성 / 매핑 정의 / 여러 필드에 걸쳐 검색하기

✅ 요구사항 반영하기

☐ 검색을 했을 때 `상품명`, `상품 설명`, `카테고리명`을 기반으로 관련된 상품을 검색해야 한다.
(
상품명 : 상품 설명 : 카테고리명의 score 가중치를 **3 : 1 : 2**로 설정하기)

Spring Data Elasticsearch에 익숙한 사람은 Spring Boot에서 코드를 작성해도 된다. 하지만 그렇지 않다면 Kibana에서 요구 사항을 만족시키는 쿼리를 작성한 뒤에 Spring Boot 코드로 옮기는 게 훨씬 편하다. 따라서 Kibana에서 요구 사항을 만족시키는 쿼리를 작성해보자.

1. **Clone 받은 프로젝트가 정상적으로 실행되는 지 확인하기**

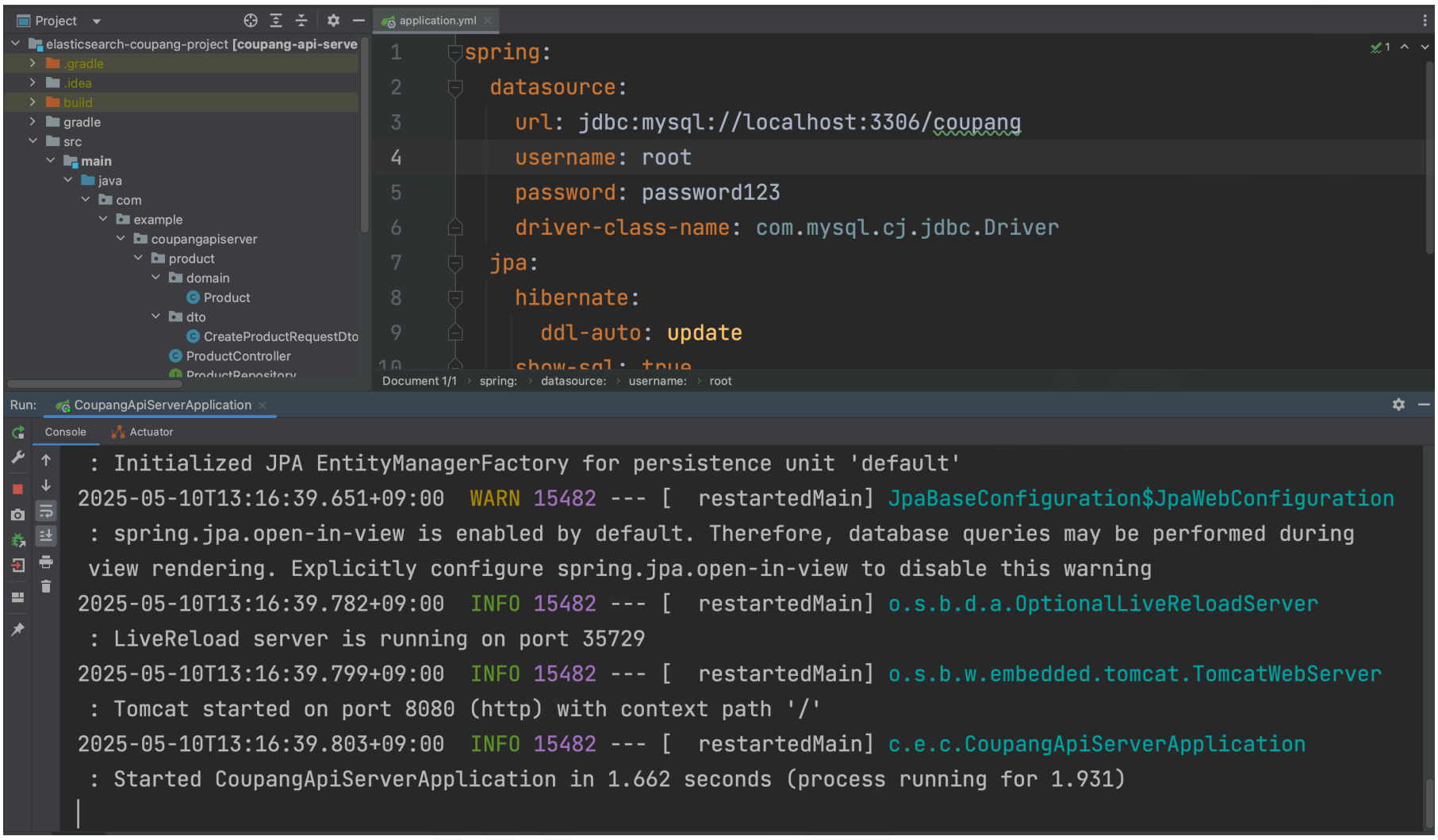
   Spring Boot 프로젝트를 시작하기 전에 로컬 환경에 MySQL이 실행되어 있어야 하고, MySQL에 `coupang`이라는 데이터베이스가 만들어져 있어야 한다. 그리고 로컬 환경에 설치된 MySQL의 `username`과 `password`에 맞게 application.yml 파일을 수정해야 한다.

   application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/coupang
    username: root
    password: password123
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

   그런 뒤에 Spring Boot 서버를 실행시켜보자. 아무런 에러 없이 잘 실행된다면 정상적으로 Spring Boot와 MySQL이 연결되서 작동하는 것이다.

khb302@naver.com



2. **인덱스 생성하기, 매핑 정의하기**

   Spring Boot에서 정의해놓은 Product 엔티티가 다음과 같은 필드를 가지고 있다. 그에 맞게 Elasticsearch에서 인덱스를 생성하고 매핑을 정의하자.

   Product 엔티티

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(columnDefinition = "TEXT")
    private String description; // 에디터가 HTML로 상품 설명을 저장

    private int price;

    private double rating;

    private String category;

    ...
}
```

   products 인덱스 생성 / 매핑 정의

```
DELETE /products

PUT /products
{
  "mappings": {
    "properties": {
      "id": {
        "type": "long"
      },
      "name": {
        "type": "text" // 유연한 검색 필요
      },
      "description": {
        "type": "text" // 유연한 검색 필요
      },
      "price": {
        "type": "integer" // 10억 이하의 정수
      },
      "rating": {
        "type": "double" // 실수(소수점을 가진 숫자 포함)
      },
      "category": {
        "type": "text" // 유연한 검색 필요
      }
    }
  }
}

GET /products
```

`상품명`, `상품 설명`, `카테고리명`을 기반으로 관련된 상품을 검색하기 위해 `name`, `description`, `category`를 `text` 타입으로 매핑했다.

3. **테스트용 데이터 넣기**

```
POST /products/_doc
{
  "id": 1,
  "name": "lg tv",
  "description": "",
  "price": 10000,
  "rating": 5.0,
  "category": ""
}

POST /products/_doc
{
  "id": 2,
  "name": "",
  "description": "lg tv",
  "price": 10000,
  "rating": 5.0,
  "category": ""
}

POST /products/_doc
{
```

```
  "id": 3,
  "name": "",
  "description": "",
  "price": 10000,
  "rating": 5.0,
  "category": "lg tv"
}
```

2. **검색하기**

```
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "lg",
      "fields": [
        "name^3",
        "description^1",
        "category^2"
      ]
    }
  }
}
```

- `상품명` (`name`), `상품 설명` (`description`), `카테고리` (`category`)에 포함된 키워드로 검색하더라도 검색이 잘 되는 걸 확인할 수 있다.
- 응답값을 보면 가중치에 따라 score가 책정되는 걸 확인할 수 있다. 그리고 score를 기준으로 데이터가 정렬돼서 조회된다.

첫 번째 요구사항을 만족시켰다. 다음 강의에서 다른 요구사항도 차근차근 만족시켜보자.

khb302@gmail.com

🎯 [실습] 대소문자 구분 없이 검색 / 단어 순서 상관없이 검색 / HTML 태그 섞이지 않게 검색

✅ 요구사항 반영하기

- [ ] 대소문자 구분없이 검색할 수 있어야 한다.
- [ ] 단어의 순서에 상관없이 검색할 수 있어야 한다.
  - ex) `삼성 노트북`이라는 상품을 `노트북 삼성`이라고 검색해도 조회가 되야 한다.
- [ ] `상품 설명`에 HTML 태그가 섞여서 저장되어 있는데, 검색할 때는 HTML 태그로 검색되지 않아야 한다.

1. **대소문자 구분없이 검색할 수 있게 만들기**

   대소문자를 구분없이 검색하려면 Analyzer의 구성요소인 token filter로 `lowercase`가 적용돼야 한다. 하지만 인덱스를 만들 때 Analyzer 또는 token filter에 대해 아무 설정도 하지 않았다.

   이전 강의(📖 Elasticsearch에 기본값으로 설정되어 있는 애널라이저(Analyzer))에서 잠깐 설명했었는데, 아무런 설정을 하지 않으면 standard analyzer가 설정된다고 했다. standard analyzer는 다음과 같은 구성으로 이루어져있다.

   ```
   "char_filter": [],
   "tokenizer": "standard",
   "filter": ["lowercase"]
   ```

   즉, 아무런 설정을 하지 않아도 `lowercase`가 적용되어 있다. 정말 그런 지 테스트해보자.

   [Analyze API를 통해 디버깅하기]

   ```
   GET /products/_analyze
   {
     "field": "name",
     "text": "Samsung TV"
   }
   ```

   응답값

   ```
   {
     "tokens": [
       {
         "token": "samsung",
         "start_offset": 0,
         "end_offset": 7,
         "type": "<ALPHANUM>",
         "position": 0
       },
       {
         "token": "tv",
         "start_offset": 8,
         "end_offset": 10,
   ```

```
      "type": "<ALPHANUM>",
      "position": 1
    }
  ]
}
```

Analyzer가 나눈 토큰을 살펴보면 lowercase가 잘 적용됐음을 알 수 있다.

(정말 검색이 잘 되는 지는 맨 마지막에 한꺼번에 테스트 할 예정이다.)

2. **단어의 순서에 상관없이 검색 잘 되게 만들기**

   데이터 유형을 `text`로 설정하면 Analyer가 값을 토큰으로 나눠서 역인덱스에 저장하기 때문에 단어의 순서에 상관없이 검색이 잘 된다. 반면, `keyword` 타입은 필드 값을 토큰으로 나누지 않고 통째로 역인덱스에 저장한다. 따라서 단어의 순서가 바뀌면 다른 값으로 인식해 검색되지 않는다.

   [기존 매핑 정의]

```
PUT /products
{
  "mappings": {
    "properties": {
      "id": {
        "type": "long"
      },
      "name": {
        "type": "text" // 유연한 검색 필요
      },
      "description": {
        "type": "text" // 유연한 검색 필요
      },
      "price": {
        "type": "integer" // 10억 이하의 정수
      },
      "rating": {
        "type": "double" // 실수(소수점을 가진 숫자 포함)
      },
      "category": {
        "type": "text" // 유연한 검색 필요
      }
    }
  }
}
```

   기존에 정의한 매핑에서 type을 text로 이미 설정해뒀기 때문에 단어의 순서가 바뀌더라도 검색이 잘 될 것이다.

   (정말 검색이 잘 되는 지는 맨 마지막에 한꺼번에 테스트 할 예정이다.)

3. **HTML 태그로 검색되지 않게 만들기**

   HTML 태그로 검색되지 않게 만드려면 역인덱스에 토큰을 저장할 때 HTML 태그는 제거하고 토큰을 저장해야 한다. 그러려면 character filter로 `html_strip`을 활용해야 한다.

   기존 인덱스에 설정되어 있는 Analyzer의 구성 요소를 수정하려면 인덱스를 재생성해야 한다. 삭제하고 다시 만들자.

```
DELETE /products

PUT /products
{
  "settings": {
    "analysis": {
      // 커스텀 애널라이저 정의
      "analyzer": {
        "products_description_analyzer": {
          "char_filter": ["html_strip"],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "id": {
        "type": "long"
      },
      "name": {
        "type": "text" // 유연한 검색 필요
      },
      "description": {
        "type": "text", // 유연한 검색 필요
        "analyzer": "products_description_analyzer"
      },
      "price": {
        "type": "integer" // 10억 이하의 정수
      },
      "rating": {
        "type": "double" // 실수(소수점을 가진 숫자 포함)
      },
      "category": {
        "type": "text" // 유연한 검색 필요
      }
    }
  }
}

GET /products
```

`description` 필드에 데이터를 넣으면 정말 HTML 태그를 제거하고 토큰으로 저장하는 지 Analyze API를 활용해 체크해보자.

```
GET /products/_analyze
{
  "field": "description",
  "text": "<h1>상품 설명</h1>"
}
```

[응답값]

```
{
  "tokens": [
    {
      "token": "상품",
      "start_offset": 4,
      "end_offset": 6,
      "type": "<HANGUL>",
      "position": 0
    },
    {
      "token": "설명",
      "start_offset": 7,
      "end_offset": 9,
      "type": "<HANGUL>",
      "position": 1
    }
  ]
}
```


응답값을 살펴보면 HTML 태그를 제거하고 토큰으로 잘 저장하고 있는 걸 확인할 수 있다.

시험 삼아 다른 필드에서도 테스트해보자.

```
GET /products/_analyze
{
  "field": "name",
  "text": "<h1>상품 설명</h1>"
}
```

[응답값]


```
{
  "tokens": [
    {
      "token": "h1",
      "start_offset": 1,
      "end_offset": 3,
      "type": "<ALPHANUM>",
      "position": 0
    },
    {
      "token": "상품",
      "start_offset": 4,
      "end_offset": 6,
      "type": "<HANGUL>",
      "position": 1
    },
    {
      "token": "설명",
      "start_offset": 7,
      "end_offset": 9,
      "type": "<HANGUL>",
      "position": 2
    },
    {
```

```
      "token": "h1",
      "start_offset": 11,
      "end_offset": 13,
      "type": "<ALPHANUM>",
      "position": 3
    }
  ]
}
```

`html_strip` 필터가 적용되지 않아서 HTML 태그도 같이 토큰으로 저장됐다.

4. **요구사항을 잘 반영했는 지 테스트해보기**

   3가지 요구사항을 전부 만족시킨 채로 셋팅했다. 데이터를 넣고 검색을 했을 때도 잘 작동하는 지 체크해보자.

[데이터 삽입하기]

```
POST /products/_doc
{
  "id": 1,
  "name": "Samsung TV",
  "description": "<strong>The Best TV</strong>",
  "price": 10000,
  "rating": 5.0,
  "category": "Home"
}
```

[검색해보기]

```
// 대소문자 섞어서 검색해보기 + 단어의 순서에 상관없이 검색해보기
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "tv SAMSUNG",
      "fields": [
        "name^3",
        "description^1",
        "category^2"
      ]
    }
  }
}

// HTML 태그로 검색해보기
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "strong",
      "fields": [
        "name^3",
        "description^1",
        "category^2"
      ]
    }
```

```
  }
}
```

- 대소문자와 상관없이 검색이 잘 되는 걸 확인할 수 있다.
- 단어의 순서와 상관없이 검색이 잘 되는 걸 확인할 수 있다.
- HTML 태그로 검색했을 때 검색이 되지 않는 걸 확인할 수 있다.

@naver.com
khb3025

🎯 [실습] 한글, 영어 검색 / 동의어 기반 검색 / 필터링

✅ 요구사항 반영하기

- [ ] 한글, 영어 둘 다 검색이 잘 돼야 한다.
- [ ] `상품명`에 대해서는 아래 값을 동의어로 인식할 수 있어야 한다.

```
samsung, 삼성
apple, 애플
노트북, 랩탑, 컴퓨터, computer, laptop, notebook
전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone
아이폰, iphone
맥북, 맥, macbook, mac
```

- [ ] 카테고리(`category`), 가격(`price`)으로 필터링을 할 수 있어야 한다.
  - 특정 카테고리 내에서만 검색할 수 있어야 한다.
  - 특정 가격 범위 내에서만 검색할 수 있어야 한다.

1. **한글, 영어 둘 다 검색이 잘 되도록 만들기**

   `상품명`, `상품 설명`, `카테고리` 필드에 들어가는 한글로 이루어진 값도 적절하게 토큰을 분리하려면 기본으로 설정되는 standard Analyzer가 아닌 Nori Analyzer를 활용해야 한다. Nori Analyzer는 다음과 같이 설정할 수 있다고 했다.

   ```
   // 방법 1
   "analyzer": "nori"

   // 방법 2 (nori analyzer의 구성을 직접 명시)
   "char_filter": [],
   "tokenizer": "nori_tokenizer",
   "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
   ```

   기존 Nori Analyzer의 구성을 요구사항에 맞게 커스텀해서 사용하려면 2번째 방법을 사용해야 한다. 2번째 방법을 활용해 매핑을 다시 정의해서 인덱스를 재생성하자.

   ```
   DELETE /products

   PUT /products
   {
     "settings": {
       "analysis": {
         // 커스텀 애널라이저 정의
         "analyzer": {
           "products_name_analyzer": {
             "char_filter": [],
             "tokenizer": "nori_tokenizer",
             "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
           },
           "products_description_analyzer": {
   ```

```
        "char_filter": ["html_strip"],
        "tokenizer": "nori_tokenizer",
        "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
      },
      "products_category_analyzer": {
        "char_filter": [],
        "tokenizer": "nori_tokenizer",
        "filter": ["nori_part_of_speech", "nori_readingform", "lowercase"]
      }
    }
  }
},
"mappings": {
  "properties": {
    "id": {
      "type": "long"
    },
    "name": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_name_analyzer"
    },
    "description": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_description_analyzer"
    },
    "price": {
      "type": "integer" // 10억 이하의 정수
    },
    "rating": {
      "type": "double" // 실수(소수점을 가진 숫자 포함)
    },
    "category": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_category_analyzer"
    }
  }
 }
}
```

잘 작동하는 지 Analyze API로 테스트해보자.

```
GET /products/_analyze
{
  "field": "name",
  "text": "특수가전제품 Samsung TV"
}

GET /products/_analyze
{
  "field": "description",
  "text": "<h1>특수가전제품 Samsung TV</h1>"
}

GET /products/_analyze
{
  "field": "category",
```

```
  "text": "특수가전제품 Samsung TV"
}
```

2. **동의어로 검색할 수 있게 만들기**

```
DELETE /products

PUT /products
{
  "settings": {
    "analysis": {
          // 필터 정의
          "filter": {
        "product_synonyms": {
          "type": "synonym",
          "synonyms": [
            "samsung, 삼성",
            "apple, 애플",
            "노트북, 랩탑, 컴퓨터, computer, laptop, notebook",
            "전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone",
            "아이폰, iphone",
            "맥북, 맥, macbook, mac"
    ]
   }
  },

// 커스텀 애널라이저 정의
"analyzer": {
  "products_name_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase",
      "product_synonyms"
    ]
  },
  "products_description_analyzer": {
    "char_filter": ["html_strip"],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase"
    ]
  },
  "products_category_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase"
    ]
  }
```

```
    }
   }
  },
  "mappings": {
   "properties": {
    "id": {
     "type": "long"
    },
    "name": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_name_analyzer"
    },
    "description": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_description_analyzer"
    },
    "price": {
     "type": "integer" // 10억 이하의 정수
    },
    "rating": {
     "type": "double" // 실수(소수점을 가진 숫자 포함)
    },
    "category": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_category_analyzer"
    }
   }
  }
 }
```

잘 작동하는 지 Analyze API로 테스트해보자.

```
GET /products/_analyze
{
  "field": "name",
  "text": "삼성 애플 노트북 전화기 아이폰 맥북"
}
```

3. **가격으로 필터링 할 수 있게 만들기**

[기존 검색 쿼리]

```
POST /products/_search
{
  "query": {
    "multi_match": {
      "query": "lg",
      "fields": [
        "name^3",
        "description^1",
        "category^2"
      ]
    }
  }
}
```

기존 검색 쿼리에서 가격(price)로 필터링 하려면 아래와 같이 쿼리를 수정하면 된다.

```
POST /products/_search
{
  "query": {
    "bool": { // 2가지 이상의 조건을 적용시켜야 할 때
      "must": { // score와 관련 있는 쿼리 (유연한 검색 필요)
        "multi_match": {
          "query": "lg",
          "fields": [
            "name^3",
            "description^1",
            "category^2"
          ]
        }
      },
      "filter": { // score와 관련 없는 쿼리 (정확한 값 비교 필요)
        "range": {
          "price": {
            "gte": 10000,
            "lte": 50000
          }
        }
      }
    }
  }
}
```

4. **카테고리로 필터링하기**

   `category`를 기준으로 필터링하려면 타입이 `keyword` 타입이어야 한다. 왜냐하면 `category`를 기준으로 필터링하려면 정확히 일치하는 값으로 비교해서 필터링을 해야 하기 때문이다.

   하지만 `category` 필드는 관련된 상품을 유연하게 검색할 때도 사용하기 때문에 `text` 타입으로도 활용할 수 있어야 한다.

   이러한 특징 때문에 `category` 필드에 `text`와 `keyword` 타입을 동시에 적용시켜야 한다.

```
DELETE /products

PUT /products
{
 "settings": {
  "analysis": {
      // 필터 정의
      "filter": {
     "product_synonyms": {
      "type": "synonym",
      "synonyms": [
       "samsung, 삼성",
                "apple, 애플",
                "노트북, 랩탑, 컴퓨터, computer, laptop, notebook",
                "전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone",
                "아이폰, iphone",
                "맥북, 맥, macbook, mac"
      ]
     }
    },
```

```
// 커스텀 애널라이저 정의
"analyzer": {
  "products_name_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase",
      "product_synonyms"
    ]
  },
  "products_description_analyzer": {
    "char_filter": ["html_strip"],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase"
    ]
  },
  "products_category_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase"
    ]
  }
 }
},
"mappings": {
  "properties": {
    "id": {
      "type": "long"
    },
    "name": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_name_analyzer"
    },
    "description": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_description_analyzer"
    },
    "price": {
      "type": "integer" // 10억 이하의 정수
    },
    "rating": {
      "type": "double" // 실수(소수점을 가진 숫자 포함)
    },
    "category": {
      "type": "text", // 유연한 검색 필요
      "analyzer": "products_category_analyzer",
      // 멀티 필드로 keyword 타입을 추가
      "fields": {
        "raw": {
```

```
        "type": "keyword"
       }
      }
     }
    }
   }
}
```

테스트를 하기 위해 더미 데이터를 먼저 넣자.

```
POST /products/_doc
{
  "id": 1,
  "name": "LG OLED TV 42인치",
  "description": "<p>선명한 화질의 LG OLED TV입니다.</p>",
  "price": 3000,
  "rating": 4.8,
  "category": "가전제품"
}

POST /products/_doc
{
  "id": 2,
  "name": "LG 냉장고 300L",
  "description": "<p>신선함을 오래 유지하는 LG 냉장고</p>",
  "price": 49000,
  "rating": 4.5,
  "category": "가전제품"
}

POST /products/_doc
{
  "id": 3,
  "name": "삼성 냉장고",
  "description": "<p>삼성의 최신 기술이 적용된 냉장고</p>",
  "price": 20000,
  "rating": 4.7,
  "category": "가전제품"
}

POST /products/_doc
{
  "id": 4,
  "name": "LG 스마트폰",
  "description": "<p>LG의 최신 스마트폰 모델</p>",
  "price": 25000,
  "rating": 4.2,
  "category": "휴대전화"
}
```

필터링 기능이 잘 작동하는 지 확인하기 위해 아래 쿼리를 실행시켜보자.

```
POST /products/_search
{
  "query": {
```

```
    "bool": {
      "must": {
        "multi_match": {
          "query": "LG",
          "fields": [
            "name^3",
            "description^1",
            "category^2"
          ]
        }
      },
      "filter": [
        {
          "term": {
            "category.raw": "가전제품"
          }
        },
        {
          "range": {
            "price": {
              "gte": 10000,
              "lte": 50000
            }
          }
        }
      ]
    }
  }
}
```

khb302501@gmail.com

🎯 [실습] 특정 조건 상위 노출 / 오타 허용 검색 / 하이라이팅 처리 / 페이지네이션

✅ 요구사항 반영하기

- [ ] 되도록이면 검색 결과 중 평점이 4.0이 넘는 상품을 상위에 노출시킬 수 있도록 한다.
- [ ] 검색어에 어느 정도 오타가 있더라도 유사한 상품을 조회할 수 있어야 한다.
  - ex) `samsung notebook`이라는 상품을 `simsung notebook`이라고 검색해도 조회가 돼야 한다.
- [ ] `상품명`에서 일치하는 검색 키워드는 하이라이팅 처리(`<b></b>`)를 해야 한다.
- [ ] 상품이 5개씩 조회되도록 페이지네이션 해야 한다.

1. **되도록이면 평점이 4.0이 넘는 상품을 상위 노출 시키기**

   이 기능을 구현하기 위해서는 bool 쿼리의 `should`를 활용하면 된다.

```
GET /products/_search
{
  "query": {
    "bool": {
      "must": {
        "multi_match": {
          "query": "LG 냉장고",
          "fields": [
            "name^3",
            "description^1",
            "category^2"
          ]
        }
      },
      "filter": [
        {
          "term": {
            "category.raw": "가전제품"
          }
        },
        {
          "range": {
            "price": {
              "gte": 10000,
              "lte": 50000
            }
          }
        }
      ],
      "should": [
        {
          "range": {
            "rating": {
```

```
          "gt": 4.0
        }
       }
      }
     ]
    }
  }
}
```

2. **어느 정도 오타가 있더라도 검색되게 만들기**

   어느 정도 오타가 있더라도 검색이 되게 만드려면 fuziness를 활용하면 된다.

```
GET /products/_search
{
  "query": {
    "bool": {
"must": {
  "multi_match": {
    "query": "마전제품",
    "fields": [
      "name^3",
      "description^1",
      "category^2"
    ],
    "fuzziness": "AUTO"
  }
},
"filter": [
  {
    "term": {
      "category.raw": "가전제품"
    }
  },
  {
    "range": {
      "price": {
        "gte": 10000,
        "lte": 50000
      }
     }
    }
   ],
   "should": [
    {
     "range": {
      "rating": {
       "gt": 4.0
      }
     }
    }
   ]
  }
 }
}
```

3. `상품명`에서 일치하는 검색 키워드는 하이라이팅 처리(`<b></b>`) 하기

```
GET /products/_search
{
  "query": {
    "bool": {
      "must": {
        "multi_match": {
          "query": "삼성",
          "fields": [
            "name^3",
            "description^1",
            "category^2"
          ],
          "fuzziness": "AUTO"
        }
      },
      "filter": [
        {
          "term": {
            "category.raw": "가전제품"
          }
        },
        {
          "range": {
            "price": {
              "gte": 10000,
              "lte": 50000
            }
          }
        }
      ],
      "should": [
        {
          "range": {
            "rating": {
              "gt": 4.0
            }
          }
        }
      ]
    }
  },
  "highlight": {
    "fields": {
      "name": {
        "pre_tags": ["<b>"],
        "post_tags": ["</b>"]
      }
    }
  }
}
```

4. **상품이 5개씩 조회되도록 페이지네이션 처리하기**

```
GET /products/_search
{
  "query": {
    "bool": {
      "must": {
        "multi_match": {
          "query": "삼성",
          "fields": [
            "name^3",
            "description^1",
            "category^2"
          ],
          "fuzziness": "AUTO"
        }
      },
      "filter": [
        {
          "term": {
            "category.raw": "가전제품"
          }
        },
        {
          "range": {
            "price": {
              "gte": 10000,
              "lte": 50000
            }
          }
        }
      ],
      "should": [
        {
          "range": {
            "rating": {
              "gt": 4.0
            }
          }
        }
      ]
    },
    "highlight": {
      "fields": {
        "name": {
          "pre_tags": ["<b>"],
          "post_tags": ["</b>"]
        }
      }
    },
    "from": 0, // 몇 번째 데이터부터 조회할 건지 = (페이지 번호 - 1) x size
    "size": 5 // 페이지당 몇 개의 데이터를 조회할 지
}
```

`검색 기능`의 요구 사항을 전부 만족시켰다. 그럼 다음 기능인 `자동 완성 기능`의 요구사항을 만족시켜보자.

@gmail.com
khb302

🎯 [실습] 자동 완성 기능

✅ 요구사항 반영하기

- [ ] `상품명`으로만 자동 완성이 되도록 해야 한다.
- [ ] 자동 완성 결과가 5개만 조회되도록 설정해야 한다.

1. **자동 완성 기능 구현하기**

   자동 완성 기능을 구현하기 위해 `search_as_you_type`을 활용하자.

```
DELETE /products

PUT /products
{
  "settings": {
    "analysis": {
        // 필터 정의
        "filter": {
       "product_synonyms": {
        "type": "synonym",
        "synonyms": [
          "samsung, 삼성",
                "apple, 애플",
                "노트북, 랩탑, 컴퓨터, computer, laptop, notebook",
                "전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone",
                "아이폰, iphone",
                "맥북, 맥, macbook, mac"
        ]
       }
     },

     // 커스텀 애널라이저 정의
    "analyzer": {
     "products_name_analyzer": {
      "char_filter": [],
      "tokenizer": "nori_tokenizer",
      "filter": [
       "nori_part_of_speech",
       "nori_readingform",
       "lowercase",
       "product_synonyms"
      ]
     },
     "products_description_analyzer": {
      "char_filter": ["html_strip"],
      "tokenizer": "nori_tokenizer",
      "filter": [
       "nori_part_of_speech",
       "nori_readingform",
```

```
        "lowercase"
       ]
      },
      "products_category_analyzer": {
       "char_filter": [],
       "tokenizer": "nori_tokenizer",
       "filter": [
        "nori_part_of_speech",
        "nori_readingform",
        "lowercase"
       ]
      }
    }
  }
},
"mappings": {
  "properties": {
    "id": {
  "type": "long"
},
"name": {
  "type": "text", // 유연한 검색 필요
  "analyzer": "products_name_analyzer",
  // 멀티 필드로 search_as_you_type 타입을 추가
  "fields": {
    "auto_complete": {
      "type": "search_as_you_type",
      "analyzer": "nori"
    }
  }
},
"description": {
  "type": "text", // 유연한 검색 필요
  "analyzer": "products_description_analyzer"
},
"price": {
  "type": "integer" // 10억 이하의 정수
},
"rating": {
  "type": "double" // 실수(소수점을 가진 숫자 포함)
      },
      "category": {
       "type": "text", // 유연한 검색 필요
       "analyzer": "products_category_analyzer",
       // 멀티 필드로 keyword 타입을 추가
       "fields": {
        "raw": {
         "type": "keyword"
        }
       }
      }
    }
  }
}
```

자동 완성 기능이 잘 구현됐는 지 테스트하기 위해 더미 데이터부터 넣자.

```
POST /products/_doc
{
  "id": 1,
  "name": "삼성 스마트 TV 65인치",
  "description": "<p>선명한 화질과 뛰어난 스마트 기능을 갖춘 삼성 TV입니다.</p>",
  "price": 4500,
  "rating": 4.6,
  "category": "가전제품"
}

POST /products/_doc
{
  "id": 2,
  "name": "삼성 스마트 냉장고",
  "description": "<p>스마트 기능과 대용량 저장공간을 제공하는 삼성 냉장고입니다.</p>",
  "price": 5000,
  "rating": 4.7,
  "category": "가전제품"
}
```

자동 완성 기능이 잘 작동하는 지 아래 쿼리를 실행시켜보자.

```
GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "삼", // 삼, 삼성, 삼성 스
      "type": "bool_prefix",
      "fields": [
        "name.auto_complete",
        "name.auto_complete._2gram",
        "name.auto_complete._3gram"
      ]
    }
  }
}

GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "스", // 스, 스마, 스마트
      "type": "bool_prefix",
      "fields": [
        "name.auto_complete",
        "name.auto_complete._2gram",
        "name.auto_complete._3gram"
      ]
    }
  }
}
```

그리고 결과가 5개까지만 조회되도록 만들자.

```
POST /products/_search
{
```

```
  "query": {
    "multi_match": {
      "query": "삼", // 삼, 삼성, 삼성 스
      "type": "bool_prefix",
      "fields": [
        "name.auto_complete",
        "name.auto_complete._2gram",
        "name.auto_complete._3gram"
      ]
    }
  },
  "size": 5
}
```


요구 사항에 있는 모든 사항을 다 반영시켰다. 다음 강의에서 지금까지 작성한 쿼리를 정리해보자.

@kfc1gg@gmail.com
khb302500

📖

최종 쿼리 / 적용 전략 및 순서

✅ 최종 쿼리

[생성해야 할 인덱스 및 매핑]

```
DELETE /products

PUT /products
{
  "settings": {
    "analysis": {
          // 필터 정의
          "filter": {
  "product_synonyms": {
    "type": "synonym",
    "synonyms": [
      "samsung, 삼성",
                "apple, 애플",
                "노트북, 랩탑, 컴퓨터, computer, laptop, notebook",
                "전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone",
                "아이폰, iphone",
                "맥북, 맥, macbook, mac"
    ]
  }
},

// 커스텀 애널라이저 정의
"analyzer": {
  "products_name_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase",
      "product_synonyms"
    ]
  },
  "products_description_analyzer": {
    "char_filter": ["html_strip"],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
      "lowercase"
    ]
  },
  "products_category_analyzer": {
    "char_filter": [],
    "tokenizer": "nori_tokenizer",
    "filter": [
      "nori_part_of_speech",
      "nori_readingform",
```

```
        "lowercase"
       ]
      }
     }
    }
  },
  "mappings": {
   "properties": {
    "id": {
     "type": "long"
    },
    "name": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_name_analyzer",
     // 멀티 필드로 search_as_you_type 타입을 추가
     "fields": {
      "auto_complete": {
       "type": "search_as_you_type",
       "analyzer": "nori"
      }
     }
    },
    "description": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_description_analyzer"
    },
    "price": {
     "type": "integer" // 10억 이하의 정수
    },
    "rating": {
     "type": "double" // 실수(소수점을 가진 숫자 포함)
    },
    "category": {
     "type": "text", // 유연한 검색 필요
     "analyzer": "products_category_analyzer",
     // 멀티 필드로 keyword 타입을 추가
     "fields": {
      "raw": {
       "type": "keyword"
      }
     }
    }
   }
  }
```

[검색 기능]

```
GET /products/_search
{
  "query": {
    "bool": {
      "must": {
        "multi_match": {
          "query": "_____",
          "fields": [
```

```
        "name^3",
        "description^1",
        "category^2"
      ],
      "fuzziness": "AUTO"
    }
  },
  "filter": [
    {
      "term": {
        "category.raw": "_____"
      }
    },
    {
      "range": {
        "price": {
          "gte": ____,
          "lte": ____
        }
      }
    }
  ],
  "should": [
    {
      "range": {
        "rating": {
          "gt": 4.0
        }
      }
    }
  ]
 }
},
"highlight": {
 "fields": {
  "name": {
   "pre_tags": ["<b>"],
   "post_tags": ["</b>"]
  }
 },
 "from": 0,
 "size": 5
}
```

[자동 완성 기능]

```
POST /products/_search
{
 "query": {
  "multi_match": {
   "query": "_______",
   "type": "bool_prefix",
   "fields": [
    "name.auto_complete",
    "name.auto_complete._2gram",
```

```
        "name.auto_complete._3gram"
      ]
    }
  },
  "size": 5
}
```

✅ 적용 전략 및 순서

1. ~~Kibana에서 요구사항에 맞는 쿼리 작성하기 (완료!)~~
2. Spring Boot에서 `products` 인덱스에 맞는 Document 정의하기
3. Spring Data Elasticsearch 라이브러리에 맞게 쿼리 작성하기
4. 더미 데이터 넣고 Spring Boot에서 잘 작동하는 지 테스트하기

다음 강의에서는 Spring Boot에서 코드를 마저 작성해보자.

kb302500@gmail.com

[실습] Spring Boot에서 인덱스에 맞게 Document 정의하기

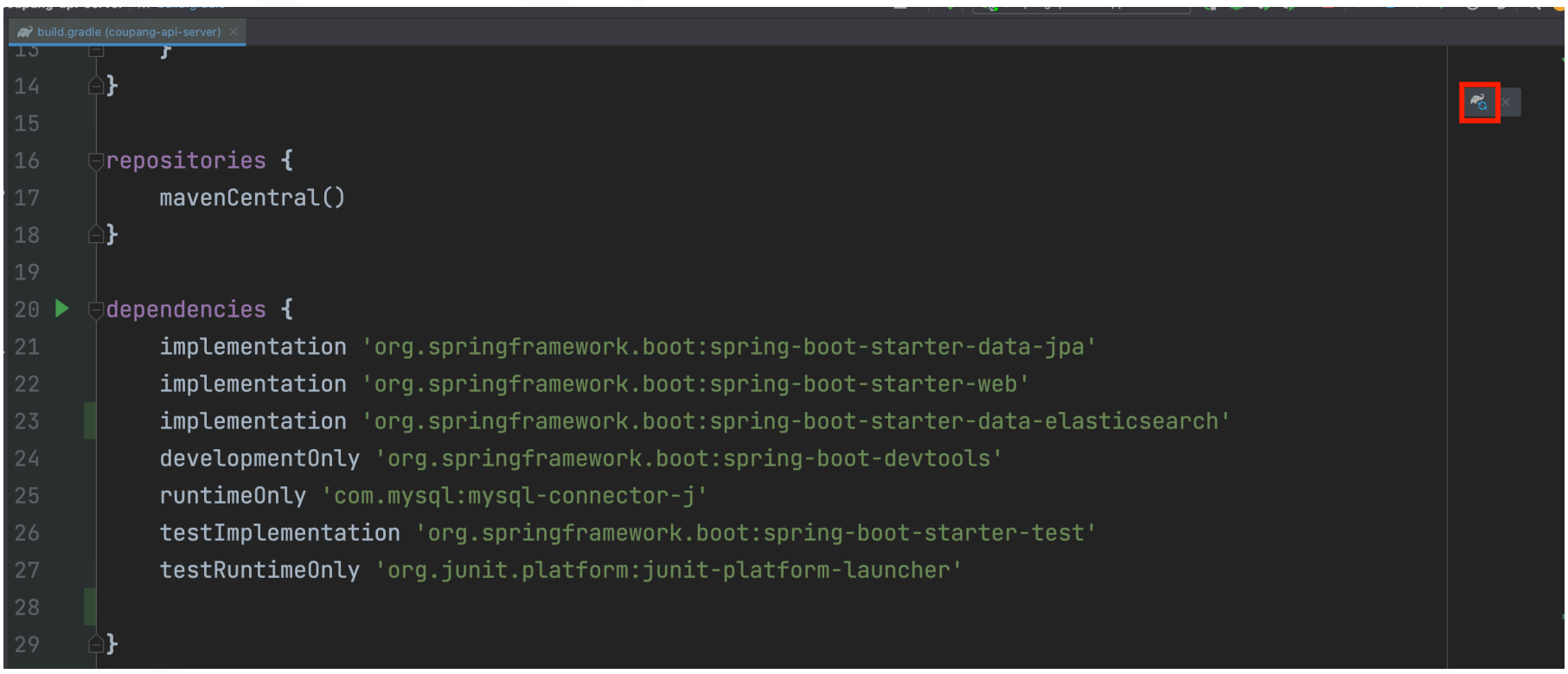
✅ Spring Boot에서 인덱스에 맞게 Document 정의하기

> Spring Data JPA에서 DB의 테이블에 맞게 `Entity`를 작성하는 것처럼, Spring Data Elasticsearch에서 인덱스에 맞게 `Document`를 작성해야 한다.

1. **Elasticsearch 의존성 설치**

   build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-elasticsearch'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.mysql:mysql-connector-j'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```



3. **application.yml에 Elasticsearch 연결을 위한 정보 작성하기**

   application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/coupang
```

```
    username: root
    password: password123
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  elasticsearch:
    uris: http://localhost:9200

logging:
  level:
    org.elasticsearch.client: TRACE
```

4. **Document 생성하기**

   Document 객체에 products 인덱스의 매핑(mapping) 정보를 반영해보자.

ProductDocument

```
@Document(indexName = "products") // products 인덱스 생성
@Setting(settingPath = "/elasticsearch/product-settings.json")
public class ProductDocument {
  @Id
  private String id;

  @MultiField(mainField = @Field(type = FieldType.Text, analyzer = "products_name_analyzer"),
      otherFields = {
          @InnerField(suffix = "auto_complete", type = FieldType.Search_As_You_Type, analyzer = "nori")
      }
  )
  private String name;

  @Field(type = FieldType.Text, analyzer = "products_description_analyzer")
  private String description;

  @Field(type = FieldType.Integer)
  private Integer price;

  @Field(type = FieldType.Double)
  private Double rating;

  @MultiField(mainField = @Field(type = FieldType.Text, analyzer = "products_category_analyzer"),
      otherFields = {
          @InnerField(suffix = "raw", type = FieldType.Keyword)
      }
  )
  private String category;

  public ProductDocument(Long id, String name, String description, Integer price, Double rating,
      String category) {
    this.id = id;
    this.name = name;
    this.description = description;
    this.price = price;
    this.rating = rating;
    this.category = category;
  }
```

```
  // getter, setter 메서드
}
```

4. **settings 파일 만들어주기**

   json 형식의 파일로 만들어야 하기 때문에 주석을 전부 지운 채로 작성해야 한다. 그리고 앞쪽에 `"settings"` 필드 안의 값만 작성해야 한다. `"mappings"`에 대한 정보조차 적지 않는다.
   (
   `"mappings"` 정보는 Document 객체에 기입하기 때문에 적을 필요가 없다.)

   src/main/resources/elasticsearch/product-settings.json

```
{
  "analysis": {
     "filter": {
     "product_synonyms": {
       "type": "synonym",
     "synonyms": [
      "samsung, 삼성",
            "apple, 애플",
            "노트북, 랩탑, 컴퓨터, computer, laptop, notebook",
            "전화기, 휴대폰, 핸드폰, 스마트폰, 휴대전화, phone, smartphone, mobile phone, cell phone",
            "아이폰, iphone",
            "맥북, 맥, macbook, mac"
     ]
   }
 },
 "analyzer": {
   "products_name_analyzer": {
     "char_filter": [],
     "tokenizer": "nori_tokenizer",
     "filter": [
       "nori_part_of_speech",
       "nori_readingform",
       "lowercase",
       "product_synonyms"
     ]
   },
   "products_description_analyzer": {
     "char_filter": ["html_strip"],
     "tokenizer": "nori_tokenizer",
     "filter": [
       "nori_part_of_speech",
       "nori_readingform",
       "lowercase"
     ]
   },
   "products_category_analyzer": {
     "char_filter": [],
     "tokenizer": "nori_tokenizer",
     "filter": [
       "nori_part_of_speech",
       "nori_readingform",
       "lowercase"
     ]
   }
 }
```

```
  }
}
```

5. **Elasticsearch Repository 생성하기**

   ProductDocumentRepository

```
public interface ProductDocumentRepository extends ElasticsearchRepository<ProductDocument, String> {
}
```

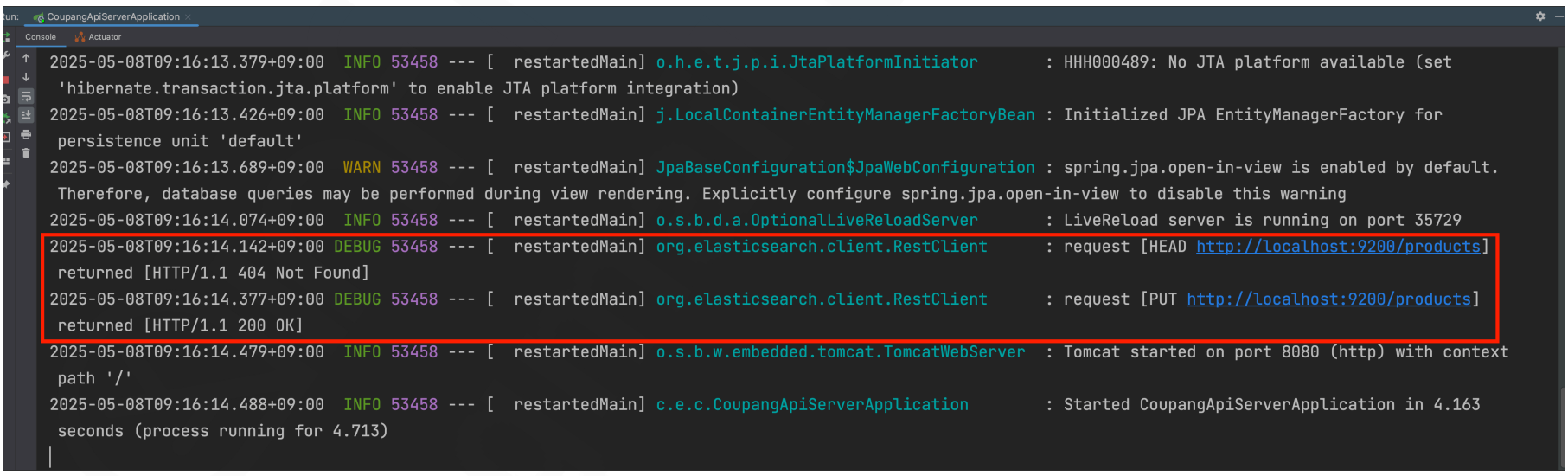
   참고) 최소 1개 이상의 ElasticsearchRepository가 존재해야, Spring Boot를 실행시킬 때 Elasticsearch와 연결을 시도한다.

6. **기존에 생성해둔 products 인덱스 삭제하기**

   Kibana에서 아래 명령어를 실행시켜 인덱스를 삭제하자.

```
DELETE /products // 인덱스 삭제

GET /products // 잘 삭제됐는 지 확인
```

7. **Spring Boot 프로젝트 실행시키기**



   위 로그가 찍혀있다면 Document 객체에 정의한 내용대로 index가 잘 생성됐다는 걸 알 수 있다.

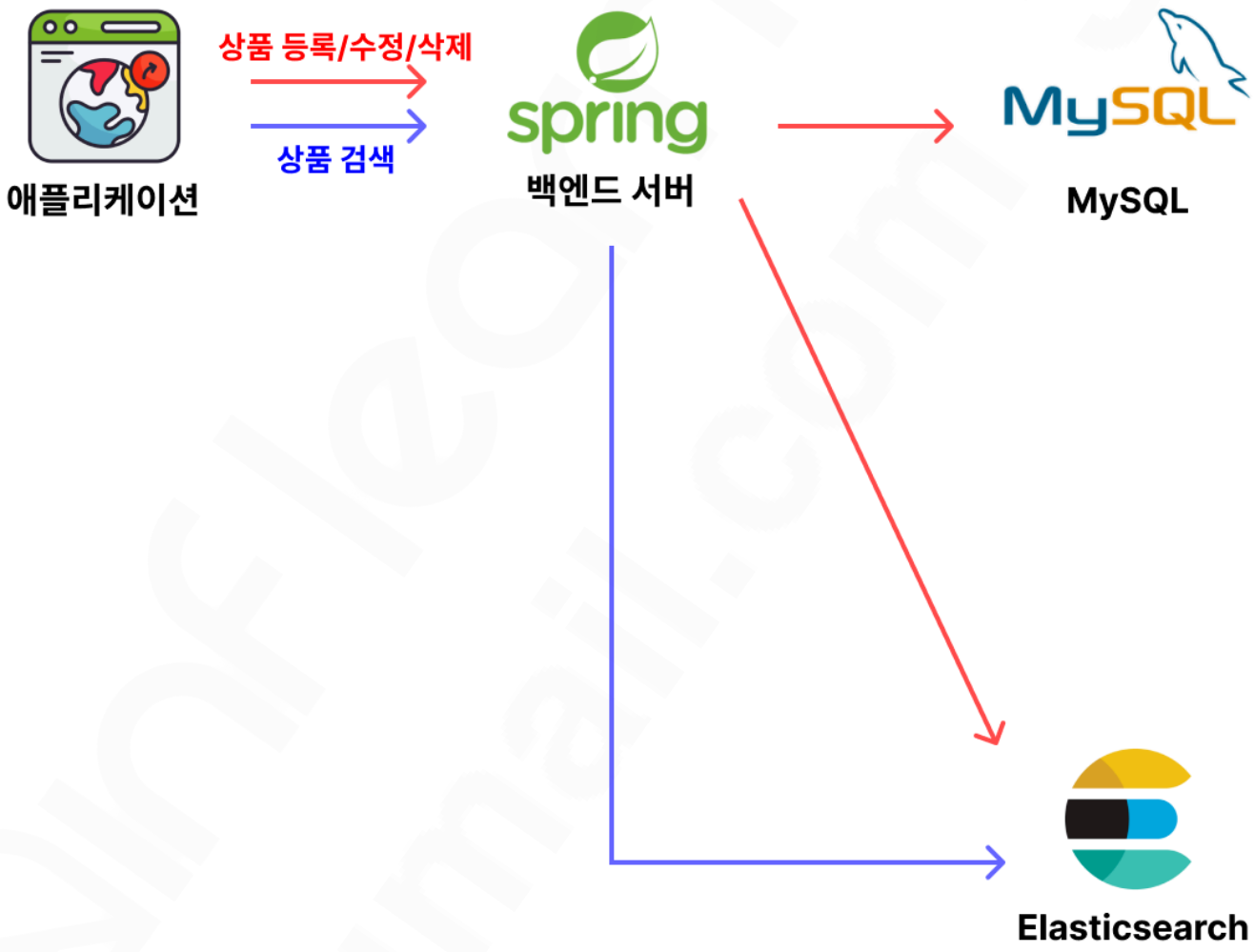
8. **index 잘 생성됐는 지 kibana로 확인하기**

```
GET /products
```

[실습] MySQL에 데이터 삽입/삭제 시 Elasticsearch에도 같이 반영되게 만들기

✅ MySQL에 데이터 삽입/삭제 시 Elasticsearch에도 같이 반영되게 만들기

이전 강의에서 설계했던 인프라 아키텍처에 맞게 작동하도록 만들어보자.



1. **ProductDocumentRepository 의존성 추가**

   ProductService

```
@Service
public class ProductService {

 private final ProductRepository productRepository;
 private final ProductDocumentRepository productDocumentRepository;

 public ProductService(ProductRepository productRepository,
    ProductDocumentRepository productDocumentRepository) {
  this.productRepository = productRepository;
  this.productDocumentRepository = productDocumentRepository;
 }
```

2. **상품 등록 로직 수정하기**

   ProductService

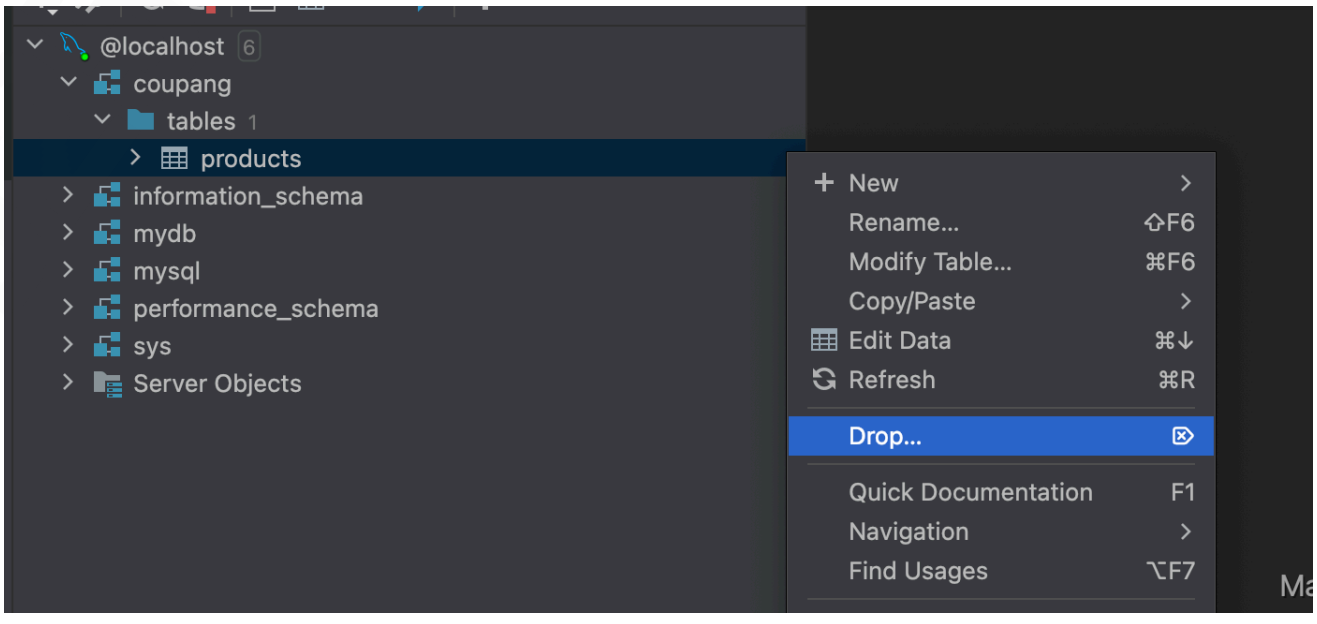
```
public Product createProduct(CreateProductRequestDto createProductRequestDto) {
    Product product = new Product(
        createProductRequestDto.getName(),
        createProductRequestDto.getDescription(),
        createProductRequestDto.getPrice(),
        createProductRequestDto.getRating(),
        createProductRequestDto.getCategory()
    );
    Product savedProduct = productRepository.save(product);

    ProductDocument productDocument = new ProductDocument(
        savedProduct.getId().toString(),
        savedProduct.getName(),
        savedProduct.getDescription(),
        savedProduct.getPrice(),
        savedProduct.getRating(),
        savedProduct.getCategory()
    );

    productDocumentRepository.save(productDocument);

    return savedProduct;
}
```

3. **상품 삭제 로직 수정하기**

```
 public void deleteProduct(Long id) {
    productRepository.deleteById(id);

    productDocumentRepository.deleteById(id.toString());
}
```

✅ 잘 작동하는 지 테스트해보기

1. **기존 MySQL 테이블 삭제하기**



2. **Kibana에서 Elasticsearch 인덱스 삭제하기**

```
DELETE /products
```

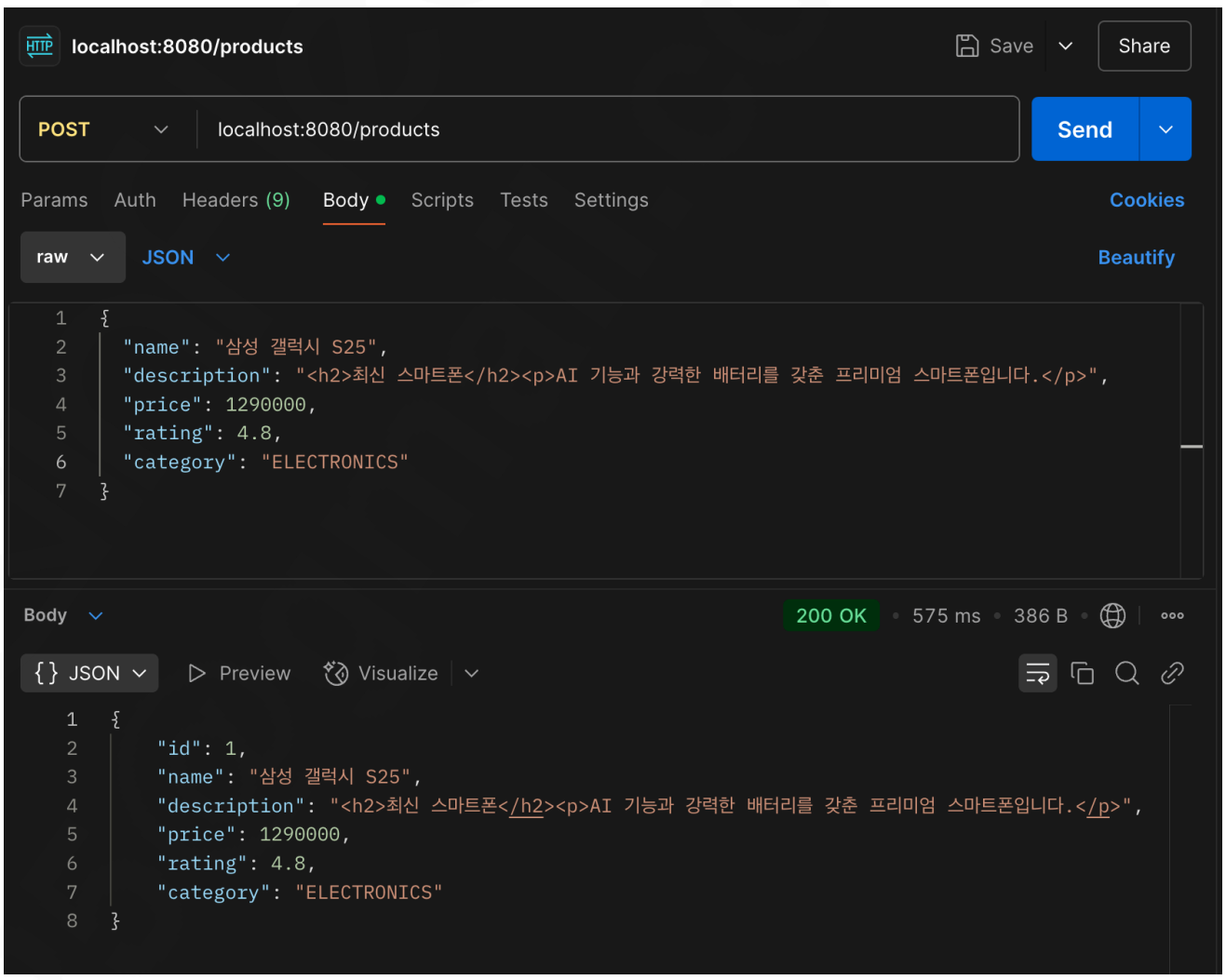
3. **Spring Boot 서버 실행시키기**

   Spring Data JPA와 Spring Data Elasticsearch에 의해 Spring Boot 프로젝트에서 정의한 객체에 맞게 MySQL에 테이블이 생성되고, Elasticsearch에 인덱스가 생성됐을 것이다.

4. **Postman으로 데이터 넣어보기**

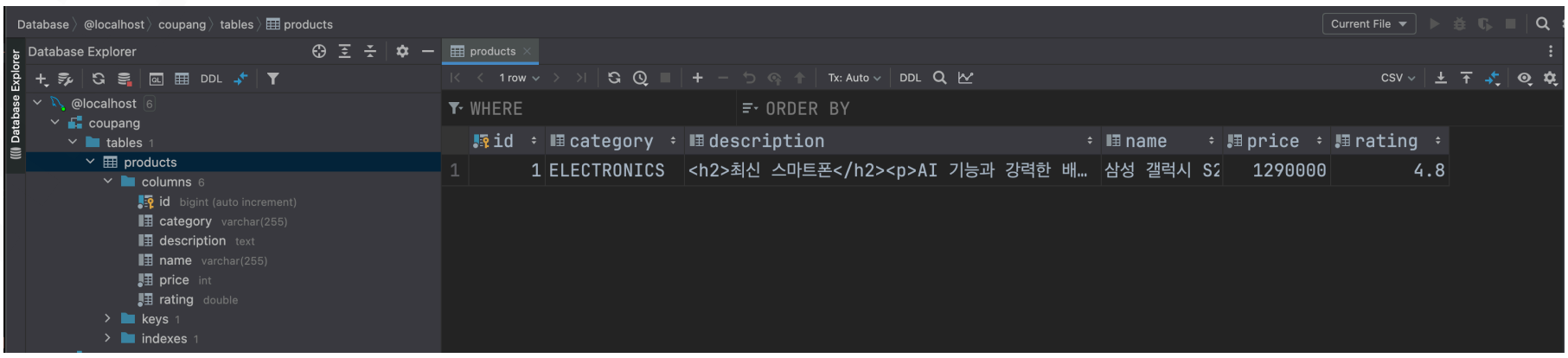
   localhost:8080/products

```
{
  "name": "삼성 갤럭시 S25",
  "description": "<h2>최신 스마트폰</h2><p>AI 기능과 강력한 배터리를 갖춘 프리미엄 스마트폰입니다.</p>",
  "price": 1290000,
  "rating": 4.8,
  "category": "ELECTRONICS"
}
```

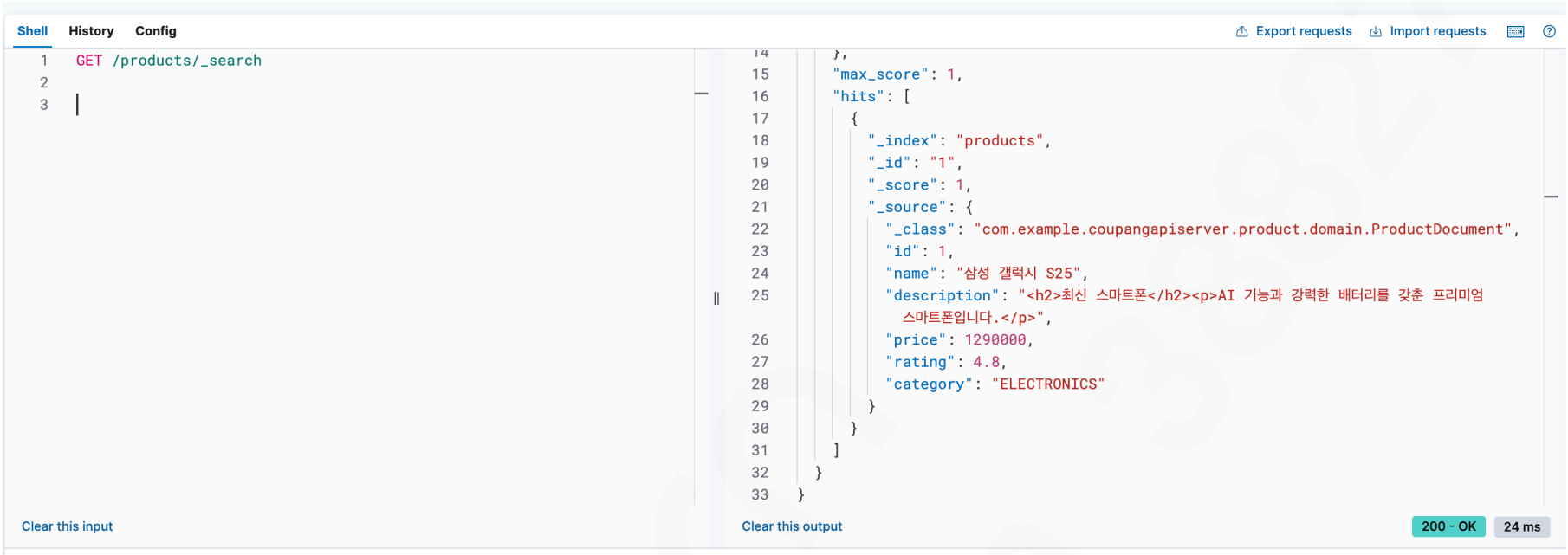


5. **데이터 잘 들어갔는 지 확인하기**

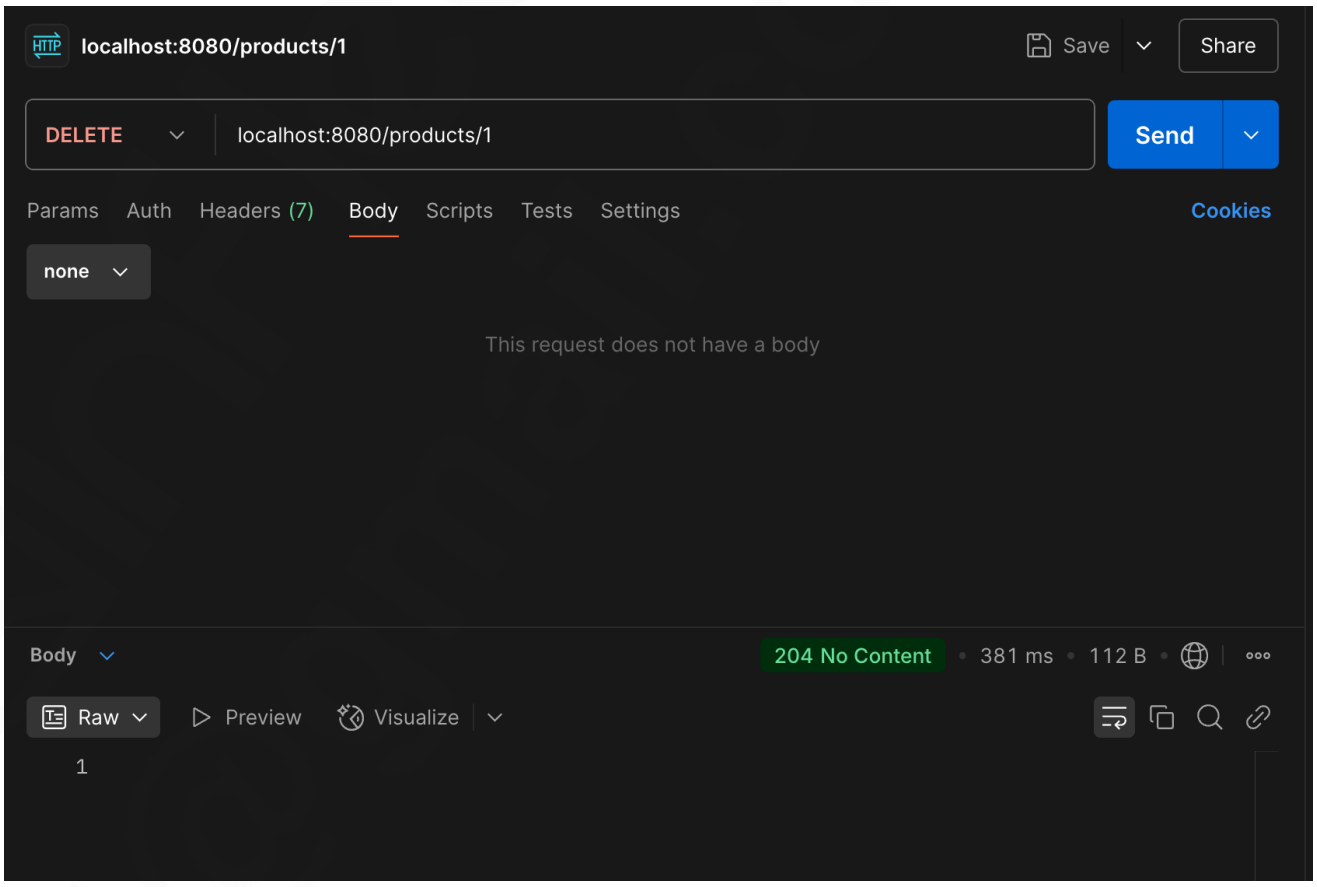
   MySQL (Datagrip)



Elasticsearch (Kibana)

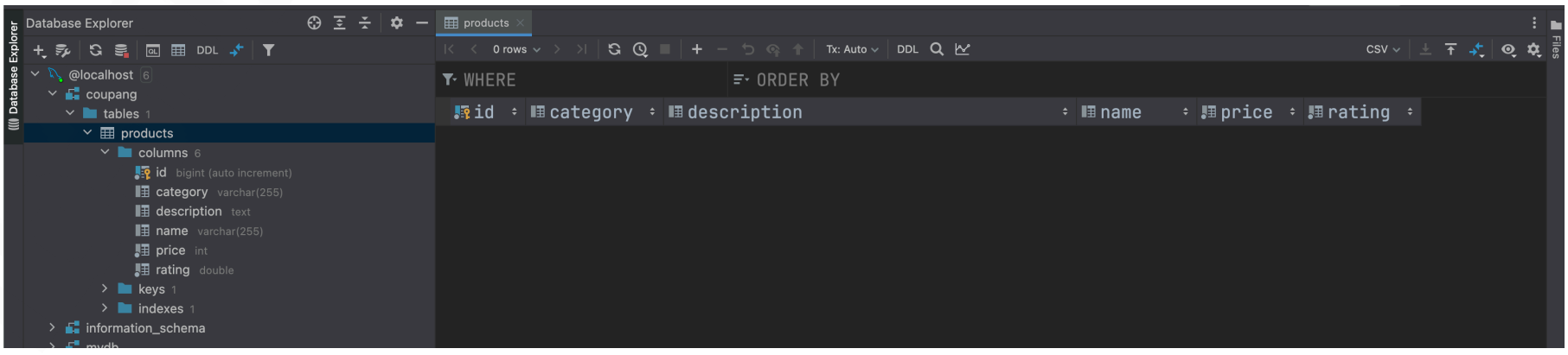


7. **Postman으로 데이터 삭제해보기**

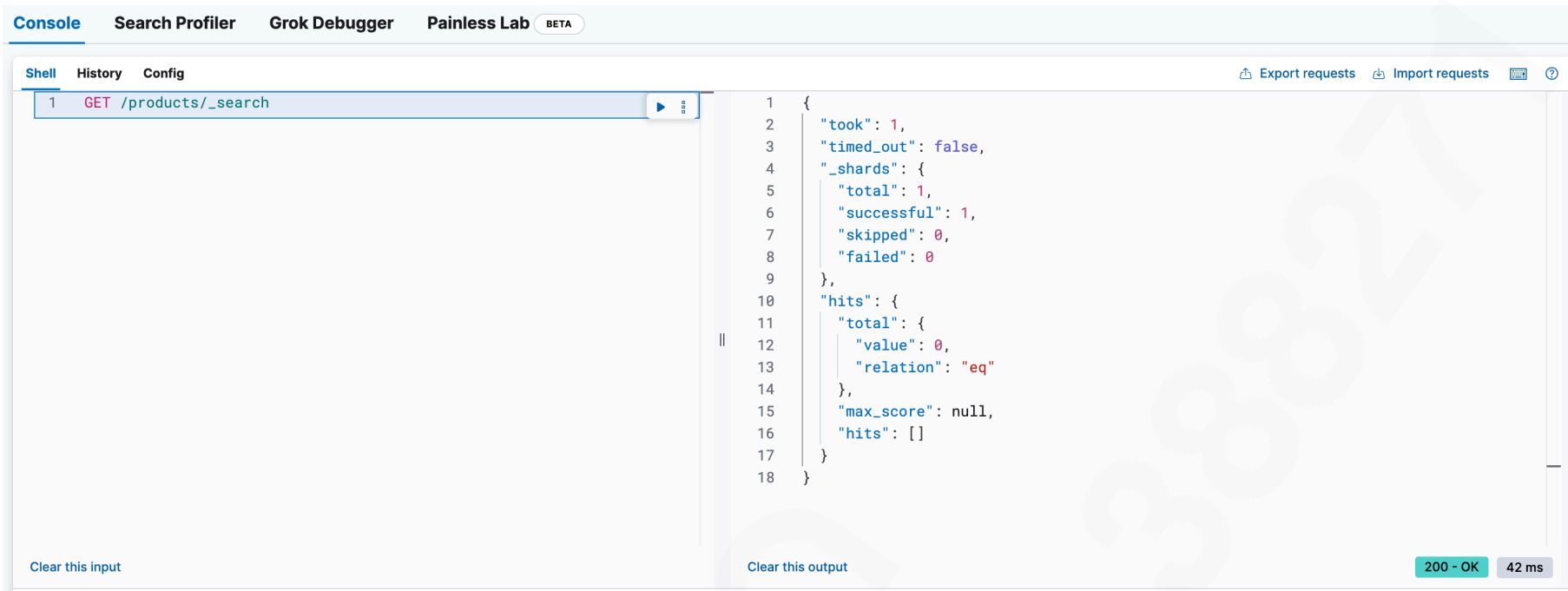


7. **데이터 잘 삭제됐는 지 확인하기**

MySQL (Datagrip)



Elasticsearch (Kibana)



Spring Boot에 Elasticsearch에 대한 코드를 추가해줌으로써 데이터 동기화 문제를 해결했다. 데이터를 삽입하면 MySQL, Elasticsearch 둘 다 동시에 삽입이 되고, 데이터를 삭제하면 MySQL, Elasticsaerch 둘 다 동시에 삭제가 된다.

그럼 다음 강의에서는 본격적으로 자동 완성 API와 검색 API를 만들어보자.

khb3025@gmail.com

🎯 [실습] Spring Boot에서 Elasticsearch 활용해 자동완성 API 만들기

✅ Spring Boot에서 Elasticsearch 활용해 자동완성 API 만들기

1. **Controller에 API 추가하기**

   ProductController

```
@RestController
@RequestMapping("products")
public class ProductController {
        ...
    @GetMapping("/suggestions")
   public ResponseEntity<List<String>> getSuggestions(@RequestParam String query) {
      List<String> suggestions = productService.getSuggestions(query);
      return ResponseEntity.ok(suggestions);
   }

        ...
}
```

2. **Service에 의존성 추가하기**

   ProductService

```
@Service
public class ProductService {

  private final ProductRepository productRepository;
  private final ProductDocumentRepository productDocumentRepository;
  private final ElasticsearchOperations elasticsearchOperations;

  public ProductService(ProductRepository productRepository,
      ProductDocumentRepository productDocumentRepository,
      ElasticsearchOperations elasticsearchOperations) {
    this.productRepository = productRepository;
    this.productDocumentRepository = productDocumentRepository;
    this.elasticsearchOperations = elasticsearchOperations;
  }

  ...
}
```

3. **Service에 메서드 만들고 로직 추가하기**

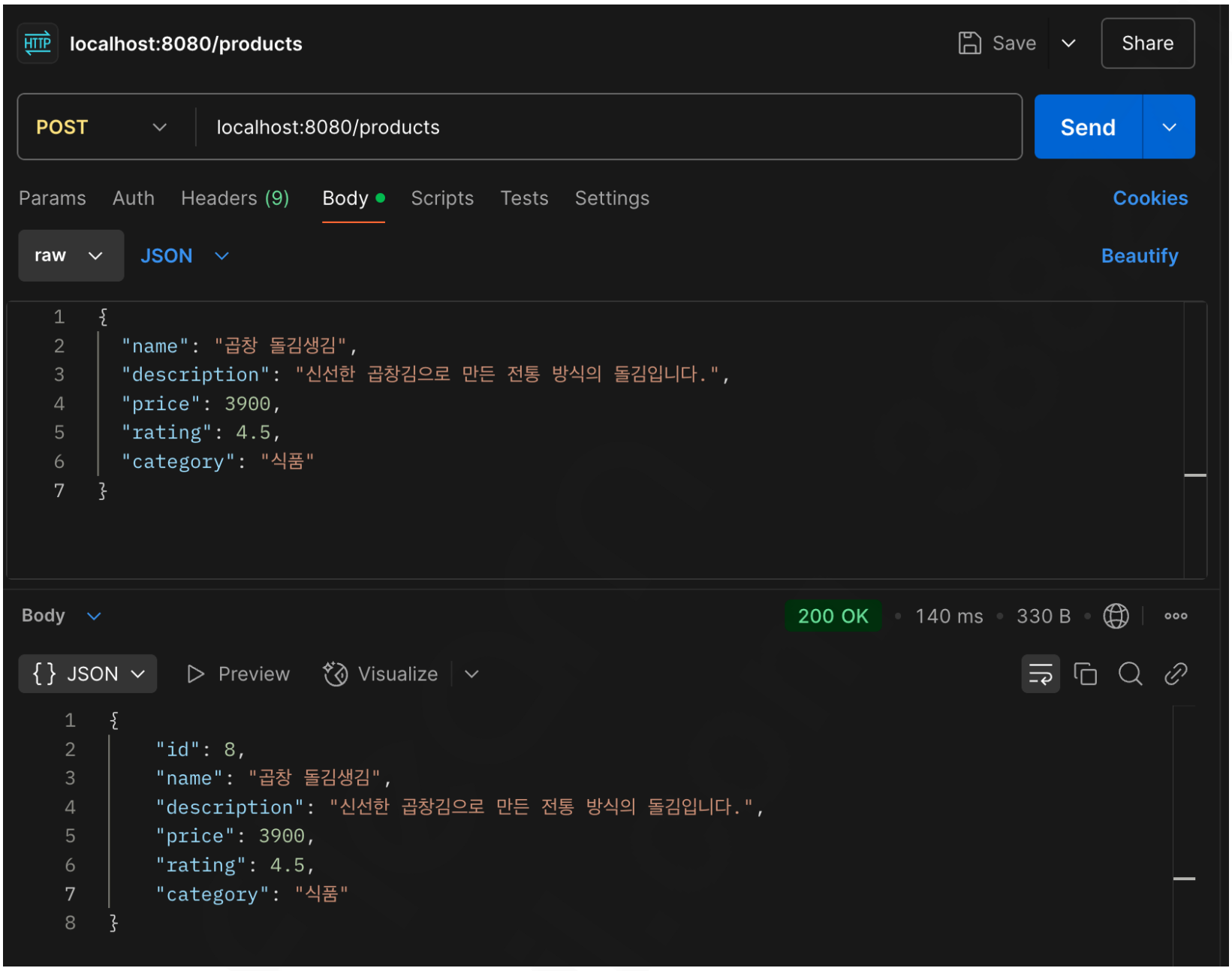
   ProductService

```
@Service
public class ProductService {
        ...
```

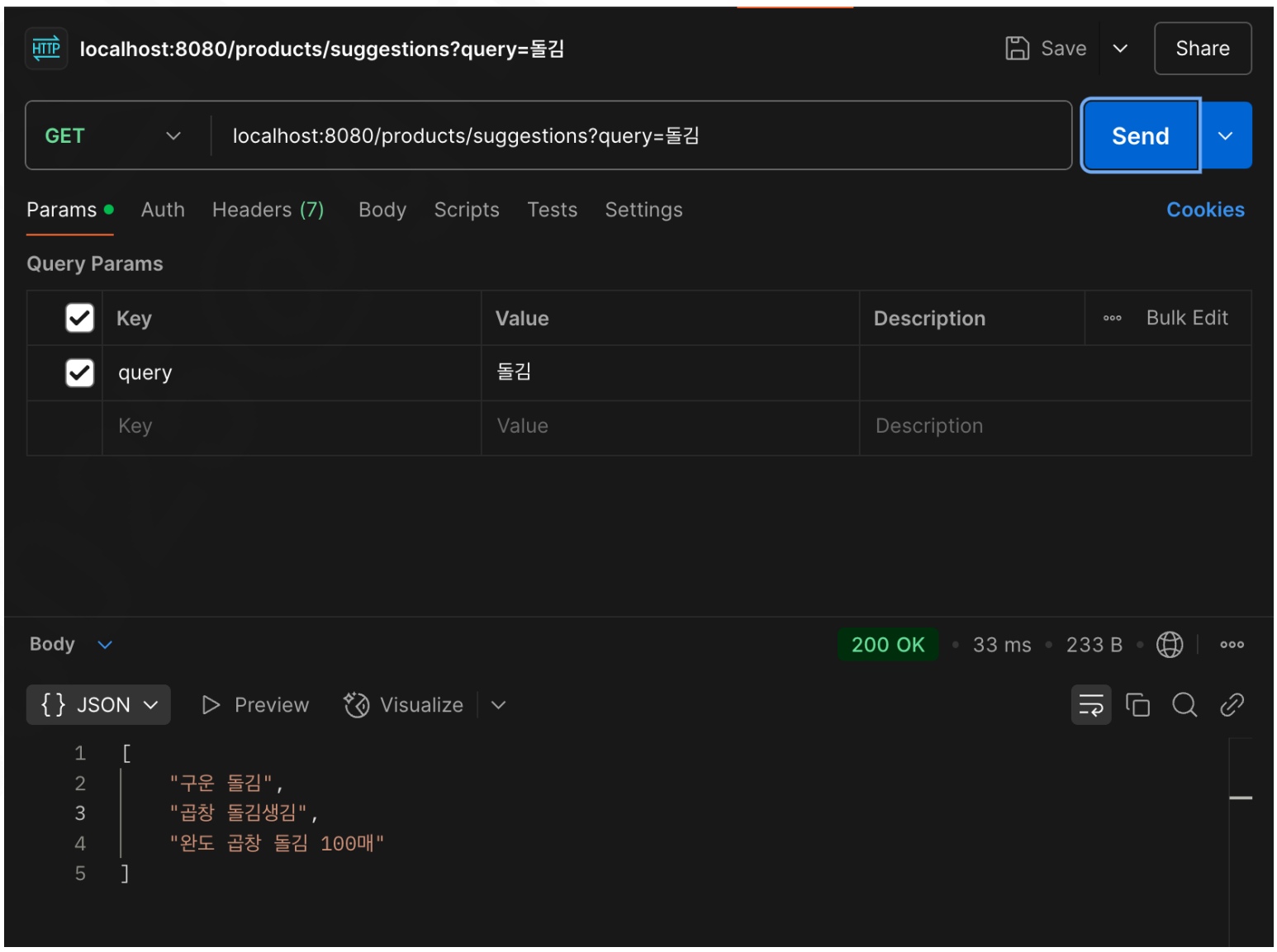
```
public List<String> getSuggestions(String query) {
    Query multiMatchQuery = MultiMatchQuery.of(m → m
        .query(query)
        .type(TextQueryType.BoolPrefix)
        .fields("name.auto_complete", "name.auto_complete._2gram", "name.auto_complete._3gram")
    )._toQuery();

    NativeQuery nativeQuery = NativeQuery.builder()
        .withQuery(multiMatchQuery)
        .withPageable(PageRequest.of(0, 5))
        .build();

    SearchHits<ProductDocument> searchHits = this.elasticsearchOperations.search(nativeQuery, ProductDocu
    return searchHits.getSearchHits().stream()
        .map(hit → {
          ProductDocument productDocument = hit.getContent();
          return productDocument.getName();
        })
        .toList();
  }

    ...

}
```

4. **Postman으로 테스트를 위한 더미 데이터 넣기**

```
{
  "name": "곱창 돌김생김",
  "description": "신선한 곱창김으로 만든 전통 방식의 돌김입니다.",
  "price": 3900,
  "rating": 4.5,
  "category": "식품"
}

{
  "name": "구운 돌김",
  "description": "불에 구워 바삭한 식감을 살린 프리미엄 돌김.",
  "price": 4500,
  "rating": 4.7,
  "category": "식품"
}

{
  "name": "완도 곱창 돌김 100매",
  "description": "완도산 곱창김 100매 구성, 김밥용으로 적합.",
  "price": 12900,
  "rating": 4.8,
  "category": "식품"
}
```



5. **Postman으로 테스트해보기**



- `돌`, `돌김`, `곱`, `곱창`으로 요청을 보냈을 때 자동 완성 API의 응답값이 잘 날라오는 걸 확인할 수있다.

🎯 [실습] Spring Boot에서 Elasticsearch 활용해 검색 API 만들기

✅ Spring Boot에 검색 API 만들기

1. **Controller에 API 추가하기**

   ProductController

```
@RestController
@RequestMapping("products")
public class ProductController {
        ...
    @GetMapping("/search")
    public ResponseEntity<List<ProductDocument>> searchProducts(
        @RequestParam String query,
        @RequestParam(required = false) String category,
        @RequestParam(defaultValue = "0") double minPrice,
        @RequestParam(defaultValue = "1000000000") double maxPrice,
        @RequestParam(defaultValue = "1") int page,
        @RequestParam(defaultValue = "5") int size
    ) {
        List<ProductDocument> products = productService.searchProducts(query, category, minPrice, maxPrice, page,
        return ResponseEntity.ok(products);
    }

        ...
}
```

2. **Service에 메서드 추가하기**

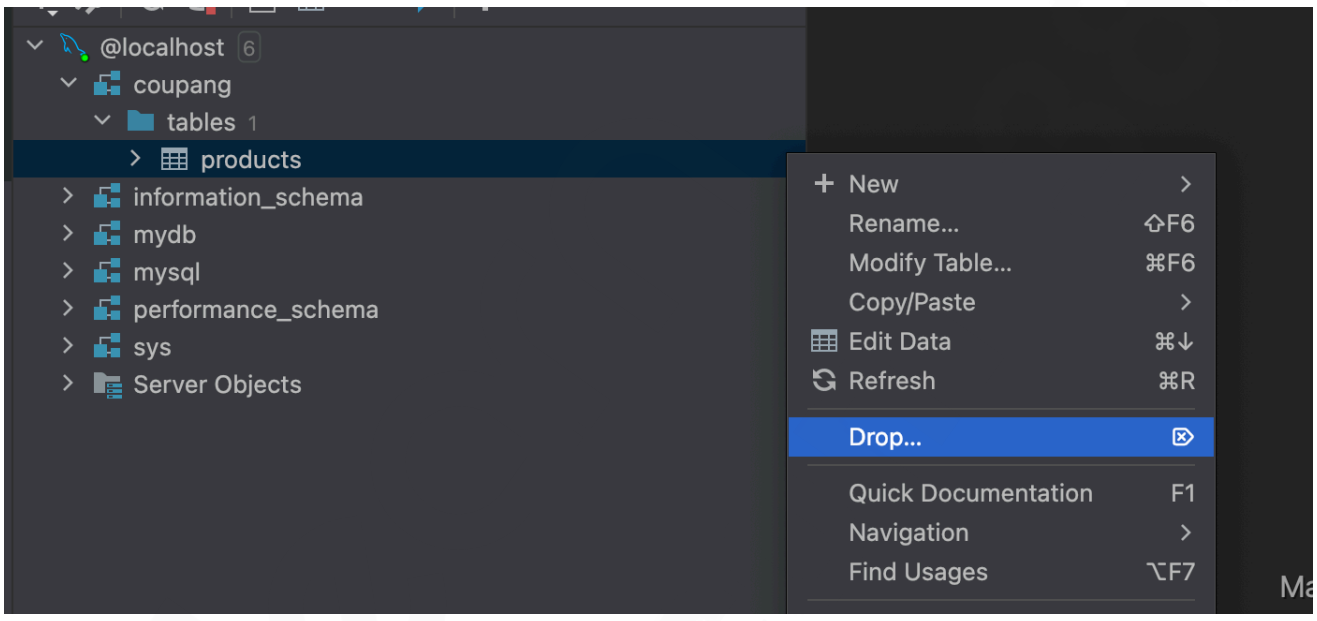
   ProductService

```
@Service
public class ProductService {
        ...

        public List<ProductDocument> searchProducts(
      String query,
      String category,
      double minPrice,
      double maxPrice,
      int page,
      int size
  ) {

    // multi_match 쿼리
    Query multiMatchQuery = MultiMatchQuery.of(m → m
        .query(query)
        .fields("name^3", "description^1", "category^2")
        .fuzziness("AUTO")
    )._toQuery();
```

```
// term filter 쿼리 : 카테고리가 정확히 일치하는 것만 필터링
List<Query> filters = new ArrayList<>();
if (category != null && !category.isEmpty()) {
  Query categoryFilter = TermQuery.of(t → t
      .field("category.raw")
      .value(category)
  )._toQuery();
  filters.add(categoryFilter);
}

// range filter: 가격 범위 필터
Query priceRangeFilter = NumberRangeQuery.of(r → r
    .field("price")
    .gte(minPrice)
    .lte(maxPrice)
)._toRangeQuery()._toQuery();
filters.add(priceRangeFilter);

// should: rating > 4.0
Query ratingShould = NumberRangeQuery.of(r → r
    .field("rating")
    .gt(4.0)
)._toRangeQuery()._toQuery();

// bool query 조합
Query boolQuery = BoolQuery.of(b → b
    .must(multiMatchQuery)
    .filter(filters)
    .should(ratingShould)
)._toQuery();

// Highlight Query 생성
HighlightParameters highlightParameters = HighlightParameters.builder()
    .withPreTags("<b>")
    .withPostTags("</b>")
    .build();
Highlight highlight = new Highlight(highlightParameters, List.of(new HighlightField("name")));
HighlightQuery highlightQuery = new HighlightQuery(highlight, ProductDocument.class);

  // NativeQuery 생성
  NativeQuery nativeQuery = NativeQuery.builder()
      .withQuery(boolQuery)
      .withHighlightQuery(highlightQuery)
      .withPageable(PageRequest.of(page - 1, size))
      .build();

  // 쿼리 실행
  SearchHits<ProductDocument> searchHits = this.elasticsearchOperations.search(nativeQuery, ProductDocument.c
  return searchHits.getSearchHits().stream()
      .map(hit → {
        ProductDocument productDocument = hit.getContent();
        String highlightedName = hit.getHighlightField("name").get(0);
        productDocument.setName(highlightedName);
        return productDocument;
      })
      .toList();
}
```

```
    ...
}
```

3. **깔끔한 테스트를 위해 기존 데이터 삭제하기**
    a. **기존 MySQL 테이블 삭제하기**

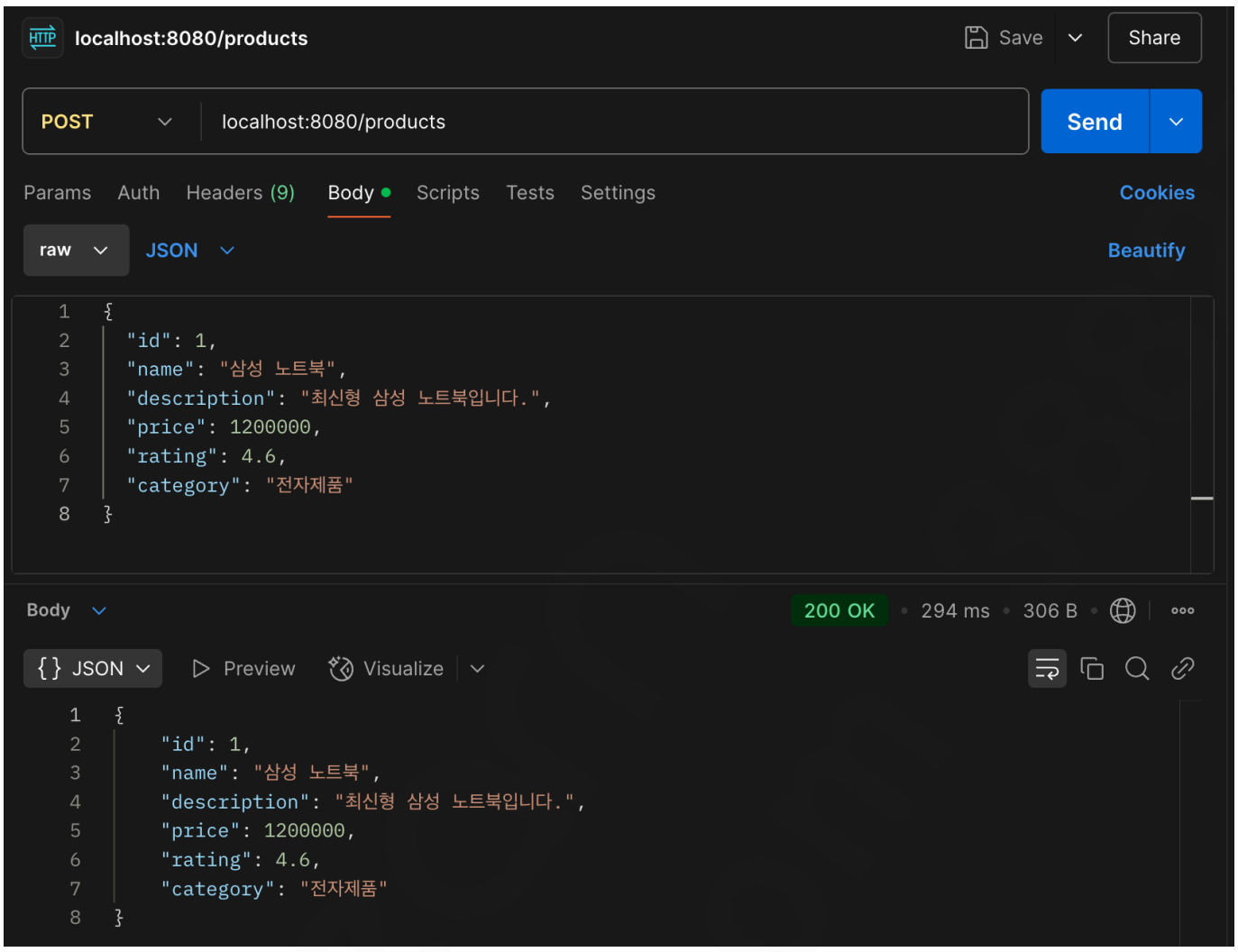


    b. **Kibana에서 Elasticsearch 인덱스 삭제하기**

```
DELETE /propducts
```

4. **Postman으로 테스트를 위한 더미 데이터 집어넣기**

```
{
  "name": "삼성 노트북 13인치",
  "description": "최신형 삼성 노트북입니다.",
  "price": 1200000,
  "rating": 4.6,
  "category": "전자제품"
}

{
  "name": "삼성 노트북 최신형 15인치",
  "description": "2025년 최신 15인치 노트북입니다.",
  "price": 2200000,
  "rating": 4.8,
  "category": "전자제품"
}

{
  "name": "삼성 스마트폰 A17",
  "description": "최신형 스마트폰, 휴대전화",
  "price": 1500000,
  "rating": 4.5,
  "category": "휴대폰"
}
```



5. **테스트해보기**

> 요구사항의 모든 내용을 다 테스트해보진 않고 몇 가지만 간단하게 테스트해보자.

1. `삼성`으로 검색해보기

```
http://localhost:8080/products/search?query=삼성
```

- 키워드에 맞게 잘 검색이 된다.

2. 카테고리 필터링 추가해보기

```
http://localhost:8080/products/search?category=전자제품&query=삼성
```

- 카테고리에 맞는 데이터만 조회가 된다.

3. 가격 범위 필터링 추가해보기

```
http://localhost:8080/products/search?category=전자제품&minPrice=0&maxPrice=1500000&query=삼성
```

- 가격 범위에 해당하는 데이터만 조회가 된다.

4. `samsung`으로 검색해보기

```
http://localhost:8080/products/search?category=전자제품&minPrice=0&maxPrice=1500000&query=samsung
```

- 동의어로도 잘 검색이 된다.

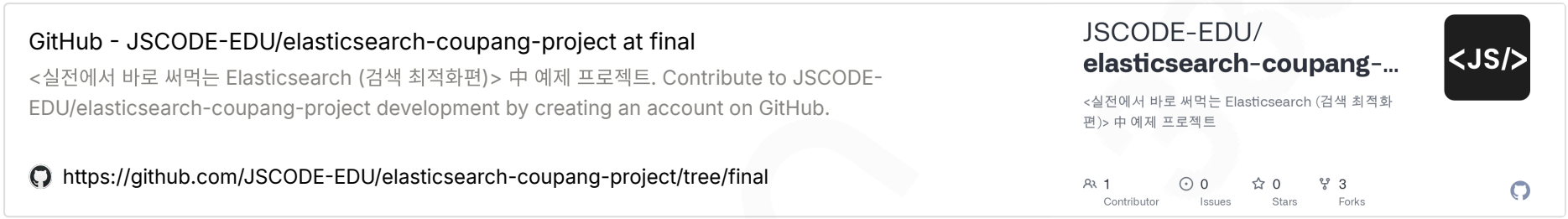
참고) 만약 테스트 코드를 작성할 수 있는 역량을 갖고 있다면, 테스트 코드를 작성해서 더 간편하고 꼼꼼하게 테스트할 수 있을 것이다.

지금까지 구현하고자 한 자동 완성 기능과 검색 기능을 다 구현했다. 다음 강의에서는 로컬 환경에서 띄운 Elasitcsearch를 활용하는 방법이 아닌, 클라우드에서 띄우는 Elasticsearch를 활용하는 방법을 배워볼 것이다.

@kfc3882@gmail.com
khb3025

[참고 자료] 프로젝트 완성본 (Github)

프로젝트 레포지토리의 `final` 브랜치에 프로젝트 완성본을 올려뒀다. 강의의 실습을 똑같이 따라했는데도 잘 작동하지 않았다면, 아래 코드와 비교해서 잘못 작성한 부분이 있는 지 체크해보자.



@gmail.com
khb3025

📖

현업에서는 Elasticsearch를 직접 구축해서 쓰진 않나요?

✅ 현업에서는 Elasticsearch를 직접 구축해서 쓰진 않나요?

현업에서는 Elasticsearch를 직접 설치해서 운영하기도 하지만, 많은 경우에는 **AWS OpenSearch**나 **Elastic Cloud** 같은 서비스를 활용해 구축한다. 왜냐하면 Elasticsearch를 구축하려면 단순히 설치만 하면 끝나는 게 아니라, 운영과 모니터링, 장애 대응, 확장, 보안 설정 등 많은 요소를 신경써서 구축하고 관리해야 하기 때문이다. 따라서 현업에서 많이 구축하는 방식에 익숙해지기 위해 Elastic Cloud를 활용해 Elasticsearch를 구축하는 방법을 알아볼 것이다.

👨🏻‍🏫 Elastic Cloud를 활용하는 방법을 배우기 전에, Elastic Cloud와 AWS OpenSearch의 차이에 대해 먼저 알아보자.

@kfc@gmail.com
khb302

📖

AWS Opensearch vs Elastic Cloud

✅ AWS OpenSearch vs Elastic Cloud

AWS OpenSearch와 Elastic Cloud의 차이를 알기 위해 OpenSearch의 역사에 대해 먼저 알아보자.

Elasticsearch를 개발한 회사에서 Elasticsearch를 처음에는 오픈 소스로 오픈했다. 하지만 2021년 1월 이후로 Elasticsearch에 새로운 라이선스를 부과하면서 자유로운 활용이 금지되었다. AWS는 이에 반발해, 오픈 소스로 활용할 수 있었던 버전인 Elasticsearch 7.10.2 버전을 포크(fork)하여 자체적인 검색엔진을 개발하기 시작했다. 그게 바로 OpenSearch이다.

이러한 유래 때문에 Elasticsearch와 OpenSearch가 가진 대부분의 기능은 비슷하다. 하지만 시간이 흐르면서 버전이 업데이트 됨에 따라 기능이나 사용법의 차이가 조금씩 생기게 됐다.

이런 배경에 의해 AWS Opensearch Service는 Opensearch라는 독자적인 검색 엔진을 사용하고, Elastic Cloud는 우리가 지금까지 배운 Elasticsearch의 검색 엔진을 사용한다. 즉, Opensearch와 Elasticsearch는 엄연히 다른 소프트웨어이다. 하지만 둘 다 현업 서비스에서 검색 엔진으로 활용하는 데는 아무 문제가 없는 완성도 있는 서비스이다.

> 그럼 이번 강의에서 AWS OpenSearch 대신에 Elastic Cloud를 활용하는 이유가 뭘까?

이번 실습에서 Elastic Cloud를 활용한 이유는 크게 2가지이다.

- AWS OpenSearch에서는 nori 플러그인을 제공하지 않는다.
- AWS OpenSearch는 Elastic Cloud에 비해 레퍼런스가 많이 없다.

이런 이유 때문에 이번 실습에서 Elastic Cloud를 활용하기로 결정했다.

참고) 당연히 Elastic Cloud 대신에 OpenSearch를 활용해서 검색 엔진을 구축해도 된다. 다만, OpenSearch와 Elasticsearch는 엄연히 다른 소프트웨어이기 때문에 지금까지 배운 명령어나 쿼리 중에 다른 부분이 있을 수 있다. 하지만 비슷한 부분이 많고 기본적인 작동 원리는 똑같기 때문에 금방 익힐 수 있을 것이다.

👨🏽‍🏫 그럼 다음 강의에서 Elastic Cloud를 활용해 Elasticsearch를 구축해보자.

[참고 자료] 실습에서 발생하는 Elastic Cloud 비용

비용이 걱정되시는 분들은 학습이 끝나자마자 최대한 빠르게 Elastic Cloud 리소스를 종료하는 걸 추천한다. 강의의 마지막 부분에 비용이 나가지 않게 Elastic Cloud 리소스를 종료하는 방법도 알려줄 예정이다.

✅ Elastic Cloud 비용

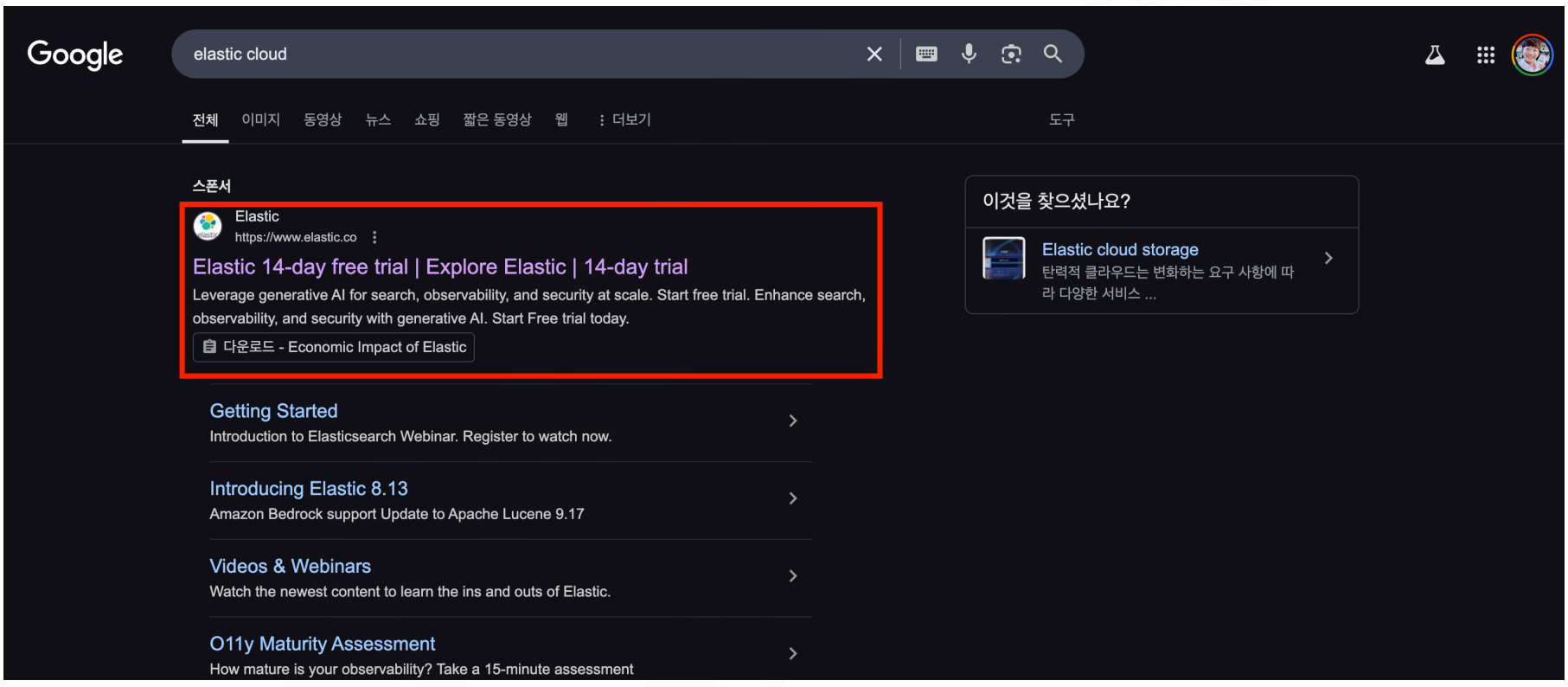
- 시간당 0.0311 USD (24시간당 약 1000원)
    - **참고)** Elastic Cloud를 처음 사용하는 유저라면 14일 간 무료로 사용할 수 있다.

@gmail.com
khb302

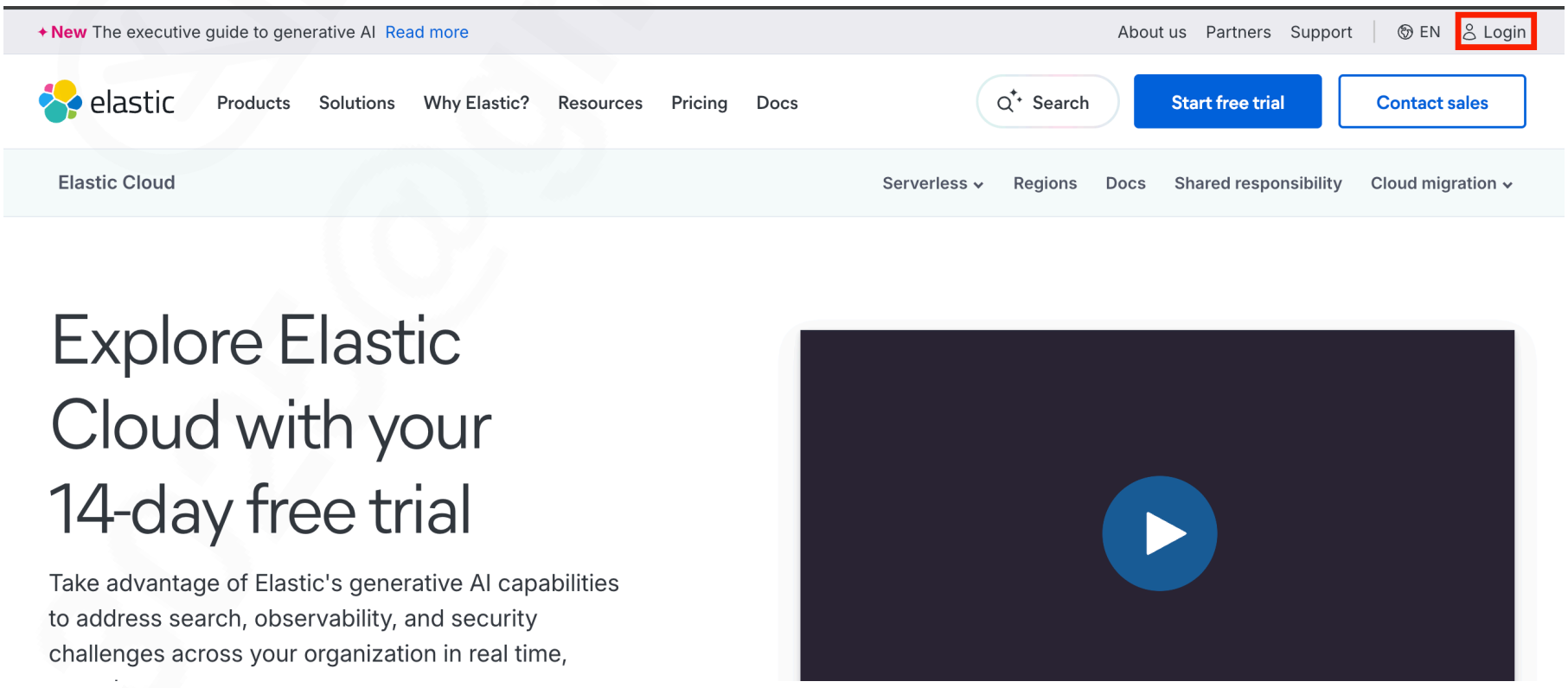
[실습] Elastic Cloud에서 Elasticsearch 생성하기 (+ 가입하기)

✅Elastic Cloud에 처음 가입하는 유저인 경우 (14일 무료 체험판 활용)

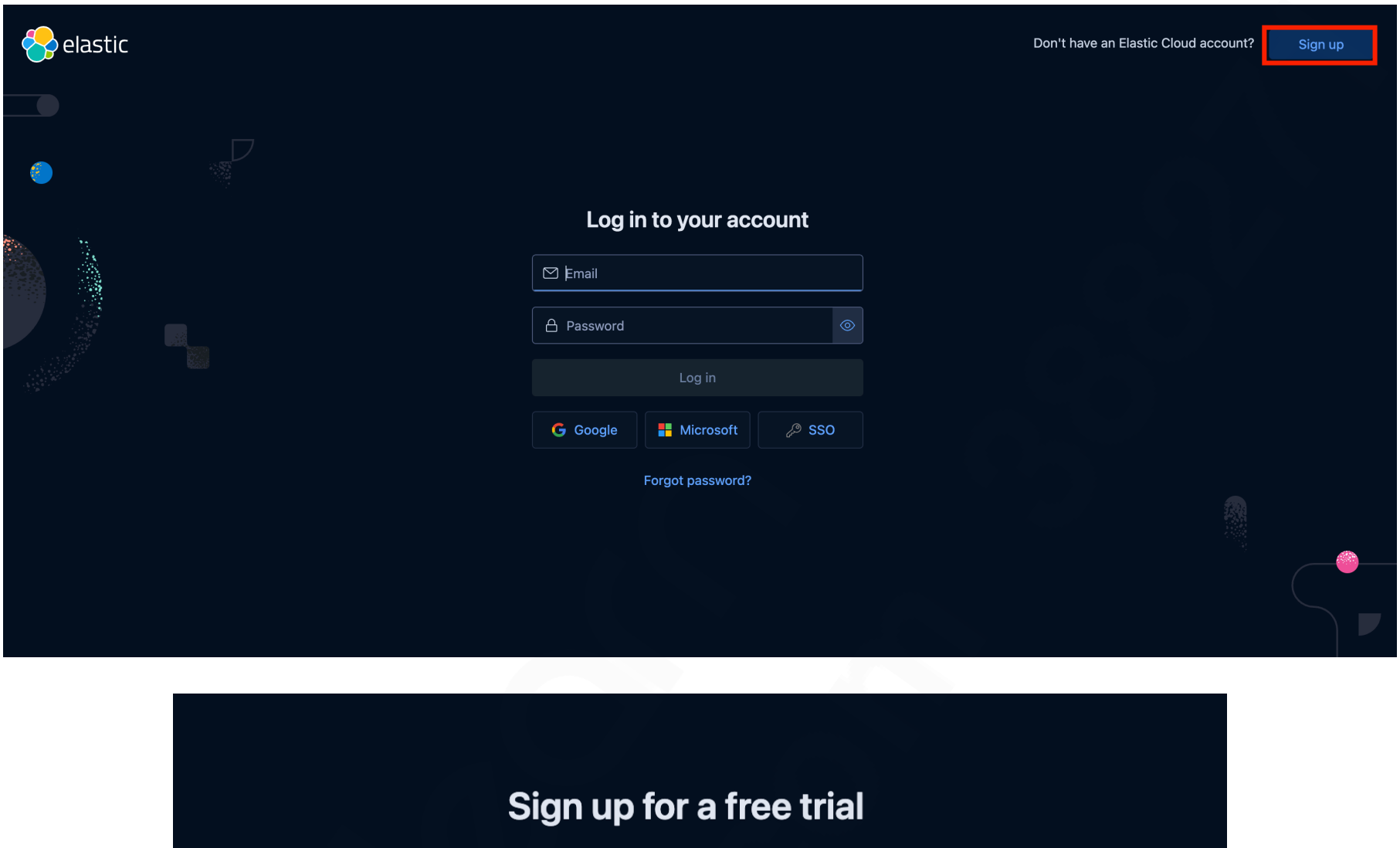
1. **Elastic Cloud 검색해서 들어가기**



2. **로그인 페이지로 들어가기**

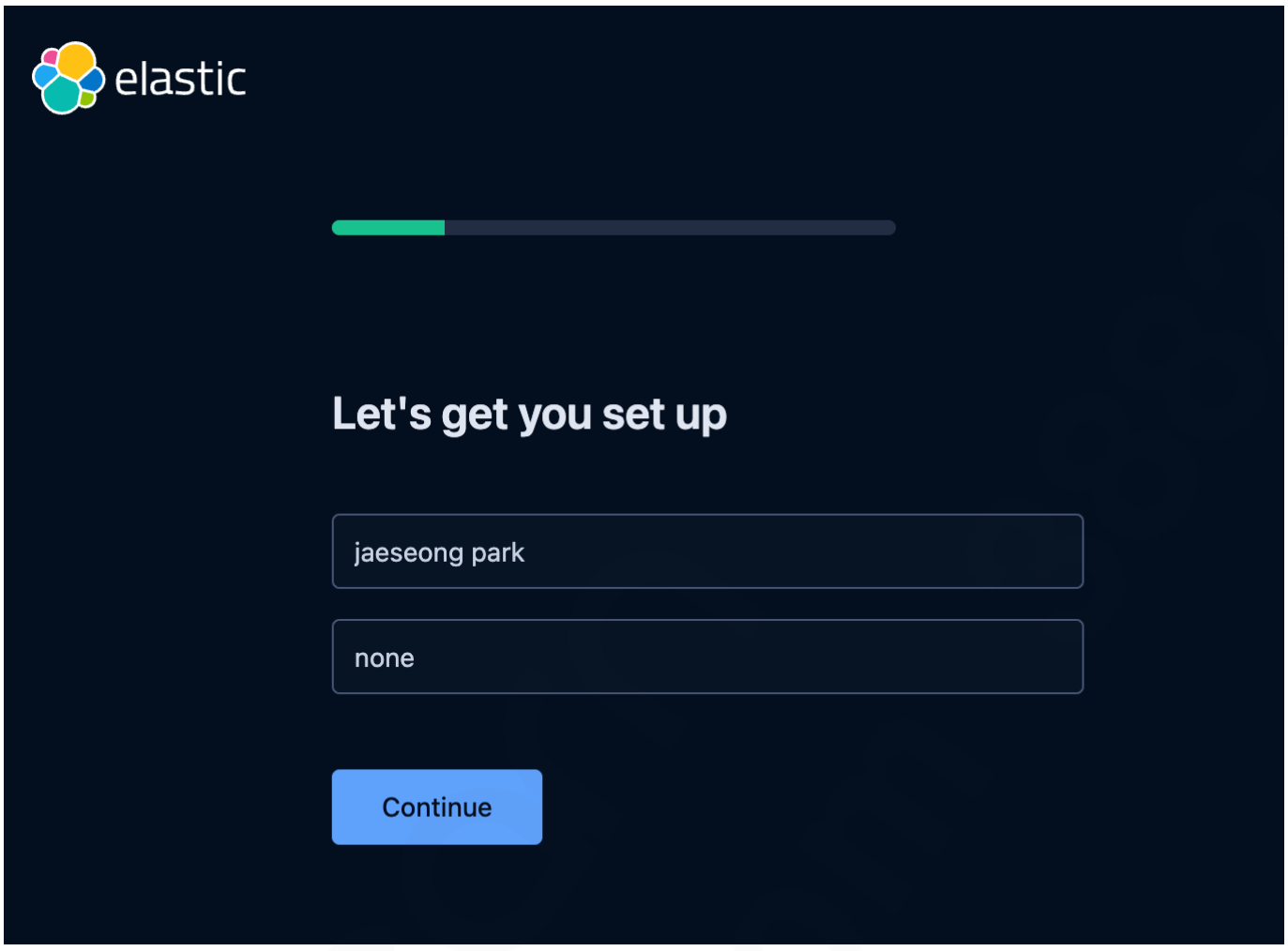


3. **회원가입 버튼 누르기**

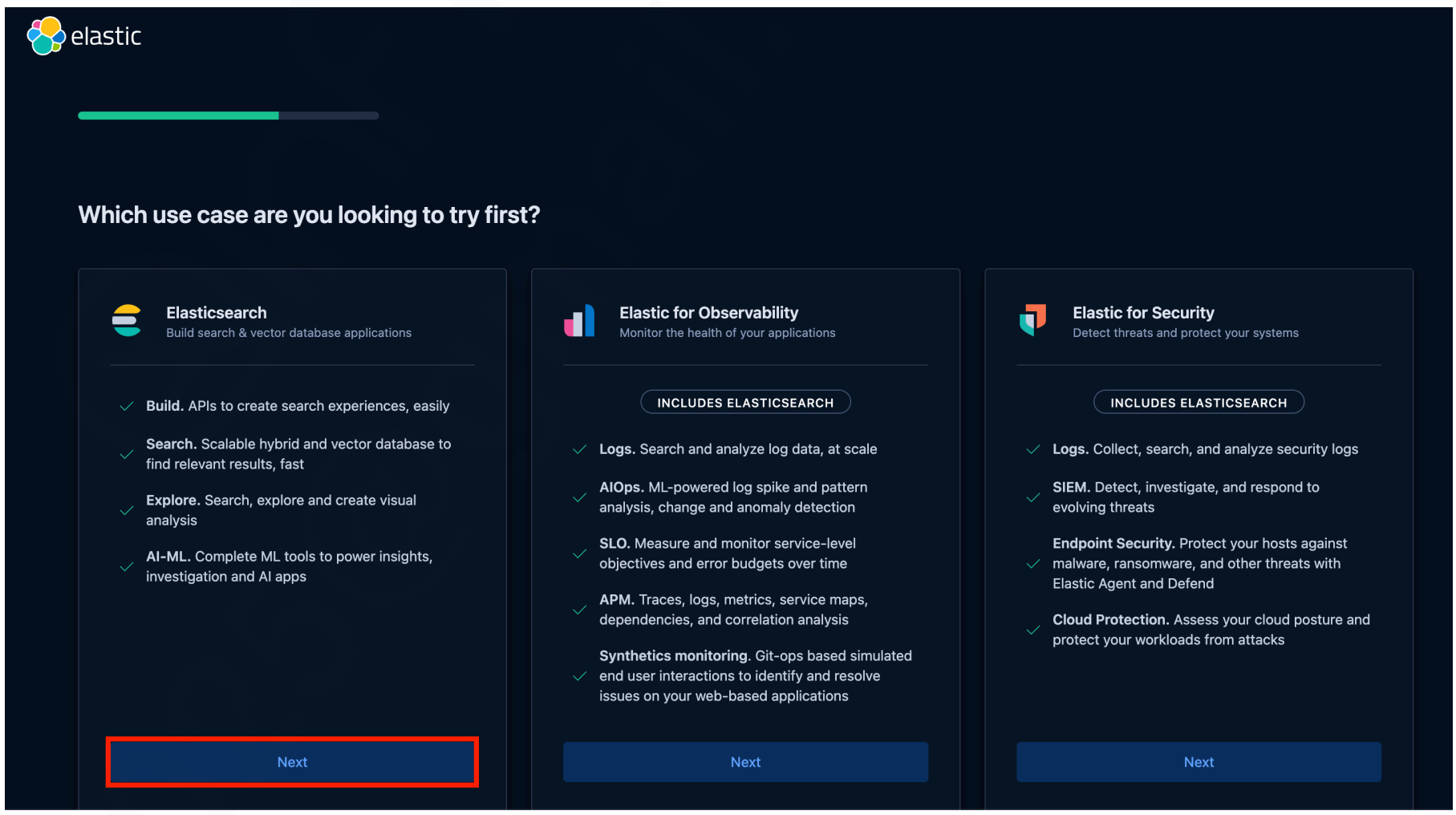


참고) Microsoft 계정으로 가입해도 상관없다.

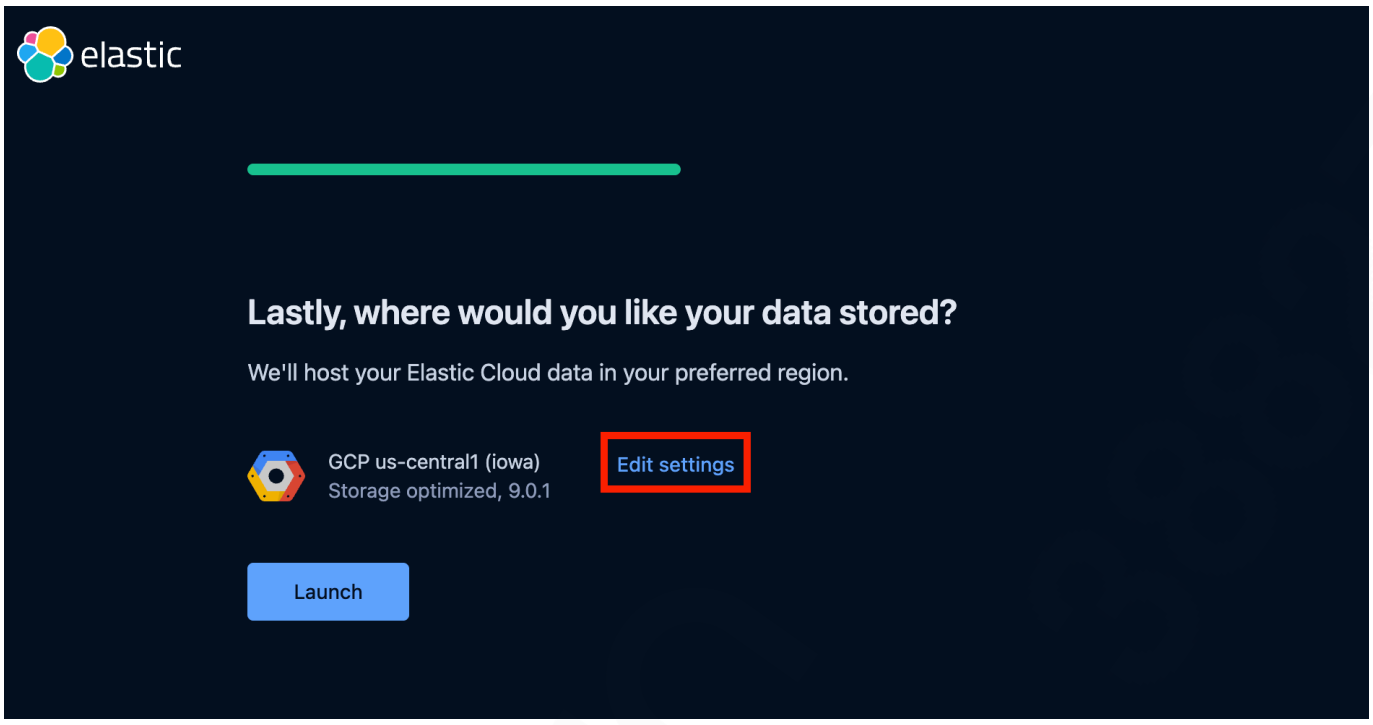
4. **기본 정보 입력하기**

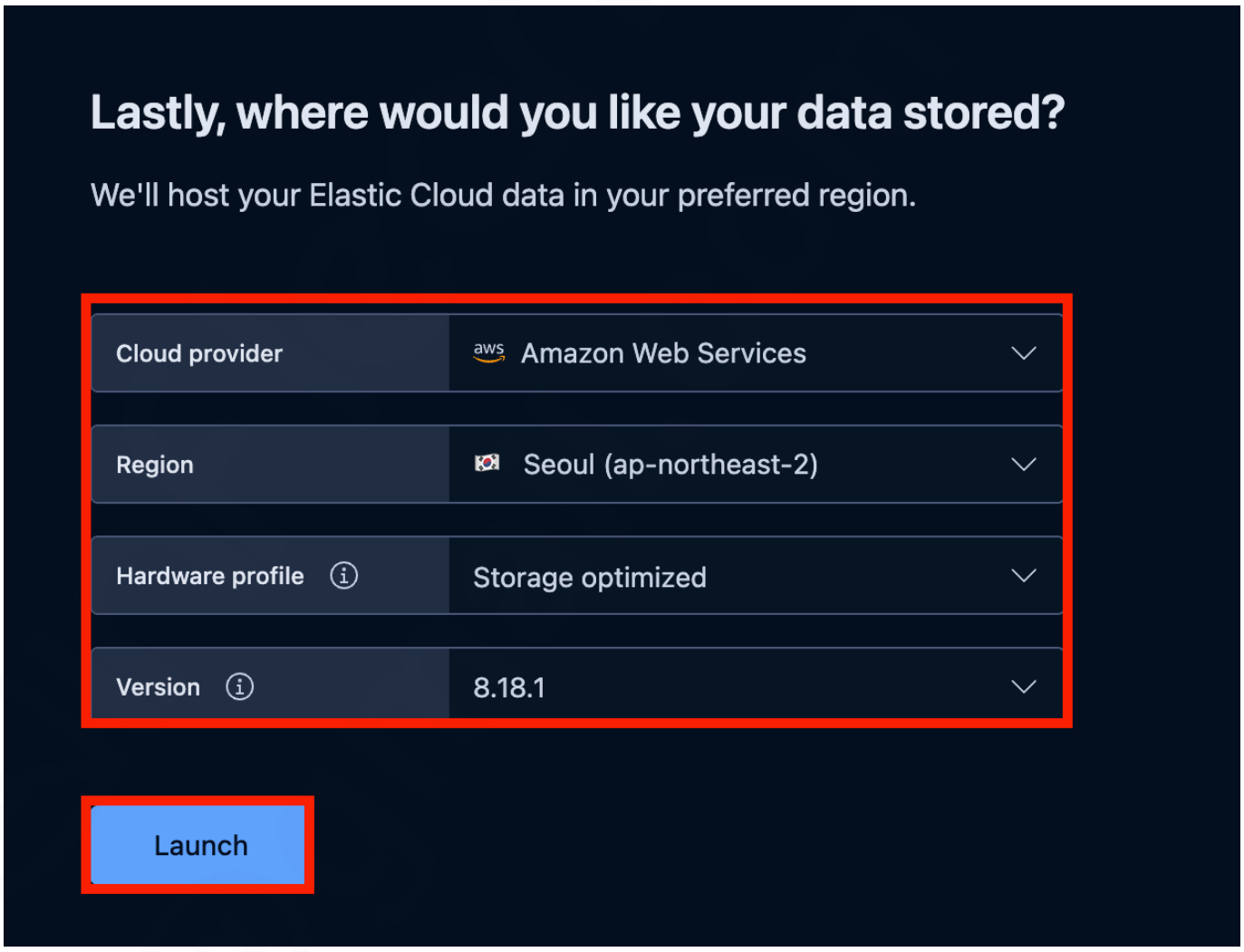


Full Name과 Company를 입력하라고 하는데 대충 입력해도 된다. 그리고 그 이후에 설문 조사 같은 항목이 나오는데 역시나 대충 클릭해도 된다. 그러다 아래 페이지가 나오면 Elasticsearch 메뉴의 Next 버튼을 클릭해주자.



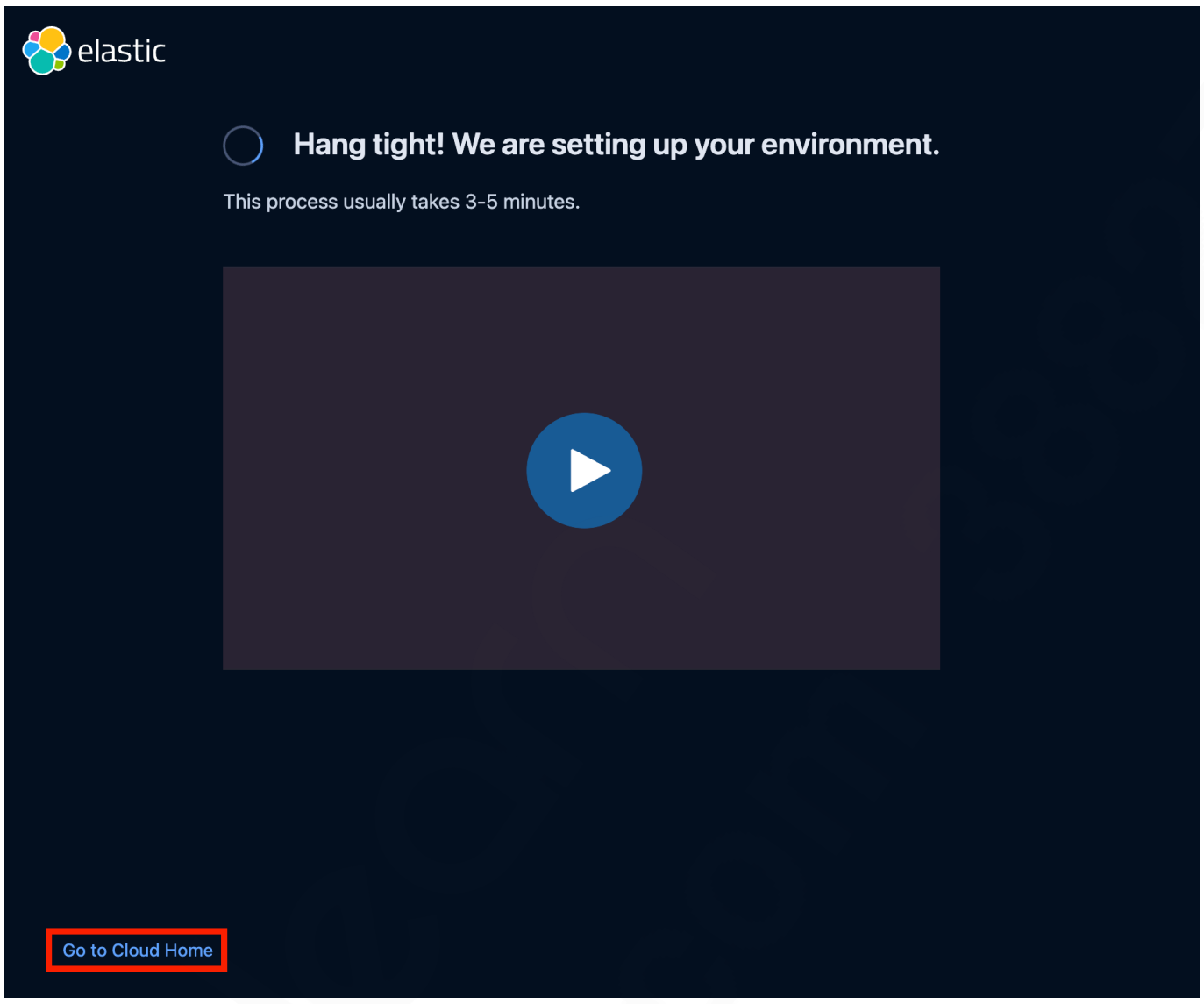
5. 생성할 Elasticsearch 기본 셋팅하기





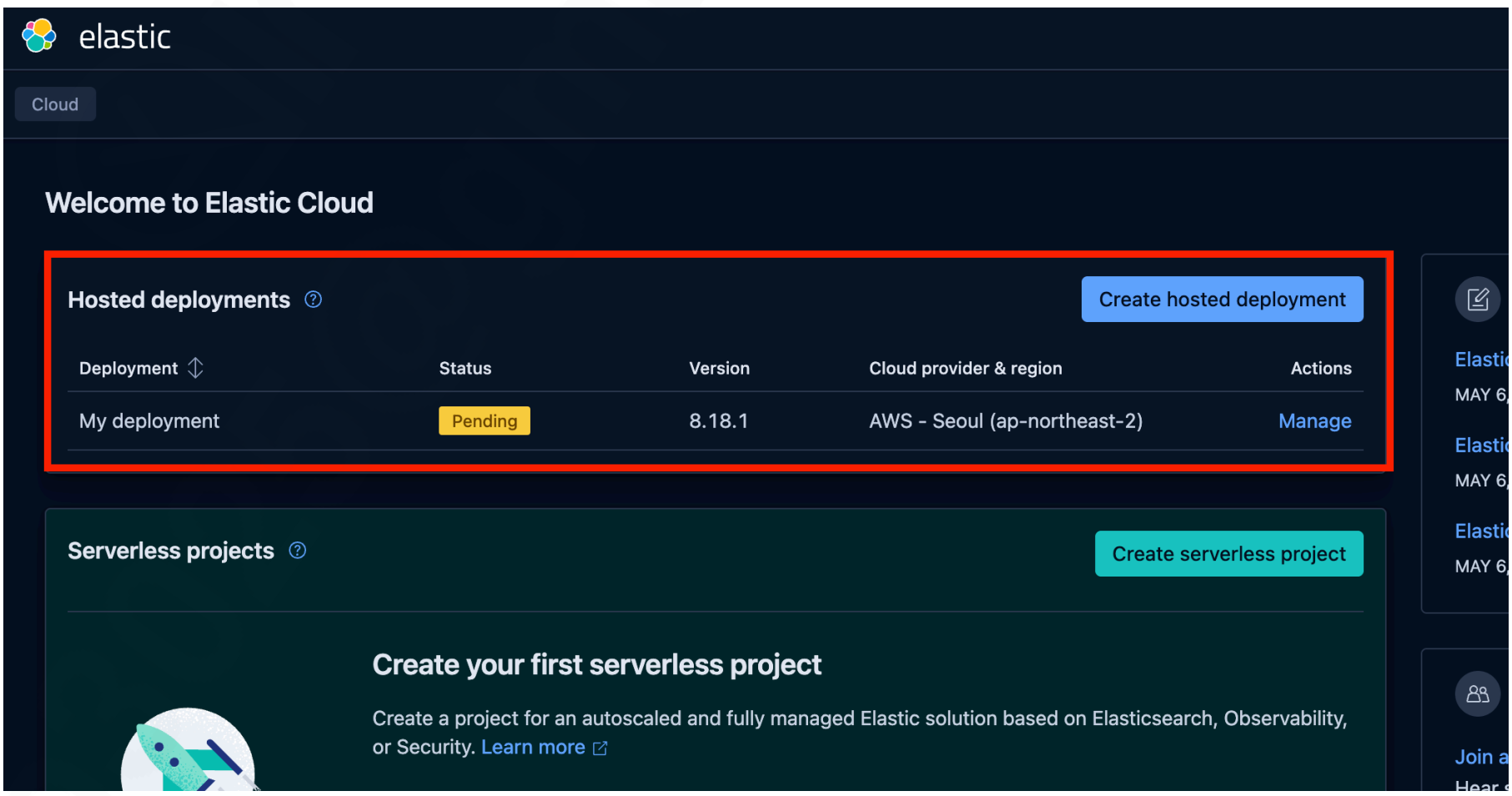
Version은 8 버전 중에 가장 최신 버전을 선택하면 된다. 그러고 Launch 버튼을 누르면 아래 화면이 뜨면서 3~5분 정도 기다려야 한다.
좌측 하단에 Go to Cloud Home 버튼을 눌러 홈으로 돌아가자.

khb3025

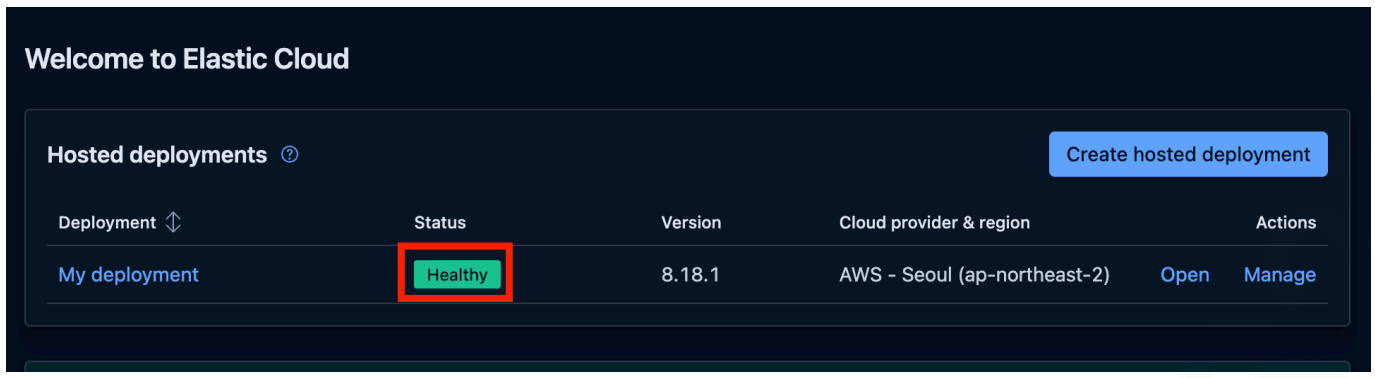


6. **생성된 Deployment 확인하기**

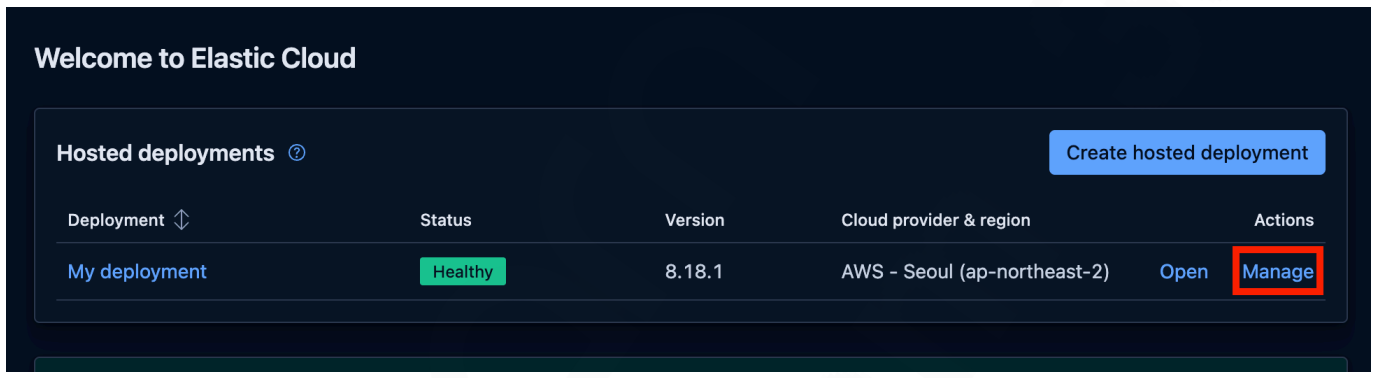
   생성된 Elasticsearch의 한 단위를 보고 Deployment라고 부른다. Status가 아직 `Pending`인 걸로 봐서 아직 생성 중이다. 조금만 더 기다리자.

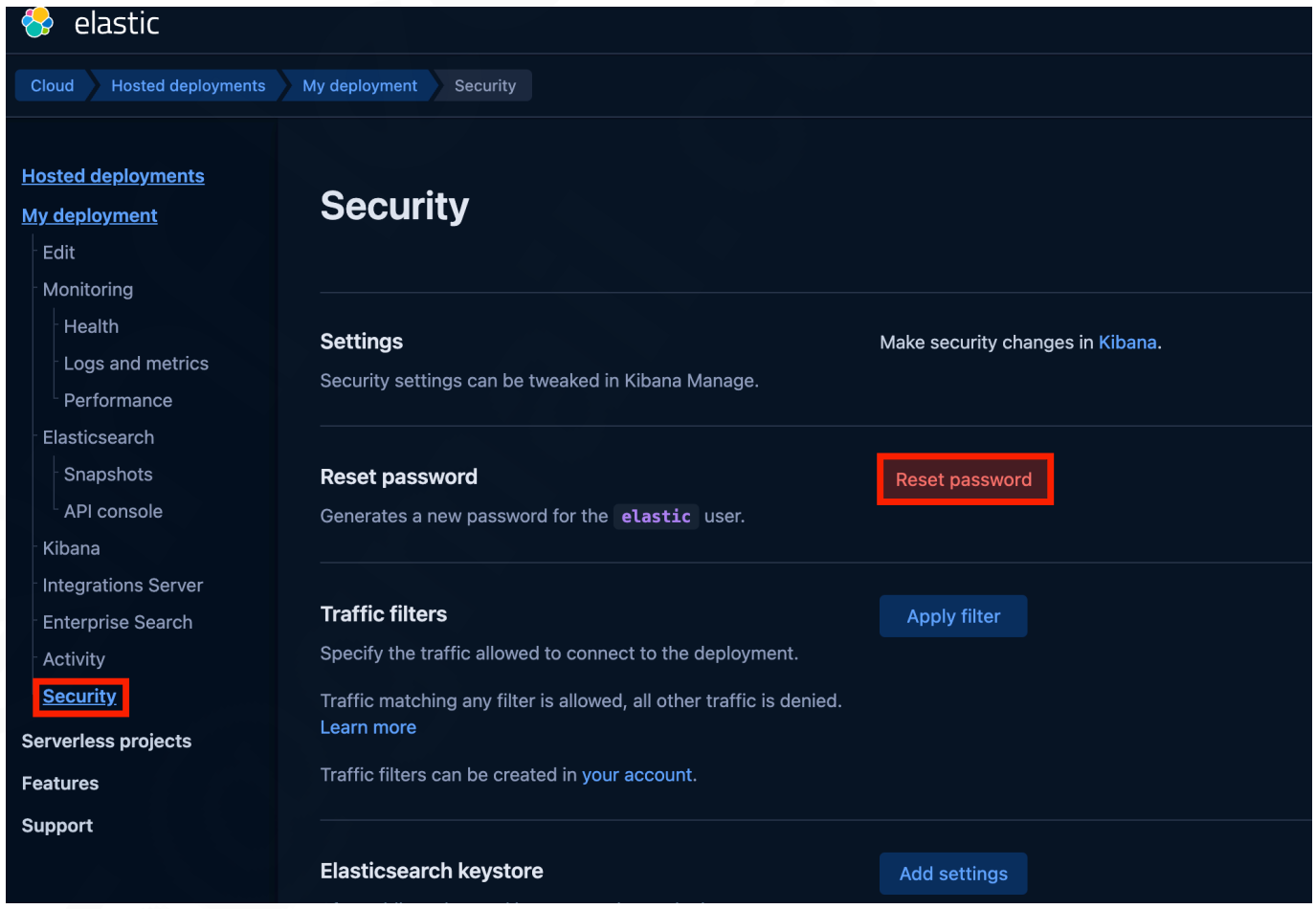


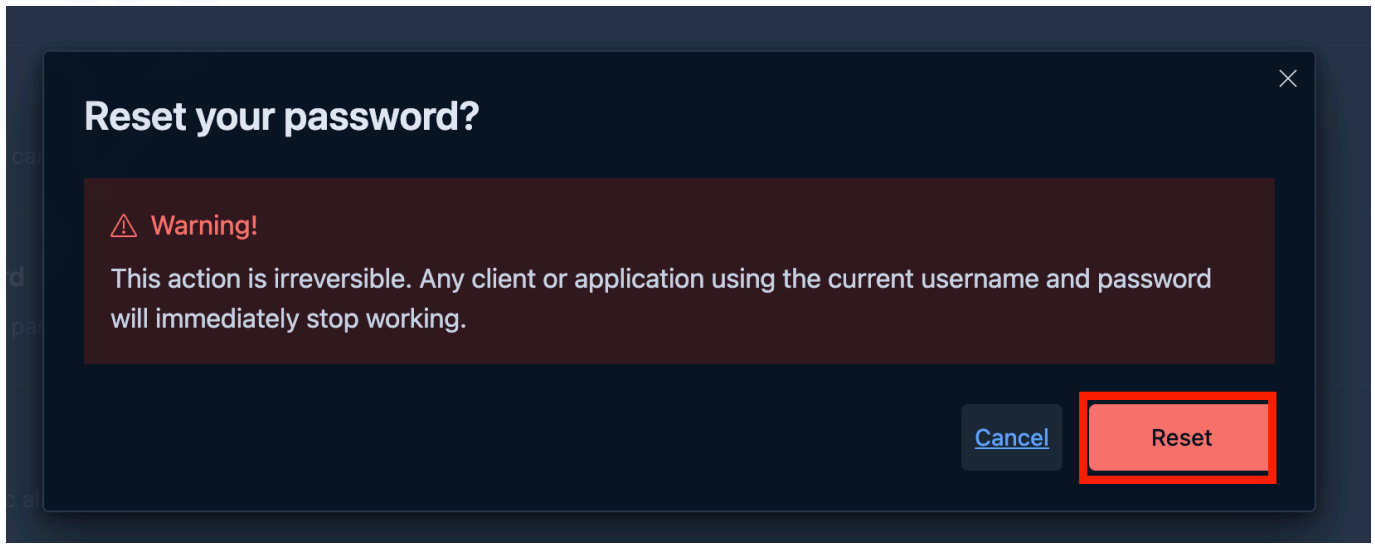
3~5분 정도 기다리면 Healthy로 바뀐다.

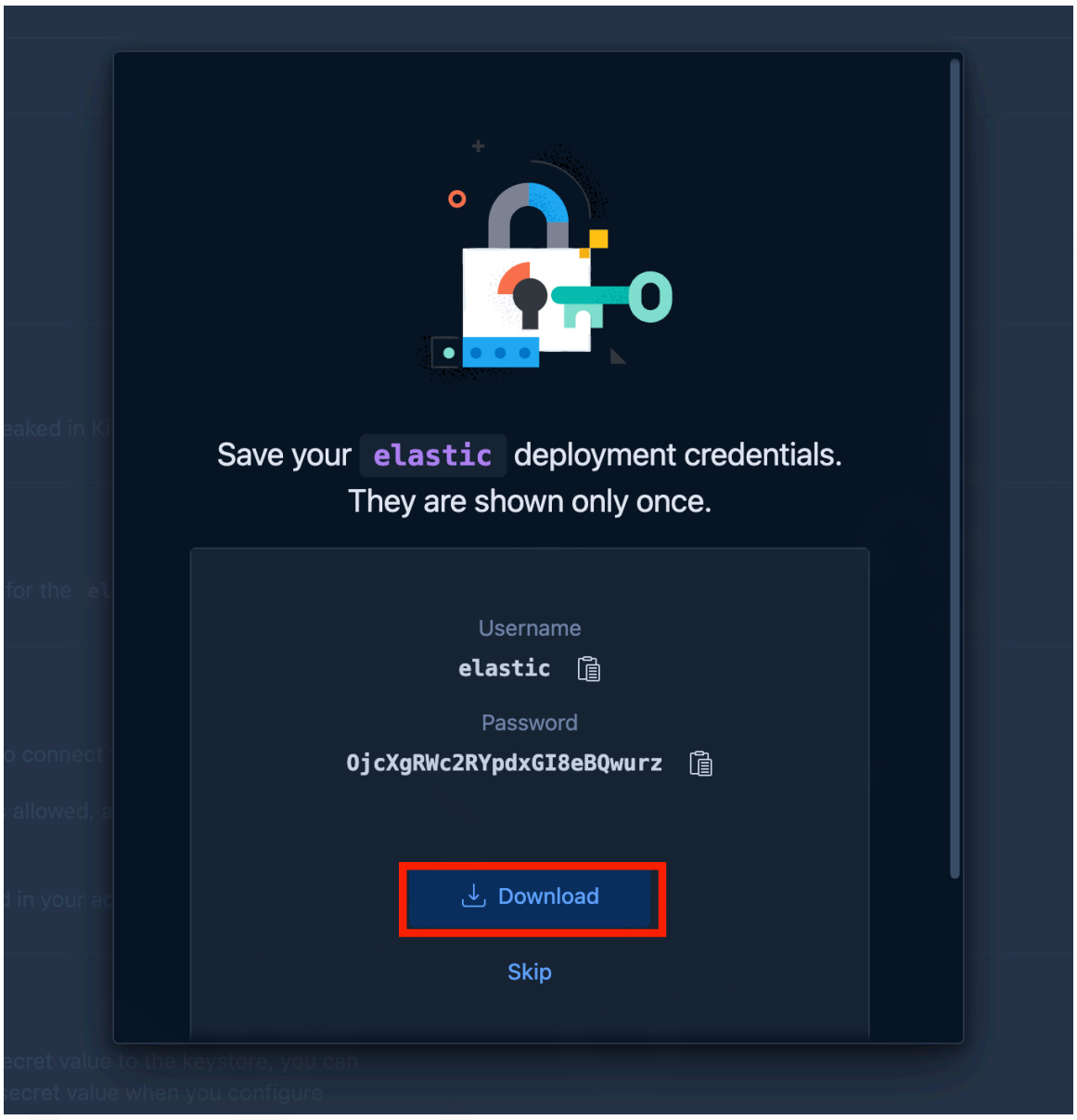


7. **Elasticsearch에 접근할 수 있는 password 셋팅하러 가기**





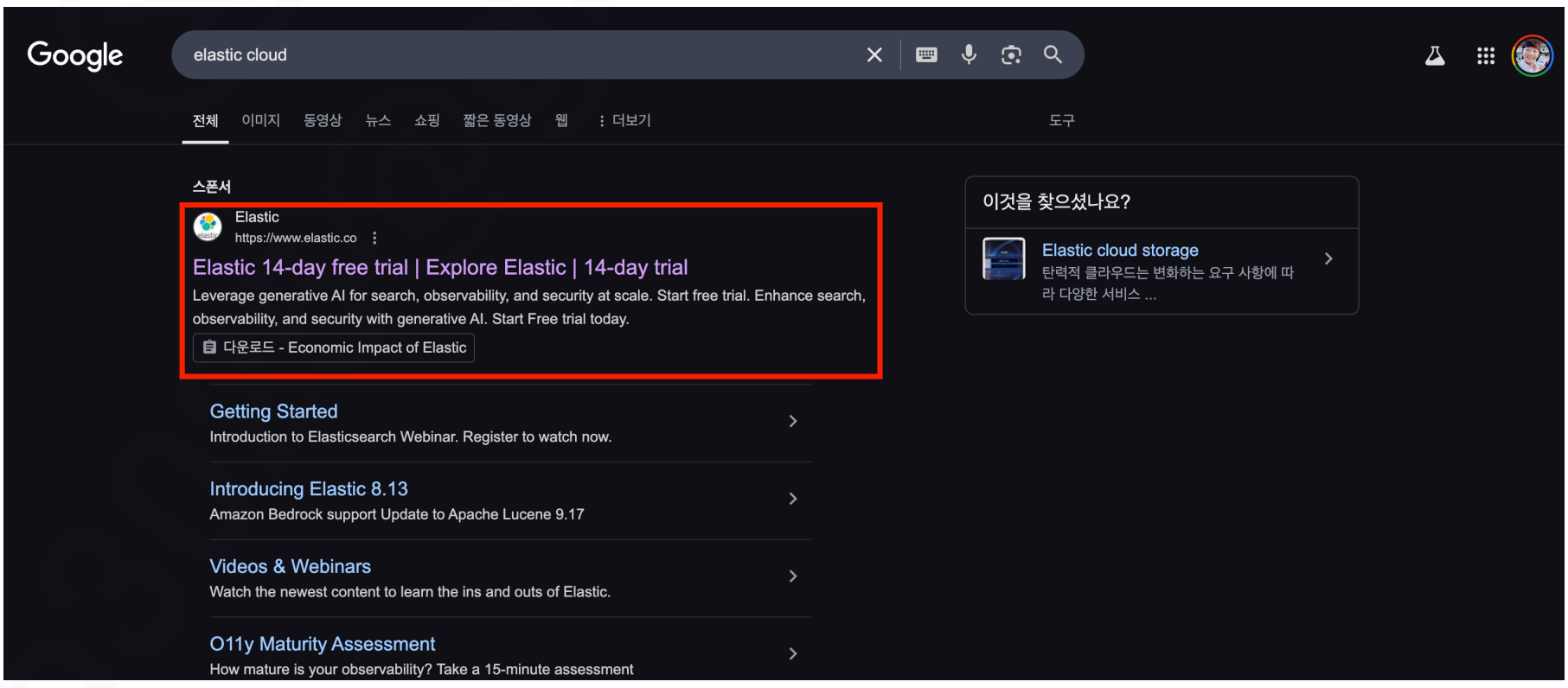




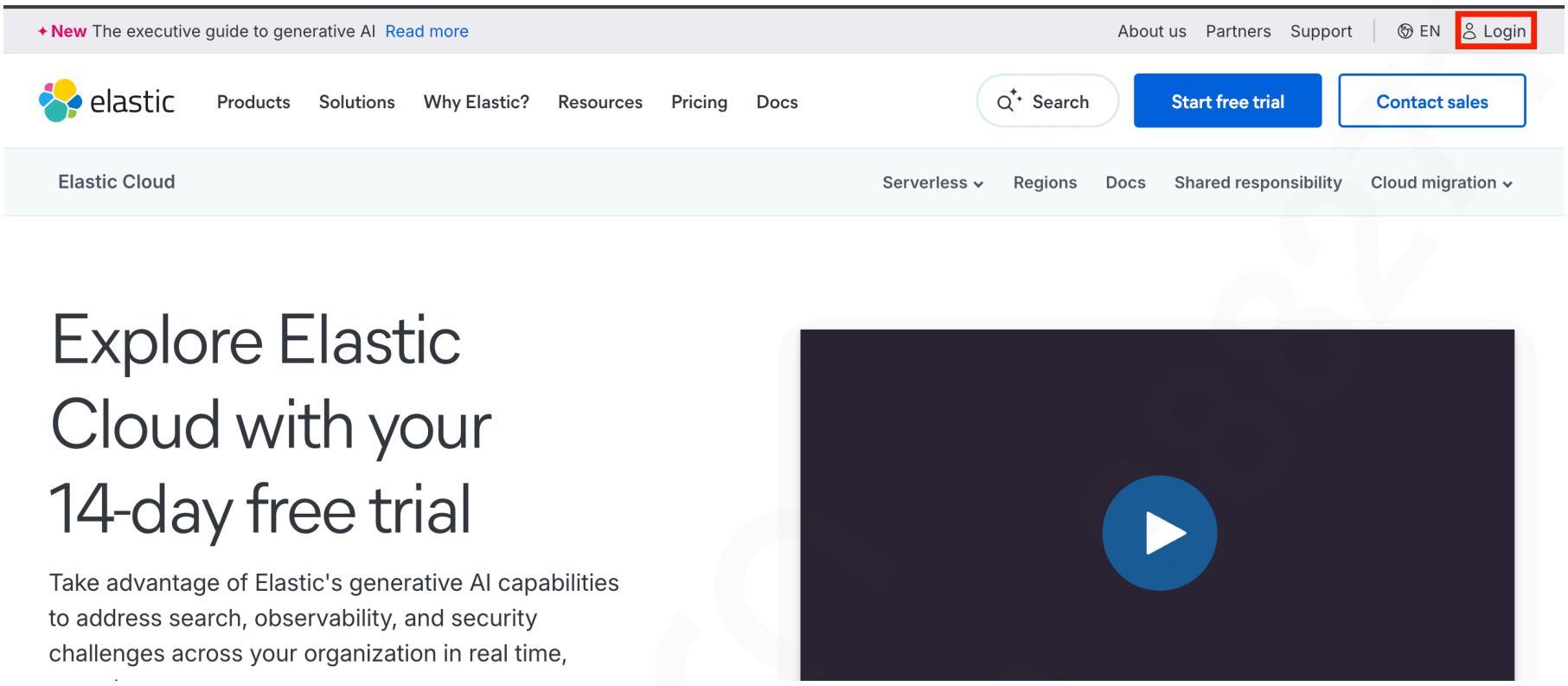
Elasticsearch에 접근할 때 사용하는 username, password이다. 잊어버리지 않게 잘 적어두거나 파일로 다운 받아두자.

✅ Elastic Cloud의 14일 체험판을 이미 쓴 유저인 경우

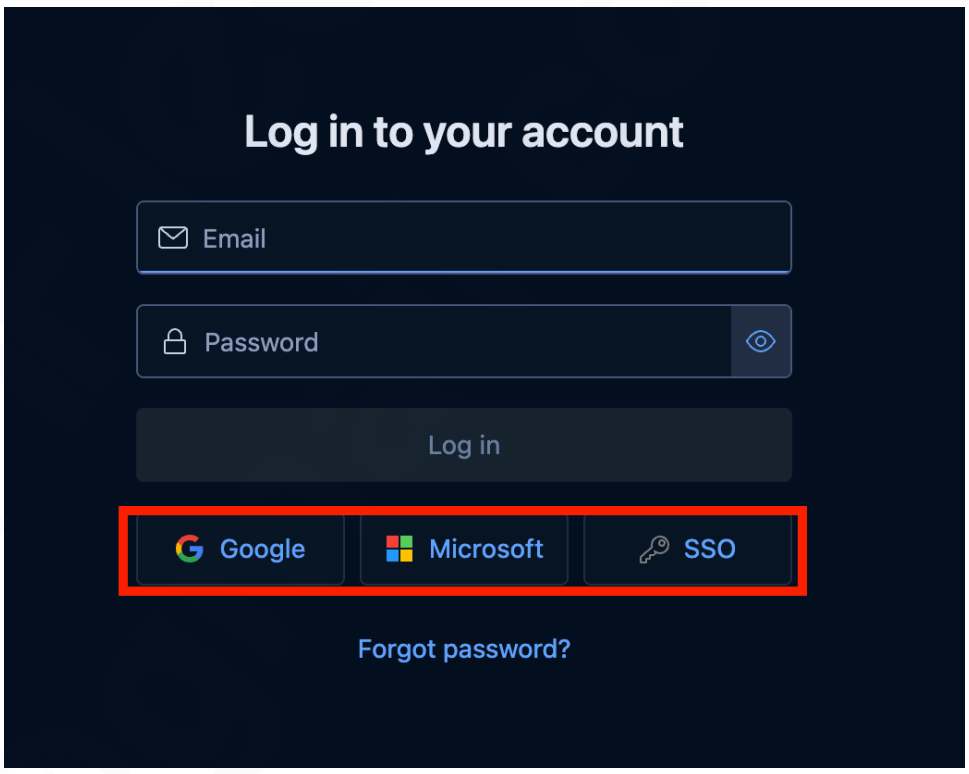
1. **Elastic Cloud 검색해서 들어가기**



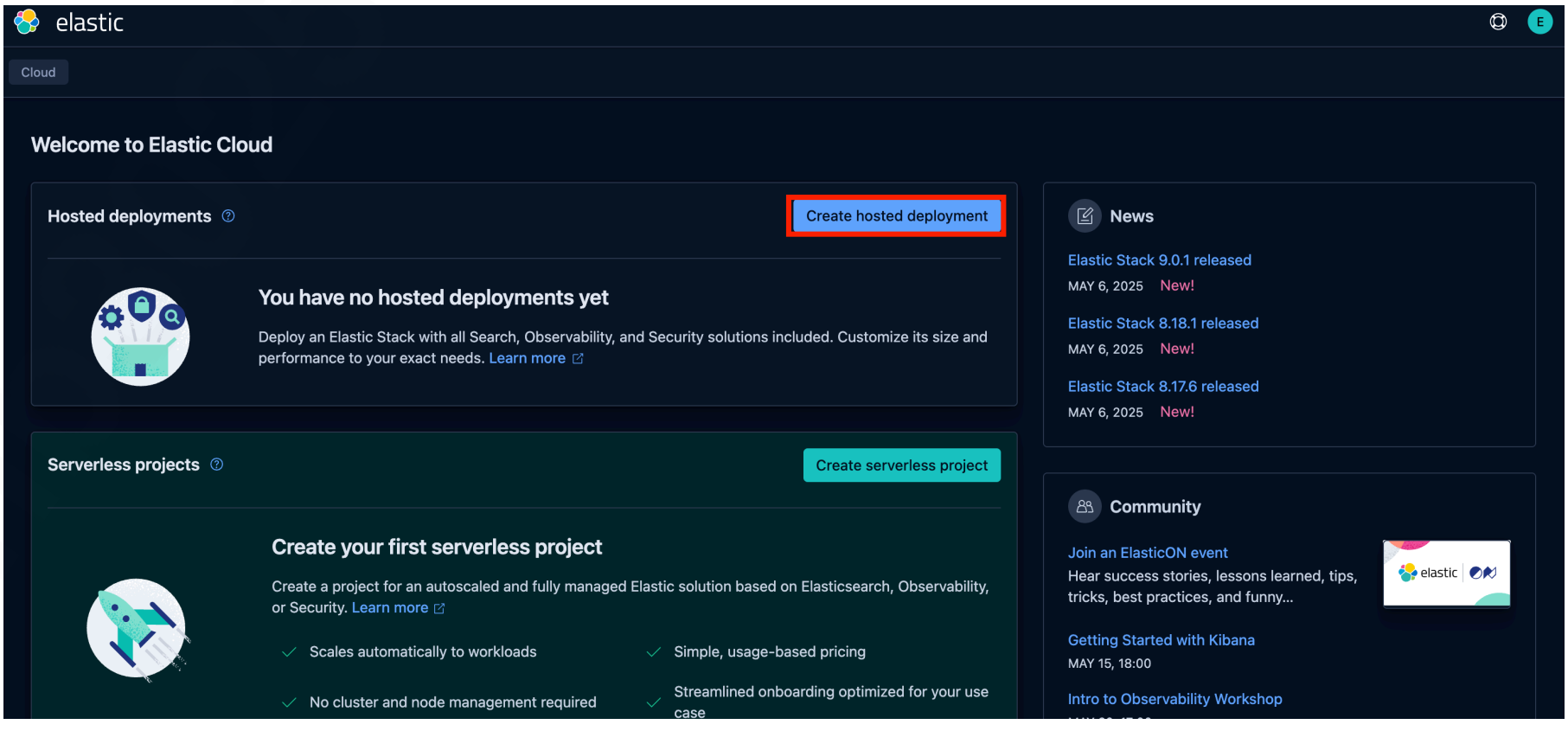
2. **로그인 페이지로 들어가기**

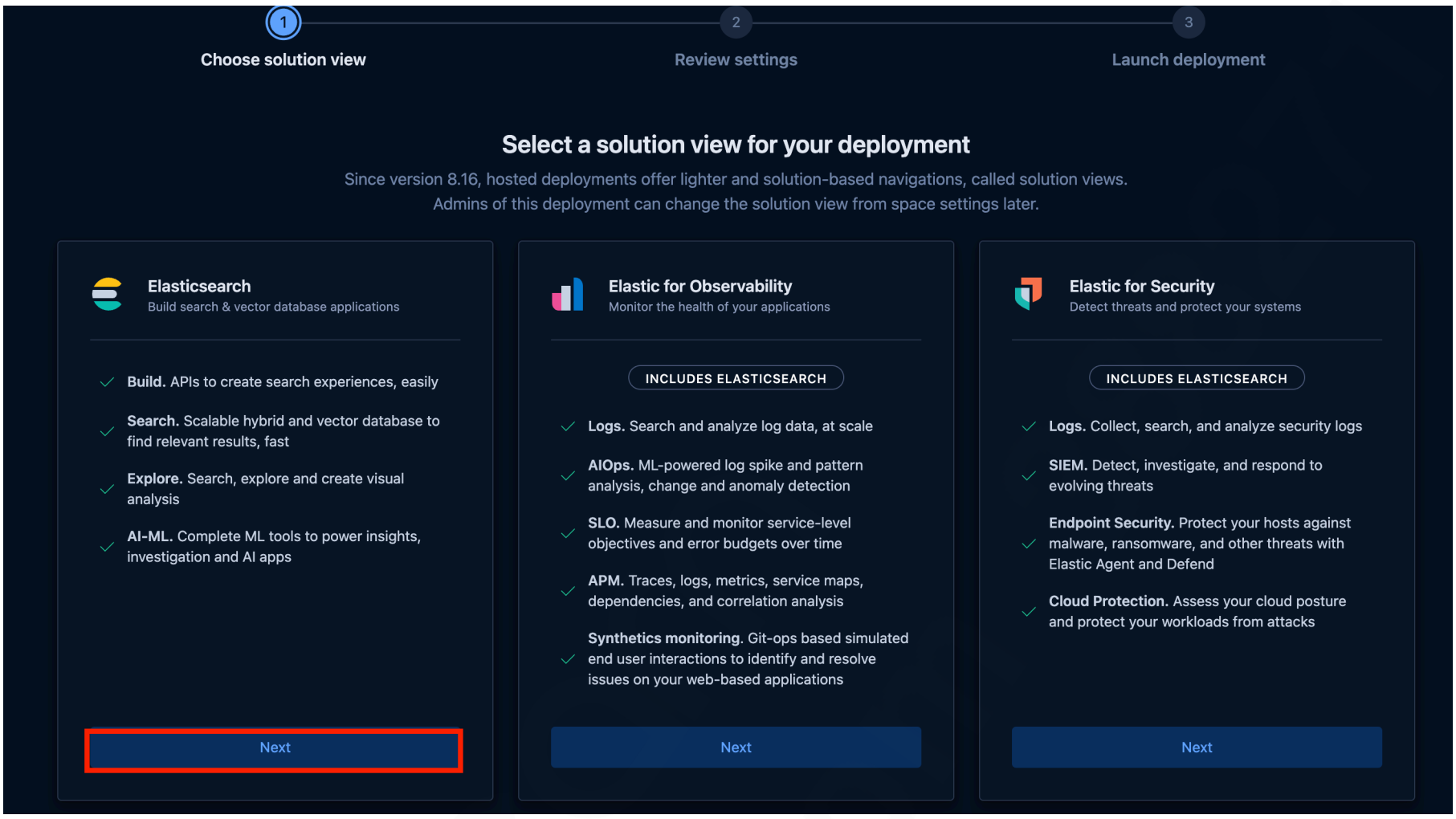


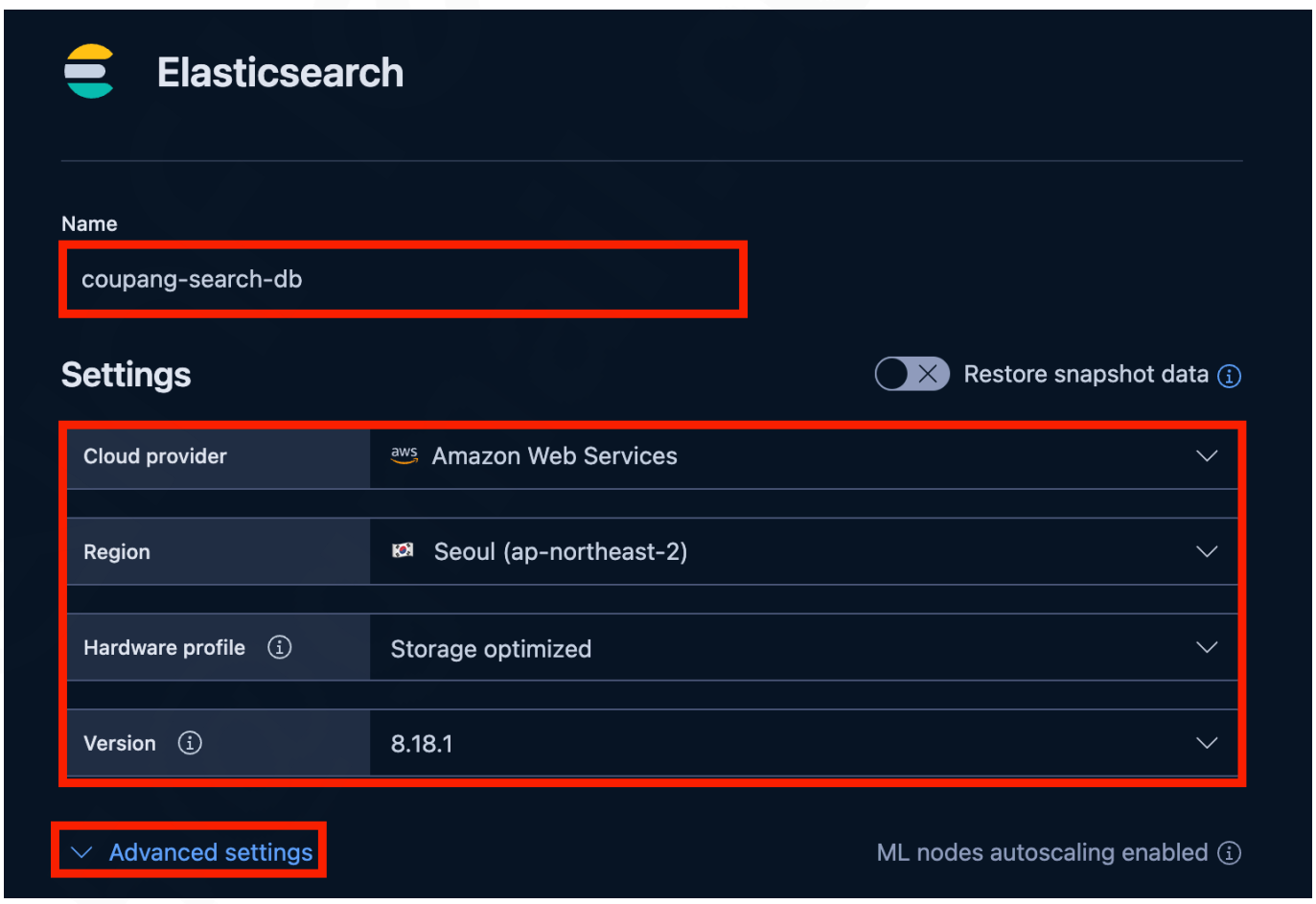
3. **로그인하기**



4. **Elasticsearch의 Deployment 생성하기**

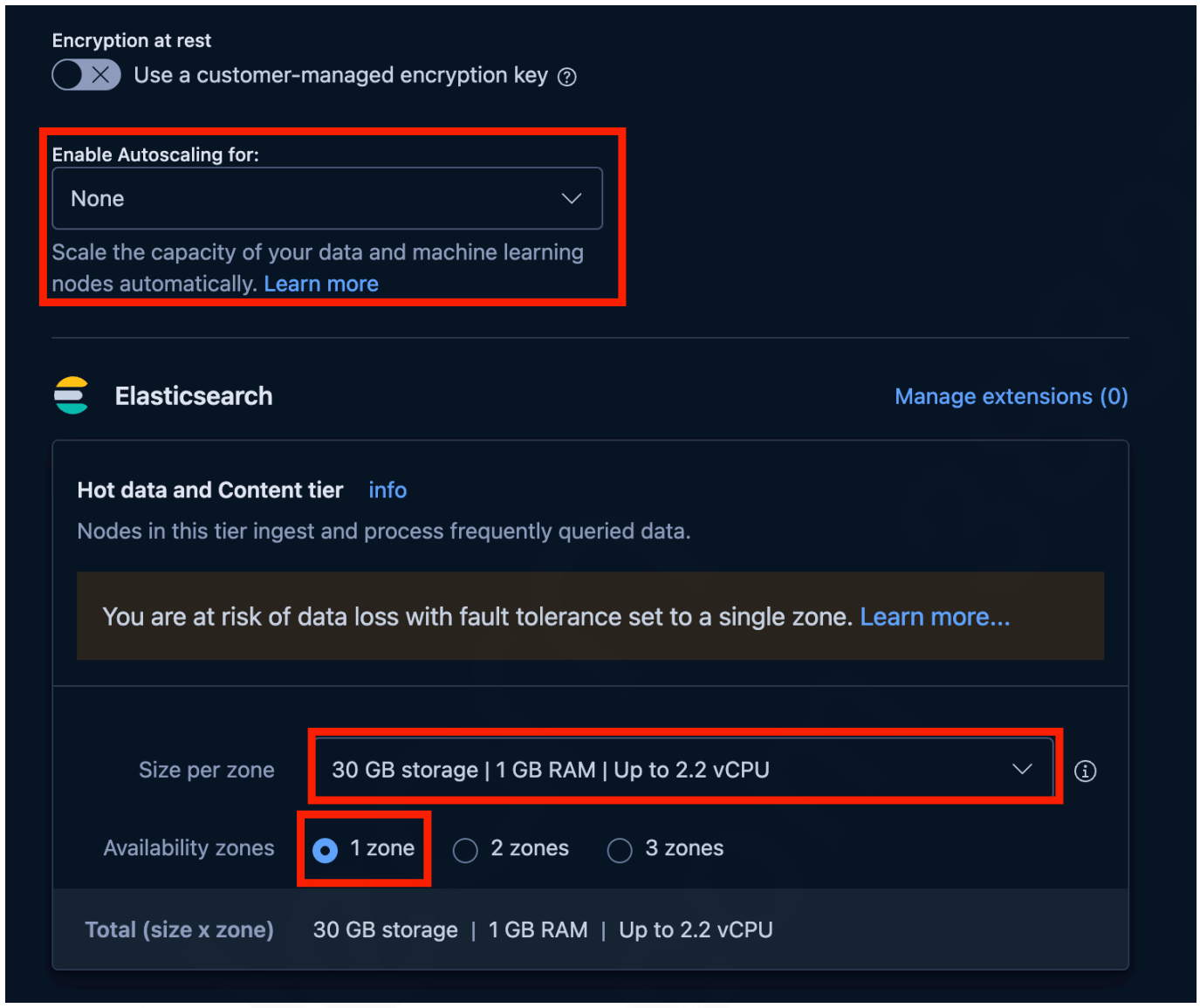






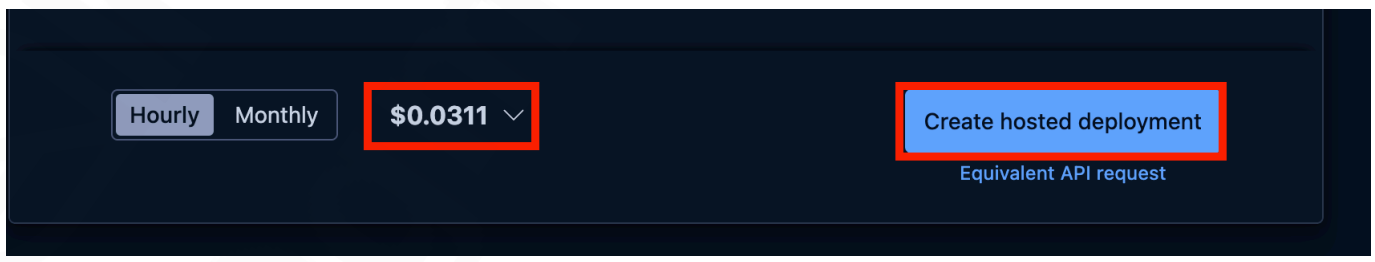
Version은 8 버전 중에 가장 최신 버전을 선택하면 된다.

khb3023



(나머지는 다 그대로 두기)

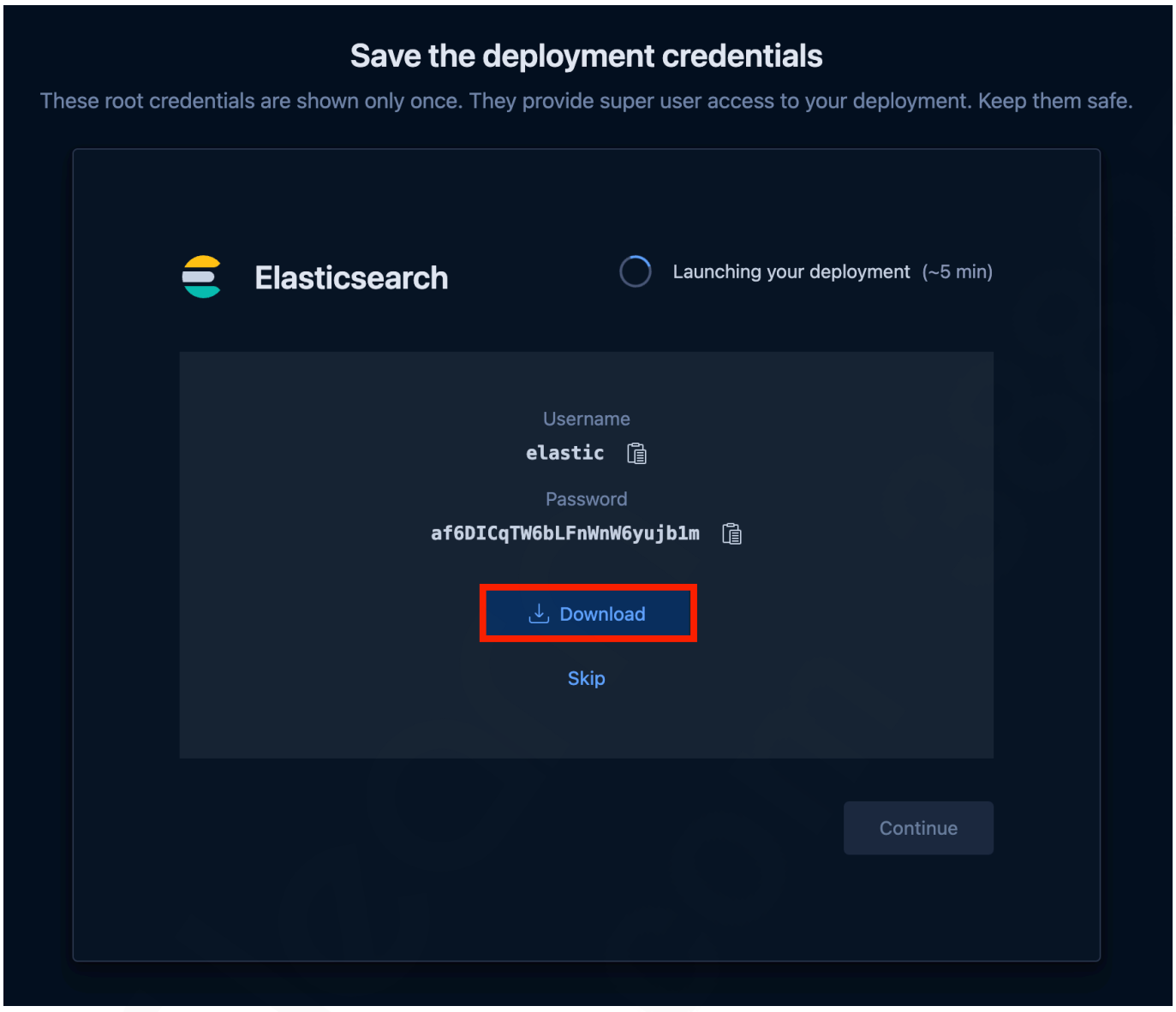
맨 아래로 쭉 내렸을 때 비용이 $0.0311이라고 되어있다면 잘 설정한 것이다. `Create hosted deployment` 버튼을 클릭하자.



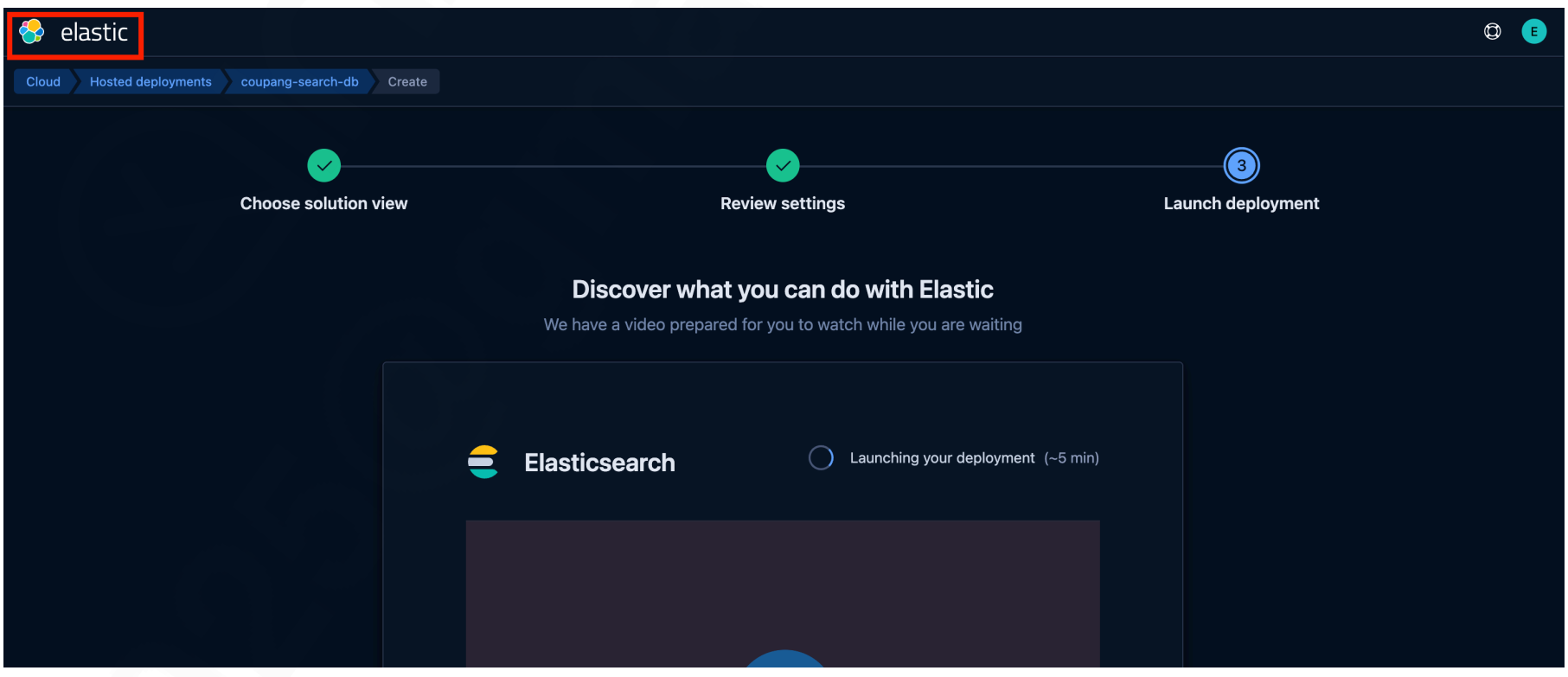
5. **Elasticsearch 비밀번호 적어두기**

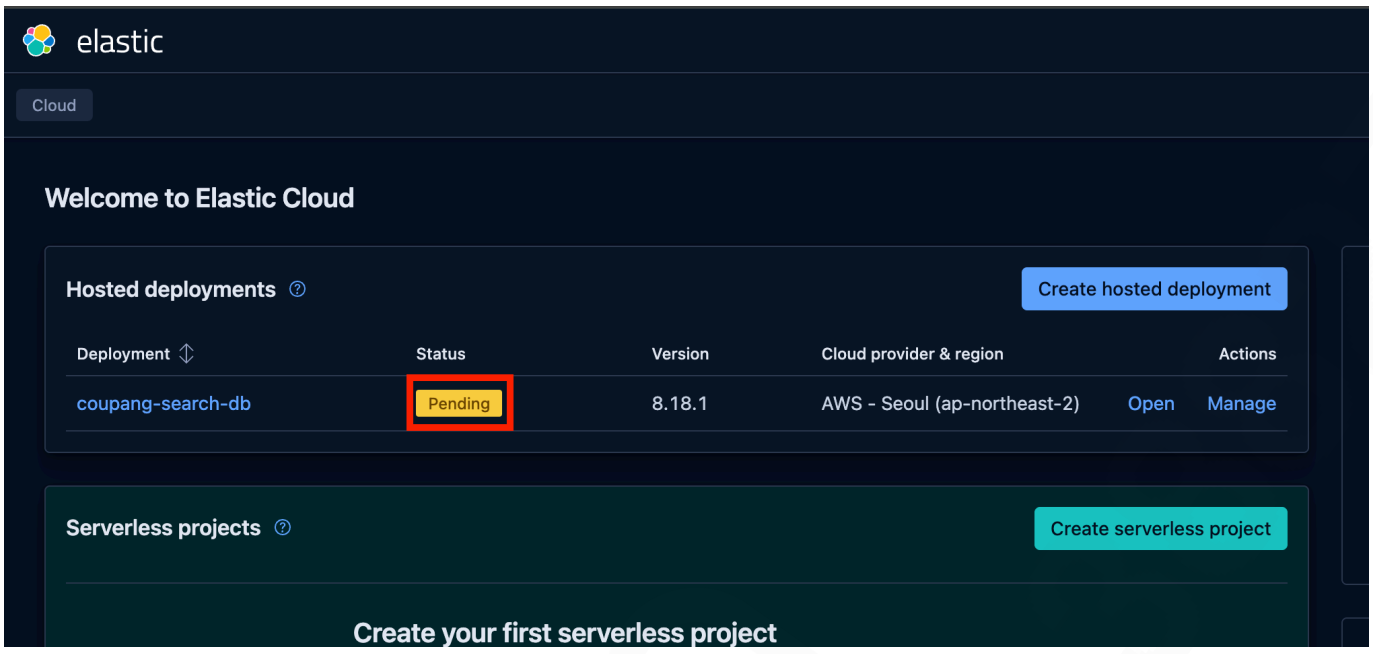
Elasticsearch에 접근할 때 사용하는 username, password이다. 잊어버리지 않게 잘 적어두거나 파일로 다운 받아두자.

khb3023


Save the deployment credentials
These root credentials are shown only once. They provide super user access to your deployment. Keep them safe.
Elasticsearch
Launching your deployment (~5 min)
Username
elastic
Password
af6DICqTW6bLFnWnW6yujb1m
Download
Skip
Continue

6. 홈으로 돌아오기


elastic
Cloud
Hosted deployments
coupang-search-db
Create
Choose solution view
Review settings
3
Launch deployment
Discover what you can do with Elastic
We have a video prepared for you to watch while you are waiting
Elasticsearch
Launching your deployment (~5 min)



Elasticsearch가 생성되기까지 5분 정도 걸린다. 5분 정도 기다렸다가 새로고침을 해보면 아래와 같이 Status가 `Healthy`로 바뀐 걸 확인할 수 있다.

Elastic Cloud를 활용해 Elasticsearch를 구축했으니, 다음 강의에서는 Spring Boot에 Elastic Cloud를 통해 생성한 Elasticsearch를 연결해보자.

khb3025@gmail.com

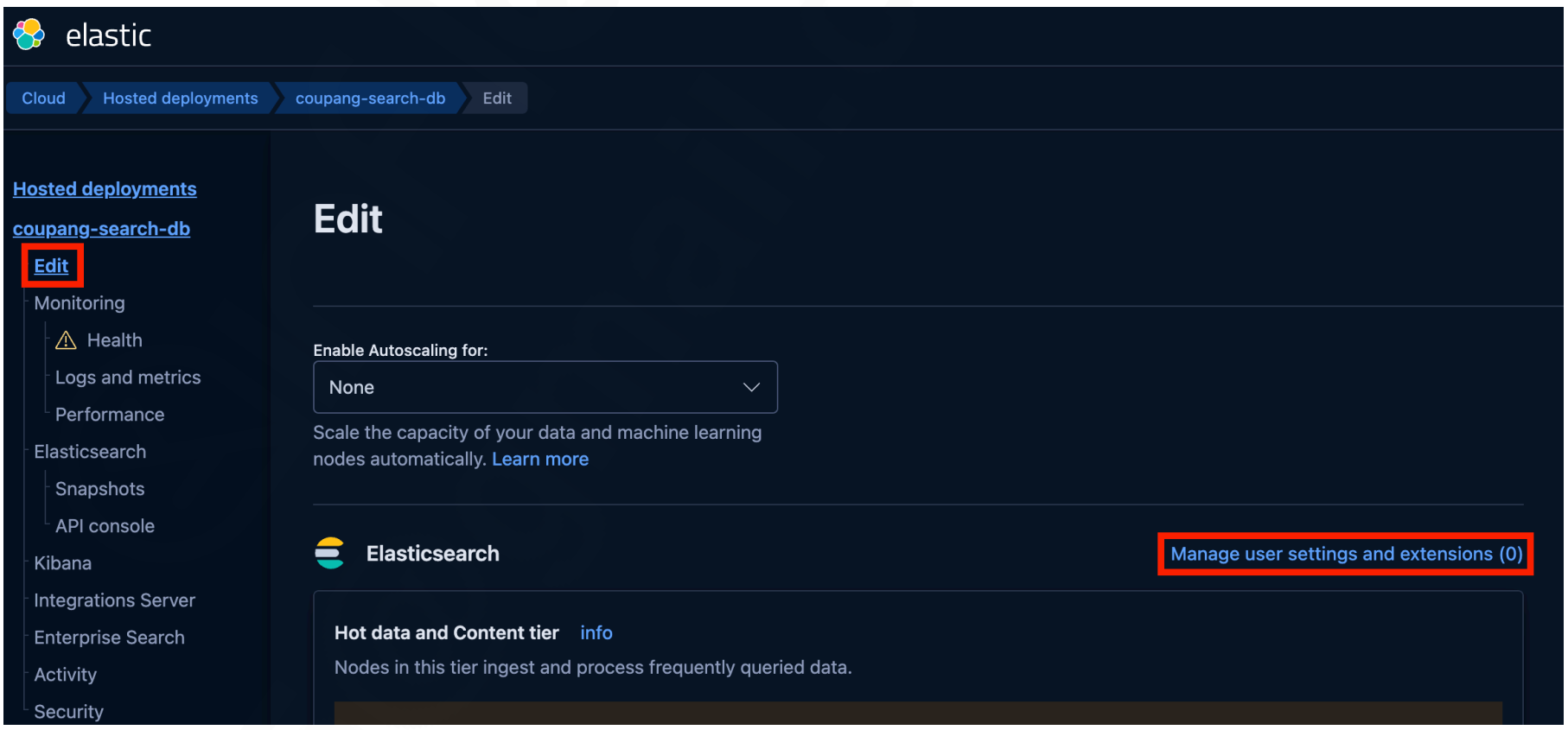
🎯

[실습] Spring Boot에 Elasitc Cloud의 Elasticsearch 연결하기

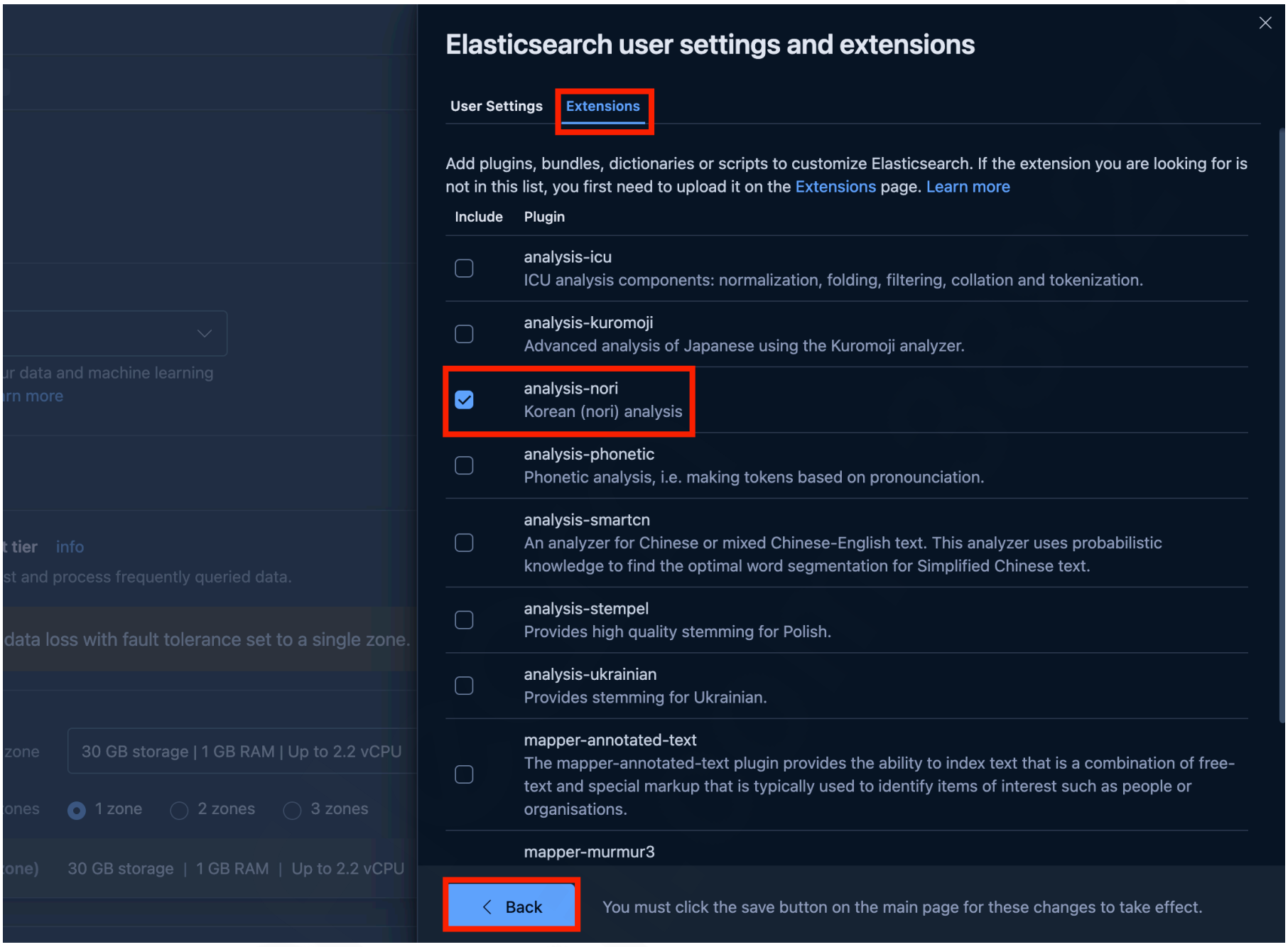
✅ Spring Boot에 Elasitc Cloud의 Elasticsearch 연결하기

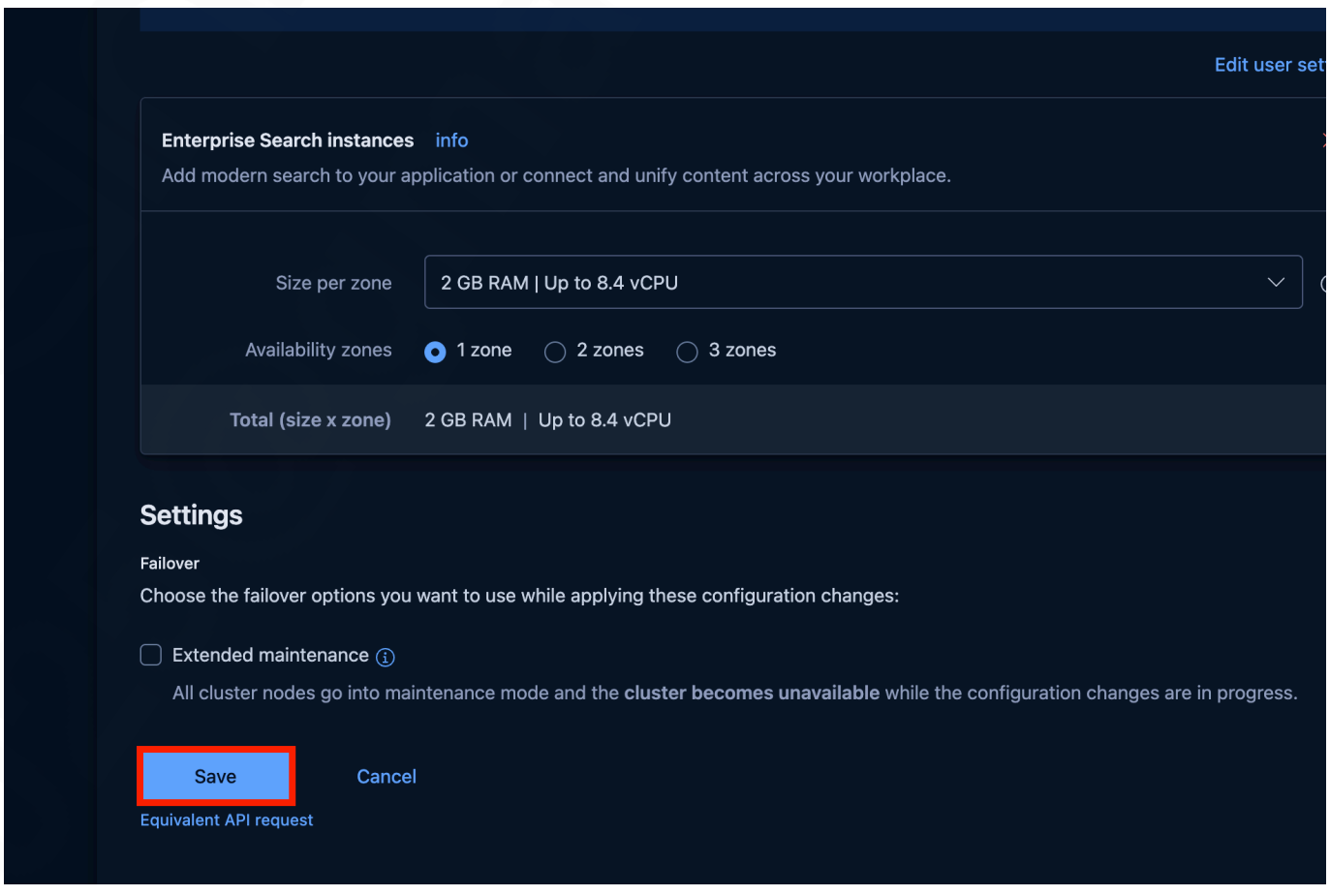
Spring Boot에 Elastic Cloud의 Elasicsearch를 연결하기 위해서는 **주소(Endpoint)**, **username**, **password** 이 3가지를 알아야 한다.
그리고 nori anaylzer를 사용하기 위해서는 **nori analyzer 플러그인을 셋팅**해주어야 한다.

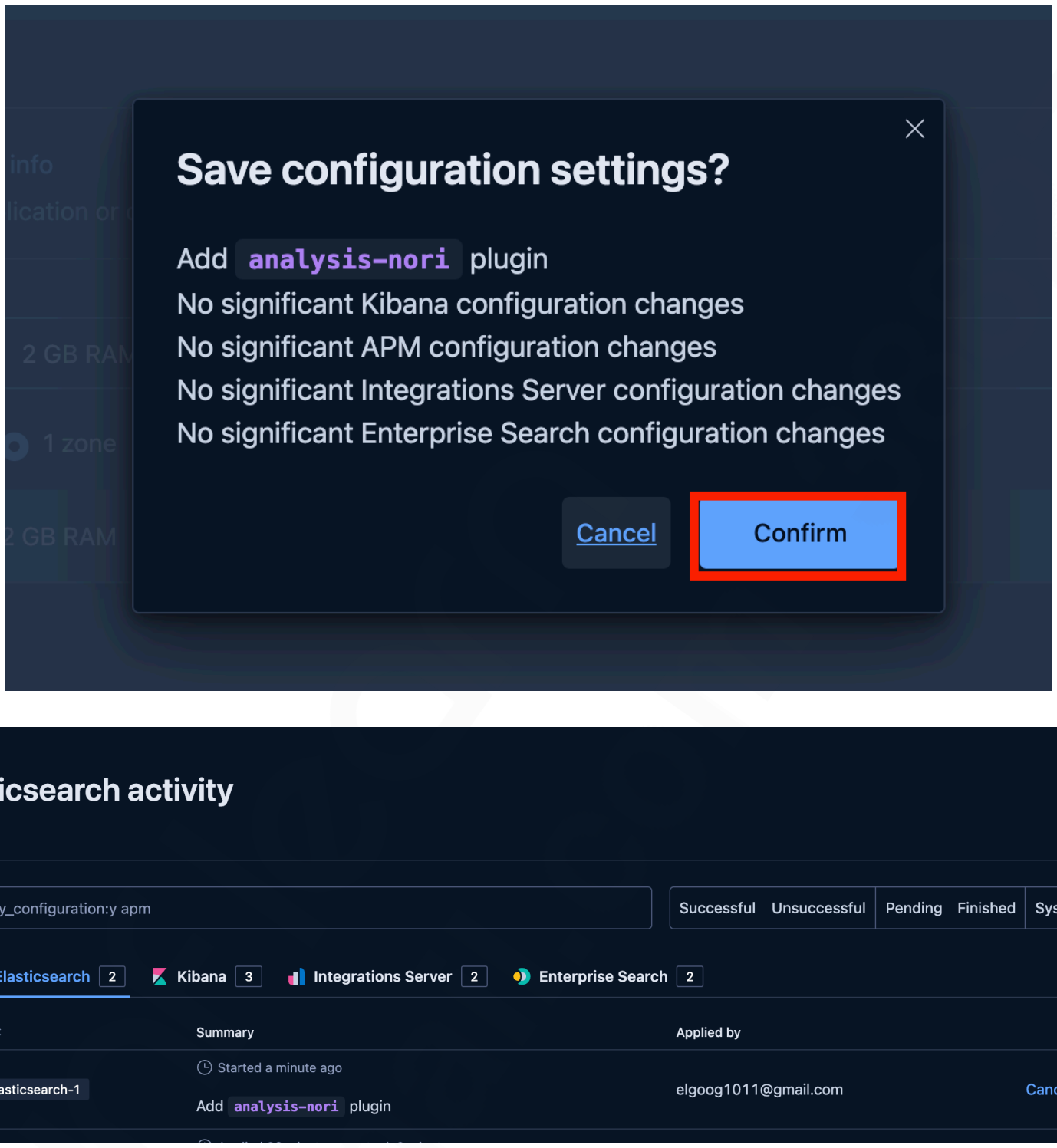
1. **nori analyzer 플러그인 셋팅하기**

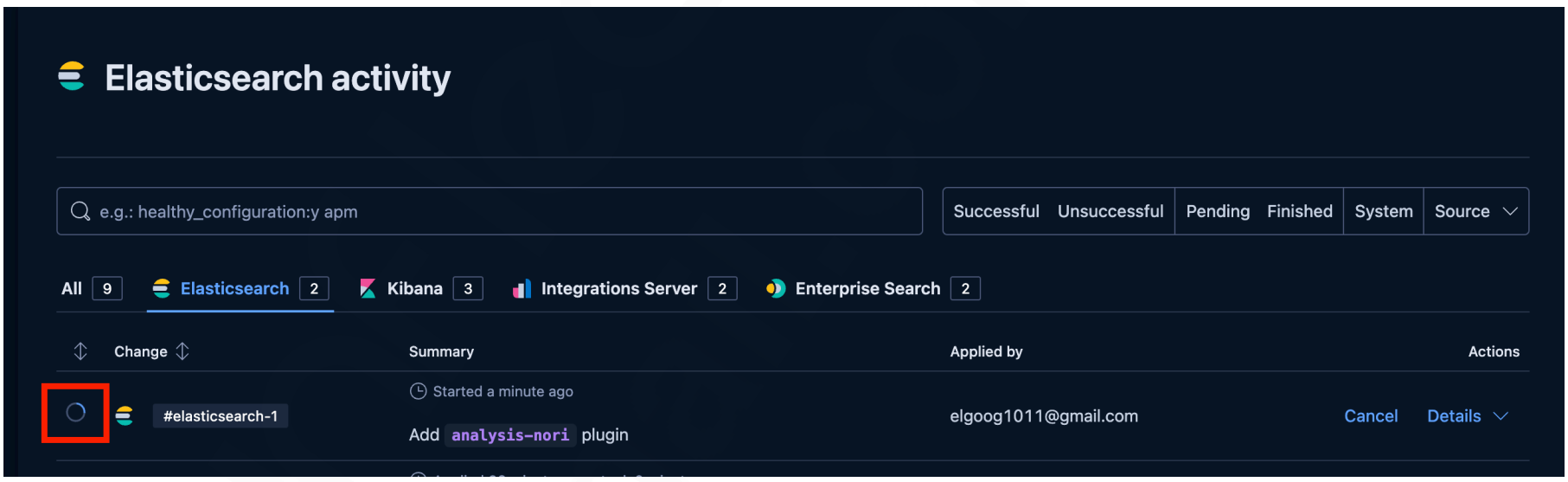


khb3025@naver.com

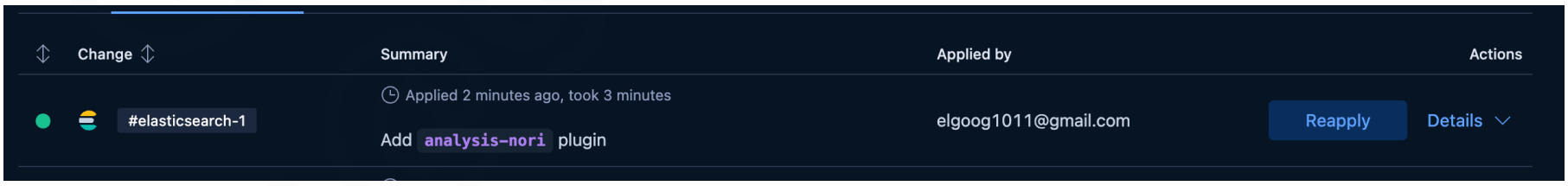

Elasticsearch user settings and extensions
User Settings
Extensions
Add plugins, bundles, dictionaries or scripts to customize Elasticsearch. If the extension you are looking for is not in this list, you first need to upload it on the Extensions page. Learn more
Include
Plugin
analysis-icu
ICU analysis components: normalization, folding, filtering, collation and tokenization.
analysis-kuromoji
Advanced analysis of Japanese using the Kuromoji analyzer.
analysis-nori
Korean (nori) analysis
analysis-phonetic
Phonetic analysis, i.e. making tokens based on pronounciation.
analysis-smartcn
An analyzer for Chinese or mixed Chinese-English text. This analyzer uses probabilistic knowledge to find the optimal word segmentation for Simplified Chinese text.
analysis-stempel
Provides high quality stemming for Polish.
analysis-ukrainian
Provides stemming for Ukrainian.
mapper-annotated-text
The mapper-annotated-text plugin provides the ability to index text that is a combination of free-text and special markup that is typically used to identify items of interest such as people or organisations.
mapper-murmur3
Back
You must click the save button on the main page for these changes to take effect.


Edit user set
Enterprise Search instances info
Add modern search to your application or connect and unify content across your workplace.
Size per zone
2 GB RAM | Up to 8.4 vCPU
Availability zones
1 zone
2 zones
3 zones
Total (size x zone)
2 GB RAM | Up to 8.4 vCPU
Settings
Failover
Choose the failover options you want to use while applying these configuration changes:
Extended maintenance
All cluster nodes go into maintenance mode and the cluster becomes unavailable while the configuration changes are in progress.
Save
Cancel
Equivalent API request



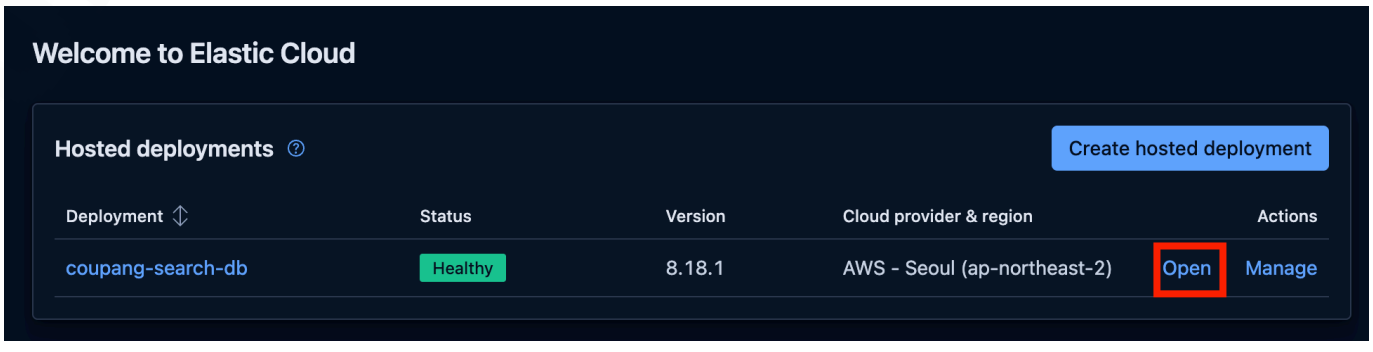


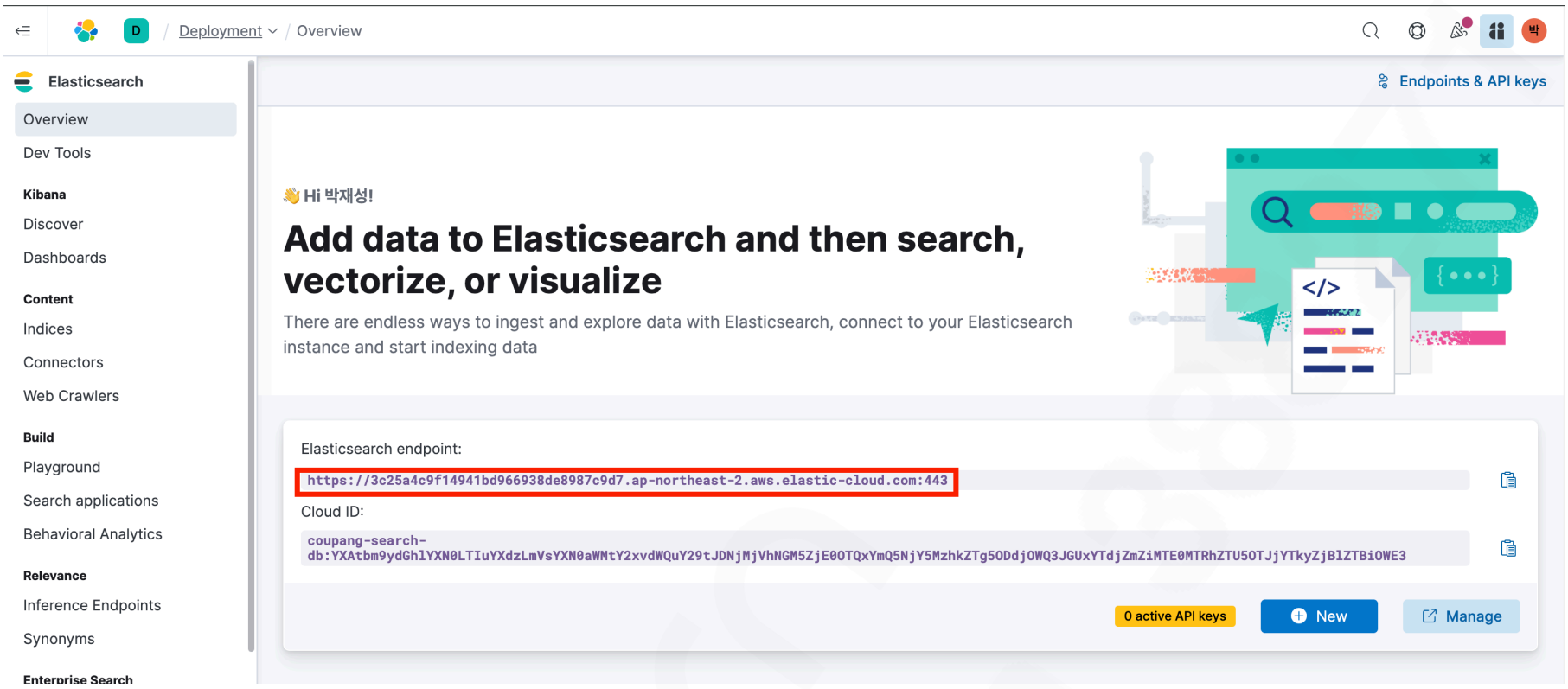
플러그인이 설치 및 셋팅되는 데 시간이 5분 정도 걸린다. 조금만 기다려주자.



위 이미지는 설치가 완료된 모습이다.

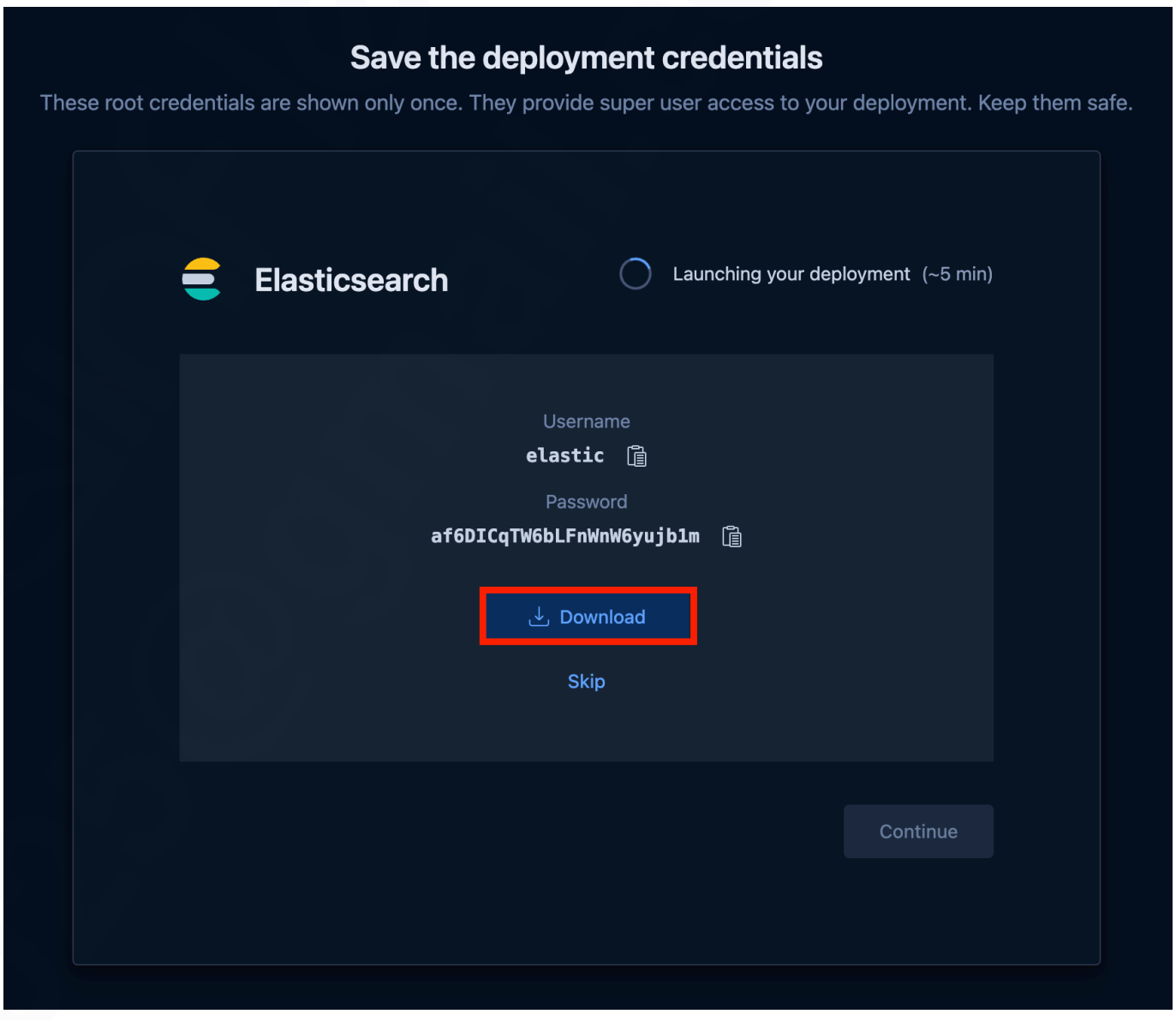
2. Elasticsearch 주소(Endpoint) 확인하기





2. **username, password 확인하기**

   이전 강의에서 elasticsearch의 deployment를 생성하면서 username이랑 password는 잘 적어두라고 얘기했었다.



3. **Spring Boot 프로젝트에 생성한 Elasticsearch 정보 입력하기**

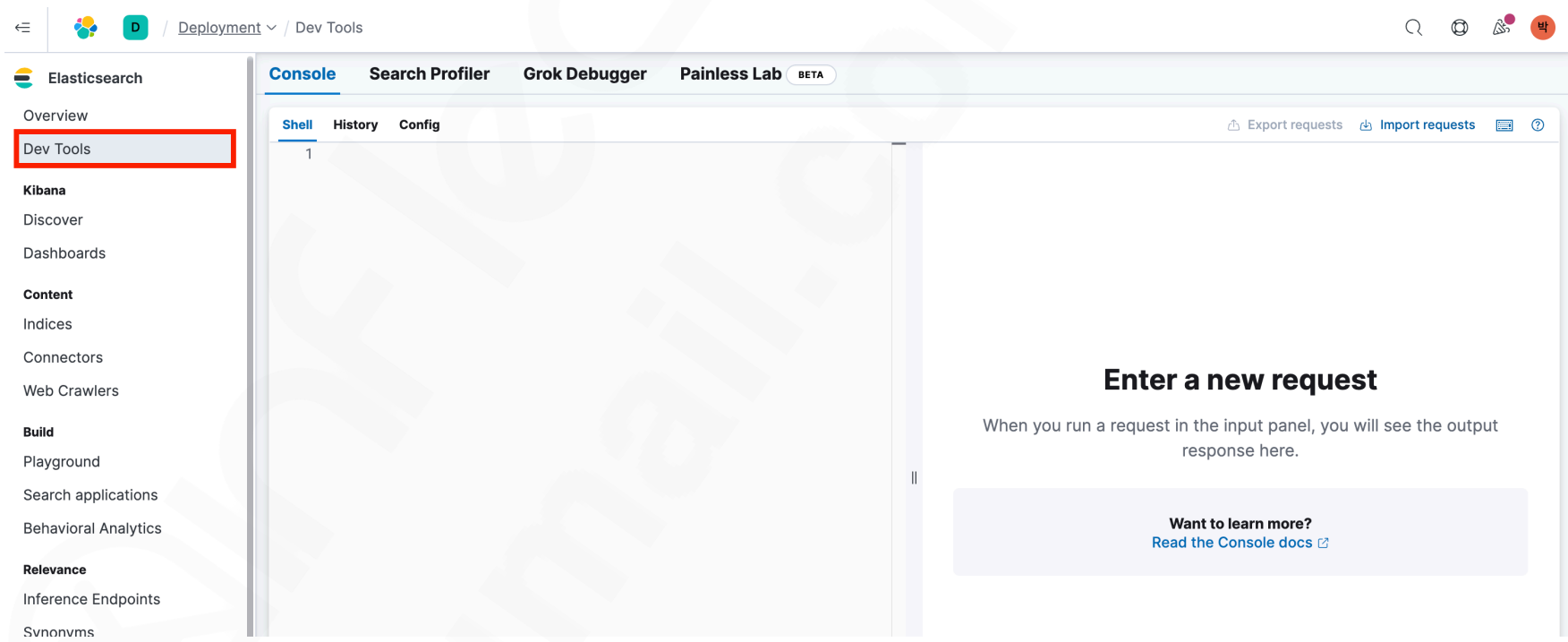
   application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/coupang
    username: root
    password: password123
    driver-class-name: com.mysql.cj.jdbc.Driver
```

```
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  elasticsearch:
    uris: https://3c25a4c9f14941bd966938de8987c9d7.ap-northeast-2.aws.elastic-cloud.com:443
    username: elastic
    password: af6DICqTW6bLFnWnW6yujb1m

logging:
  level:
    org.elasticsearch.client: TRACE
```

자신이 생성한 Elasticsearch의 값에 맞게 바꿔서 작성하도록 하자.

5. **Elasticsearch에 잘 연결됐는 지 테스트해보기**
   a. **Kibana에서 인덱스 조회해보기**




```
GET /products
```

   b. **Postman 통해서 데이터 삽입 후 검색해보기**

localhost:8080/products

```
{
  "name": "삼성 스마트폰 A17",
  "description": "최신형 스마트폰, 휴대전화",
  "price": 1500000,
  "rating": 4.5,
  "category": "휴대폰"
}
```

Kibana에서 데이터 조회해보기

```
GET /products/_search
```

지금까지 Elasticsearch의 기능에 대해 공부하고, 그 기능을 Spring Boot에서 적용시켜보며, 실제 Elastic Cloud의 Elasticserch에까지 연결시켜서 성공적으로 프로젝트를 완성했다.

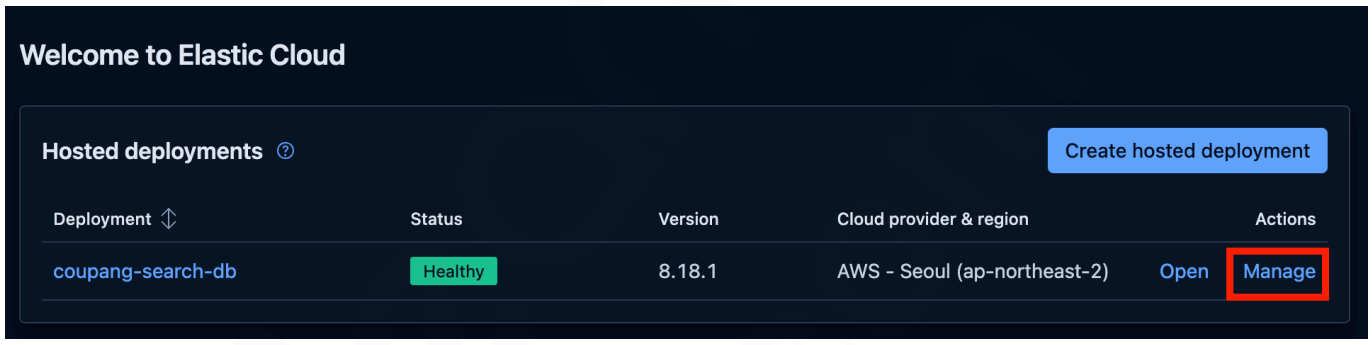
@kfc38821@gmail.com
khb302500

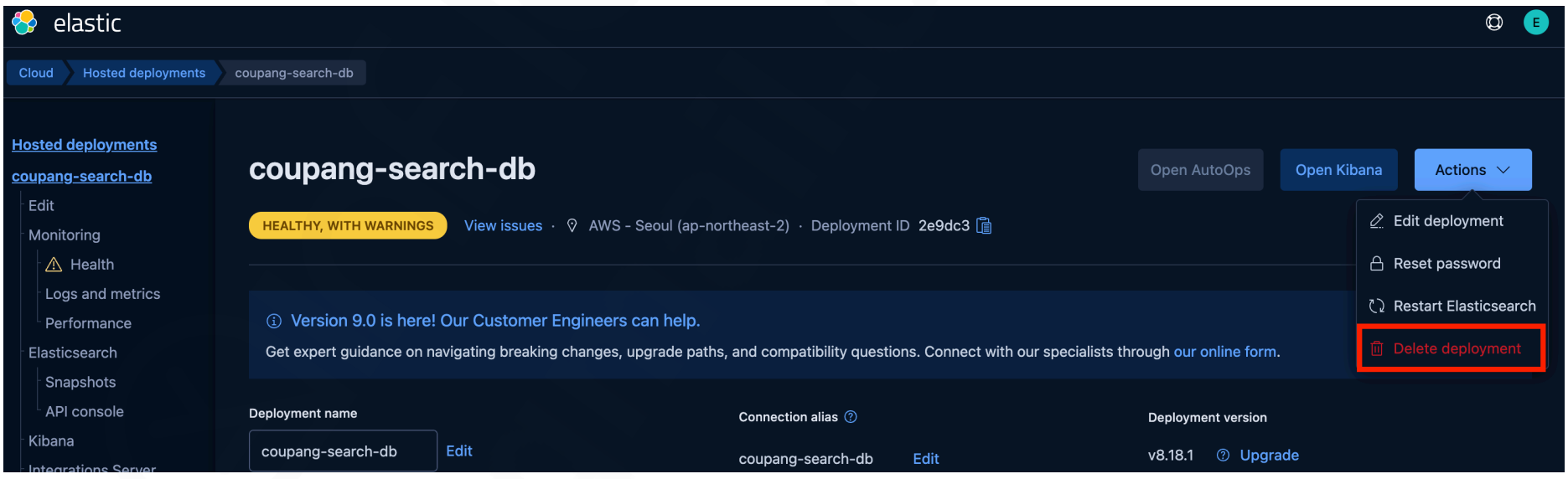
📖

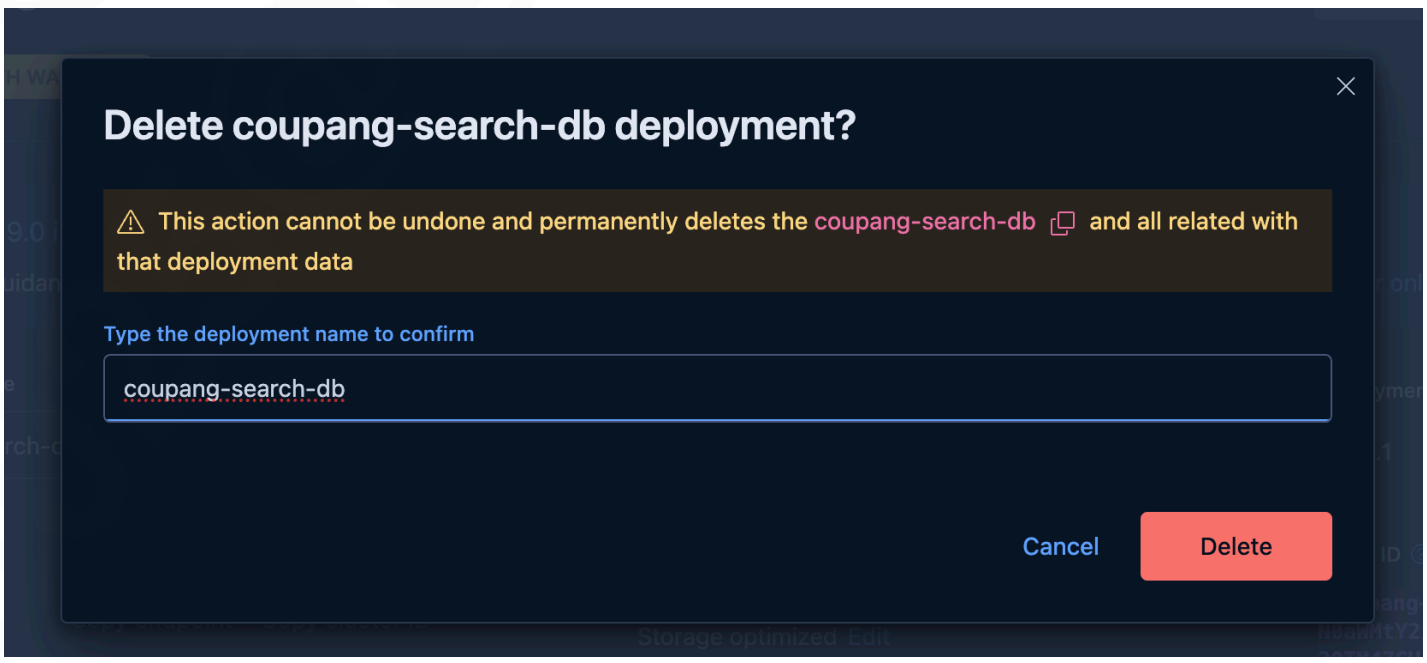
[보충 강의] 비용 나가지 않게 Elastic Cloud 리소스 정리하기

✅ 비용 나가지 않게 Elastic Cloud 리소스 정리하기

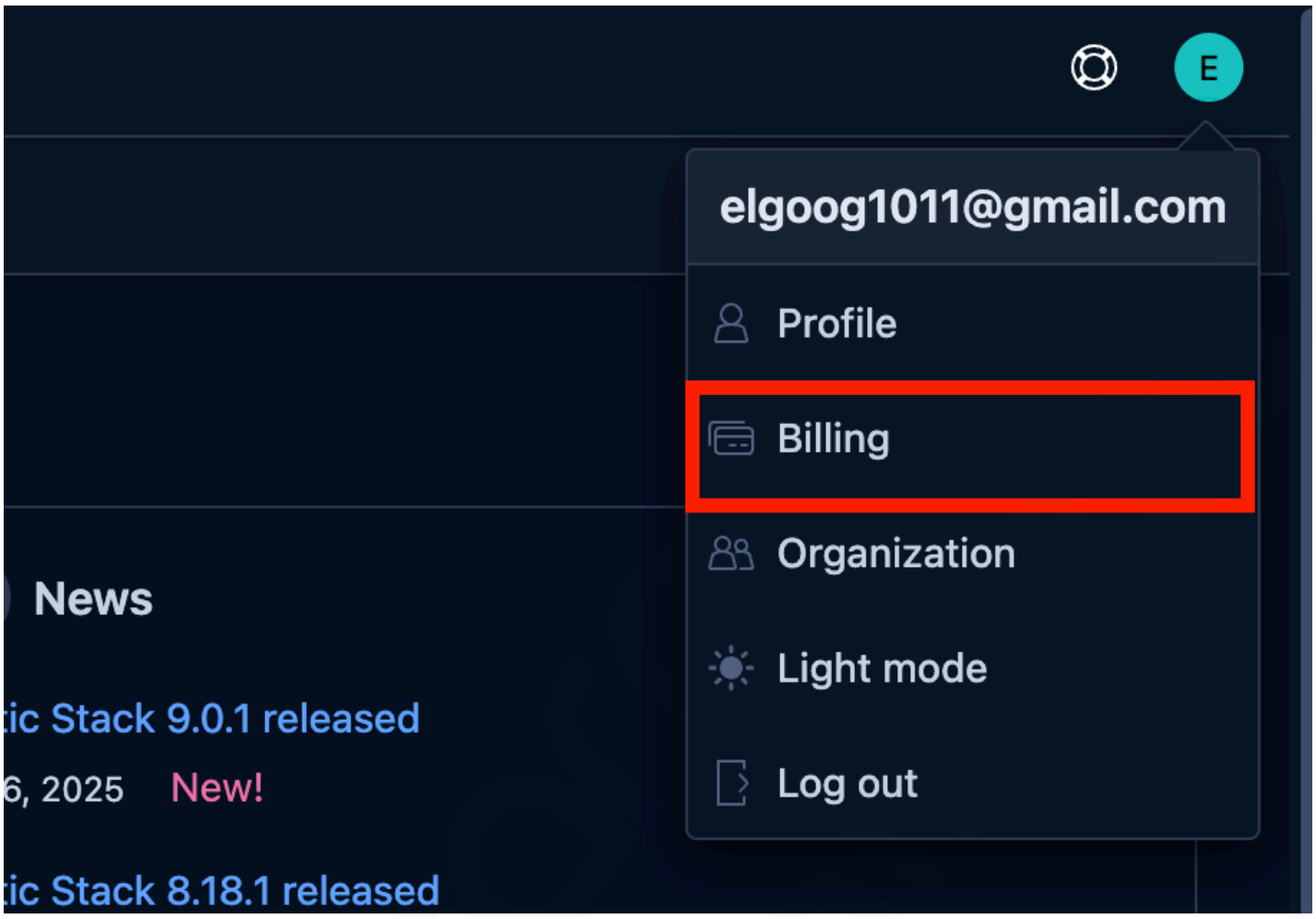
> 14일 무료 체험판을 사용하신 분은 별도로 리소스를 종료하지 않아도 된다. 14일이 지나면 알아서 체험판이 종료된다.

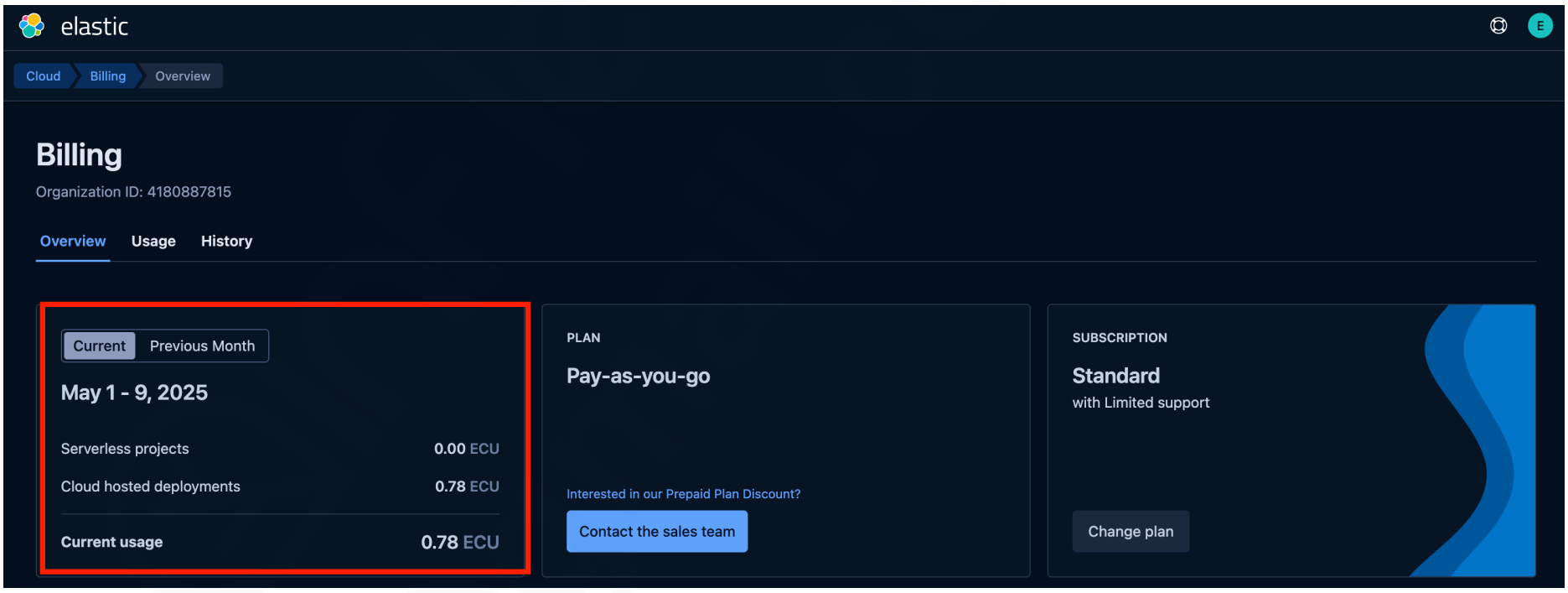






✅ 혹시나 비용이 나가는 건 아닌 지 체크하는 방법


E
elgoog1011@gmail.com
Profile
Billing
Organization
Light mode
Log out
News
ic Stack 9.0.1 released
6, 2025
New!
ic Stack 8.18.1 released


elastic
E
Cloud
Billing
Overview
Billing
Organization ID: 4180887815
Overview
Usage
History
Current
Previous Month
May 1 - 9, 2025
Serverless projects
0.00 ECU
Cloud hosted deployments
0.78 ECU
Current usage
0.78 ECU
PLAN
Pay-as-you-go
Interested in our Prepaid Plan Discount?
Contact the sales team
SUBSCRIPTION
Standard
with Limited support
Change plan

kb302508